BrandShield

Documentation

Digital Risk Protection platform — architecture, API, data model, features and operations

Generated from the repository source on 2026-10-23

Contents

1. Overview
2. Documentation index
3. Architecture
4. Tech stack
5. Setup and deployment
6. Configuration
7. API reference
8. Data model
9. Auth and security
10. Frontend
11. Background jobs and integrations
12. Testing
13. Troubleshooting
14. Contributing
15. Glossary
16. Open questions
17. Feature: Authentication and accounts
18. Feature: Brand registry
19. Feature: Scanning and detection
20. Feature: Social and app threats
21. Feature: Web threats
22. Feature: Look-alike Lab
23. Feature: Candidate import
24. Feature: Investigations
25. Feature: Source Health and providers
26. Feature: Model Evaluation
27. Feature: Settings and audit
28. Feature: Dashboard
29. Feature: Brand Studio
30. Feature: Voice assistant
31. Feature: Landing page

README.MD

BrandShield

BrandShield is a Digital Risk Protection platform. You describe your brand (name, aliases, official social handles, official apps, domains, logo). BrandShield scans public sources for look-alike **social profiles, fake apps and look-alike websites**, scores every suspect with a reason for every point, excludes your own official assets, and prepares evidence-backed reports. **A person reviews every detection and approves every report; nothing is ever sent or taken down automatically** (notifications and takedown e-mails are stored preferences only, `backend/app/api/settings.py`). It also ships **Brand Studio**, an AI toolkit for naming, logos, palettes and copy, which registers a new brand in BrandShield in one step. One login serves both, and every account owns its own brand (`backend/app/models/brand.py:14`, `owner_id`).

Who it is for: brand-protection and security analysts who need to find and triage impersonation (analyst role), administrators who run the sources and settings (admin role), and read-only stakeholders (viewer role). Roles: `backend/app/config/deps.py:46`.

Full documentation index: **docs/index.md**.

Features

Area	What it does	Guide
Overview	Counts, trends and the scan trigger	dashboard
Scanning and detection	Collect, de-duplicate, allowlist, name/logo/text signals, score, human review	scanning-and-detection
Social and app threats	Fake profiles, pages and apps (Social Threats, App Threats pages)	social-and-app-threats
Web threats	Look-alike domains: registrable-domain comparison, punycode, Safe Browsing	web-threats
Look-alike Lab	Name tester, test cases, logo comparison (CLIP or perceptual hashes)	lookalike-lab
Candidate import	Add suspects by hand or from CSV/JSON	candidate-import
Accounts	Sign-in, sign-up, roles, demo accounts	authentication-and-accounts
Brand Registry	Brand profile, official handles/apps/domains/logo (the allowlist)	brand-registry
Investigations	Status, resolution, notes and history per detection	investigations
Reports	Evidence-backed report drafts; a person approves	not written yet (open questions)
Source Health and providers	Collector status, credits, text/image providers (admin)	source-health-and-providers
Model Evaluation	Synthetic evaluation of the name matcher (admin)	model-evaluation
Settings and audit	Data mode, schedule, thresholds, audit log (admin)	settings-and-audit
Brand Studio	AI naming, logo, palette, copy under `/studio/`	brand-studio
Voice agent	Spoken navigation and form filling (Chrome/Edge)	voice-agent
Landing page	Public marketing page at `/` for signed-out visitors	landing-page

Routes and which are admin-only: `frontend/src/App.tsx:89-179`.

Prerequisites

Need	Version	Source
Python	3.11+ (3.12 used by Docker, Vercel and CI lint)	`backend/pyproject.toml:10`, `backend/.python-version`, `backend/Dockerfile:8`
Node.js	22	`frontend/Dockerfile:8`, `.github/workflows/ci.yml:36`
Docker with compose plugin	optional, for the container stack	`docker-compose.yml`
PostgreSQL / Redis	optional; SQLite is the default database and scans run in a thread without Redis	`backend/app/config/config.py:28`, `:110-111`

Quick start

1. Create the backend settings file and fill the three required values (`JWT_SECRET` 16+ characters, `ADMIN_EMAIL`, `ADMIN_PASSWORD`):

   ```
   copy backend\.env.example backend\.env
   python -c "import secrets; print(secrets.token_urlsafe(48))"    # use as JWT_SECRET
   ```

   `JWT_SECRET` has no default, so the API will not start without it (`backend/app/config/config.py:40`).

2. Start the API (first run creates `backend/.venv`, installs `requirements.txt`, applies migrations, then serves on port 8000):

```
npm run backend
```

3. In a second terminal start the web apps (first run runs `npm install` in `frontend/` and `studio/`):

```
npm run frontend
```

4. Open **http://localhost:5173** (BrandShield; Brand Studio is at `/studio/` on the same address). API docs: http://localhost:8000/docs.

Both commands work from the project root (`package.json:5-8`) and from the parent folder `ScamMind AI New/` (its `package.json` forwards them with `npm --prefix "ScamMind AI"`). What they do, ports and port checks: docs/setup-and-deployment.md.

With no live source configured the app is in **sample mode**: detections are fictional and a "Sample data" badge is shown (`backend/app/core/collectors/selection.py:68-100`). Sign in with `ADMIN_EMAIL` / `ADMIN_PASSWORD` , the demo button if `DEMO_EMAIL` / `DEMO_PASSWORD` are set, or create an account at `/register` .

Commands

Task	Command	Where
API (migrate + reload server)	`npm run backend`	project root
Web apps (BrandShield 5173, Studio 3000)	`npm run frontend`	project root
Backend tests	`python -m pytest -q`	`backend/` (use `.venv`)
Backend lint	`python -m ruff check .`	`backend/`
Frontend lint / type-check / tests / build	`npm run lint` / `npm run typecheck` / `npm test` / `npm run build`	`frontend/`
Studio type-check / tests / build	`npm run lint` / `npm test` / `npm run build`	`studio/`
Browser tests	`npm run e2e:install` once, then `npm run test:e2e`	`frontend/`
Docker stack	`docker compose up --build` (after `copy .env.example .env`)	project root
Apply migrations by hand	`python -m scripts.migrate`	`backend/`
Pipeline self-check	`python -m scripts.canary`	`backend/`

Details: docs/testing.md and docs/setup-and-deployment.md.

Project layout

Path	Contents
`backend/`	FastAPI app: `app/api` routers, `app/core` (engine, collectors, pipeline, providers), `app/services` , `app/models` , `alembic/` migrations (0001-0012), `scripts/` , `tests/`
`frontend/`	React 19 + TypeScript + Vite app (`src/pages` , `src/components` , `src/api` , `src/hooks`), `e2e/` Playwright specs, `nginx/` for the container
`studio/`	Brand Studio (React + Vite + Tailwind, base `/studio/` , `studio/vite.config.ts:11`)
`docs/`	Documentation: index, plus existing guides (`API.md` , `DECISIONS.md` , `SECURITY.md` , `DEPLOY_VERCEL.md` , `USER_GUIDE.md` , `FEATURE_STATUS.md` , ...)
`scripts/dev.mjs`	The two one-command dev launchers
`vercel.json`	Single Vercel project with three services and a daily cron
`docker-compose.yml` , `docker-compose.dev.yml`	api, worker, db, redis, frontend, studio
`.github/workflows/ci.yml`	CI pipeline

Health and safety notes

- `GET /health` (readiness), `GET /health/live` (liveness): `backend/app/api/health.py:22-33` .
- Passwords use bcrypt, tokens are JWT with server-side refresh rotation, secrets are redacted from logs (`backend/app/config/logging.py`). Read docs/SECURITY.md before exposing the app.
- The in-memory rate limiters are per process (`backend/app/config/ratelimit.py`).
- Every account sees only its own brand and detections; another account's records answer 404 (`backend/tests/api/test_tenant_isolation.py`).

More documentation

docs/index.md · setup and deployment · testing · troubleshooting · contributing · glossary · API · Vercel guide · user guide · feature status

DOCS/INDEX.MD

BrandShield documentation index

Plain-language summary. BrandShield finds fake social profiles, fake apps and look-alike websites that imitate a brand, explains every score, and leaves each decision to a person. This folder explains how it is built and how to run it. New to the project? Read the README, then architecture, then the feature you care about.

Start here

Document	What it covers
../README.md	What the app does, quick start (`npm run backend` , `npm run frontend`), commands
architecture.md	Components, diagrams (context, components, deployment), folders, design decisions
tech-stack.md	Every language, library, service and tool, with versions
setup-and-deployment.md	Local setup, Docker, Vercel and Supabase, CI/CD, rollback, logging
configuration.md	Every environment variable and config file (names only)

Reference

Document	What it covers
api-reference.md	Every endpoint: auth, request, response, status codes
data-model.md	Tables, relationships (ER diagram), migrations 0001-0012
auth-and-security.md	Login and tokens, roles, tenant isolation, demo accounts, known risks
frontend.md	Pages, routing, state, components, styling, API integration
background-jobs-and-integrations.md	Scheduler, worker, cron, and every third-party service
testing.md	Test strategy, how to run, structure
troubleshooting.md	Common errors, causes and fixes
contributing.md	Code style, how to add endpoints, pages, migrations, collectors
glossary.md	Domain terms as this code uses them
open-questions.md	Things that need your decision, and defects found

Features

Feature	Document
Sign-in, accounts, roles, demo accounts	features/authentication-and-accounts.md
Brand profile, multiple brands, official assets	features/brand-registry.md
Scanning and detection (collectors, scoring)	features/scanning-and-detection.md
Social Threats and App Threats	features/social-and-app-threats.md
Web Threats (look-alike domains, Safe Browsing)	features/web-threats.md
Look-alike Lab (names, URLs, logos)	features/lookalike-lab.md
Candidate import (manual and file)	features/candidate-import.md
Investigations	features/investigations.md
Reports	*not written yet, see open-questions.md section 5*
Source Health and providers	features/source-health-and-providers.md
Model Evaluation	features/model-evaluation.md
Settings and audit	features/settings-and-audit.md
Dashboard	features/dashboard.md
Brand Studio	features/brand-studio.md
Voice assistant	features/voice-agent.md
Landing page	features/landing-page.md

Older documents in this folder

These predate the set above and may be out of date; where they differ, the code and the documents above win: API.md, COLLECTORS.md, DATABASE_SCHEMA.md (generated), DECISIONS.md, DEPENDENCIES.md, DEPLOY_VERCEL.md, DONE_AUDIT.md, EXTENSION_PLAN.md, FEATURE_STATUS.md, FRONTEND_CONTRACT.md,

SECURITY.md, SPEC_FROM_REFERENCE.md, STUDIO.md, STUDIO_INTEGRATION_CONTRACT.md, SUBMISSION.md, USER_GUIDE.md, schema.sql.

DOCS/ARCHITECTURE.MD

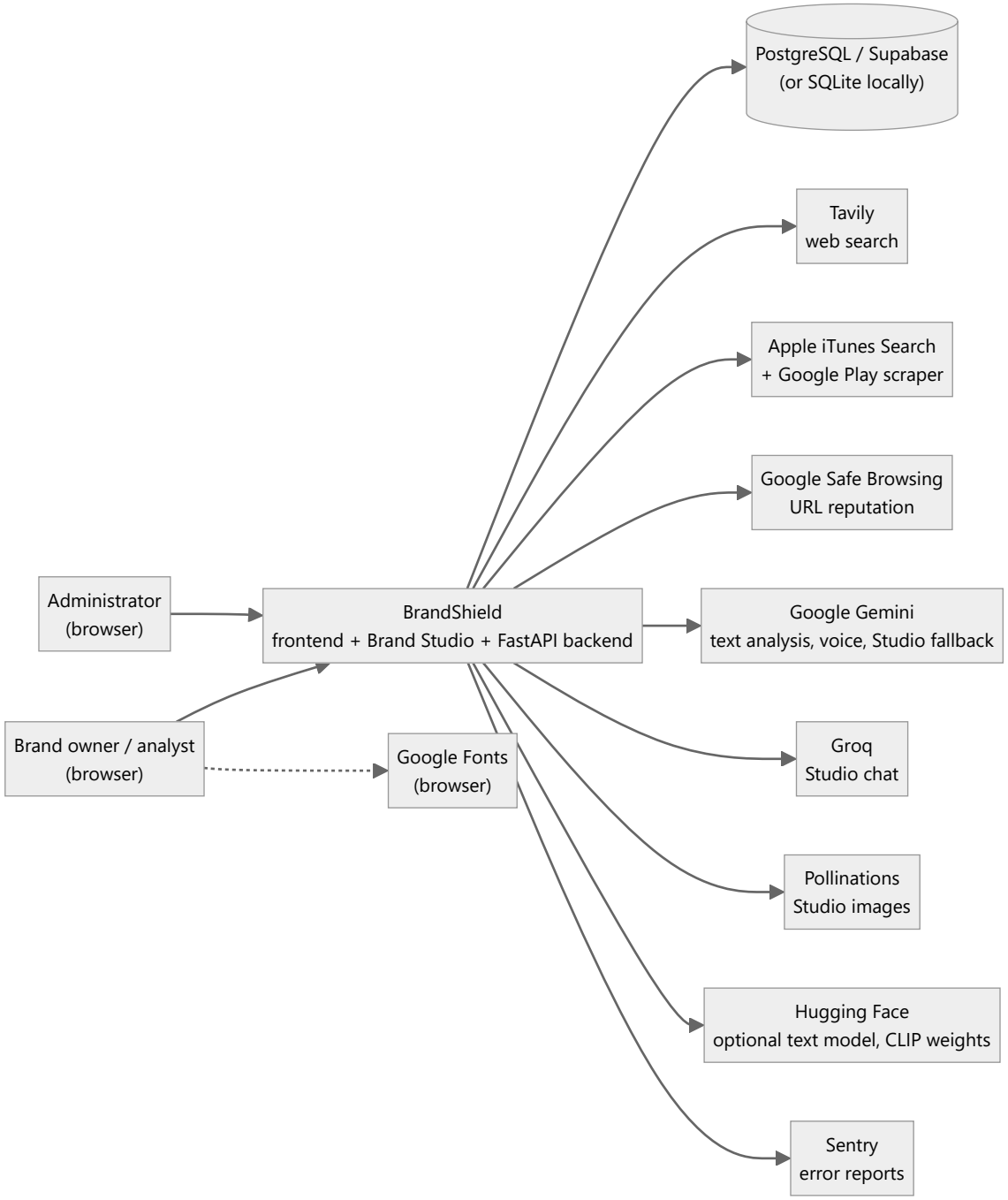
Architecture

Plain-language summary

- BrandShield helps a company find fake social-media accounts, fake mobile apps and look-alike websites that imitate its brand.
- Each person who signs up gets their own private workspace (their own brand). A background "scan" collects possible fakes, scores them, and shows them to a human, who decides what is real. Nothing is reported automatically.
- It is made of a Python server (the "backend"), a React website (the "frontend") and a second React tool, Brand Studio, for creating brand names, logos and content.
- Without any outside keys it runs on built-in sample data. Real web search, AI and link-reputation checks switch on only when keys are provided.
- It can run on Vercel (three services on one domain plus a daily scheduled call) or with Docker Compose (database, queue, API, worker, web servers).

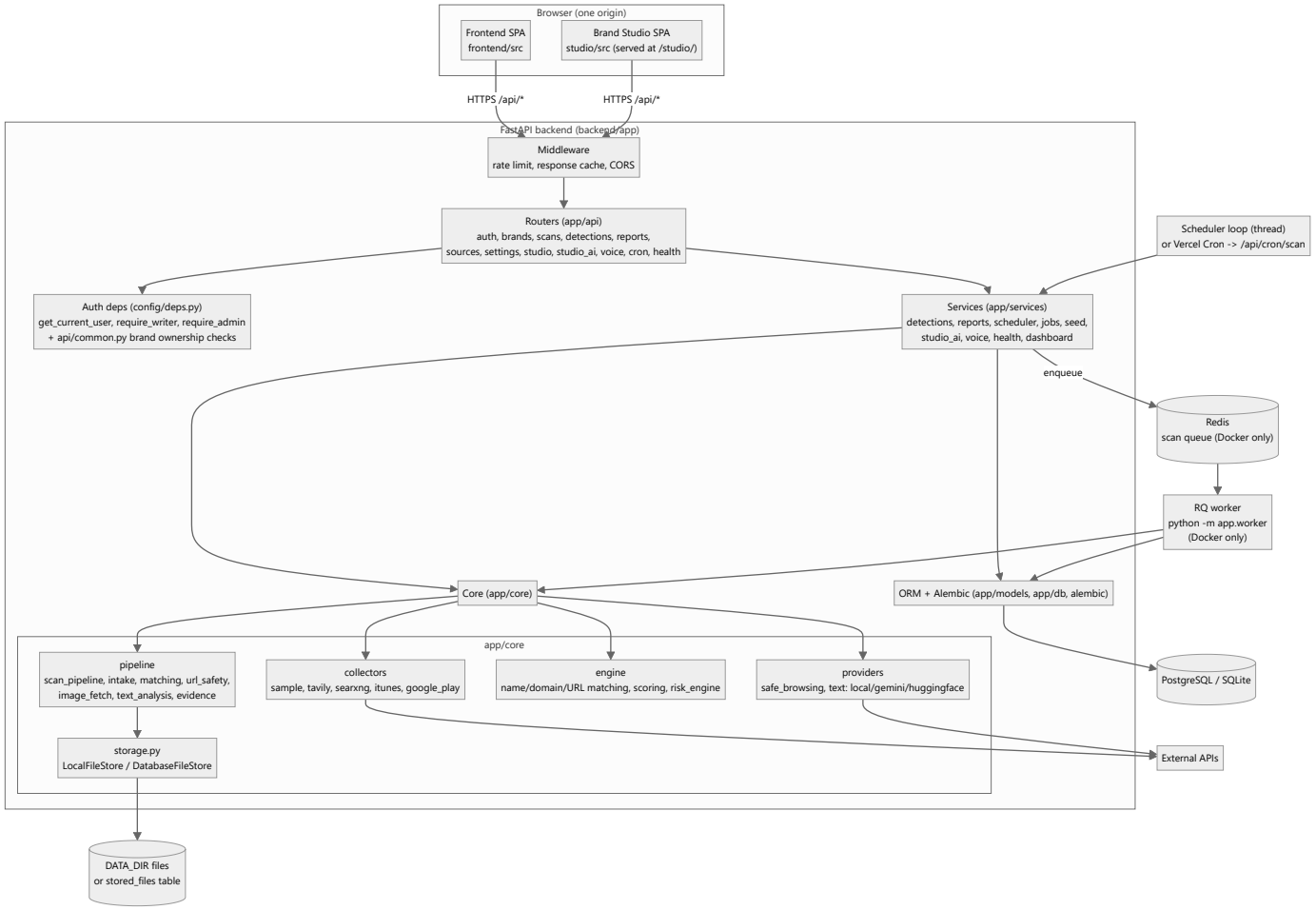
1. System context

Who and what BrandShield talks to. Every external service is optional and is called only from the backend (never from the browser), except Google Fonts which the browser loads directly.



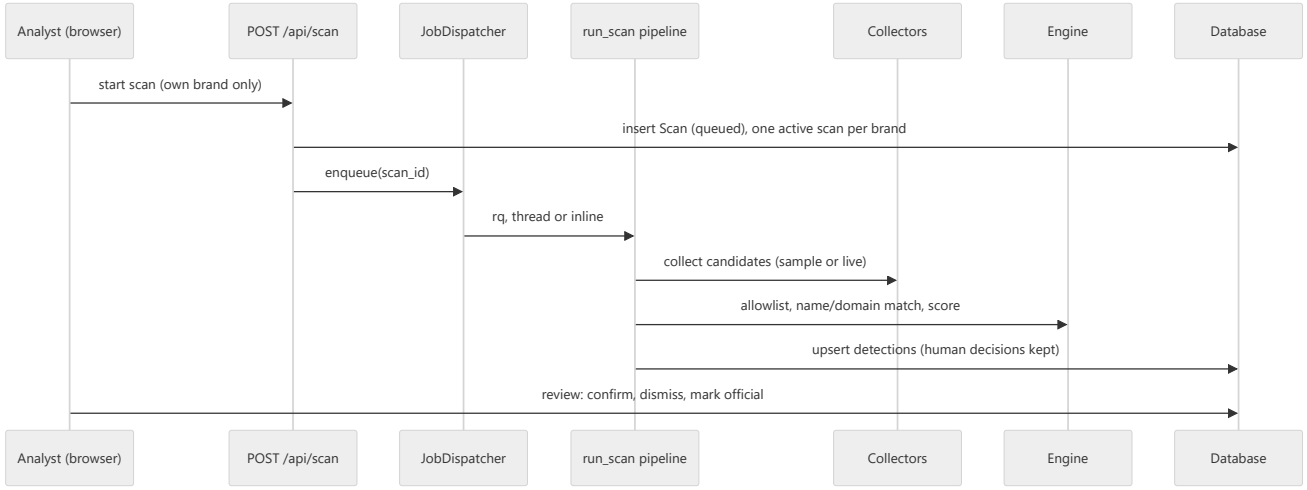
External system	What it is used for	Evidence
PostgreSQL / Supabase	All data; SQLite is the default for local runs	`backend/app/config/config.py:28` , `backend/app/db/session.py:10-45`
Tavily	Live web search collector	`backend/app/core/collectors/tavily.py:38`
iTunes / Google Play	App-store candidates	`backend/app/core/collectors/itunes.py:34` , `backend/app/core/collectors/google_play.py`
Google Safe Browsing v4	Reputation of candidate URLs (URL is sent, never opened)	`backend/app/core/providers/safe_browsing.py:1-3,42`
Gemini	Optional scan text analysis; voice agent; Studio chat fallback	`backend/app/core/providers/text/gemini.py:36` , `backend/app/services/studio_ai.py:40,152`
Groq / Pollinations	Studio chat / image generation	`backend/app/services/studio_ai.py:39,41`
Hugging Face	Optional text provider; CLIP model download	`backend/app/core/providers/text/huggingface.py` , `backend/requirements-extra.txt:6,8`
Sentry	Optional scrubbed error reporting	`backend/app/config/observability.py:1-8,79-102`

2. Component diagram



Piece	Responsibility	Evidence
`create_app`	Wires settings, limiters, collectors, file store, scan context, middleware, routers; seeds admin and demo users at startup; optionally starts the scheduler	`backend/app/main.py:50-184`
Middleware	Per-IP and per-token rate limit for `/api/*`; short read cache; CORS	`backend/app/main.py:119-167`
Routers	Thin HTTP layer under prefix `/api`; health routes live outside it	`backend/app/main.py:170-183`
Auth deps	Bearer JWT -> `User`; role checks; `require_writer` = admin or analyst, `require_admin`	`backend/app/config/deps.py:28-57`
Ownership helpers	Brand lookups scoped to `brands.owner_id`; foreign brand is a 404	`backend/app/api/common.py:22-58`
Scan pipeline	collect, de-duplicate, allowlist, name match, prefilter, logo fetch, text analysis, score, upsert detections	`backend/app/core/pipeline/scan_pipeline.py:1-16`
Engine	Pure, deterministic scoring and explanations; never confirms anything	`backend/app/core/README.md:1-9`, `backend/app/core/engine/risk_engine.py:19`
Job dispatcher	Runs a queued scan via RQ, a thread, or inline	`backend/app/services/jobs.py:1-9,25-65`
Scheduler	Pure decision function plus an in-process loop; credit-aware interval stretching	`backend/app/services/scheduler.py:1-14`
File store	Logos, candidate images, evidence snapshots, Studio images	`backend/app/core/storage.py:1-10`

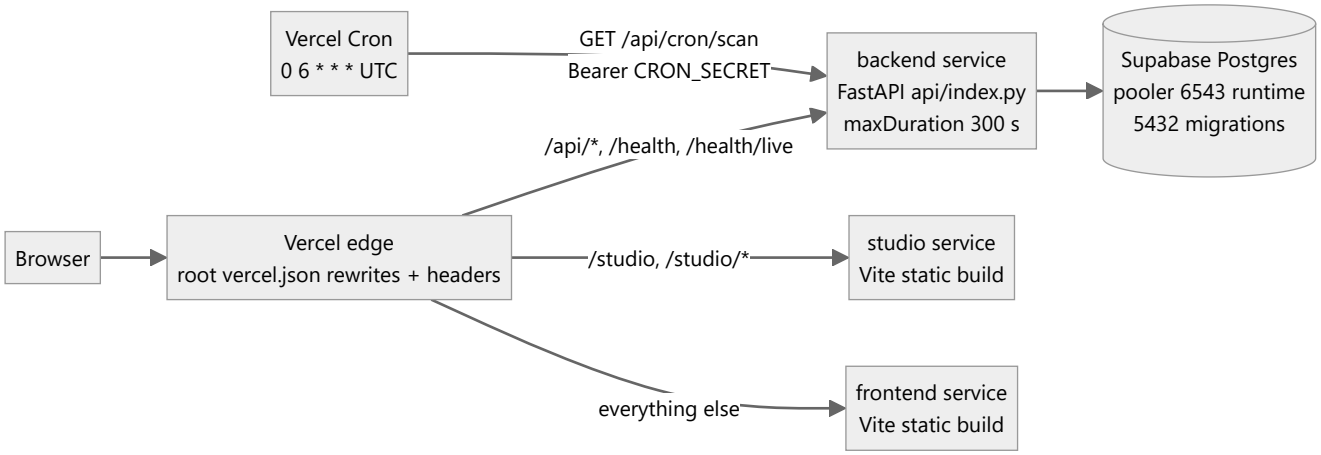
2.1 Life of a scan



Evidence: `request_scan` creates the queued row, enforces ownership and a partial unique index for one active scan per brand, and calls the dispatcher (`backend/app/core/pipeline/scan_pipeline.py:146-190`); the executor choice is in `backend/app/services/jobs.py:25-48`; "a human decision survives a re-scan" is stated at `scan_pipeline.py:10-12`.

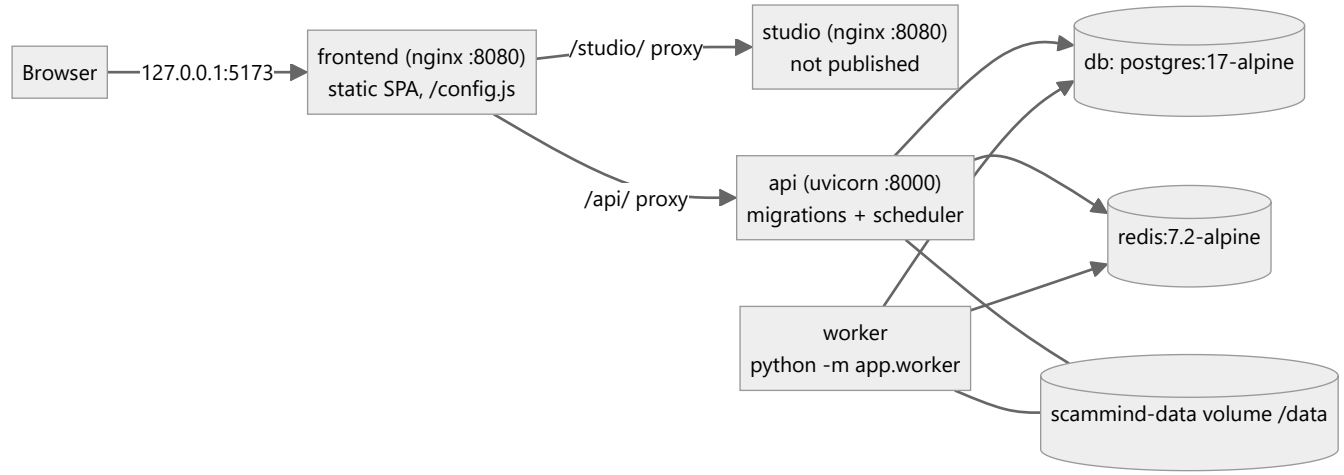
3. Deployment diagrams

3.1 Vercel (one project, three services, one domain)



Evidence: services and rewrites `vercel.json:3-81`; cron `vercel.json:82-87`; entrypoint `backend/api/index.py:18-40` and `backend/pyproject.toml:57-59`; Supabase ports `backend/app/config/config.py:35-37`, `backend/app/db/session.py:30-31`. On Vercel the backend runs with serverless defaults (inline scans, no scheduler, no response cache, files in the database): `backend/app/config/config.py:175-190`. The Vercel deployment is described in `docs/DEPLOY_VERCEL.md:9-12` as unverified (never deployed); Vercel Services are labelled Beta there.

3.2 Docker Compose



Evidence: services `docker-compose.yml:37-161`; frontend nginx proxies `/studio/` and `/api/` (`frontend/nginx/default.conf.template:38-70`); api publishes `127.0.0.1:${API_PORT:-8000}` (`docker-compose.yml:85-86`); only `studio` is internal-only (`docker-compose.yml:157-159`) unless the dev overlay publishes it (`docker-compose.dev.yml:16-20`). The `worker` has `SCHEDULER_ENABLED=false` and `SCAN_EXECUTOR=rq` (`docker-compose.yml:22,111`).

3.3 Local development

`npm run backend` (API on 8000) and `npm run frontend` (BrandShield on 5173, Studio on 3000; BrandShield's Vite dev server proxies `/studio` and `/api`) (`scripts/dev.mjs:79-110`, `frontend/vite.config.ts:8-15`). Without `REDIS_URL` scans run in an in-process thread (`backend/app/services/jobs.py:25-30`).

4. Folder and module structure

```
ScamMind AI/
  backend/            FastAPI + SQLAlchemy 2 + Alembic
  frontend/           BrandShield React + Vite app (also holds the Playwright e2e tests)
  studio/             Brand Studio React + Vite app (base /studio/)
  docs/               Documentation (existing docs plus generated schema)
  scripts/            dev.mjs: one-command local runner
  .github/workflows/  ci.yml
  vercel.json         Vercel services, routing, cron, security headers
  docker-compose.yml  Full stack; docker-compose.dev.yml adds localhost ports
```

4.1 `backend/`

Path	Purpose
`backend/app/main.py`	App factory `create_app()`; lifespan seeding and scheduler start (`main.py:50-184`)
`backend/app/worker.py`	RQ worker entrypoint; `run_scan_job(scan_id)` (`worker.py:80-101`)
`backend/app/api/`	HTTP routers: `auth`, `brands`, `candidates`, `scans`, `detections`, `dashboard`, `lookalike`, `reports`, `evaluation`, `sources`, `settings`, `providers`, `audit`, `studio` (saved items), `studio_ai` (AI proxy), `voice`, `cron`, `health`; `common.py` holds the ownership helpers
`backend/app/config/`	`config.py` (settings), `security.py` (bcrypt, JWT), `deps.py` (auth dependencies), `ratelimit.py`, `response_cache.py`, `logging.py` (JSON + redaction), `observability.py` (Sentry), `errors.py`, `time.py`
`backend/app/db/`	`session.py` (engine/pool per backend), `request_cache.py` (per-session memo), `base.py`
`backend/app/models/`	SQLAlchemy models: `user.py` (users, refresh tokens), `brand.py` (brands and registry), `detection.py`, `ops.py` (sources, scans, settings, reports, audit log, api usage, look-alike cases, stored files), `studio.py`
`backend/app/schemas/`	Pydantic request/response models (`auth.py`, `brand.py`)
`backend/app/services/`	Business logic between routers and core: detection review actions, reports, dashboard aggregates, scheduler, job dispatcher, seed (admin/demo), Studio AI proxy, voice agent, health, canary, settings and data-mode helpers
`backend/app/core/`	Detection features, free of HTTP and persistence (`core/README.md:1-9`)
`backend/app/core/engine/`	Pure scoring: normalization, name/domain/URL matching, allowlist, `scoring.py`, `risk_engine.py`, logo/image hashes, text signals, synthetic generation/evaluation
`backend/app/core/collectors/`	Candidate sources (Tavily, SearXNG, iTunes, Google Play, sample), query variants, credit ledger, orchestrator, selection of the collector chain
`backend/app/core/pipeline/`	Scan pipeline and steps: intake, matching, `url_safety.py`, `image_fetch.py`, screenshot hook, logo handling, text analysis, evidence, bands
`backend/app/core/providers/`	External checks behind interfaces: `safe_browsing.py`, `text/` (local, Gemini, Hugging Face, cache, prompt, schema), authorized-access stub
`backend/app/core/storage.py`	File store interface and two implementations
`backend/alembic/versions/`	Migrations 0001 to 0012 (initial schema through `0012_active_brand.py`)
`backend/api/index.py`	Vercel entrypoint (`api.index:app`)
`backend/scripts/`	`migrate.py` (advisory-locked upgrade), `canary.py`, `e2e_server.py`, `evaluate.py`, `export_schema.py`, `live_check.py`, `seed_test_brand.py`, `studio_ai_check.py`
`backend/tests/`	pytest suite: `api/`, `config/`, `core/`, `db/`, `services/`, fixtures, Postgres support (`pg_support.py`)
`backend/data/`	Local runtime data (SQLite DB, logos, evidence, models); git-ignored (`.gitignore`: `backend/data/`)
`backend/Dockerfile`, `docker-entrypoint.sh`	One image for `api`, `worker`, `migrate` roles

4.2 `frontend/src/`

Path	Purpose
`App.tsx`, `main.tsx`	Routes and guards (`RequireAuth`, `RequireAdmin`), theme bootstrap (`App.tsx:38-78`)
`api/`	`client.ts` (base URL, token storage, refresh-on-401 with Web Lock), `auth.ts`, `sm.ts` (endpoint wrappers), `queryCache.ts`, `types.ts`, `format.ts`, `sample.ts`
`pages/`	One file per screen: Login, Register, ProtectBrand, Setup, Dashboard, BrandProfile, SocialMonitoring, AppStoreMonitoring, WebThreats, Investigations, LookalikeLab, Reports, SourceHealth, Settings, Evaluation
`components/`	`layout/` (AppLayout, Sidebar, Header, BrandSwitcher), `monitor/`, `charts/`, `common/`, `voice/` (voice agent button)
`hooks/`	Data hooks (`useApi`, `useBrand`, `useScan`, `useMonitor`, `useVoiceAgent`, ...)
`landing/`	Public marketing page shown at `/` for signed-out visitors (`App.tsx:26-36`)
`utils/`	Theme, passwords policy mirror, voice executor/snapshot, Studio link helpers
`styles/`	Tokens and global CSS
`frontend/e2e/`	Playwright specs: `smoke`, `demo-login`, `register`, `multi-brand`, `studio-flow`, `voice`, `visual`, `protect-hover`
`frontend/nginx/`	Container web server config and runtime `config.js` writer

4.3 `studio/src/`

Path	Purpose
`App.tsx`, `main.tsx`	Studio shell; dev-only redirect to BrandShield's origin (`main.tsx:9-12`)
`components/`	Tools: BrandNameGenerator, LogoGenerator, BrandingAssistant, ContentGenerator, ColorPaletteGenerator, BrandAudit, BrandChecker, ReviewAnalyzer, SentimentAnalysis, Analytics, Collections, ProtectBrand, LogoUpload, SuccessFlow
`contexts/`	Auth (reads the shared BrandShield session), Brand, Collections, Theme
`lib/`	`session.ts` (same storage keys as BrandShield), `api.ts` (refresh with the same Web Lock name), `ai.ts` (calls the backend AI proxy), `items.ts` (saved items), `brandshield.ts`, `useAiStatus.ts`
`services/gemini.ts`	Despite its name, wraps the backend `/api/studio/ai/*` proxy (`gemini.ts:1-17`)
`studio/nginx/default.conf`	Container web server config

4.4 Other top-level folders

Path	Purpose
`docs/`	`API.md`, `DATABASE_SCHEMA.md` and `schema.sql` (generated by `backend/scripts/export_schema.py:1-4`), `DECISIONS.md`, `SECURITY.md`, `DEPENDENCIES.md`, `DEPLOY_VERCEL.md`, `COLLECTORS.md`, `STUDIO*.md`, `USER_GUIDE.md`, and others
`scripts/dev.mjs`	Cross-platform dev runner (ports 8000, 5173, 3000)
`e2e`	There is no top-level `e2e/` folder; the browser tests are in `frontend/e2e/` (`frontend/playwright.config.ts:31-32`)

5. Design patterns and key decisions

5.1 Tenant ownership through `brands.owner_id`

Every brand has a nullable `owner_id` pointing at `users.id` (`backend/app/models/brand.py:14`). All brand-scoped data hangs off a brand, so ownership of a brand implies ownership of its detections, scans, reports and images.

- `get_brand_or_404` returns the brand only if `brand.owner_id == user.id`; another account's brand is a plain 404, never 403, so existence does not leak (`backend/app/api/common.py:22-27`).
- `own_brand` picks the user's selected (`users.active_brand_id`) or oldest non-archived brand (`common.py:30-40`); `resolve_brand` makes `brand_id` optional everywhere (`common.py:43-50`); `owns_brand` guards objects such as images (`common.py:53-58` , used at `backend/app/api/candidates.py:109-115`).
- `request_scan` refuses a brand the user does not own (`scan_pipeline.py:146-160`). Studio items are scoped by `user_id` (`backend/app/api/studio.py:103-119`).
- Migration 0010 added the column and gave existing brands to the first admin (`backend/alembic/versions/0010_brand_owner.py:1-11`).
- Still global (shared by all accounts): settings, data mode, provider choice, source health and collector state, credit counters (`docs/DECISIONS.md:208` ; code: `backend/app/services/app_settings.py:59-66` , `backend/app/api/sources.py:125-158`). See `docs/auth-and-security.md` for the consequences.

5.2 Per-request memo

`memo(db, key, load)` stores repeated identical reads (brand lookups, settings rows) in `Session.info` , and clears the memo on every flush, commit and rollback, so it never outlives a write (`backend/app/db/request_cache.py:1-30`). It exists because a remote database makes every query a network round trip (`request_cache.py:3-5`). Used for the ownership helpers (`backend/app/api/common.py:24,40,57`).

5.3 Response cache

A tiny in-process cache serves heavy read endpoints (dashboard, lists, sources, settings, detections, brands, reports, scans, providers) for `RESPONSE_CACHE_SECONDS` (default 15). Keys include a hash of the `Authorization` header (so answers that differ by role or user do not mix); any POST/PUT/PATCH/DELETE clears everything; scan start/finish also clears it (`backend/app/config/response_cache.py:1-18,43-45` , `backend/app/main.py:137-158`). It is per process and is turned off on Vercel (`config.py:184`).

5.4 Frozen scoring weights

The original social and app weights are frozen as `WEIGHTS_V1` and every such detection is stamped algorithm version `1.0` ; a test pins them to the numbers in `scoring.py` . New web signals use separate weights `WEIGHTS_V2_WEB` and algorithm version `2.0` (`backend/app/core/engine/risk_engine.py:41-43` , docstring `risk_engine.py:9-18`). The social maxima are name 34, logo 28, added words 12, bio 28, account 18; app maxima name 24, icon 30, publisher 22, description 17, store 10 (`backend/app/core/engine/scoring.py:34-35`). Severity cut-offs are configurable but default to LOW below 40, MEDIUM 40-69, HIGH 70-84, CRITICAL 85+ (`risk_engine.py:46-54` , `backend/app/config/config.py:166-168`). `docs/DECISIONS.md:134` records that the `weights_v2_web` numbers still "need your approval".

5.5 Human in the loop

- The engine "only scores; a human approves or rejects every detection" (`backend/app/core/engine/scoring.py:14` ; `risk_engine.py:19`).
- A re-scan never resets a human decision (confirmed, dismissed, reviewing, marked official) (`scan_pipeline.py:10-12`).
- Review statuses a user may set: reviewing, confirmed, dismissed, plus mark-official (`backend/app/services/detections.py:33-35`); these routes require `require_writer` (`backend/app/api/detections.py:163-195`).
- Reports start as drafts and need an explicit approve step (`backend/app/api/reports.py:195-196` , `backend/app/services/reports.py:1-3`).
- AI output can only add a bounded number of points and never triggers an action (`backend/app/core/providers/text/prompt.py:1-9`); the voice agent asks for a spoken yes before destructive clicks or unrequested submits (`backend/app/services/voice.py:1-18`).

5.6 Sample versus live data

`data_mode` is `live` , `sample` or `auto` (default `auto`), stored in the `settings` table (`backend/app/services/data_mode.py:7-16`). `auto` becomes live only when a Tavily key, a SearXNG URL or the Google Play package is configured; otherwise sample (`backend/app/core/collectors/selection.py:68-97`). Sample candidates are synthetic

and are labelled as such on every signal (`risk_engine.py:21-22`); sample URLs are never sent to Safe Browsing and nothing is fetched for them (`backend/app/core/collectors/sample.py:273-275` , `scan_pipeline` logo fetch is live-only per `config.py:116`). Changing the global mode is admin-only (`backend/app/api/sources.py:239-250`).

5.7 Inline scans on Vercel

A serverless function can be frozen once its response is sent, so background threads and an RQ worker do not work (`docs/DEPLOY_VERCEL.md:210`). On Vercel the executor defaults to `inline` , the scan runs inside the request, the in-process scheduler is off, and Vercel Cron calls `GET /api/cron/scan` (`backend/app/config/config.py:181-187` , `backend/app/services/jobs.py:41-42` , `backend/app/api/cron.py:1-6,32-46`). The function limit is `maxDuration: 300` (`vercel.json:10`). The cron handler runs the same decision function as the scheduler, including credit-aware interval stretching, plus a 60-minute slack because Hobby cron can fire anywhere in the hour (`backend/app/api/cron.py:22-24`).

5.8 Database-backed file store

`FileStore` is a small protocol (`write` , `write_if_absent` , `read` , `exists` , `delete`) with two implementations chosen by `STORAGE_BACKEND` : `LocalFileStore` under `DATA_DIR` , and `DatabaseFileStore` using the `stored_files` table, needed on Vercel where the filesystem is read-only and not shared. Both reject paths that could escape the store (`backend/app/core/storage.py:1-10,45-52,169-176`). Files are content-addressed and served with a SHA-256-based ETag (`backend/app/api/common.py:13-15`).

5.9 Other recurring patterns

Pattern	Where
App factory with injectable collaborators (tests pass fakes for the URL checker, collectors, fetcher)	`backend/app/main.py:50-58`
Optional features behind lazy imports and feature flags	`backend/app/services/jobs.py:8-9` , `backend/requirements-extra.txt:1-2`
A failing data source never blocks a scan; the scan ends `partial`	`scan_pipeline.py:15-16`
Advisory-locked migrations safe for many replicas	`backend/scripts/migrate.py:1-7,35-42`
Append-only audit log for mutating actions	`backend/app/services/audit.py` , `backend/app/api/audit.py:36-84`
Single-flight token refresh shared across tabs and both apps via a Web Lock	`frontend/src/api/client.ts:225-264` , `studio/src/lib/api.ts:93-99`
Shared login between BrandShield and Studio through same-origin `localStorage` keys `scammind.auth` / `scammind.user`	`frontend/src/api/client.ts:59-60` , `studio/src/lib/session.ts:1-4`
One origin for everything (Vercel rewrites or nginx proxy) so the SPA calls relative `/api`	`vercel.json:44-81` , `frontend/nginx/default.conf.template:60-70` , `frontend/src/api/client.ts:23-27`

DOCS/TECH-STACK.MD

Technology stack

Plain-language summary

- BrandShield has three parts: a Python web service (the "backend"), a React web app (the "frontend"), and a second React app called Brand Studio that is served under `/studio/`.
- Data lives in a PostgreSQL database (Supabase in the Vercel setup) or a local SQLite file for development.
- Outside services (Tavily, Google Gemini, Groq, Pollinations, Google Safe Browsing, Hugging Face, Sentry) are all optional and switched on only by keys.
- Versions below are the constraints written in the repo's dependency files (for example `>=0.115,<1` means "0.115 or newer, but below 1.0"). They are **not** the exact installed versions; no lock file exists for the Python side.

1. Languages and runtimes

Technology	Version (as written)	Where it is used	Source
Python	`>=3.11` required; Docker image `python:3.12-slim`; CI tests 3.11 and 3.12; Vercel default noted as 3.12	Backend	`backend/pyproject.toml:10`, `backend/Dockerfile:8,21`, `.github/workflows/ci.yml:26,55,98`, `docs/DEPLOY_VERCEL.md` (platform facts table)
TypeScript	frontend `~6.0.2`; studio `~5.8.2`	Both React apps	`frontend/package.json:36`, `studio/package.json:33`
Node.js	`22` (CI and both Dockerfiles)	Build and tooling	`.github/workflows/ci.yml:35-37`, `frontend/Dockerfile:8`, `studio/Dockerfile:8`
Node (root helper)	no version pinned	`scripts/dev.mjs` uses only Node built-ins	`scripts/dev.mjs:5-9`, `package.json:1-9`
Shell / SQL	POSIX `sh`; Alembic migrations 0001-0012	Container entrypoint, DB schema	`backend/docker-entrypoint.sh`, `backend/alembic/versions/`

2. Backend (Python) runtime dependencies

All from `backend/requirements.txt` (also read by Vercel through `pyproject.toml`, `backend/pyproject.toml:16-17`).

Package	Version constraint	Why / where	Source line
FastAPI	`>=0.115,<1`	Web framework; routers in `backend/app/api/*`, app factory `create_app` in `backend/app/main.py:50`	`backend/requirements.txt:1`
Uvicorn (`standard`)	`>=0.30,<1`	ASGI server (`uvicorn app.main:create_app --factory`)	`requirements.txt:2`, `backend/docker-entrypoint.sh:20`
SQLAlchemy	`>=2.0,<2.1`	ORM, 2.0 style models in `backend/app/models/*`	`requirements.txt:3`
Alembic	`>=1.13,<2`	Schema migrations (`backend/alembic/versions/0001...0012`), run via `backend/scripts/migrate.py`	`requirements.txt:4`
psycopg (`binary`)	`>=3.2,<4`	PostgreSQL driver (URLs `postgresql+psycopg://`)	`requirements.txt:5`
pydantic	`>=2.7,<3`	Request/response schemas	`requirements.txt:6`
pydantic-settings	`>=2.3,<3`	`Settings` from environment (`backend/app/config/config.py:17`)	`requirements.txt:7`
email-validator	`>=2.1,<3`	`EmailStr` on login/sign-up (`backend/app/schemas/auth.py:7,47`)	`requirements.txt:8`
httpx	`>=0.27,<1`	All outbound HTTP (collectors, Safe Browsing, Studio AI, SSRF-safe fetchers)	`requirements.txt:9`
PyJWT	`>=2.8,<3`	Access and refresh tokens (`backend/app/config/security.py:6,35`)	`requirements.txt:10`
bcrypt	`>=4.1,<6`	Password hashing, 12 rounds (`security.py:11,15-16`)	`requirements.txt:11`
Pillow	`>=10.3` (no upper cap)	Logo/image validation, hashing, re-encode (`backend/app/core/pipeline/logo.py`, `image_fetch.py`)	`requirements.txt:12`
redis (client)	`>=5,<7`	Scan queue connection and worker health (`backend/app/worker.py:92-95`)	`requirements.txt:13`
rq	`>=1.16,<3`	Background job queue for scans (`backend/app/services/jobs.py`)	`requirements.txt:14`
python-multipart	`>=0.0.9`	File uploads (logos, images, imports)	`requirements.txt:15`
tldextract	`>=5.1,<6`	Registrable domain via the bundled public-suffix snapshot, no runtime download	`requirements.txt:16`
rapidfuzz	`>=3.9,<4`	String-similarity signal for domains; voice fast path (`backend/app/services/voice.py:32`)	`requirements.txt:17`
google-genai	`>=1.20,<3`	Gemini function calling for the voice agent	`requirements.txt:18`
idna	`>=3.7,<4`	IDN/punycode handling	`requirements.txt:19`

2.1 Optional backend extras (`backend/requirements-extra.txt`)

Installed only when you run `pip install -r requirements-extra.txt` or build Docker with `INSTALL_EXTRAS=true` (`backend/Dockerfile:15-17`). Each is imported lazily and enabled by a setting.

Package	Version constraint	Why / where	Source line
sentry-sdk	`>=2.0,<3`	Error reporting when `SENTRY_DSN` is set (`backend/app/config/observability.py`)	`requirements-extra.txt:3`
ImageHash	`>=4.3,<5`	Optional image-similarity backend (`IMAGE_SIMILARITY_BACKEND=imagehash`); pulls numpy, scipy, PyWavelets	`requirements-extra.txt:4`
google-genai	`>=2.30,<3`	Gemini text provider (`SCAM_LLM_PROVIDER=gemini`)	`requirements-extra.txt:5`
huggingface_hub	`>=1.0,<2`	Optional Hugging Face text provider (capped below 2 because fastembed needs `<2.0`)	`requirements-extra.txt:6`
google-play-scraper	`>=1.2,<2`	Google Play search adapter	`requirements-extra.txt:7`
fastembed	`>=0.9,<1`	CLIP image embeddings for the Look-alike Lab (ONNX, no torch; model downloads on first use into `DATA_DIR/models`)	`requirements-extra.txt:8`

Needs confirmation: `google-genai` appears in both files with different constraints (`>=1.20,<3` in `requirements.txt:18` , `>=2.30,<3` in `requirements-extra.txt:5`). Confirm which version range is actually intended when both files are installed (pip must satisfy both, so effectively `>=2.30,<3`).

2.2 Backend dev dependencies (`backend/requirements-dev.txt`)

Package	Constraint	Use	Source
pytest	`>=8,<9`	Test runner (`backend/pyproject.toml:19-22`)	`requirements-dev.txt:2`
ruff	`>=0.5`	Linter (rules E, F, I, UP, B; line length 140)	`requirements-dev.txt:3` , `backend/pyproject.toml:24-31`

3. Frontend (BrandShield, `frontend/package.json`)

Package	Constraint	Why	Source line
react, react-dom	`^19.2.8`	UI library	`frontend/package.json:21-22`
react-router-dom	`^7.18.4`	Client-side routing and guards (`frontend/src/App.tsx:2`)	`package.json:23`
lucide-react	`^1.55.0`	Icons	`package.json:20`
d3-geo (+ `@types/d3-geo`)	`^3.1.1`	Map drawing for the dashboard (data in `frontend/public/states.geojson`)	`package.json:18-19`
Vite	`^8.3.0`	Dev server and bundler (`frontend/vite.config.ts`)	`package.json:37`
@vitejs/plugin-react	`^6.1.1`	React support in Vite	`package.json:33`
Vitest	`^5.0.4`	Unit/component tests (jsdom)	`package.json:38`
jsdom	`^29.1.1`	Test DOM	`package.json:34`
@testing-library/react / jest-dom / user-event	`^16.3.3` / `^7.0.1` / `^14.6.7`	Component testing helpers	`package.json:27-29`
@playwright/test	`^1.64.0`	Browser end-to-end tests (`frontend/e2e/*.spec.ts`)	`package.json:26`
oxlint	`^1.81.0`	Linter	`package.json:35`
TypeScript	`~6.0.2`	Type-check (`tsc -b`)	`package.json:36`
@types/node, @types/react, @types/react-dom	`^24.13.3` , `^19.2.18` , `^19.2.7`	Type definitions	`package.json:30-32`

Scripts: `dev` (vite), `build` (`tsc -b && vite build`), `lint` (oxlint), `typecheck` , `test` (`vitest run`), `test:e2e` (`playwright test`) (`frontend/package.json:6-16`). Fonts: Manrope from Google Fonts (`frontend/index.html:8-10`). The frontend uses no state-management or HTTP library; it has its own fetch wrapper (`frontend/src/api/client.ts`).

4. Brand Studio (`studio/package.json`)

Line numbers below refer to `studio/package.json` .

Package	Constraint	Why	Line
react, react-dom	`^19.0.0`	UI	21, 23
Vite	`^6.4.4`	Bundler; `base: '/studio/'` (`studio/vite.config.ts:10`)	34
@vitejs/plugin-react	`^5.0.4`	React in Vite	16
Tailwind CSS + @tailwindcss/vite	`^4.1.14`	Styling	15, 32
tailwind-merge, clsx	`^3.5.0` , `^2.1.1`	Class helpers	25, 18
chart.js + react-chartjs-2	`^4.5.1` , `^5.3.1`	Analytics charts	17, 22
motion	`^12.23.24`	Animations	20
react-markdown	`^10.1.0`	Rendering AI answers	24
lucide-react	`^0.546.0`	Icons	19
Vitest, jsdom, Testing Library	`^3.2.4` , `^26.1.0` , `^6.6.3` / `^16.3.0`	Tests	28, 29, 31, 35
TypeScript	`~5.8.2`	`tsc --noEmit` is the "lint" script	33

Studio has no AI SDK of its own: all AI calls go through the BrandShield backend (`studio/src/services/gemini.ts:1-6` , `studio/src/lib/ai.ts`). Font: Inter from Google Fonts (`studio/index.html:9-12`).

5. Datastores, queue, cache

Technology	Version	Role	Source
PostgreSQL	`postgres:17-alpine` image (Compose and CI service); Supabase-hosted Postgres in the Vercel deployment (version chosen by Supabase, not pinned in the repo)	Primary database in production	`docker-compose.yml:39` , `.github/workflows/ci.yml:78` , `docs/DEPLOY_VERCEL.md`
SQLite	Python stdlib	Default for local runs and most tests (`sqlite:///.../data/scammind.db` , WAL mode, foreign keys on)	`backend/app/config/config.py:28` , `backend/app/db/session.py:12-43`
Supabase	hosted service	Managed Postgres; runtime uses the transaction pooler (port 6543, prepared statements off), migrations use a direct/session URL (5432)	`backend/app/db/session.py:30-31` , `backend/app/config/config.py:35-37`
Redis	`redis:7.2-alpine` (Compose only)	Backing store for the RQ scan queue; AOF on, password required	`docker-compose.yml:55-67`
RQ	`rq>=1.16,<3`	Job queue; worker is `python -m app.worker`	`backend/app/worker.py:93-100`
In-process thread runner	stdlib	Scan runner when no Redis	`backend/app/services/jobs.py:61-65`
In-process response cache	custom	15 s cache of heavy read endpoints, dropped by any write	`backend/app/config/response_cache.py:1-18`
Per-session memo	custom	Repeated identical reads within one request	`backend/app/db/request_cache.py:1-30`
File store	custom	Local files under `DATA_DIR` or rows in `stored_files` table	`backend/app/core/storage.py:1-10,169-176`
Local named volumes	Docker	`pgdata` , `redisdata` , `scammind-data`	`docker-compose.yml:163-166`

There is no separate cache server: Redis is used for the queue only (`docs` /code show no other Redis use).

6. Cloud and third-party services

Service	Used for	Required?	Where	Key variable
Vercel	Hosting: three "services" on one domain plus Cron	Optional (one deployment target; documented as never deployed)	`vercel.json`, `docs/DEPLOY_VERCEL.md:7-12`	`VERCEL` (set by platform), `CRON_SECRET`
Supabase	Managed Postgres	Optional	see above	`DATABASE_URL`, `MIGRATION_DATABASE_URL`
Tavily	Live web search collector (`POST https://api.tavily.com/search`)	Optional	`backend/app/core/collectors/tavily.py:38`	`TAVILY_API_KEY`
SearXNG	Optional self-hosted search fallback	Optional	`backend/app/core/collectors/searxng.py`	`SEARXNG_BASE_URL`
Apple iTunes Search API	App store collector, no key	Used in live mode	`backend/app/core/collectors/itunes.py:34`	none
Google Play (scraper)	App store collector	Optional package	`backend/app/core/collectors/google_play.py`	none
Google Safe Browsing v4	URL reputation (`threatMatches:find`)	Optional	`backend/app/core/providers/safe_browsing.py:42`	`GOOGLE_SAFE_BROWSING_API_KEY`
Google Gemini	Scan text analysis, Studio chat fallback, voice agent	Optional	`backend/app/services/studio_ai.py:40`, `backend/app/core/providers/text/gemini.py:36`	`GEMINI_API_KEY`, `VOICE_GEMINI_API_KEY`
Groq	Studio chat (primary)	Optional	`backend/app/services/studio_ai.py:39`	`GROQ_API_KEY`
Pollinations	Studio image generation	Optional	`backend/app/services/studio_ai.py:41`	`POLLINATIONS_API_KEY`
Hugging Face Inference	Optional text provider; also hosts the CLIP model weights downloaded by fastembed	Optional	`backend/app/core/providers/text/huggingface.py`, `backend/requirements-extra.txt:6,8`	`HUGGINGFACE_API_TOKEN`
Sentry	Error reporting	Optional	`backend/app/config/observability.py`	`SENTRY_DSN`
Google Fonts	Manrope and Inter web fonts (allowed in CSP)	Browser loads it	`frontend/index.html:8-10`, `vercel.json:114`	none
GitHub Actions	CI	n/a	`.github/workflows/ci.yml`	none (no secrets)

7. Containers and web servers

Item	Version	Source
Backend image base	`python:3.12-slim` (multi-stage, non-root uid 10001, venv in `/opt/venv`)	`backend/Dockerfile:8,21,25-26`
Frontend and Studio runtime	`nginxinc/nginx-unprivileged:1.27-alpine` (port 8080)	`frontend/Dockerfile:17`, `studio/Dockerfile:15`
Frontend/Studio build image	`node:22-alpine`	`frontend/Dockerfile:8`, `studio/Dockerfile:8`
Docker Compose project	name `scammind`	`docker-compose.yml:13`

8. Dev tooling and CI

Tool	Purpose	Where
ruff	Python lint (`ruff check .` in `backend/`)	`.github/workflows/ci.yml:29-33`, `backend/pyproject.toml:24-55`
oxlint	Frontend lint (`npm run lint`)	`ci.yml:42-44`, `frontend/.oxlintrc.json`
tsc	Type-check (`tsc -b` frontend; `tsc --noEmit` studio)	`ci.yml:45-47,147-149`
pytest	Backend tests; SQLite by default, Postgres via `TEST_DATABASE_URL`	`ci.yml:66-68,110-112`, `backend/tests/conftest.py`
Vitest	Frontend and Studio unit tests	`ci.yml:127-129,150-152`
Playwright (Chromium)	Browser end-to-end tests against the real API with sample data	`ci.yml:157-198`, `frontend/playwright.config.ts`
Vite	Dev servers and production builds	`frontend/vite.config.ts`, `studio/vite.config.ts`
Canary script	`python -m scripts.canary` : in-memory detection self-check	`ci.yml:69-71`, `backend/scripts/canary.py:1-5`
Docker buildx	Image builds in CI (no push)	`ci.yml:201-231`

8.1 GitHub Actions jobs (`.github/workflows/ci.yml`)

Triggers: push to `main` , and pull requests (`ci.yml:6-9`). Read-only permissions and per-ref concurrency (`ci.yml:11-16`).

Job	What it does	Depends on	Lines
`lint`	ruff on backend; oxlint and type-check on frontend (Python 3.12, Node 22)	none	19-47
`test-server`	pytest on SQLite for Python 3.11 and 3.12, plus canary	none	49-71
`test-server-postgres`	Postgres 17 service; runs migrations twice (second must be a no-op), then pytest on Postgres	none	73-112
`test-frontend`	Vitest and production build	none	114-132
`test-studio`	tsc, Vitest, build	none	134-155
`e2e`	Playwright with real API on SQLite and sample collector; uploads traces on failure	test-server, test-frontend, test-studio	157-198
`build`	Builds the three Docker images without pushing	lint, test-server, test-frontend, test-studio	201-231
`publish` (commented out)	Optional push to ghcr.io	disabled	233-261

The workflow file states it was written and YAML-checked locally but never run (`ci.yml:3-5`).

9. Why these choices (as recorded in the code)

- **SQLite + Postgres:** same SQLAlchemy models run on both; SQLite keeps local setup to zero services (`backend/app/db/session.py:10-45`).
- **Optional everything external:** a fresh install runs on sample data with no keys (`backend/app/core/collectors/selection.py:77-100`).
- **Lazy imports:** `rq` , `redis` , Sentry, Gemini, Hugging Face and CLIP are imported only when used, so the app and tests start without them (`backend/app/services/jobs.py:8-9` , `backend/requirements-extra.txt:1-2`).
- **httpx with pinned-IP connections:** outbound fetches resolve the host once and connect to that IP (see `docs/auth-and-security.md` , SSRF).

DOCS/SETUP-AND-DEPLOYMENT.MD

Setup and deployment

Plain-language summary

- To run BrandShield on your own computer you need Python, Node, one settings file (`backend/.env`) and two commands: `npm run backend` and `npm run frontend` .
- The same code can also run as a Docker stack (api, worker, database, Redis, web) or as one Vercel project with a Supabase database.
- Whatever you choose, the database tables are created by migrations (`alembic`). If the API says the database is not ready, the migration step has not been run.
- Health pages (`/health` , `/health/live`) tell you whether the app is up. Logs are structured JSON with secrets removed.
- Several parts of the Docker and Vercel setup were written but never run by the authors. They are marked below.

1. Local setup (no Docker)

Step	Command (from the project root unless stated)	Source
1. Settings file	`copy backend\.env.example backend\.env` (Linux/macOS: `cp backend/.env.example backend/.env`)	`backend/.env.example`
2. Fill required values	`JWT_SECRET` (16+ chars), `ADMIN_EMAIL` , `ADMIN_PASSWORD`	`backend/app/config/config.py:40,45-46`
3. Generate a secret	`python -c "import secrets; print(secrets.token_urlsafe(48))"`	`docs/DEPLOY_VERCEL.md` section 3.3
4. Virtual env + packages	done automatically by `npm run backend` on first run, or by hand: `cd backend` , `python -m venv .venv` , `.\.venv\Scripts\python -m pip install -r requirements-dev.txt`	`scripts/dev.mjs:91-100`
5. Migrations	`cd backend` then `.\.venv\Scripts\python -m scripts.migrate` (or `python -m alembic upgrade head` ; `npm run backend` runs the latter)	`backend/scripts/migrate.py:45-51` , `scripts/dev.mjs:102`
6. Web packages	done automatically by `npm run frontend` on first run, or `npm install` in `frontend/` and `studio/`	`scripts/dev.mjs:50-56,83-84`

Notes

- The Settings class reads `backend/.env` and also a `.env` in the folder above `backend/` (the project root), in that order (`backend/app/config/config.py:18-24`). The root `.env` is the Docker compose file (see section 3); do not put compose-only values there for a non-Docker run.
- Blank values count as "not set" (`env_ignore_empty=True` , `config.py:23`).
- The default database is a SQLite file `backend/data/scammind.db` ; `DATA_DIR` defaults to `backend/data` (`config.py:28-29`). Without `REDIS_URL` scans run in an in-process thread (`config.py:110-111`).
- `backend/.env` is gitignored (`.gitignore` : `.env` , `.env.*` , `!backend/.env.example`). Never commit it.
- Optional extras (Sentry, CLIP logo model, Google Play scraper, ImageHash): `pip install -r requirements-extra.txt` (`backend/requirements-extra.txt`). Not needed to run or test.

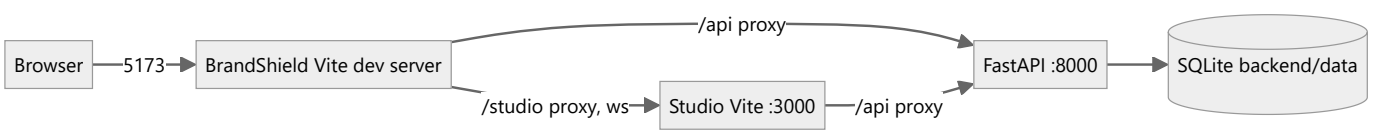
2. The two dev commands (`scripts/dev.mjs`)

`package.json:5-8` maps `npm run backend` to `node scripts/dev.mjs backend` and `npm run frontend` to `node scripts/dev.mjs frontend` . The parent folder `ScamMind AI New/package.json` forwards both with `npm --prefix "ScamMind AI" run ...` .

	`npm run backend`	`npm run frontend`
Port check	refuses to start if 8000 is in use (`scripts/dev.mjs:89`)	refuses if 5173 or 3000 is in use (`:80`)
First run	creates `backend/.venv` and pip-installs `requirements.txt` if missing (`:93-100`)	`npm install` in `frontend/` and `studio/` if `node_modules` is missing (`:83-84`)
Migrations	runs `python -m alembic upgrade head` ; stops with "Database migration failed: check DATABASE_URL in backend/.env" on failure (`:102-106`)	none
Server	`uvicorn app.main:create_app --factory --port 8000 --reload --reload-dir app` (`:108`)	Studio `npm run dev` (port 3000) and BrandShield `npm run dev` (port 5173) (`:87-88`)
Open	http://localhost:8000/docs, `/health/live` (`:107`)	http://localhost:5173 (Studio at `/studio/` ; port 3000 redirects there) (`:85`)
Stop	Ctrl+C, or when any child exits, the rest stop (`:26-31`); on Windows `taskkill /T /F` (`:39`)	same

How the port check works: it tries to connect to the port on `127.0.0.1` and `::1` ; a successful connection means busy (`:57-68`).

How the pieces fit in dev:



- BrandShield's own pages call the API directly at `http://127.0.0.1:8000` in development (`frontend/src/api/client.ts:27`).

- `/studio` is proxied to port 3000 and `/api` to port 8000 so Studio shares BrandShield's origin and login (`frontend/vite.config.ts:10-13` , `studio/vite.config.ts:11-21`).
- Opening Studio's port 3000 directly redirects to `http://localhost:5173/studio/` (`studio/src/lib/devRedirect.ts:16-21`).

3. Docker

Files: `docker-compose.yml` , `docker-compose.dev.yml` , `backend/Dockerfile` , `backend/docker-entrypoint.sh` , `frontend/Dockerfile` , `frontend/nginx/` , `studio/Dockerfile` , `studio/nginx/default.conf` .

> The Docker files state they were **never built or run** where they were written (`backend/Dockerfile:5` , `frontend/Dockerfile:5` , `studio/Dockerfile:6`). CI builds the three images without running them (`.github/workflows/ci.yml:201-231`).

Quick commands

Task	Command
Create compose settings	`cp .env.example .env` (root `.env.example`), fill `POSTGRES_PASSWORD` , `REDIS_PASSWORD` , `JWT_SECRET` , `ADMIN_EMAIL` , `ADMIN_PASSWORD` (compose refuses to start without them: `docker-compose.yml:20-23,79-80`)
Start everything	`docker compose up --build`
Start with db/redis/studio ports published on localhost	`docker compose -f docker-compose.yml -f docker-compose.dev.yml up --build`
One-off migration	`docker compose run --rm api migrate` (`docker-compose.yml:82` comment; `backend/docker-entrypoint.sh:26`)

Services (`docker-compose.yml`)

Service	Image / build	Role	Notes
`db`	`postgres:17-alpine` (`:39`)	PostgreSQL, volume `pgdata`	healthcheck `pg_isready` (`:47-52`)
`redis`	`redis:7.2-alpine` (`:55`)	RQ queue, password via `REDISCLI_AUTH`	appendonly on (`:60`)
`api`	`./backend` Dockerfile, tag `brandshield-server:local` (`:70-74`)	FastAPI on `127.0.0.1:${API_PORT:-8000}`	runs migrations then uvicorn; scheduler on by default (`:83`); healthcheck `/health/live` (`:95`)
`worker`	same image, command `worker` (`:106-108`)	RQ worker (`python -m app.worker`)	scheduler forced off (`:111`); starts after api is healthy (`:114-116`)
`frontend`	`./frontend` Dockerfile (`:132-136`)	nginx on `127.0.0.1:${FRONTEND_PORT:-5173}` -> 8080	`API_URL` written to `/config.js` at start (`:140`)
`studio`	`./studio` Dockerfile (`:153-156`)	nginx, not published; frontend nginx proxies `/studio/`	`docker-compose.dev.yml:18-19` publishes it on `${STUDIO_PORT:-3000}`

Compose forces `SCAN_EXECUTOR=rq` and `APP_ENV=production` unless overridden (`docker-compose.yml:16,22`); the dev overlay sets `APP_ENV=local` and `LOG_LEVEL=DEBUG` on api and worker (`docker-compose.dev.yml:10-17`) and publishes `db` and `redis` on `127.0.0.1` (`:4-9`).

Images

Image	Build	Source
Server (api, worker, migrate)	two stages: venv with `requirements.txt` (+ `requirements-extra.txt` if `--build-arg INSTALL_EXTRAS=true`), then slim runtime, non-root uid 10001, `VOLUME /data` , healthcheck on `/health/live`	`backend/Dockerfile:8-42`
Entrypoint roles	`api` (migrate unless `RUN_MIGRATIONS=false` , then uvicorn with `--proxy-headers`), `worker` , `migrate`	`backend/docker-entrypoint.sh:15-27`
Frontend	Vite build on `node:22-alpine` , served by `nginx-unprivileged:1.27-alpine` on 8080	`frontend/Dockerfile:8-23`
Studio	Vite build (base `/studio/`) placed in `/usr/share/nginx/html/studio`	`studio/Dockerfile:8-21`

nginx

Behaviour	Source
Security headers (nosniff, DENY framing, HSTS, CSP, Permissions-Policy) on every response	`frontend/nginx/default.conf.template:13-20`
`/config.js` served from `/tmp/runtime-config/config.js` , never cached	`default.conf.template:23-27`
`/healthz` returns `ok`	`default.conf.template:29-33`
`/studio/` proxied to `http://studio:8080` ; `/studio` redirects to `/studio/`	`default.conf.template:38-56`
`/api/` proxied to `http://api:8000` , 6 MB body, 180 s read timeout (for Studio image generation)	`default.conf.template:60-70`
`/assets/` cached one year; everything else falls back to `index.html`	`default.conf.template:73-82`
`API_URL` must start with `http://` or `https://` and contain no credentials/quotes, else the container refuses to start	`frontend/nginx/40-runtime-config.sh:10-17`

The worker healthcheck in `docker-compose.yml:122` imports `get_settings` from `app.config.config` (`backend/app/config/config.py`) and calls `check_worker` (`backend/app/services/health.py:99`). It previously imported a non-existent `app.core.config` ; that was corrected. Not verified by running Docker.

4. Building

Target	Command	Output
BrandShield	`npm run build` in `frontend/` (`tsc -b && vite build` , `frontend/package.json:8`)	`frontend/dist`
Brand Studio	`npm run build` in `studio/` (`vite build` , `studio/package.json:48`)	`studio/dist` (base `/studio/`)
Backend	no build step; Docker image or `pip install -r requirements.txt`	n/a

In a production build the frontend calls the API on the same origin (relative `/api/...`) unless `API_URL` (runtime, `/config.js`) or `VITE_API_URL` (build time) is set (`frontend/src/api/client.ts:22-27`).

5. Environments

	Dev	Stage	Prod
How	`npm run backend` + `npm run frontend`	**Needs confirmation:** the repo defines no staging environment; Vercel preview deployments and a second compose stack are possible but not configured	Docker (generic host) or Vercel
`APP_ENV`	`local` (default, `config.py:26`)	n/a	`production` (compose default `docker-compose.yml:16` ; Dockerfile `:23`)
Database	SQLite file	n/a	PostgreSQL (compose `db` , or Supabase on Vercel)
Scan executor	thread (no Redis)	n/a	`rq` with worker (compose) or `inline` (Vercel)
Scheduler	in-process loop (`config.py:122`)	n/a	exactly one process (compose api) or Vercel Cron
Data	sample unless a live source is configured	n/a	`auto` : live if Tavily key, SearXNG URL or google-play package is configured (`backend/app/core/collectors/selection.py:92`)

`APP_ENV=production` makes a missing or old migration show as `degraded` in `/health` (`backend/app/services/health.py:66-80`). `APP_ENV=test` disables the scheduler (`backend/app/main.py:86`).

6. Vercel (one project, three services)

Read `docs/DEPLOY_VERCEL.md` for the full step list. **That guide itself says the deployment is unverified** (Services is a Beta feature; nobody had deployed it). Verified-from-code summary:

`vercel.json`

Part	What it does	Source
`services.backend`	root `backend` , FastAPI, entrypoint `api.index:app` , `maxDuration` 300 s, excludes tests/scripts/data from the bundle	`vercel.json:4-14`
`services.frontend`	root `frontend` , Vite; rewrite serves `/index.html` for non-file paths (SPA routes)	`vercel.json:15-24`
`services.studio`	root `studio` , Vite; rewrites map `/studio/assets/*` and `/studio/<file.ext>` to built files, other `/studio/...` to `/index.html`	`vercel.json:25-42`
Top-level `rewrites` (order matters)	`/api/(.*)` , `/health` , `/health/live` -> backend; `/studio` , `/studio/(.*)` -> studio; `/(.*)` -> frontend	`vercel.json:44-80`
`crons`	`GET /api/cron/scan` daily `0 6 * * *` (UTC)	`vercel.json:82-87`
`headers`	security headers + CSP (`connect-src 'self'`), immutable cache for `/assets/*` and `/studio/assets/*` , `no-cache` for `/config.js`	`vercel.json:88-150`

`/docs` , `/redoc` and `/openapi.json` are not routed to the backend (they fall through to the frontend).

Vercel-mode defaults (only when `VERCEL` is set)

`backend/app/config/config.py:175-190` : unless you set them explicitly, `SCAN_EXECUTOR=inline` , `SCHEDULER_ENABLED=false` , `RESPONSE_CACHE_SECONDS=0` , `STORAGE_BACKEND=database` , `DATA_DIR=/tmp/brandshield` . Database pool shrinks to 1 + 1 connections with a 300 s recycle (`backend/app/db/session.py:23-26`). `backend/api/index.py` builds the app at module level, seeds the admin/demo users at cold start (never raises) and never starts the scheduler thread (`backend/api/index.py:19-40`).

Because scans run inline, a scan lives inside one request and is bounded by `maxDuration` (300 s). A scan left `running` after a timeout is failed after `SCAN_STALE_MINUTES` (default 30, `config.py:114`).

Environment variables to set on Vercel

Name	Required	Notes
`DATABASE_URL`	yes	Supabase transaction pooler (port 6543), form `postgresql+psycopg://...`
`JWT_SECRET`	yes	16+ characters (`config.py:40`)
`ADMIN_EMAIL` , `ADMIN_PASSWORD`	for the first sign-in	admin is created at cold start if missing; an existing password is never overwritten
`APP_ENV`	recommended	`production`
`CRON_SECRET`	for scheduled scans	without it `/api/cron/scan` answers 503; wrong secret gives 401 (`backend/app/api/cron.py:34-38`)
`MIGRATION_DATABASE_URL`	for you only	session/direct URL (port 5432) used by migrations, not needed at runtime (`config.py:37,217-222`)
`DEMO_EMAIL` / `DEMO_PASSWORD` , `DEMO_ADMIN_EMAIL` / `DEMO_ADMIN_PASSWORD`	optional	public demo accounts; use throwaway passwords
`TAVILY_API_KEY` , `GEMINI_API_KEY` , `GOOGLE_SAFE_BROWSING_API_KEY` , `GROQ_API_KEY` , `POLLINATIONS_API_KEY` , `VOICE_GEMINI_API_KEY`	optional	live sources and AI features; backend-only, never `VITE_` -prefixed
`SCAN_INTERVAL_MINUTES`	optional	e.g. `1440` for daily
Also enable	"Enable access to System Environment Variables" so `VERCEL=1` exists	`docs/DEPLOY_VERCEL.md` section 3.3

Leave `VITE_API_URL` unset (same origin) and do not set `STORAGE_BACKEND` , `SCAN_EXECUTOR` , `SCHEDULER_ENABLED` , `RESPONSE_CACHE_SECONDS` (the guide's advice; the code defaults already match). `CORS_ORIGINS` is only needed for local development or a separate frontend host (`backend/.env.example` comment).

Database migration step (manual)

Nothing in the Vercel build runs migrations. Run them from your computer after every release that adds a migration:

```
cd backend
$env:MIGRATION_DATABASE_URL = "<session pooler string, port 5432>"
.\.venv\Scripts\python -m scripts.migrate
```

`scripts.migrate` reads only `MIGRATION_DATABASE_URL` or `DATABASE_URL` (`backend/scripts/migrate.py:47` , `config.py:219-222`) and on PostgreSQL holds an advisory lock so concurrent runs are safe (`migrate.py:32-42`). Skipping it gives a 500 on first request and `/health` shows `migration: degraded` .

Supabase notes

Topic	Fact	Source
Runtime pooler	`DATABASE_URL` should use the **transaction pooler (port 6543)**. For `:6543/` URLs the code sets `prepare_threshold=None` (no server-side prepared statements)	`backend/app/db/session.py:30-31`
Session pooler limits	Supabase session mode allows only 15 clients in total; this is why the pool is small and why a session-mode `DATABASE_URL` on serverless exhausts connections (see troubleshooting)	`session.py:19-22`
Migrations	use session/direct URL (port 5432) via `MIGRATION_DATABASE_URL`	`config.py:35-37`
Driver prefix	URL must start `postgresql+psycopg://` ; plain `postgresql://` raises `ModuleNotFoundError: psycopg2` per the Vercel guide	`docs/DEPLOY_VERCEL.md` section 6
Stored files	on Vercel logos, candidate images and evidence live in the `stored_files` table and count toward database size	`config.py:32-34` , migration `0009_stored_files.py`

Limits on Vercel

No CLIP model (hash fallback), no RQ worker or threads, rate limiters are per instance, 4.5 MB body limit, cold starts (`docs/DEPLOY_VERCEL.md` section 5).

7. Generic container host

Needs managed PostgreSQL and Redis, a TLS proxy, and a persistent volume for `DATA_DIR` shared by api and worker (`docker-compose.yml:88,113`). Run `migrate` as a pre-deploy job and set `RUN_MIGRATIONS=false` on extra api replicas (safe either way: advisory lock). Exactly one process runs the scheduler: `SCHEDULER_ENABLED=true` on one api, `false` elsewhere (`docker-compose.yml:83,111`). Set `CORS_ORIGINS` to the frontend's https origin and `FORWARDED_ALLOW_IPS` to the proxy address (`backend/docker-entrypoint.sh:20`). Probes: liveness `/health/live` , readiness `/health` .

8. CI/CD pipeline (`.github/workflows/ci.yml`)

Triggers: push to `main` and every pull request (`:6-9`); runs are cancelled when superseded (`:14-16`); read-only token (`:11-12`). No secrets are used (`:1-2`). The workflow notes it was never run on GitHub Actions at the time of writing (`:3`).

Job	What it runs	Depends on	Lines
`lint`	`ruff check .` (backend), `npm run lint` (oxlint) and `npm run typecheck` (frontend)	none	19-47
`test-server`	`python -m pytest -q` on Python 3.11 and 3.12, then `python -m scripts.canary`	none	49-71
`test-server-postgres`	postgres:17 service, `scripts.migrate` twice (second must be a no-op), `pytest -q` with `TEST_DATABASE_URL`	none	73-112
`test-frontend`	`npm test`, `npm run build`	none	114-132
`test-studio`	`npx tsc --noEmit`, `npx vitest run`, `npm run build`	none	134-155
`e2e`	Playwright (Chromium) against a real sample-mode API; uploads traces on failure	test-server, test-frontend, test-studio	157-199
`build`	builds server, studio and frontend Docker images, no push	lint, test-server, test-frontend, test-studio	201-231
`publish`	commented-out template pushing to ghcr.io	would need build, e2e, test-server-postgres	233-261

There is no automatic deployment job. **Needs confirmation:** how Vercel deploys are triggered (Git integration on the Vercel side is assumed by `docs/DEPLOY_VERCEL.md` but not configured in the repo).

9. Rollback

Layer	What the repo says	Source
Application	roll back application code first; keep previous image tags	guidance from the former README deployment section (replaced; see git history), not enforced by code
Database	every migration 0001-0012 defines a `downgrade()`; avoid downgrading a database that holds real data without a backup	`backend/alembic/versions/*.py` (one `def downgrade` each)
Data	back up the database and `DATA_DIR` (evidence snapshots with SHA-256, report files, logos) before releases	`docker-compose.yml:88,113`
Demo emergency	an admin can switch data mode to Sample in Settings to continue on fictional data	`backend/app/api/settings.py:136,181-182`

Downgrade command (manual): `python -m alembic downgrade -1` from `backend/` (standard Alembic; `backend/alembic.ini`). **Needs confirmation:** a Vercel "instant rollback" procedure is not documented in the repo.

10. Monitoring and logging

Item	Detail	Source
Logs	JSON lines to stdout: `ts`, `level`, `logger`, `message`, optional `exc`; level from `LOG_LEVEL`	`backend/app/config/logging.py:45-64`
Redaction	configured secret values, `password/token/api_key/authorization` pairs, bearer tokens and JWTs are replaced by `[REDACTED]`	`logging.py:7-28`, `config.py:197-202`
`GET /health` (also `/api/health`)	public; `status` ok / degraded / down plus one word per component (database, migration, redis, worker, data_dir, collectors); HTTP 503 only if the database is down; makes no third-party calls	`backend/app/api/health.py:22-27`, `backend/app/services/health.py:226-230`
`GET /health/live`	process liveness, no dependencies	`health.py:30-33`
`GET /api/health/details`	admin only: reasons, migration revision, worker count, disk	`health.py:36-40`
`POST /api/health/canary` / `python -m scripts.canary`	in-memory pipeline self-check (no network, no real data)	`health.py:43-49`, `ci.yml:69-71`
Sentry	optional: set `SENTRY_DSN` and `pip install -r requirements-extra.txt` (or `INSTALL_EXTRAS=true` build arg); events are scrubbed before sending; without the package one warning is logged	`backend/app/config/observability.py:1-12,79-87`, `docker-compose.yml:73`
Rate limit	600 requests/min per IP and token, in memory; 429 "Too many requests. Slow down and retry shortly."	`main.py:119-135`, `config.py:163`
Probes	Docker healthchecks use `/health/live` (api) and `/healthz` (nginx)	`docker-compose.yml:95`, `frontend/Dockerfile:23`

DOCS/CONFIGURATION.MD

Configuration reference

Plain-language summary

- BrandShield is configured only through environment variables (and, for local runs, a `backend/.env` file that is never committed).
- Only one value is truly mandatory for the backend to start: `JWT_SECRET`. Everything else has a safe default or is optional.
- Features that call outside services (web search, AI, link reputation) stay off until the matching key is set. Without keys the app runs on built-in sample data.
- This page lists every variable by area: what it does, its default, whether it is required, and the file that reads it. No real values appear here.
- Secrets are named but never shown. Put them in `.env` locally or in the Vercel / Docker environment, never in code.

1. How configuration is loaded

Mechanism	Detail	Source
Backend settings class	One pydantic `Settings` class; every field maps to an environment variable of the same name, case-insensitive. Extra variables are ignored.	`backend/app/config/config.py:17-24`
`.env` files read	`backend/.env` and then the repo-root `.env` (the root one is what `docker-compose.yml` uses).	`backend/app/config/config.py:19`
Blank means unset	`NAME=` left empty (as in the `.env.example` files) is treated as "not set", so the default applies.	`backend/app/config/config.py:23`
Only required field	`jwt_secret` has no default and must be at least 16 characters; the server will not start without it.	`backend/app/config/config.py:40`
Migration-only settings	`DatabaseSettings` reads only `DATABASE_URL` and `MIGRATION_DATABASE_URL`, so Alembic and `scripts/migrate.py` run without `JWT_SECRET`. `MIGRATION_DATABASE_URL` wins when set.	`backend/app/config/config.py:210-222`, `backend/scripts/migrate.py:47`, `backend/alembic/env.py:12`
Cached	`get_settings()` is `lru_cache`d, so changes need a restart.	`backend/app/config/config.py:205-207`
Vercel mode	When `VERCEL` is set, five defaults change (see section 2.1) unless you set them explicitly.	`backend/app/config/config.py:12-14`, `backend/app/config/config.py:175-190`
Secrets type	Secret values use pydantic `SecretStr`; `secret_values` lists the ones masked in logs (see `docs/auth-and-security.md`).	`backend/app/config/config.py:196-202`

Templates (names only): `backend/.env.example` (non-Docker backend), `.env.example` (root, used by Docker Compose), `frontend/.env.example`, `studio/.env.example`.

1.1 Which `.env` goes with which way of running

How you run	File read	Notes
`npm run backend` (root)	`backend/.env`	`scripts/dev.mjs` creates `backend/.venv`, installs requirements, applies migrations, starts uvicorn on port 8000. `scripts/dev.mjs` (backend branch)
`npm run frontend` (root)	none required	Starts the BrandShield Vite server on 5173 and Brand Studio on 3000.
`docker compose up`	root `.env`	Compose passes a fixed list of variables into the containers (`docker-compose.yml:15-35`, `:77-84`). Variables not listed there (for example `GROQ_API_KEY`) are **not** forwarded to the container.
Vercel	Project environment variables	Same names as the backend table below. See `docs/DEPLOY_VERCEL.md`.
Tests	Set in `backend/tests/conftest.py`	Throwaway values; shell secrets are popped so a real key never reaches a test run (`backend/tests/conftest.py:4-19`).

2. Backend variables

Defaults come from `backend/app/config/config.py` (line numbers below). "Req" = required.

2.1 Core

Variable	Purpose	Default	Req	Where read
`APP_ENV`	`local`, `test` or `production`. `test` also keeps the in-process scheduler off. Also the default Sentry environment name.	`local` (Docker image sets `production`)	no	`config.py:26`, `main.py:86`, `backend/Dockerfile:22-23`
`LOG_LEVEL`	Root log level for the JSON logger.	`INFO`	no	`config.py:27`, `backend/app/config/logging.py:58-64`
`VERCEL`	Set by the Vercel platform (not by you). Switches on serverless defaults.	unset	no	`config.py:12-14`, `backend/app/db/session.py:23`
`PORT`	Port used by the container health check and entrypoint.	`8000`	no	`backend/Dockerfile:22-23,39`, `backend/docker-entrypoint.sh:20`
`FORWARDED_ALLOW_IPS`	Which proxy IPs uvicorn trusts for `X-Forwarded-For` (affects the client IP seen by rate limiters).	`127.0.0.1`	no	`backend/docker-entrypoint.sh:21`
`RUN_MIGRATIONS`	Container `api` role applies migrations before start unless `false`.	`true`	no	`backend/docker-entrypoint.sh:17`, `docker-compose.yml:82`
`INSTALL_EXTRAS`	Docker build arg: `true` installs `requirements-extra.txt` (Sentry, CLIP, etc.).	`false`	no	`backend/Dockerfile:15-17`, `docker-compose.yml:73`

Vercel-mode defaults (applied only when `VERCEL` is set and the variable is not given explicitly) — `backend/app/config/config.py:181-187`:

Setting	Value on Vercel	Why (code comment)
`SCAN_EXECUTOR`	`inline`	no background threads or RQ worker; the scan runs inside the request
`SCHEDULER_ENABLED`	`false`	no in-process loop; Vercel Cron calls `/api/cron/scan`
`RESPONSE_CACHE_SECONDS`	`0`	a per-instance cache would go stale across instances
`STORAGE_BACKEND`	`database`	the function file system is read-only (except `/tmp`) and not shared
`DATA_DIR`	`/tmp/brandshield`	only `/tmp` is writable

2.2 Database

Variable	Purpose	Default	Req	Where read
`DATABASE_URL`	SQLAlchemy URL. SQLite file for local runs; `postgresql+psycopg://...` for Postgres/Supabase.	`sqlite:///<backend>/data/scammind.db`	no	`config.py:28`, `config.py:216`
`MIGRATION_DATABASE_URL`	Direct or session-mode Postgres URL (port 5432) used only by Alembic / `scripts/migrate.py`, so `DATABASE_URL` can be the Supabase transaction pooler (port 6543).	unset (falls back to `DATABASE_URL`)	no	`config.py:37`, `config.py:217-222`
`DB_POOL_SIZE`	Postgres pool size.	`5`; `1` on Vercel	no	`backend/app/db/session.py:24`
`DB_MAX_OVERFLOW`	Extra connections above the pool.	`2`; `1` on Vercel	no	`backend/app/db/session.py:25`
`DB_POOL_RECYCLE`	Seconds before a connection is recycled.	`1800`; `300` on Vercel	no	`backend/app/db/session.py:26`
`DB_PRE_PING`	`1` / `true` / `yes` enables a liveness ping per checkout.	off	no	`backend/app/db/session.py:27`
`TEST_DATABASE_URL`	Tests only: run the suite against a throwaway Postgres database (each test engine gets a random schema).	unset	no	`backend/tests/pg_support.py:22-28`, `.github/workflows/ci.yml:92`

Notes: a URL containing `:6543/` automatically disables server-side prepared statements (`backend/app/db/session.py:30-31`). TCP keepalives are always set for Postgres (`session.py:29`).

2.3 Authentication and tokens

Variable	Purpose	Default	Req	Where read
`JWT_SECRET`	HMAC key that signs access and refresh tokens. Minimum 16 characters. **Secret.**	none	**yes**	`config.py:40` , `backend/app/config/security.py:35,48`
`JWT_ALGORITHM`	JWT signing algorithm.	`HS256`	no	`config.py:41`
`ACCESS_TOKEN_MINUTES`	Access token lifetime.	`15`	no	`config.py:42` , `security.py:39`
`REFRESH_TOKEN_DAYS`	Refresh token lifetime.	`7`	no	`config.py:43` , `security.py:43`
`ADMIN_EMAIL`	Email of the real administrator created at start if absent.	unset (no admin created)	no (needed to have an admin)	`config.py:45` , `backend/app/services/seed.py:14-24`
`ADMIN_PASSWORD`	Password for that admin. Never overwrites an existing account's password. **Secret.**	unset	no	`config.py:46` , `seed.py:16,21`
`ADMIN_NAME`	Display name of the admin.	`Administrator`	no	`config.py:51`
`LOGIN_MAX_FAILURES`	Failed logins allowed per email+IP in the window.	`5`	no	`config.py:53` , `main.py:99`
`LOGIN_WINDOW_SECONDS`	Length of that window.	`900`	no	`config.py:54` , `main.py:99`
`API_RATE_LIMIT_PER_MINUTE`	General limit for `/api/*` , per client IP and per bearer token. `0` = off.	`600`	no	`config.py:163` , `main.py:106,119-135`

2.4 Demo accounts

Both demo accounts are deliberately public: their credentials are served by `GET /api/auth/demo` (`backend/app/api/auth.py:82-96`). Use throwaway passwords.

Variable	Purpose	Default	Req	Where read
`DEMO_EMAIL`	Public demo account (role `analyst`). Both `DEMO_EMAIL` and `DEMO_PASSWORD` must be set. Ignored if equal to `ADMIN_EMAIL` .	unset (no demo)	no	`config.py:47` , `seed.py:27-60`
`DEMO_PASSWORD`	Its password; a changed value is applied at the next start. **Public by design.**	unset	no	`config.py:48` , `seed.py:52-54`
`DEMO_ADMIN_EMAIL`	Second public account: a guarded administrator. Must differ from `ADMIN_EMAIL` and `DEMO_EMAIL` .	unset	no	`config.py:49` , `seed.py:63-97`
`DEMO_ADMIN_PASSWORD`	Its password. **Public by design.**	unset	no	`config.py:50` , `seed.py:89-91`

2.5 Sign-up

Variable	Purpose	Default	Req	Where read
`SIGNUP_ENABLED`	Turns `POST /api/auth/register` on or off (off returns 403).	`true`	no	`config.py:57` , `backend/app/api/auth.py:107-108`
`SIGNUP_MAX_PER_IP_PER_HOUR`	Sign-ups per client IP per hour (in memory, per process).	`5`	no	`config.py:58` , `main.py:100`
`SIGNUP_MAX_PER_EMAIL_PER_HOUR`	Sign-ups per email per hour (in memory, per process).	`3`	no	`config.py:59` , `main.py:101`

2.6 CORS and web

Variable	Purpose	Default	Req	Where read
`CORS_ORIGINS`	Comma-separated allowed origins. No wildcard; credentials are off.	`http://localhost:5173`	no	`config.py:30,192-194` , `main.py:160-167`
`CORS_ORIGIN_REGEX`	Extra allowed origins as a regular expression (for example preview deployments).	unset	no	`config.py:31` , `main.py:163`
`RESPONSE_CACHE_SECONDS`	TTL of the short in-process cache for heavy read endpoints. `0` = off.	`15` ; `0` on Vercel	no	`config.py:123` , `main.py:137-158` , `backend/app/config/response_cache.py:15-18`

2.7 Uploads, imports and URL checks

Variable	Purpose	Default	Where read
`LOGO_MAX_BYTES`	Max brand logo upload size.	`2097152` (2 MB)	`config.py:61` , `backend/app/api/brands.py:222`
`CANDIDATE_IMAGE_MAX_BYTES`	Max candidate image / app icon upload.	`2097152`	`config.py:84` , `backend/app/api/candidates.py:84-86`
`IMPORT_MAX_BYTES`	Max CSV/JSON candidate import size.	`1048576` (1 MB)	`config.py:85` , `candidates.py:63-66`
`IMPORT_MAX_ROWS`	Max rows per import.	`500`	`config.py:86` , `candidates.py:68-70`
`URL_CHECK_TIMEOUT_SECONDS`	Timeout for the SSRF-safe reachability checker.	`5.0`	`config.py:62` , `main.py:107-111`
`URL_CHECK_MAX_REDIRECTS`	Redirect hops followed (each re-validated).	`3`	`config.py:63` , `main.py:107-111`
`URL_CHECK_MAX_BYTES`	Response bytes read before stopping.	`65536`	`config.py:64` , `main.py:107-111`

2.8 Scanning and collectors

Variable	Purpose	Default	Where read
`TAVILY_API_KEY`	Tavily web-search key. Without it the Tavily collector reports `no_key`. **Secret.**	unset	`config.py:91`, `backend/app/core/collectors/selection.py:83`
`TAVILY_MONTHLY_BUDGET`	Credits per UTC month the app allows itself (free plan is 1000).	`900`	`config.py:92`
`TAVILY_MAX_QUERIES_PER_SCAN`	Max Tavily queries per scan.	`12`	`config.py:93`
`TAVILY_SEARCH_DEPTH`	`basic` (1 credit) or `advanced` (2).	`basic`	`config.py:94`
`TAVILY_MAX_RESULTS`	Results per Tavily call.	`10`	`config.py:95`
`SEARXNG_BASE_URL`	Optional SearXNG instance. Must be a public host unless the next flag is set.	unset	`config.py:96`, `backend/app/core/collectors/searxng.py:79-93`
`SEARXNG_ALLOW_PRIVATE_HOSTS`	Operator opt-in for a SearXNG on a private network.	`false`	`config.py:97`, `searxng.py:82-87`
`SEARXNG_MAX_QUERIES_PER_SCAN`	Max SearXNG queries per scan.	`12`	`config.py:98`
`ITUNES_COUNTRY`	Apple App Store country code.	`in`	`config.py:99`
`ITUNES_CALLS_PER_MINUTE`	Own throttle for iTunes search.	`20`	`config.py:100`
`ITUNES_CACHE_TTL_SECONDS`	iTunes answer cache.	`3600`	`config.py:101`
`ITUNES_MAX_QUERIES_PER_SCAN`	Max iTunes queries per scan.	`8`	`config.py:102`
`GOOGLE_PLAY_COUNTRY`	Google Play country code (needs the optional `google-play-scraper` package).	`in`	`config.py:103`
`GOOGLE_PLAY_MAX_QUERIES_PER_SCAN`	Max Google Play queries per scan.	`6`	`config.py:104`
`SOURCE_SLOW_THRESHOLD_MS`	A source slower than this is shown as "slow".	`4000`	`config.py:105`
`COLLECTOR_TIMEOUT_SECONDS`	Per-collector HTTP timeout.	`15.0`	`config.py:106`
`REACHABILITY_CHECKS_PER_SCAN`	`>0` runs SSRF-safe reachability checks of that many result URLs.	`0` (off)	`config.py:107`
`REDIS_URL`	Redis for the RQ scan queue. Unset means in-process thread runner. May contain a password, so treated as a **secret**.	unset	`config.py:110`, `backend/app/services/jobs.py:25-30`, `backend/app/worker.py:89-95`
`SCAN_EXECUTOR`	`auto` (rq if `REDIS_URL` else thread), `rq`, `thread` or `inline`.	`auto`; `inline` on Vercel	`config.py:111`, `jobs.py:25-32`
`SCAN_QUEUE_NAME`	RQ queue name.	`scans`	`config.py:112`, `worker.py:96`
`SCAN_JOB_TIMEOUT_SECONDS`	RQ job timeout.	`900`	`config.py:113`, `jobs.py:79`
`SCAN_STALE_MINUTES`	A queued/running scan older than this is treated as dead and failed.	`30`	`config.py:114`
`SCAN_PREFILTER`	Skip logo/AI steps for candidates with no name signal or keyword hit.	`true`	`config.py:115`
`SCAN_LOGO_FETCH`	SSRF-safe download of candidate logos/icons (live data only).	`true`	`config.py:116`
`SCAN_LOGO_MAX_BYTES`	Cap per downloaded candidate logo.	`2097152`	`config.py:117`
`SCAN_LOGO_TIMEOUT_SECONDS`	Timeout per logo download.	`5.0`	`config.py:118`
`SCAN_MAX_CANDIDATES`	Max candidates per scan.	`500`	`config.py:119`
`SCAN_MAX_LOGO_FETCHES`	Max logo downloads per scan.	`25`	`config.py:120`
`SCREENSHOTS_ENABLED`	Playwright screenshot hook (the package is not installed by default).	`false`	`config.py:121`
`IMAGE_SIMILARITY_BACKEND`	`hash`, `multihash`, `imagehash` or `clip` for logo/icon scoring.	`hash`	`config.py:81`
`LOGO_COMPARE_CLIP`	Look-alike Lab logo tester only: `auto` or `off`.	`auto`	`config.py:83`
`SEVERITY_MEDIUM_MIN` / `SEVERITY_HIGH_MIN` / `SEVERITY_CRITICAL_MIN`	Severity cut-offs (LOW below medium). Must satisfy 0 < medium < high < critical <= 100.	`40` / `70` / `85`	`config.py:166-168`, `backend/app/core/engine/risk_engine.py:46-54`
`GOOGLE_SAFE_BROWSING_API_KEY`	Safe Browsing Lookup v4 key. No key: reputation is "unavailable", no request is made. **Secret.**	unset	`config.py:132`, `backend/app/core/providers/safe_browsing.py:12`
`SAFE_BROWSING_TIMEOUT_SECONDS`	Request timeout.	`5.0`	`config.py:133`
`SAFE_BROWSING_DAILY_BUDGET`	Our own request cap per UTC day.	`5000`	`config.py:134`
`SAFE_BROWSING_MAX_URLS_PER_SCAN`	URLs sent per scan.	`100`	`config.py:135`

Variable	Purpose	Default	Where read
`SAFE_BROWSING_CACHE_TTL_SECONDS`	Verdict cache.	`3600`	`config.py:136`

2.9 Providers and AI (scan text analysis)

Hosted text analysis is **off** by default: no candidate text leaves the server unless `SCAM_LLM_ENABLED=true` (`config.py:66-69`).

Variable	Purpose	Default	Where read
`SCAM_LLM_ENABLED`	Master switch for hosted text analysis.	`false`	`config.py:68`, `backend/app/core/pipeline/text_analysis.py:53`
`SCAM_LLM_PROVIDER`	Default provider: `gemini`, `huggingface`, `local` or `off`. An admin can change it in Settings (stored in the database).	`gemini`	`config.py:69`, `backend/app/api/providers.py:50-57`
`GEMINI_API_KEY`	Google Gemini key (scan text, Studio chat fallback, voice fallback). **Secret.**	unset	`config.py:71`, `backend/app/core/providers/text/factory.py:31`
`GEMINI_MODEL`	Gemini model id for scan text analysis.	`gemini-3.5-flash-lite`	`config.py:72`
`HUGGINGFACE_API_TOKEN`	Hugging Face Inference token. **Secret.**	unset	`config.py:74`, `factory.py:33`
`HUGGINGFACE_MODEL`	Hugging Face model id. No default: unavailable until set.	unset	`config.py:75`
`TEXT_EMBEDDINGS_ENABLED`	Embeddings switch (also needs a model id and key).	`false`	`config.py:77`, `factory.py:38-44`
`GEMINI_EMBEDDING_MODEL`	Embedding model id. No default (unverified).	unset	`config.py:78`
`TEXT_PROVIDER_TIMEOUT_SECONDS`	Hosted provider timeout.	`10.0`	`config.py:87`
`TEXT_PROVIDER_MAX_RETRIES`	Retries after the first attempt (5xx/timeout only).	`2`	`config.py:88`

2.10 Brand Studio AI proxy (`/api/studio/ai/*`)

Keys stay on the server; the Studio front end never sees them (`backend/app/api/studio_ai.py:1-3`, `studio/vite.config.ts:7-8`).

Variable	Purpose	Default	Where read
`GROQ_API_KEY`	Groq chat key (primary). Without it, chat falls back to Gemini. **Secret.**	unset	`config.py:139`, `backend/app/services/studio_ai.py:94,152`
`POLLINATIONS_API_KEY`	Pollinations image-generation key. **Secret.**	unset	`config.py:140`, `studio_ai.py:230`
`STUDIO_GROQ_MODEL`	Groq model id.	`openai/gpt-oss-120b`	`config.py:141`
`STUDIO_IMAGE_MODEL`	Pollinations model id.	`black-forest-labs/flux.1-schnell`	`config.py:142`
`STUDIO_AI_RATE_PER_MINUTE`	Per-user rate limit, chat + image together (in memory).	`20`	`config.py:143`, `main.py:103`
`STUDIO_DAILY_TEXT_QUOTA`	Per user per UTC day, stored in `api_usage`. `0` = no limit.	`200`	`config.py:151`
`STUDIO_DAILY_IMAGE_QUOTA`	Same for images.	`50`	`config.py:152`
`STUDIO_AI_TIMEOUT_SECONDS`	Chat call timeout.	`45.0`	`config.py:153`
`STUDIO_IMAGE_TIMEOUT_SECONDS`	Image call timeout.	`120.0`	`config.py:154`
`STUDIO_AI_MAX_RETRIES`	Retries (timeouts, connection errors, 5xx).	`2`	`config.py:155`
`STUDIO_AI_RETRY_BACKOFF_SECONDS`	Backoff between retries.	`0.5`	`config.py:156`
`STUDIO_MAX_MESSAGES` / `STUDIO_MAX_CHARS` / `STUDIO_MAX_TOKENS` / `STUDIO_MAX_PROMPT_CHARS`	Request size limits.	`40` / `30000` / `4096` / `2000`	`config.py:157-160`

2.11 Voice agent (`/api/voice`)

Variable	Purpose	Default	Where read
`VOICE_GEMINI_API_KEY`	Gemini key for the voice agent; falls back to `GEMINI_API_KEY`. Simple commands work with no key. **Secret.**	unset	`config.py:145`, `backend/app/services/voice.py`
`VOICE_GEMINI_MODEL`	Flash model with function calling.	`gemini-3-flash-preview`	`config.py:146`
`VOICE_GEMINI_FALLBACK_MODEL`	Tried once if the first refuses quickly.	`gemini-flash-latest`	`config.py:147`
`VOICE_TIMEOUT_SECONDS`	Gemini refuses deadlines under 10 s.	`10.0`	`config.py:148`
`VOICE_RATE_PER_MINUTE`	Per-user limit (in memory).	`30`	`config.py:149`, `main.py:104`
`VOICE_THINKING_LEVEL`	`minimal`, `low`, ...; empty = model default.	`minimal`	`config.py:150`

2.12 Storage

Variable	Purpose	Default	Where read
`STORAGE_BACKEND`	`local` (files under `DATA_DIR`) or `database` (rows in `stored_files`). Any other value raises at start.	`local`; `database` on Vercel	`config.py:34`, `backend/app/core/storage.py:169-176`
`DATA_DIR`	Local file root (logos, candidate images, evidence, CLIP models).	`<backend>/data`; `/data` in Docker; `/tmp/brandshield` on Vercel	`config.py:29`, `backend/Dockerfile:22-23`, `docker-compose.yml:18`

2.13 Scheduler and cron

Variable	Purpose	Default	Where read
`SCHEDULER_ENABLED`	Run the in-process scheduler loop (never started when `APP_ENV=test`). Exactly one process should run it.	`true`; `false` on Vercel and for the Docker worker	`config.py:122`, `main.py:86-88`, `docker-compose.yml:83,111`
`SCAN_INTERVAL_MINUTES`	Overrides the Settings page schedule. `0` = off.	unset	`config.py:124`
`SCHEDULER_POLL_SECONDS`	How often the loop wakes.	`60`	`config.py:125`, `main.py:87`
`SCHEDULER_MAX_BRANDS_PER_RUN`	Most brands scanned per scheduler/cron pass.	`5`	`config.py:126`
`SCHEDULER_MAX_QUERIES_PER_SCAN`	Credit estimate override; defaults to `TAVILY_MAX_QUERIES_PER_SCAN`.	unset	`config.py:127`
`CRON_SECRET`	Bearer secret for `GET /api/cron/scan`. Unset: endpoint returns 503. **Secret.**	unset	`config.py:38`, `backend/app/api/cron.py:32-38`

The cron schedule itself is `0 6 * * *` (daily 06:00 UTC) in `vercel.json:82-87`.

2.14 Observability

Variable	Purpose	Default	Where read
`SENTRY_DSN`	Enables Sentry (needs `sentry-sdk` from `requirements-extra.txt`). **Secret.**	unset (off)	`config.py:171`, `backend/app/config/observability.py:79-102`
`SENTRY_ENVIRONMENT`	Environment tag.	`APP_ENV`	`config.py:172`, `observability.py:92`
`SENTRY_TRACES_SAMPLE_RATE`	0.0 to 1.0 (clamped).	`0.0`	`config.py:173`, `observability.py:93`

2.15 Scripts and tests (not used by the running app)

Variable	Used by	Purpose	Source
`E2E_DB_PATH`, `E2E_DATA_DIR`	`backend/scripts/e2e_server.py`	Temp SQLite file and data dir for browser tests. Required there.	`e2e_server.py:25-26`
`E2E_PORT`	same	API port (default `8765`).	`e2e_server.py:87`
`E2E_VIEWER_EMAIL`, `E2E_VIEWER_PASSWORD`	same	Optional read-only user seeded for role tests.	`e2e_server.py:54-56`
`HF_HUB_DISABLE_SYMLINKS_WARNING`	CLIP loader	Set to `1` by default (Windows noise).	`backend/app/core/engine/logo_similarity.py:258`

`backend/scripts/e2e_server.py` ignores `backend/.env` and clears third-party keys from the environment (`e2e_server.py:4-6,20-24`).

---

3. Frontend (BrandShield, `frontend/`)

Variable	Purpose	Default	Req	Where read
`VITE_API_URL`	API origin, baked in at build time. Leave unset on Vercel's single-domain setup (relative `/api/...`).	dev: `http://127.0.0.1:8000`; production build: same origin (`''`)	no	`frontend/src/api/client.ts:23-27`, `frontend/.env.example:1-3`
`API_URL` (container, runtime)	nginx writes it to `/config.js` as `window.__SCAMMIND_CONFIG__.API_URL` at container start, and uses it in the CSP. Must start with `http://` or `https://` and contain no credentials, quotes or spaces. Takes precedence over `VITE_API_URL`.	`http://localhost:8000` (Dockerfile)	no	`frontend/nginx/40-runtime-config.sh:1-20`, `frontend/Dockerfile:18`, `frontend/nginx/default.conf.template:20`
`PUBLIC_API_URL` (compose)	Feeds both `VITE_API_URL` build arg and runtime `API_URL`.	`http://localhost:8000`	no	`docker-compose.yml:136,140`
`VITE_DEMO_EMAIL`, `VITE_DEMO_PASSWORD`	Optional local fallback for the "Use demo credentials" button when the server advertises none. Put only in `.env.local`; never on a public deployment.	unset	no	`frontend/src/pages/LoginPage.tsx:21-23`, `frontend/.env.example:4-6`
`VITE_STUDIO_URL`	Only for setups where Studio is on another origin.	`/studio/`	no	`frontend/src/utils/studio.ts:2-3`
`STUDIO_DEV_URL`	Dev server only: where `/studio` is proxied.	`http://127.0.0.1:3000`	no	`frontend/vite.config.ts:11`
`STUDIO_API_PROXY`	Dev server only: where `/api` is proxied (Studio calls relative `/api`).	`http://127.0.0.1:8000`	no	`frontend/vite.config.ts:13`

Runtime file: `frontend/public/config.js` is an empty stub in development (`window.__SCAMMIND_CONFIG__ = {}`); in the nginx image it is replaced by the generated file (`frontend/nginx/default.conf.template:23-27`).

4. Brand Studio (`studio/`)

Studio needs no secrets (`studio/.env.example:1`); nothing from the environment is injected into its bundle (`studio/vite.config.ts:7-8`).

Variable	Purpose	Default	Where read
`STUDIO_API_PROXY`	Dev server only, when Studio is opened directly on port 3000.	`http://127.0.0.1:8000`	`studio/vite.config.ts:23`, `studio/.env.example:2-3`
`VITE_SHIELD_URL`	Dev only: where BrandShield runs so opening Studio's port redirects there.	`http://localhost:5173`	`studio/src/main.tsx:9-12`, `studio/.env.example:5-6`

Fixed Studio settings (not variables): Vite `base: '/studio/'`, dev/preview host `127.0.0.1`, port `3000` with `strictPort` (`studio/vite.config.ts:10,17-25`).

5. Playwright / end-to-end (`frontend/playwright.config.ts`)

Variable	Purpose	Default	Source
`E2E_API_PORT`	Test API port.	`8765`	`frontend/playwright.config.ts:13`
`E2E_WEB_PORT`	Test BrandShield dev-server port.	`5174`	`:14`
`E2E_STUDIO_PORT`	Test Studio dev-server port (not 3000, to avoid a developer's own Studio).	`3100`	`:15`
`E2E_TMP_DIR`	Temp directory shared by main process and workers.	new temp dir	`:17-18`
`E2E_PYTHON`	Python used to start the backend.	`backend/.venv/...python` (CI sets `python`)	`:20`, `.github/workflows/ci.yml:162-163`
`E2E_ADMIN_EMAIL`, `E2E_ADMIN_PASSWORD`	Throwaway admin for the run (password randomly generated if unset).	`e2e-admin@example.com` + random	`:23-24`
`E2E_VIEWER_EMAIL`, `E2E_VIEWER_PASSWORD`	Throwaway viewer.	`e2e-viewer@example.com` + random	`:25-26`
`E2E_DB_PATH`	SQLite file for the test API.	`<tmp>/e2e.sqlite`	`:27`
`E2E_API_URL`, `E2E_WEB_URL`	Derived from the ports above and read by specs.	derived	`:28-29`
`CI`	Enables list+github+html reporters and trace retention.	unset	`:38,44`

The config also sets, for the test backend only, fixed demo-account values (`DEMO_EMAIL`, `DEMO_ADMIN_EMAIL` and their passwords) and a random `JWT_SECRET` per run (`playwright.config.ts:55-71`). These are test fixtures, not production values.

6. Dev-runner ports (`scripts/dev.mjs`)

Command	Starts	Port	Source
`npm run backend`	uvicorn `app.main:create_app --factory --reload`	8000	`scripts/dev.mjs` (backend branch: `requireFreePorts([8000])`)
`npm run frontend`	Vite for BrandShield	5173	`scripts/dev.mjs` (`requireFreePorts([5173, 3000])`)
`npm run frontend`	Vite for Brand Studio	3000 (path `/studio/`)	same

The script exits with a message if a port is busy. Ports are fixed in the script and are not configurable by environment variable. Root `package.json:5-8` defines the two npm scripts.

7. Docker Compose (`docker-compose.yml`, `.env.example`)

Required (compose refuses to start without them, `${VAR:?}`): `POSTGRES_PASSWORD`, `REDIS_PASSWORD`, `JWT_SECRET`, `ADMIN_EMAIL`, `ADMIN_PASSWORD` (`docker-compose.yml:20-23,43,59,79-80`, `.env.example:4-9`).

Variable	Purpose	Default	Source
`POSTGRES_USER` / `POSTGRES_DB`	Database user / name.	`scammind` / `scammind`	`docker-compose.yml:20,42-44`
`POSTGRES_PASSWORD`	Postgres password (URL-encode special characters). **Secret.**	required	`docker-compose.yml:20,43`
`REDIS_PASSWORD`	Redis password, passed via `REDISCLI_AUTH` not the command line. **Secret.**	required	`docker-compose.yml:21,58-60`
`API_PORT`	Host port for the API (bound to 127.0.0.1).	`8000`	`docker-compose.yml:86`
`FRONTEND_PORT`	Host port for the frontend/nginx (bound to 127.0.0.1).	`5173`	`docker-compose.yml:142`
`PUBLIC_API_URL`	See section 3.	`http://localhost:8000`	`docker-compose.yml:136,140`
`DB_PORT`, `REDIS_PORT`, `STUDIO_PORT`	Only in `docker-compose.dev.yml`: publish Postgres, Redis and Studio on localhost.	`5432`, `6379`, `3000`	`docker-compose.dev.yml:4,7,17-20`
Forwarded to containers	`APP_ENV` (default `production`), `LOG_LEVEL`, `CORS_ORIGINS`, `ACCESS_TOKEN_MINUTES`, `REFRESH_TOKEN_DAYS`, `TAVILY_API_KEY`, `TAVILY_MONTHLY_BUDGET`, `SEARXNG_BASE_URL`, `GEMINI_API_KEY`, `SCAM_LLM_ENABLED`, `SENTRY_*`, `SCHEDULER_ENABLED`, `SCAN_INTERVAL_MINUTES`, `ADMIN_NAME`, `RUN_MIGRATIONS`. Fixed by compose: `DATA_DIR=/data`, `SCAN_EXECUTOR=rq`.	see file	`docker-compose.yml:15-35,77-84`

Compose-fixed behaviour: the `api` container runs migrations and the scheduler by default; the `worker` has `SCHEDULER_ENABLED=false` (`docker-compose.yml:82-83,111`). Containers drop all capabilities and set `no-new-privileges` (`docker-compose.yml:100-103,127-130`).

Variables that exist in `backend/.env.example` but are **not** forwarded by Compose (so they only take effect on a non-Docker run unless you edit the compose file): for example `GROQ_API_KEY`, `POLLINATIONS_API_KEY`, `VOICE_*`, `DEMO_*`, `GOOGLE_SAFE_BROWSING_API_KEY`, `CRON_SECRET`, `STORAGE_BACKEND`, `SIGNUP_*` (compare `docker-compose.yml:15-35,77-84` with `backend/.env.example`).

Needs confirmation: the comment at `docker-compose.yml:11` says the `backend/` folder and its old compose file "belong to an older product and are not used here", while the `api` and `worker` images are built from `./backend` (`docker-compose.yml:71`). The comment looks stale; confirm and fix or remove it.

8. Vercel (`vercel.json`)

Item	Value	Source
Services	`backend` (FastAPI, root `backend`, entrypoint `api.index:app`, `maxDuration` 300, test/data/cache folders excluded), `frontend` (Vite, root `frontend`, SPA rewrite), `studio` (Vite, root `studio`, served under `/studio`)	`vercel.json:3-43`
Public routing	`/api/*`, `/health`, `/health/live` go to backend; `/studio` and `/studio/*` to studio; everything else to frontend (first match wins)	`vercel.json:44-81`
Cron	`GET /api/cron/scan` at `0 6 * * *`	`vercel.json:82-87`
Headers	Security headers on every path; immutable one-year cache on `/assets/*` and `/studio/assets/*`; `no-cache` on `/config.js`	`vercel.json:88-145` (details in `docs/auth-and-security.md`)
Python entrypoint	`[tool.vercel] entrypoint = "api.index:app"`; dependencies are read from `requirements.txt` through `pyproject.toml`	`backend/pyproject.toml:57-59`, `backend/pyproject.toml:16-17`

Environment variables for the Vercel project are the backend variables above (`DATABASE_URL` = Supabase transaction pooler on port 6543, `MIGRATION_DATABASE_URL` = port 5432); see `docs/DEPLOY_VERCEL.md` for the full table. The Vercel deployment is documented there as unverified (never deployed), so treat the Vercel column of any default above as the code's intent.

9. Other config files

File	Purpose	Source
`backend/pyproject.toml`	Packaging, pytest (`testpaths=["tests"]` , `pythonpath=["."]`), ruff (line length 140, py311, rules E F I UP B)	`backend/pyproject.toml:19-55`
`backend/alembic.ini` , `backend/alembic/env.py`	Alembic; URL comes from `DatabaseSettings().migration_url`	`backend/alembic/env.py:3,12`
`frontend/.oxlintrc.json`	oxlint plugins react, typescript, oxc; two rules	`frontend/.oxlintrc.json:1-8`
`frontend/tsconfig.app.json`	Strict-ish TS config (target es2023, `noUnusedLocals` , `erasableSyntaxOnly`)	`frontend/tsconfig.app.json:1-30`
`frontend/nginx/default.conf.template` , `studio/nginx/default.conf`	Container web servers with security headers, SPA fallback, `/studio` and `/api` proxy	see `docs/auth-and-security.md`
`.github/workflows/ci.yml`	CI jobs; needs no secrets (throwaway Postgres password inside the job)	`.github/workflows/ci.yml:1-3,90-93`

DOCS/API-REFERENCE.MD

BrandShield API reference

Plain-language summary

BrandShield's backend is a web service that the website (and the Brand Studio tool) talk to. Everything a person can do on screen, such as signing in, registering a brand, running a scan, reviewing a suspected fake account or downloading a report, is one call to an address (an "endpoint") listed on this page. Each endpoint says who may call it (anyone, any signed-in user, only analysts and admins, or only admins), what to send, and what comes back. Mistakes always come back in one predictable shape, and heavy read-only pages are briefly remembered (cached) so the app feels fast. This page is written from the code, not from older docs: where the code was unclear, a line starting `**Needs confirmation:**` says so.

Companion files: `docs/API.md` (older, long narrative reference, may be outdated), `docs/data-model.md` (tables and migrations). All paths below are relative to the repo root; `backend/app/api/X.py:N` means line N of that file.

1. Conventions

1.1 Registration and prefixes

The app is built in `backend/app/main.py:50-184`. All routers are mounted under the prefix `/api` (`main.py:170-181`), except the health router, which is mounted at the root and declares its own `/health...` and `/api/health...` paths (`main.py:182`, `backend/app/api/health.py:22-43`).

Router module	Prefix inside `/api`	Source
`auth`	(none; paths start `/auth`, plus `/me`)	`backend/app/api/auth.py`
`brands`	`/brands`	`backend/app/api/brands.py:48`
`audit`	(none; `/audit`)	`backend/app/api/audit.py`
`cron`	`/cron`	`backend/app/api/cron.py:20`
`sources`	(none; `/sources`, `/settings/data-mode`)	`backend/app/api/sources.py`
`scans`	(none; `/scan`, `/scans`, `/jobs`)	`backend/app/api/scans.py`
`detections`	`/detections`	`backend/app/api/detections.py:16`
`dashboard`	(none; `/dashboard`)	`backend/app/api/dashboard.py`
`lookalike`	`/lookalike`	`backend/app/api/lookalike.py:126`
`settings`	`/settings`	`backend/app/api/settings.py:21`
`reports`	`/reports`	`backend/app/api/reports.py:19`
`evaluation`	`/evaluation`	`backend/app/api/evaluation.py:20`
`candidates` (+ `images`)	`/candidates`, `/candidate-images`	`backend/app/api/candidates.py:23-24`
`providers`	`/providers`	`backend/app/api/providers.py:20`
`studio`	`/studio`	`backend/app/api/studio.py:29`
`studio_ai`	`/studio/ai`	`backend/app/api/studio_ai.py:27`
`voice`	(none; `/voice`)	`backend/app/api/voice.py`
`health`	root (own paths)	`backend/app/api/health.py`

1.2 Authentication and roles

- Sign-in gives a short-lived access token (JWT, `ACCESS_TOKEN_MINUTES`, default 15) and a rotating refresh token (`REFRESH_TOKEN_DAYS`, default 7) (`backend/app/config/config.py:41-43`, `backend/app/config/security.py:26-43`).
- Send the access token as `Authorization: Bearer <access_token>`. A missing header gives `401 Not authenticated`; a bad, expired or wrong-type token, or an inactive user, gives `401 Invalid or expired token`; both carry `WWW-Authenticate: Bearer` (`backend/app/config/deps.py:28-43`).
- Roles are `admin`, `analyst`, `viewer` (`backend/app/models/user.py:17`). The guards are in `backend/app/config/deps.py:46-57`:

Label used on this page	Guard	Who may call	Failure
public	none	anyone	n/a
any signed-in	`get_current_user`	admin, analyst, viewer	401
writer	`require_writer` = `require_roles("admin","analyst")`	analyst and admin (viewers are read-only)	403 `You do not have permission to do this`
admin	`require_admin`	admin only	403
cron secret	`Authorization: Bearer <CRON_SECRET>`	the scheduler only	401 / 503

- The shared **demo administrator** (see `GET /api/auth/demo`) is an admin that is blocked from global changes with `403 Disabled for the shared demo administrator: ...` (`backend/app/services/seed.py:113-116`).

1.3 Brand ownership (multi-tenant rule)

Every brand belongs to one account (`brands.owner_id`). Any endpoint that takes a `brand_id`, or an object belonging to a brand (detection, scan, report, image, test case) answers **404** (never 403) when the object is not the caller's own, so existence does not leak (`backend/app/api/common.py:22-58`). Even admins only see their own brands. When `brand_id` is optional (most endpoints) it defaults to the account's *active* brand: the one chosen with `POST /api/brands/{id}/select`, else the oldest non-archived one (`backend/app/api/common.py:30-50`); with none, `404 No brand profile yet. Create one first.`

1.4 Error format

Defined in `backend/app/config/errors.py:1-32`. Every error body has a `detail`:

Status	Body	Notes
any `HTTPException`	`{"detail": "<message>"}`	`errors.py:17-20`
422 validation	`{"detail": [{"loc": ["body","email"], "msg": "...", "type": "..."}]}`	`Value error,` prefix is removed (`errors.py:12-13, 22-27`)
500	`{"detail": "Internal server error"}`	details are logged, never returned (`errors.py:29-32`)
429 (general limiter)	`{"detail": "Too many requests. Slow down and retry shortly."}` + `Retry-After: 60`	`main.py:132-134`

A few 409 bodies carry extra fields: a second scan returns `job_id`, `scan_id`, `scan` (`backend/app/api/scans.py:41-43`); a paid source retry without `confirm=true` returns `code: "confirmation_required"`, `estimated_credits`, `provider`, `credits_remaining` (`backend/app/api/sources.py:209-214`).

Example (real validation error shape, from `RegisterRequest` password rules, `backend/app/schemas/auth.py:30-42, 66-71`):

```
{"detail": [{"loc": ["body"], "msg": "That password is too easy to guess. Choose a longer or less common one.", "type": "value_error"}]}
```

1.5 Rate limits

Limiter	Default	Scope	Response	Source
General API	`API_RATE_LIMIT_PER_MINUTE` = 600 (0 = off)	every `/api/*` request except `OPTIONS`; counted per client IP **and** per hash of the `Authorization` header	429, `Retry-After: 60`	`config.py:163`, `main.py:119-135`
Login failures	5 failures per 900 s	key `email	ip`; only failed logins count; reset on success	429 `Too many failed attempts. Try again later.`	
Sign-up per IP	5 per hour	IP	429 `Too many sign-up attempts. Try again later.`	`config.py:58`, `auth.py:107-110`
Sign-up per email	3 per hour	email	same 429	`config.py:59`
Studio AI	20 per minute (chat + image together)	per user	429 + `Retry-After: 30`	`config.py:143`, `studio_ai.py:63-68`
Studio AI daily quota	200 text / 50 image per UTC day (0 = no limit)	per user, kept in `api_usage`	429 `You have used today's text AI allowance (200). It resets at midnight UTC.`	`config.py:151-152`, `studio_ai.py:67-68`
Voice	30 per minute	per user	429 + `Retry-After: 20`	`config.py:149`, `voice.py:55-56`

All limiters are in memory and per process (`backend/app/config/ratelimit.py:35-37`): with several workers each counts separately. The rate-limit middleware only exempts paths that do not start with `/api`, so `/health` and `/health/live` are exempt but `/api/health` is **not**, even though the comment at `main.py:121` says "/health* ... exempt" (`main.py:124`).

1.6 Response cache

`backend/app/config/response_cache.py` and `backend/app/main.py:137-158`.

- Only `GET` requests whose exact path is in `CACHEABLE` are cached, and only `200` answers (`response_cache.py:15-18`, `main.py:143-154`):

  `/api/dashboard`, `/api/sources`, `/api/settings`, `/api/detections`, `/api/detections/excluded`, `/api/detections/false-positives`, `/api/brands`, `/api/reports`, `/api/lookalike/test-cases`, `/api/scans`, `/api/scans/latest`, `/api/providers`.

  Paths with ids (for example `/api/brands/5`, `/api/detections/12`) are never cached.

- Key = (path, query string, SHA-256 of the `Authorization` header), so each sign-in has its own entries (`response_cache.py:24-25`).
- TTL `RESPONSE_CACHE_SECONDS`, default 15; `0` disables it (`config.py:123`). On Vercel the default becomes 0 (`config.py:184`).
- A hit returns the stored body with the header `X-Cache: hit` (`main.py:147`).

- Any non-`GET/HEAD/OPTIONS` request clears the whole cache (`main.py:155-158`), and so does a scan starting or finishing (`backend/app/core/pipeline/scan_pipeline.py:140`). Max 400 entries, then it is cleared (`response_cache.py:19, 38-39`).
- Because `cache_reads` is registered after `rate_limit` , it wraps it (Starlette runs the last-added middleware outermost): a cache hit is served **before** the rate limiter and **before** the token is validated, for up to the TTL, but only to a caller that presents byte-identical `Authorization` text to one that already got a 200.

Needs confirmation: whether serving a hit without re-validating an expired token is intended; the tests (`backend/tests/api/test_response_cache.py:43-50`) only cover "no token and wrong token are not served".

1.7 Pagination and filtering conventions

Offset paging with `limit` and `offset` query parameters; lists return `total` (count before paging) and `items` .

Endpoint	`limit` default (range)	`offset`	Extra echo in response
`GET /api/detections`	50 (1-200)	yes	`limit` , `offset` echoed
`GET /api/audit`	50 (1-200)	yes	`total` , `items`
`GET /api/scans`	20 (1-100)	yes	`total` , `items` only
`GET /api/reports`	10 (1-100)	yes	`total` , `items` only
`GET /api/studio/items`	100 (1-500)	yes	`limit` , `offset` echoed

Not paged (whole list returned): `/api/brands` , accounts, apps, `/api/detections/excluded` , `/api/detections/false-positives` , `/api/lookalike/test-cases` , `/api/sources` .

Other conventions: timestamps are ISO-8601 UTC with a trailing `Z` (`backend/app/config/time.py:16-20`); the date filters are spelled `from` / `to` (the Python parameter is `from_` , aliased); `source=live|sample|all` selects the data mode, defaulting to the current effective mode (`backend/app/api/detections.py:52-56`); a request body that is optional can be omitted entirely.

---

2. Auth (`backend/app/api/auth.py`)

Schemas: `backend/app/schemas/auth.py` . `UserOut = {id, name, email, role, active, default_brand_id}` (`auth.py` schema 78-85); `TokenResponse = {access_token, refresh_token, token_type:"bearer", expires_in (seconds), user}` (88-93); `OkResponse = {ok: true, detail: ""}` (96-98).

Method	Path	Auth	Request	Success	Errors	Code
POST	`/api/auth/login`	public	body `LoginRequest {email: EmailStr, password: 1-256 chars, <=72 bytes}`	200 `TokenResponse`	401 `Incorrect email or password` (also inactive user); 429 too many failures; 422	`auth.py:56`
GET	`/api/auth/demo`	**public**	none	200 see below, `Cache-Control: no-store`	none	`auth.py:82`
POST	`/api/auth/register`	public	body `RegisterRequest {name 1-80 chars after cleanup, email (lower-cased, <=254), password}`	201 `TokenResponse` (new user has role `analyst`)	403 `Sign-up is disabled` ; 409 `An account with this email already exists.` ; 429; 422 password rules	`auth.py:99`
POST	`/api/auth/refresh`	public (refresh token in body)	body `RefreshRequest {refresh_token: min 10 chars}`	200 new `TokenResponse` ; old refresh token is revoked (rotation)	401 `Invalid or expired refresh token`	`auth.py:148`
POST	`/api/auth/logout`	public (refresh token in body)	body `RefreshRequest`	200 `OkResponse` ; always 200, even for an already-invalid token	none	`auth.py:159`
GET	`/api/me` and `/api/auth/me`	any signed-in	none	200 `UserOut` (`default_brand_id` = the active brand)	401	`auth.py:173-174`

Details worth knowing:

- Password rules for sign-up: at least 10 characters, at most 72 bytes, not in a built-in list of trivial passwords, not all digits, at least 5 distinct characters, not equal to the name, the email or its local part (`backend/app/schemas/auth.py:27-42`).
- Refresh-token reuse: presenting an already-rotated token revokes **all** of that user's refresh tokens (`auth.py:132-140`).
- Login timing is equalised for unknown emails with a dummy hash (`auth.py:26` , `auth.py:69`).
- Every login attempt (success, failure) and sign-up is written to the audit log (`auth.py:72, 77, 122`).

GET /api/auth/demo (recent addition)

Deliberately public: it feeds the login page's demo buttons (`auth.py:82-96`). Behaviour (`auth.py:83-96` , `backend/app/services/seed.py:27-31, 100-105`):

- If `DEMO_EMAIL` and `DEMO_PASSWORD` are set (and the email differs from `ADMIN_EMAIL`), it returns the shared analyst demo credentials.
- If `DEMO_ADMIN_EMAIL` / `DEMO_ADMIN_PASSWORD` are set **and** that account really exists, is an active admin and the password verifies, it also returns the guarded demo administrator.

- The real admin is never advertised.

Response shapes taken from `backend/tests/api/test_demo_admin.py:30-39`:

```
{"enabled": false, "admin_enabled": false}
```

```
{"enabled": false, "admin_enabled": true, "admin": {"email": "<DEMO_ADMIN_EMAIL>", "password": "<DEMO_ADMIN_PASSWORD>"}}
```

With the analyst demo on, `enabled` is `true` and `email` / `password` are present.

Example (auth group)

```
curl -X POST http://localhost:8000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email": "admin@example.com", "password": "<password>"}'
```

```
{
  "access_token": "<jwt>",
  "refresh_token": "<jwt>",
  "token_type": "bearer",
  "expires_in": 900,
  "user": {"id": 1, "name": "Administrator", "email": "admin@example.com", "role": "admin", "active": true, "default_brand_id": null}
}
```

(`expires_in = ACCESS_TOKEN_MINUTES * 60`, `auth.py:51`; `name` default `ADMIN_NAME` = "Administrator", `config.py:51`.)

```
curl -X POST http://localhost:8000/api/auth/register -H "Content-Type: application/json" \
  -d '{"name": "Una Studio", "email": "una@example.com", "password": "<10+ char password>"}'
# -> 201, same TokenResponse shape, user.role == "analyst"   (backend/tests/api/test_studio_items.py:16)
curl http://localhost:8000/api/me -H "Authorization: Bearer <access_token>"
```

---

3. Brands (`backend/app/api/brands.py`)

Schemas: `backend/app/schemas/brand.py`. `BrandSummary = {id, name, active: bool}` (`schema/brand.py:273-276`). Limit: at most `MAX_BRANDS_PER_ACCOUNT = 10` non-archived brands per account (`brands.py:46`).

Method	Path	Auth	Request	Success	Errors	Code
GET	`/api/brands`	any signed-in	none	200 `list[BrandSummary]` (own, non-archived brands, oldest first; `active` marks the current one)		brands
POST	**`/api/brands/{brand_id}/select`**	any signed-in	path `brand_id`	200 `BrandSummary` with `active: true` ; stores `users.active_brand_id`	404 not yours / archived	brands
POST	`/api/brands`	writer	body `BrandIdentity`	201 `BrandOut` ; the new brand becomes the active brand	409 cap of 10; 422	brands
GET	`/api/brands/{brand_id}`	any signed-in		200 `BrandOut`	404	brands
PUT	`/api/brands/{brand_id}`	writer	body `BrandIdentity`	200 `BrandOut` (+ `refresh` object if identity changed)	404, 422	brands
POST	`/api/brands/{brand_id}/logo`	writer	multipart `file` (<= `LOGO_MAX_BYTES` , 2 MB)	200 `LogoOut`	`LogoError` status (415/413/422 per validator), 404	brands
GET	`/api/brands/{brand_id}/logo`	any signed-in		200 image bytes (`ETag` , `X-Content-Type-Options: nosniff`)	404 `No logo uploaded` / `Logo file is missing`	brands
GET	`/api/brands/{brand_id}/accounts`	any signed-in		200 `AccountList {total, items}`	404	brands
POST	`/api/brands/{brand_id}/accounts`	writer	body `AccountCreate {platform, handle}`	201 `AccountOut` (+ `warning` if the URL check failed)	409 duplicate; 422	brands
DELETE	`/api/brands/{brand_id}/accounts/{account_id}`	writer		200 `OkResponse`	404	brands
GET	`/api/brands/{brand_id}/apps`	any signed-in		200 `AppList {total, items}`	404	brands
POST	`/api/brands/{brand_id}/apps`	writer	body `AppCreate`	201 `AppOut` ; the same package on another store merges stores instead of 409	409 `... is already an official app` ; 422	brands
PATCH	`/api/brands/{brand_id}/apps/{app_id}`	writer	body `AppUpdate {apple_app_id}`	200 `AppOut`	404, 422	brands
DELETE	`/api/brands/{brand_id}/apps/{app_id}`	writer		200 `OkResponse`	404	brands
DELETE	`/api/brands/{brand_id}`	**admin**	query `confirm` = exact brand name	200 `OkResponse` ; soft delete (`archived_at`)	409 `Type the exact brand name in \` confirm`` to archive it`	brands
POST	`/api/brands/{brand_id}/restore`	**admin**		200 `BrandOut`	409 cap of 10	brands
POST	`/api/brands/{brand_id}/logos`	writer	multipart `file` , query `label`	201 `ExtraLogoOut`	409 `This logo is already registered` ; 422 `At most 10 extra logos`	brands
GET	`/api/brands/{brand_id}/logos/{logo_id}/file`	any signed-in		200 image bytes	404 `Logo not found`	brands
DELETE	`/api/brands/{brand_id}/logos/{logo_id}`	writer		200 `OkResponse`	404	brands
POST	`/api/brands/{brand_id}/publishers`	writer	body `PublisherCreate {store: google_play|app_store|any, name<=120, publisher_id<=120}` (needs name or id)	201 `PublisherOut`	409 duplicate; 422	brands
DELETE	`/api/brands/{brand_id}/publishers/{publisher_id}`	writer		200 `OkResponse` (also drops its verification)	404	brands
PUT	`/api/brands/{brand_id}/verification`	writer	body `VerificationIn {asset_type: domain|account|app|publisher|logo,`	200 `VerificationOut`	404 `No such <type> is`	brands

Method	Path	Auth	Request	Success	Errors	Code
			`asset_key, status: unverified|verified, method, note<=300}`		`registered for this brand` ; 422 (verified needs a `method` from `dns_txt, website_link, store_listing, platform_badge, manual_review, other`)	

`BrandIdentity` request body (`schema/brand.py:7-38`): `name` (1-120, required), `industry` (<=120), `aliases` , `keywords` , `domains` (each list <= 200 items; domains are normalised and deduplicated, max 50 after cleanup), `description` (<=2000, optional). `PUT` leaves `description` untouched when the field is omitted (`brands.py:176-177`).

`BrandOut` fields (`schema/brand.py:250-270`): `id, name, description, archived, extra_logos[], publishers[], verifications[], industry, aliases, keywords, domains, logo (LogoOut|null), official_accounts[], official_apps[], profile_health {percent, message, checks[], missing[]}, allowlist {accounts, apps, domains}, official_assets, updated_at, source, refresh` .

Identity changes (`name` , `aliases` , `keywords` , `domains`) retire earlier detections and rebuild the Look-alike Lab's built-in cases; in sample mode a new scan is started automatically (`brands.py:162, 181-205`). The response then carries `refresh: {"results_retired": <n>, "rescan": "needed"|"started", "message": "..."}` .

Account platforms and app stores are validated against `ACCOUNT_PLATFORMS` / `APP_STORES` in `backend/app/services/validators.py` (error text lists the allowed values, `schema/brand.py:45-51, 75-85`).

Example (brands group)

```
curl -X POST http://localhost:8000/api/brands \
  -H "Authorization: Bearer <access_token>" -H "Content-Type: application/json" \
  -d '{"name": "Acme Bank", "aliases": [], "keywords": [], "domains": []}'   # body from backend/tests/api/test_response_cache.py:46
# -> 201 BrandOut (id, name "Acme Bank", profile_health, allowlist, ...)
curl http://localhost:8000/api/brands -H "Authorization: Bearer <access_token>"
```

```
[{"id": 1, "name": "Acme Bank", "active": true}]
```

```
curl -X POST http://localhost:8000/api/brands/2/select -H "Authorization: Bearer <access_token>"
```

```
{"id": 2, "name": "Second Brand", "active": true}
```

After `select` , calls that omit `brand_id` (dashboard, detections, scans, reports, ...) work on brand 2, and `GET /api/me` reports `default_brand_id: 2` (`backend/app/api/common.py:30-40` , `auth.py:29-36`).

---

4. Candidates (`backend/app/api/candidates.py`)

Manual entry, CSV/JSON import and image upload. Writers only, except fetching an image. No URL is ever opened (`candidates.py:1`). Schemas live in `backend/app/core/pipeline/candidate_intake.py` : `SocialIn` (`:53`), `AppIn` (`:77`), `WebIn` (`:98`).

Method	Path	Auth	Request	Success	Errors	Code
POST	`/api/candidates/social`	writer	query `brand_id?` ; body `SocialIn {platform, handle, display_name?, url?, bio?, followers?, following?, posts?, created_at?, verified?, image_id?}`	201 per-row result dict	422 row error message	`candidates.py:36`
POST	`/api/candidates/app`	writer	body `AppIn {store, package_id, name, publisher?, publisher_id?, url?, description?, installs?, rating_count?, image_id?}`	201	422	`candidates.py:41`
POST	`/api/candidates/web`	writer	body `WebIn {url, title?, note?}`	201	422	`candidates.py:46`
POST	`/api/candidates/import`	writer	query `kind=social|app|web` , `format=csv|json` , `dry_run` (default false), `brand_id?` ; multipart `file`	200 partial-success report (valid rows saved, bad rows reported)	413 file > `IMPORT_MAX_BYTES` (1 MB); parse errors; rows capped at `IMPORT_MAX_ROWS` (500)	`candidates.py:51`
POST	`/api/candidate-images`	writer	query `purpose=candidate_image|app_icon` , `brand_id?` ; multipart `file` (<= 2 MB)	201 `{id, format, width, height, size_bytes, sha256, purpose, url, note}` (de-duplicated by hash)	`LogoError` status	`candidates.py:75`
GET	`/api/candidate-images/{image_id}/file`	any signed-in		200 image bytes	404 `Image not found`	`candidates.py:109`

The return of the per-row endpoints is `ci.run_rows(...)["results"][0]` (`candidates.py:27-33`); see `candidate_intake.py` for its keys. **Needs confirmation:** the exact per-row result keys (`status` is `error` on failure per `candidates.py:31` ; other values are defined in `run_rows` , not re-verified here).

```
curl -X POST "http://localhost:8000/api/candidates/import?kind=social&format=csv&dry_run=true" \
  -H "Authorization: Bearer <access_token>" -F "file=@candidates.csv"
```

5. Dashboard (`backend/app/api/dashboard.py`)

Method	Path	Auth	Request	Success	Code
GET	`/api/dashboard`	any signed-in	query `brand_id?`	200 object (cached)	`dashboard.py:19`

Response keys: from `dashboard.build` (`backend/app/services/dashboard.py:73-110`): `brand {id,name}` , `data_mode` , `is_sample` , `includes_sample` , `kpis {total_detections, social_impersonations, suspicious_apps, web_threats, official_assets}` (each `{value, delta_week, trend}` , `official_assets` only `{value}`), `counts {by_status, by_band, by_severity, by_type, by_category, cleared}` , `by_type[]` , `by_platform[]` , `total_open` , `needs_review[]` , `recent[]` ; the router adds `sources[]` , `sources_online` , `sources_total` , `last_scan_at` , `last_scan` , `next_scan_at` , `schedule` , `credits` (only in live mode) (`dashboard.py:33-48`).

```
curl http://localhost:8000/api/dashboard -H "Authorization: Bearer <access_token>"
```

6. Detections (`backend/app/api/detections.py`)

Statuses a person can set: `reviewing` , `confirmed` , `dismissed` (aliases `keep_reviewing` , `confirm` , `dismiss` : `backend/app/services/detections.py:34-35, 308-312`). `new` , `cleared` and `marked_official` are set by the system / mark-official. Resolutions: `true_positive` , `false_positive` , `resolved` , `monitoring` .

Method	Path	Auth	Request	Success	Errors	Code
GET	`/api/detections`	any signed-in	query below	200 `{total, limit, offset, data_mode, is_sample, includes_sample, counts, items[]}` (cached)	422 bad status/type	`detections.py:59`
GET	`/api/detections/excluded`	any signed-in	`category?`, `brand_id?`	200 `{total, data_mode, is_sample, items[]}` (items skipped because they matched the allowlist) (cached)		`detections.py:100`
GET	`/api/detections/false-positives`	any signed-in	`category?`, `source?`, `brand_id?`	200 `{total, data_mode, is_sample, items[]}` (cached)		`detections.py:114`
GET	`/api/detections/{id}`	any signed-in		200 detail (+ `data_mode`, `is_sample`)	404 `Detection not found`	`detections.py:129`
GET	`/api/detections/{id}/evidence`	any signed-in		200 `{page_url, captured_at, sha256, hash_verified, snapshot, screenshot_available, screenshot_sha256, history}`	404	`detections.py:139`; `services/detections.py:286-293`
GET	`/api/detections/{id}/evidence/screenshot`	any signed-in		200 `image/png`	404 `No screenshot was captured (screenshots are off by default)`	`detections.py:146`
POST	`/api/detections/{id}/status`	writer	body `StatusBody {status, note<=1000}`	200 detail + `audit_id`	422, 409 (marked official)	`detections.py:163`
POST	`/api/detections/{id}/resolution`	writer	body `ResolutionBody {resolution, note<=1000}`	200 detail + `audit_id`	422	`detections.py:170`
POST	`/api/detections/{id}/confirm`	writer	optional body `NoteBody {note}`	200 as `status`		`detections.py:186`
POST	`/api/detections/{id}/keep-reviewing`	writer	optional `NoteBody`	200		`detections.py:187`
POST	`/api/detections/{id}/dismiss`	writer	optional `NoteBody`	200		`detections.py:188`
POST	`/api/detections/{id}/mark-official`	writer	optional body `MarkOfficialBody {note<=1000, publisher<=120}`	200 `{detection, allowlist_entry, official_assets, audit_id}`	409	`detections.py:191`

`GET /api/detections` query parameters (`detections.py:60-77`): `category` (social|apps|web), `type`, `platform`, `band` (high|medium|low), `severity` (LOW|MEDIUM|HIGH|CRITICAL), `resolution`, `status` (repeatable), `source` (live|sample|all), `q` (<=200), `min_score` (0-100), `sort` (`score_desc` default, `score_asc`, `found_desc`, `found_asc`, `last_seen_desc`), `from`, `to`, `limit`, `offset`, `brand_id`.

List item (`summary_row`, `services/detections.py:178-202`): `id, category, type, platform, name, handle, package_id, publisher, profile_url, score, risk_band, severity, status, resolution, summary, tags, found_at, last_seen_at, stale, source, origin`.

Note: the shortcut routes `confirm`, `keep-reviewing`, `dismiss` are registered with `add_api_route` (`detections.py:177-188`).

Example (detections group)

```
curl "http://localhost:8000/api/detections?category=social&band=high&status=new&limit=20&sort=score_desc" \
  -H "Authorization: Bearer <access_token>"
```

```
{"total": 1, "limit": 20, "offset": 0, "data_mode": "sample", "is_sample": true, "includes_sample": true, "counts": {},
 "items": [{"id": 12, "category": "social", "type": "fake_profile", "platform": "instagram", "name": "...", "score": 88,
            "risk_band": "high", "severity": "CRITICAL", "status": "new", "resolution": null, "origin": "synthetic", "source": "sample"}]}
```

(`counts` content comes from `svc.list_detections`, not re-verified; the values above are illustrative of the real keys.)

```
curl -X POST http://localhost:8000/api/detections/12/status \
  -H "Authorization: Bearer <access_token>" -H "Content-Type: application/json" \
  -d '{"status": "confirmed", "note": "Uses our logo and asks for card numbers"}'
```

7. Scans, jobs, cron (`backend/app/api/scans.py`, `cron.py`)

Method	Path	Auth	Request	Success	Errors	Code
POST	`/api/scan`	writer	optional body `ScanBody {scope: all|social|apps (default all), brand_id?}`	**202** `{job_id, scan_id, status, scan}`	409 `Create the brand profile before scanning.`; 409 `A scan is already running` + `job_id`, `scan_id`, `scan`; 404 foreign brand	`scans.py:25`
GET	`/api/jobs/{job_id}`	any signed-in		200 `scan_view`	404 `Job not found`	`scans.py:48`
GET	`/api/scan/status`	any signed-in	`brand_id?`	200 `{last_scan_at, last_scan_status, last_full_sync_at, next_scan_at, schedule, running_job_id, data_mode, is_sample, scheduler}`		`scans.py:56`
GET	`/api/scans`	any signed-in	`brand_id?`, `limit` (20, 1-100), `offset`	200 `{total, items[]}` (cached)		`scans.py:83`
GET	`/api/scans/latest`	any signed-in	`brand_id?`	200 `{scan: scan_view|null}` (cached)		`scans.py:98`
GET	`/api/scans/{scan_id}`	any signed-in		200 `scan_view`	404 `Scan not found`	`scans.py:105`
GET	`/api/cron/scan`	**cron secret**	header `Authorization: Bearer <CRON_SECRET>`	200 `{ran, skipped, reason, scan_id}`	503 `Scheduled scans via cron are not configured (set CRON_SECRET).`; 401 `Invalid cron credentials`	`cron.py:32`

`scan_view` (`backend/app/core/pipeline/scan_pipeline.py:926-954`): `id, job_id, kind:"scan", brand_id, scope, status` (`queued|running|completed|partial|failed`), `state` (older vocabulary `queued|running|done|error`), `progress, message, steps, result, counts, source_report, timings, errors, started_by, data_mode, is_sample, created_at, started_at, finished_at`.

Only one queued/running scan per brand is allowed, enforced by the DB (`uq_scans_one_active_per_brand`, see `data-model.md`). The cron endpoint reuses the scheduler decision and runs one inline scan with `started_by = "scheduler"` (`cron.py:1-6, 32-46`), looking `CRON_SLACK` = 60 minutes ahead (`cron.py:24`).

```
curl -X POST http://localhost:8000/api/scan -H "Authorization: Bearer <access_token>" \
  -H "Content-Type: application/json" -d '{"scope": "all"}'      # payload from backend/tests/api/test_scan_api.py:36
```

```
{"job_id": "<job id>", "scan_id": 3, "status": "queued", "scan": {"id": 3, "kind": "scan", "scope": "all", "status": "queued", "state": "queued", "progress
```

(Status can already be `completed` when the executor is `inline`, `scans.py:44`.)

8. Sources and data mode (`backend/app/api/sources.py`)

Method	Path	Auth	Request	Success	Errors	Code
GET	`/api/sources`	any signed-in	`kind=social|store`	200 `{data_mode, requested_data_mode, summary, items[], tavily_credits, reputation}` (cached)		`sources.py:125`
PATCH	`/api/sources/{source_id}`	writer	body `MonitoredBody {monitored: bool}`	200 source item	404 `Source not found`	`sources.py:160`
POST	`/api/sources/{source_id}/retry`	writer	query `confirm` (bool, default false)	**202** `{job_id, source_id, state:"done", status, source, reports, estimated_credits}`	404; 409 not monitored / already online / no brand / no collector; 409 `confirmation_required` when the retry can spend paid credits	`sources.py:179`
GET	`/api/settings/data-mode`	any signed-in		200 selection (`sel.as_dict()`)		`sources.py:233`
PUT	`/api/settings/data-mode`	**admin**	body `DataModeBody {data_mode: live|sample|auto}`	200 selection	403 for the demo admin unless `sample`	`sources.py:239`

Source item keys: `id, slug, name, kind, description, status (online|slow|unavailable), monitored, message, last_sync_at, detail, retry_in_minutes, next_retry_at, stale_since, cached_at, cached_age_seconds, collectors, source, retry_estimate` (`sources.py:103-122`).

9. Settings (`backend/app/api/settings.py`)

Method	Path	Auth	Request	Success	Errors	Code
GET	`/api/settings`	any signed-in		200 view (cached)		`settings.py:125`
PUT	`/api/settings`	**admin**	body `SettingsIn {data_mode?: live|sample|auto, monitoring?, notifications?}`	200 view	403 (demo admin limits), 422	`settings.py:130`
POST	`/api/settings/sensitivity-preview`	any signed-in	body `{sensitivity: int}` within `SENSITIVITY_MIN..MAX`	200 `{sensitivity, threshold, would_review, total, data_mode, is_sample, message}`	422	`settings.py:194`

- `monitoring` : `schedule` (`off|5m|15m|1h|1d`), `sources` (map platform -> bool; unknown platform is a 422), `sensitivity` , `rules` (map of known rule names -> bool), `impersonation_words` (lower-cased, each <= 30 chars, <= 50 words) (`settings.py:24-67`).
- `notifications` : `email_alerts` , `high_risk_immediate` , `weekly_digest` , `webhook_url` (must be `https://...` or empty, <= 500) (`settings.py:70-83`). Stored only: the view says `"delivery": "not_implemented"` (`settings.py:117-119`).
- The webhook URL is shown only to a non-demo admin (`settings.py:118`).
- View keys: `data_mode, effective_data_mode, monitoring{...,threshold}, scheduler, notifications, account{organization,user}, updated_at` (`settings.py:105-121`).

```
curl -X PUT http://localhost:8000/api/settings -H "Authorization: Bearer <admin access token>" -H "Content-Type: application/json" \
  -d '{"monitoring": {"schedule": "1d", "sensitivity": 70}}'
```

10. Look-alike Lab (`backend/app/api/lookalike.py`)

Method	Path	Auth	Request	Success	Errors	Code
POST	`/api/lookalike/test`	any signed-in	body `TestBody {name 1-200, brand_id?, platform?}`	200 matcher verdict (pure, no side effects)	422	`lookalike.py:70`
POST	`/api/lookalike/test-url`	any signed-in	body `UrlBody {url 1-2048, brand_id?}`	200 (never opens the URL)	422	`lookalike.py:77`
POST	`/api/lookalike/test-logo`	any signed-in	multipart `file` , optional `reference` ; query `brand_id?`	200 comparison; nothing stored	422 `NO_LOGO` when no reference exists; `LogoError`	`lookalike.py:83`
GET	`/api/lookalike/logo-engine`	any signed-in		200 CLIP availability		`lookalike.py:113`
GET	`/api/lookalike/test-urls`	any signed-in	`brand_id?`	200 generated example domains		`lookalike.py:119`
GET	`/api/lookalike/test-cases`	any signed-in	`brand_id?`	200 `{total, items[{id,name,expected,builtin,source}]}` (cached)		`lookalike.py:125`
POST	`/api/lookalike/test-cases`	**admin**	body `CaseBody {name, expected: flagged|not_flagged|official}` , `brand_id?`	201 case	409 `This test case already exists`	`lookalike.py:133`
DELETE	`/api/lookalike/test-cases/{case_id}`	**admin**		200 `{ok: true}`	404	`lookalike.py:147`
POST	`/api/lookalike/test-cases/run`	writer	`brand_id?`	200 run result (`summary` , `items` , ...)		`lookalike.py:158`

```
curl -X POST http://localhost:8000/api/lookalike/test -H "Authorization: Bearer <access_token>" \
  -H "Content-Type: application/json" -d '{"name": "Acme Bank Support"}'
```

Needs confirmation: the exact keys of the verdict returned by `lookalike.evaluate_name` (`backend/app/core/pipeline/lookalike.py`) were not enumerated here.

11. Reports (`backend/app/api/reports.py`)

Reports are drafts created from detections; approval only records a person's sign-off and sends nothing (`reports.py:110-111, 195-196`). HTML responses carry `Content-Security-Policy: default-src 'none'; style-src 'unsafe-inline'` (`reports.py:20`).

Method	Path	Auth	Request	Success	Errors	Code
GET	`/api/reports/preview`	any signed-in	query `from, to, types (comma list), status, ids (comma list), include_evidence, brand_id`	200 `{title, brand, from, to, data_mode, is_sample, includes_sample, stats, include_evidence, rows[]}`	422 bad ids	`reports.py:81`
GET	`/api/reports/preview.html`	any signed-in	same	200 HTML		`reports.py:96`
POST	`/api/reports`	writer	body `ReportIn`	201 report object	422 `format: pdf` not available; 422 `No detections match...`	`reports.py:110`
GET	`/api/reports`	any signed-in	`limit` (10, 1-100), `offset` , `brand_id?`	200 `{total, items[]}` (cached)		`reports.py:138`
GET	`/api/reports/{id}`	any signed-in		200 report + `rows[]`	404 `Report not found`	`reports.py:157`
GET	`/api/reports/{id}/preview`	any signed-in		200 HTML	404	`reports.py:165`
GET	`/api/reports/{id}/export.csv`	any signed-in		200 CSV attachment	404	`reports.py:175`
GET	`/api/reports/{id}/export.html`	any signed-in		200 HTML attachment	404	`reports.py:181`
GET	`/api/reports/{id}/download`	any signed-in		200 file in the report's own format	404	`reports.py:187`
POST	`/api/reports/{id}/approve`	writer		200 report object	409 `This report is already approved`	`reports.py:195`

`ReportIn` (`reports.py:23-34`): `detection_ids` (<= 1000), `from` / `to` (dates), `types[]` , `status` , `include_evidence` (default true), `format` (`csv|html|pdf` , default `html` ; `pdf` is refused with 422), `title` (<= 200), `brand_id` . Report object (`reports.py:52-60`): `id, title, filename, format, status, size_bytes, created_by, created_at, approved_by, approved_at, from, to, count, detection_ids, stats, include_evidence, includes_sample, source, download_url, preview_url, csv_url, html_url` .

```
curl -X POST http://localhost:8000/api/reports -H "Authorization: Bearer <access_token>" -H "Content-Type: application/json" \
  -d '{"detection_ids": [12, 14], "format": "csv", "title": "Brand Protection Report"}'
```

12. Evaluation (`backend/app/api/evaluation.py`)

Method	Path	Auth	Request	Success	Code
GET	`/api/evaluation`	any signed-in	`brand_id?`	200 object	`evaluation.py:94`
POST	`/api/evaluation/run`	writer	`brand_id?`	200 same object, synthetic cache refreshed; audited	`evaluation.py:101`

Keys (`evaluation.py:81-91`): `by_category[]` , `data_mode` , `is_sample` , `label` , `synthetic {seed, per_technique, brands, computed_at, by_technique[], overall, confusion, dataset, algorithm_version, ml_classifier, false_positives, cached}` , `lookalike_tests` , `reviewed {confirmed, dismissed, marked_official, pending}` , `detection {true_positives, false_positives, precision, definition}` , `by_type[]` . Precision = confirmed / (confirmed + dismissed); recall is not measurable (`evaluation.py:87-89`).

13. Providers (`backend/app/api/providers.py`)

Method	Path	Auth	Request	Success	Errors
GET	`/api/providers`	any signed-in		200 `{active_text_provider, hosted_text_analysis_enabled, text_providers[{name,label,configured,model,needs_key,reason,active}], embeddings, image_similarity_backend, note}` (cached)	
PUT	`/api/providers/active`	**admin**	body `{provider: off|local|gemini|huggingface}`	200 same view	403 demo admin

Keys are read from the environment and never returned (`providers.py:41`).

14. Audit (`backend/app/api/audit.py`)

Method	Path	Auth	Request	Success	Code
GET	`/api/audit`	writer	`from` , `to` (datetimes), `actor` , `action` , `limit` (50, 1-200), `offset`	200 `{total, items[{id, ts, actor, actor_id, action, target, detail, source}]}` , newest first	`audit.py:52`
GET	`/api/audit/export.csv`	writer	none	200 CSV attachment `audit-log.csv` , columns `ts, actor, action, target, detail`	`audit.py:75`

Admins see every row; analysts see only their own actions; the shared demo admin is treated like an analyst (`audit.py:36-49`). Viewers get 403. CSV cells starting with `= + - @ TAB CR` are prefixed with `'` (`audit.py:70-72`).

15. Health (`backend/app/api/health.py`)

Method	Path	Auth	Success	Notes	Code
GET	`/health` and `/api/health`	public	200 `{status, version, database:{status}, components:{...}}`	**503** only when the database is down	`health.py:22-27`
GET	`/health/live`	public	200 `{status:"ok", version}`	no dependency touched	`health.py:30-33`
GET	`/api/health/details`	**admin**	200/503 `{status, version, environment, checks}`	collector reasons, migration revision, worker count, free disk	`health.py:36-40`
POST	`/api/health/canary`	**admin**	200 canary result with `passed` , `failures`	in-memory fake brand; audited	`health.py:43-50`

Components are `database, migration, collectors, redis, worker, data_dir` (`backend/app/services/health.py:202-216`); overall status is `down` if the database is down, else `degraded` if any component is bad (`:220-223`).

```
curl http://localhost:8000/health
```

```
{"status": "ok", "version": "<VERSION>", "database": {"status": "ok"}, "components": {"database": "ok", "migration": "ok", "collectors": "ok", "redis": "..
```

Needs confirmation: the literal status words for non-database components (`ok` / `degraded` / `down` / `unknown` /other) were not all enumerated from `backend/app/services/health.py` .

16. Studio items (`backend/app/api/studio.py`)

Per-user saved items for Brand Studio (replaces Firebase collections). Owner-only: another user's item is a plain 404 (`studio.py:165-169`). Types: `logo, palette, brand_name, content, audit, sentiment` (`backend/app/models/studio.py:9`). Limits: label <= 200 chars, `data` <= 64 KB JSON, image <= 2 MB, <= 500 items per user (`studio.py:31-35`).

Method	Path	Auth	Request	Success	Errors	Code
GET	`/api/studio/items`	any signed-in	`type?` , `limit` (100, 1-500), `offset`	200 `{total, limit, offset, items[]}` , newest first	422 bad `type`	`studio.py:103`
POST	`/api/studio/items`	any signed-in	JSON `{type, label, data?, image_base64?}` or multipart (`type` , `label` , `data` as JSON text, `file`)	201 item	413 too large; 415 other content type; 409 item cap; 422	`studio.py:121`
DELETE	`/api/studio/items/{item_id}`	any signed-in		200 `OkResponse` `{ok:true, detail:""}`	404 `Item not found`	`studio.py:172`
GET	`/api/studio/items/{item_id}/image`	any signed-in		200 image, `Cache-Control: private, max-age=300`	404 `This item has no image`	`studio.py:183`

Item shape (`studio.py:38-48` , asserted in `backend/tests/api/test_studio_items.py:55`): `{id: "<string>", userId: "<string>", type, label, data, url (/api/studio/items/<id>/image or null), timestamp (ms), created_at}` .

```
curl -X POST http://localhost:8000/api/studio/items -H "Authorization: Bearer <access_token>" -H "Content-Type: application/json" \
  -d '{"type": "palette", "label": "Ocean palette", "data": {"paletteName": "Ocean", "mood": "calm", "colors": [{"name": "Deep", "hex": "#123456", "role":
```

```
{"id": "1", "userId": "2", "type": "palette", "label": "Ocean palette", "data": {"paletteName": "Ocean"}, "url": null, "timestamp": 1760000000000, "created
```

(data abbreviated; payload from `backend/tests/api/test_studio_items.py:12`.)

17. Studio AI (`backend/app/api/studio_ai.py`)

A server-side proxy so provider keys never reach the browser. Signed-in users only (any role). Limits come from `backend/app/config/config.py:138-160`.

Method	Path	Auth	Request	Success	Errors	Code
GET	`/api/studio/ai/status`	any signed-in		200 `{chat{configured, providers[], used_today, daily_limit}, image{configured, provider:"pollinations", used_today, daily_limit, sizes[]}, limits{...}}`		`studio_ai.py:75`
POST	`/api/studio/ai/chat`	any signed-in	body `ChatBody {messages[1..200] of {role: system|user|assistant|model, content or text}, response_format: json|text (default text), temperature 0-2 (0.7), max_tokens?}`	200 `{text, provider, model}`	422 too many messages (>40) / `max_tokens` > 4096; 413 total chars > 30000; 503 `AI chat is not set up on the server.`; 429	`studio_ai.py:88`
POST	`/api/studio/ai/image`	any signed-in	body `{prompt 1-20000 (<= 2000 after cleanup), size}`, query `format=json|png`	200 `{image_base64, data_url, media_type, width, height, provider, model}`; `format=png` returns the raw PNG	422 bad size; 413 prompt too long; 503 not set up; 429	`studio_ai.py:109`

Allowed sizes: `512x512, 768x768, 1024x1024, 1024x768, 768x1024, 1280x720, 720x1280` (`studio_ai.py:30`). A failed provider call refunds the daily quota (`studio_ai.py:105, 128`).

```
curl -X POST http://localhost:8000/api/studio/ai/chat -H "Authorization: Bearer <access_token>" -H "Content-Type: application/json" \
  -d '{"messages": [{"role": "user", "content": "Suggest three brand names for a coffee roaster"}], "response_format": "text"}'
```

```
{"text": "...", "provider": "groq", "model": "openai/gpt-oss-120b"}
```

(`provider` is `groq` or `gemini`, `studio_ai.py:79`; default Groq model `STUDIO_GROQ_MODEL`, `config.py:141`.)

18. Voice (`backend/app/api/voice.py`) - recent addition

`POST /api/voice`, any signed-in user, per-user limit 30 per minute (`voice.py:54-56`). The browser sends a compact snapshot of the visible page controls and text; the server (Gemini key stays server-side) answers with UI actions the browser performs, then calls again with a fresh snapshot until `finish` or `ask` (`voice.py:1-8`). Fields whose label or name look sensitive (passwords etc., `SENSITIVE` regex in `backend/app/services/voice.py`) are removed server-side before planning (`voice.py:59-60`). Simple commands such as "scroll down" or "open dashboard" work with no AI key (`config.py:144`, `backend/tests/api/test_voice.py:22-26`).

Request body `VoiceBody` (`voice.py:41-50`):

Field	Type	Limit
`text`	string	<= 1000, required
`route`	string	<= 200, default `/`
`title`	string	<= 120
`routes`	list of `{path (<=100), spoken_names[] (<=10)}`	<= 50
`snapshot`	list of `Elem {i: "v<digits>", t, l, n, v, r, d, e, o[], c, off}`	<= 200 elements
`done_steps`	list of objects	<= 40
`pending`	object or null	the question asked last turn
`last_turns`	list of `{role: text}` maps	<= 6
`last_fill`	object or null	

Response (`backend/app/services/voice.py:75-77`): `{actions[], reply, ask, finish, cancel_pending, source}`; `source` defaults to `"fast"`. Errors: 401, 422, 429 (`Too many voice commands. Wait a moment and try again.`, `Retry-After: 20`).

Real example from `backend/tests/api/test_voice.py:11-17, 23-26`:

```
curl -X POST http://localhost:8000/api/voice -H "Authorization: Bearer <access_token>" -H "Content-Type: application/json" \
  -d '{"text": "open dashboard", "route": "/setup", "title": "Setup",
       "routes": [{"path": "/dashboard", "spoken_names": ["dashboard"]}, {"path": "/reports", "spoken_names": ["reports"]}],
       "snapshot": [{"i": "v1", "t": "input", "l": "Company name", "n": "brand_name"}], "pending": null}'
```

```
{"actions": [{"type": "navigate", "path": "/dashboard"}], "reply": "...", "ask": null, "finish": true, "cancel_pending": false, "source": "fast"}
```

(`actions` from the test assertion at `test_voice.py:24` ; `reply` , `finish` values shown are indicative.)

Needs confirmation: exact `reply` / `finish` values for the fast "navigate" path.

---

19. Endpoint index (quick lookup)

Total: 101 method+path registrations, counted from the route decorators in `backend/app/api/*.py` (98 `@router.<verb>` / `@images.<verb>` decorators plus the 3 `add_api_route` shortcuts at `detections.py:186-188`). `/me` + `/auth/me` and `/health` + `/api/health` are each one function registered under two paths, so they count twice.

Auth levels at a glance:

- public: `/health` , `/health/live` , `/api/health` , `/api/auth/login` , `/api/auth/register` , `/api/auth/refresh` , `/api/auth/logout` , `/api/auth/demo` , `/api/cron/scan` (needs secret)
- admin only: `PUT /api/settings` , `PUT /api/settings/data-mode` , `PUT /api/providers/active` , `DELETE /api/brands/{id}` , `POST /api/brands/{id}/restore` , `POST|DELETE /api/lookalike/test-cases...` , `GET /api/health/details` , `POST /api/health/canary`
- writer (analyst + admin): all brand edits, scans, detection decisions, candidates, reports create/approve, evaluation run, audit read, source monitor/retry, lookalike cases run
- any signed-in: all other reads, `POST /api/brands/{id}/select` , lookalike testers, sensitivity preview, studio items, studio AI, voice

DOCS/DATA-MODEL.MD

BrandShield data model

Plain-language summary

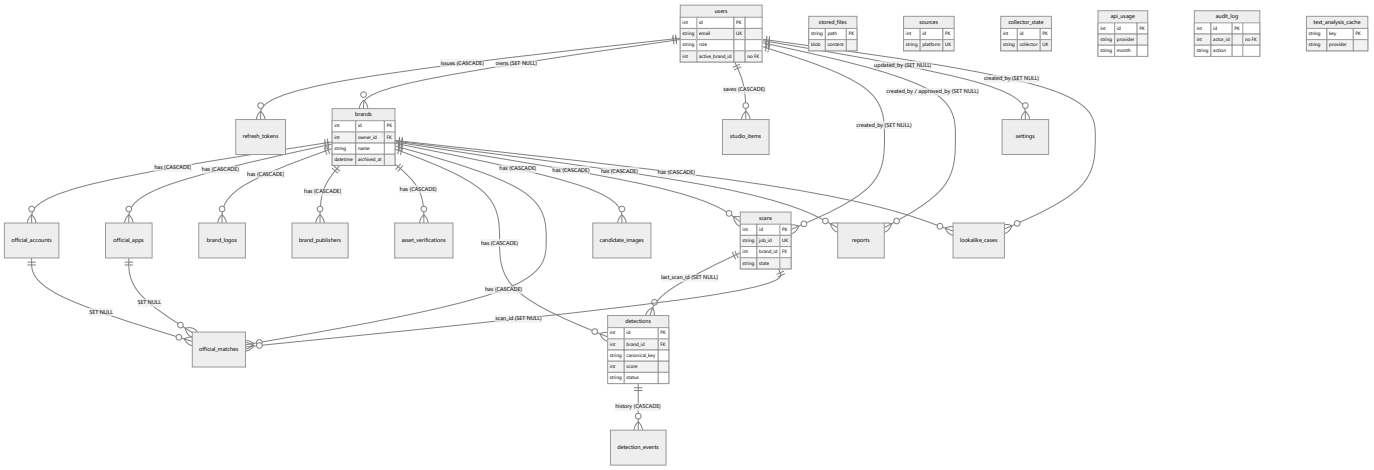
BrandShield keeps everything it knows in a database of 23 tables. Think of them as spreadsheets: one lists the people who can sign in, one lists the brands each person protects (with their official accounts, apps and logos), one lists every suspected fake that scans found, and others record who did what (the audit log), scan history, saved reports and Brand Studio items. The structure is changed over time by numbered "migrations" (0001 to 0012) that are applied with one command. The app works on a simple file database (SQLite) on a laptop and on PostgreSQL (for example Supabase) in production; uploaded images live either in a folder on disk or inside the database itself. This page is written from the code (`backend/app/models/*.py` , `backend/alembic/versions/*.py`).

Companion files: `docs/api-reference.md` (endpoints), `docs/DATABASE_SCHEMA.md` and `docs/schema.sql` (auto-generated, older). Citations are `path:line` . "Python default" means the ORM fills the value when a row is created through the app; "server default" means the database column itself has a default (only those added with `server_default` in a migration or model).

1. Overview

- ORM: SQLAlchemy 2 declarative models, all registered through `backend/app/models/__init__.py:1-31` .
- Constraint and index names follow one naming convention (`ix_` , `uq_` , `fk_` , `pk_`) defined at `backend/app/db/base.py:5-11` .
- 23 tables, matching `EXPECTED_TABLES` in `backend/tests/db/test_migration.py:14-18` , plus Alembic's own `alembic_version` .
- Timestamps are `DateTime(timezone=True)` ; the Python default is `utcnow()` (`backend/app/config/time.py:7-8`).
- JSON columns use SQLAlchemy `JSON` (JSON on both databases).
- Ownership: users own brands (`brands.owner_id`); everything about a brand cascades from `brands` . Brand Studio items belong directly to a user.

Entity relationship diagram



Tables with **no foreign key** to anything (standalone): `sources` , `collector_state` , `api_usage` , `audit_log` , `text_analysis_cache` , `stored_files` . Soft references with no FK on purpose: `users.active_brand_id` (`backend/alembic/versions/0012_active_brand.py:5-6`), `audit_log.actor_id` ("log outlives users", `ops.py:136`), `asset_verifications.verified_by_id` (`brand.py:130`), `detection_events.actor_id` , `candidate_images.created_by` , `collector_state.cache_brand_id` .

2. Tables

Notation: **NN** = NOT NULL, **null** = nullable. Types are SQLAlchemy types; `String(n)` is VARCHAR(n). All `id` columns are `Integer` primary keys. File and line references point at the model class.

2.1 Accounts (`backend/app/models/user.py`)

`users` (`user.py:10-21`)

Column	Type	Null	Default	Notes
id	Integer	NN	auto	PK
name	String(120)	NN		
email	String(254)	NN		unique + indexed (`ix_users_email`), stored lower-case
password_hash	String(100)	NN		bcrypt
role	String(20)	NN	`"viewer"` (Python)	`admin` / `analyst` / `viewer`
active	Boolean	NN	`True` (Python)	inactive users cannot sign in
created_at	DateTime(tz)	NN	`utcnow`	
active_brand_id	Integer	null		the brand the account is working on; no FK, re-validated on every read (`backend/app/api/common.py:30-40`); added by migration 0012

Self-service sign-up creates role `analyst` (`backend/app/api/auth.py:114`).

`refresh_tokens` (`user.py:24-34`)

Column	Type	Null	Default	Notes
id	Integer	NN	auto	PK
jti	String(64)	NN		unique + indexed
user_id	Integer	NN		FK `users.id` ON DELETE CASCADE, indexed
expires_at	DateTime(tz)	NN		
revoked_at	DateTime(tz)	null		set on rotation or logout
created_at	DateTime(tz)	NN	`utcnow`	

2.2 Brand registry (`backend/app/models/brand.py`)

`brands` (`brand.py:10-40`)

Column	Type	Null	Default	Notes
id	Integer	NN	auto	PK
owner_id	Integer	null		FK `users.id` ON DELETE SET NULL, indexed (`ix_brands_owner_id`); migration 0010
name	String(120)	NN		
industry	String(120)	NN	`""`	
aliases	JSON	NN	`[]`	list of strings
keywords	JSON	NN	`[]`	
domains	JSON	NN	`[]`	normalised domains
logo_path	String(300)	null		relative path in the file store
logo_format	String(10)	null		`png` / `jpg` / `webp`
logo_width, logo_height, logo_size_bytes	Integer	null		
logo_sha256	String(64)	null		
description	Text	null		brand's own public description (migration 0006)
archived_at	DateTime(tz)	null		soft delete; nothing is erased (migration 0006)
source	String(10)	NN	`"live"`	`live` / `sample`
created_at	DateTime(tz)	NN	`utcnow`	
updated_at	DateTime(tz)	NN	`utcnow` , on update `utcnow`	

Relationships (all `cascade="all, delete-orphan"`): `accounts` , `apps` , `logos` , `publishers` , `verifications` (`brand.py:36-40`).

`official_accounts` (`brand.py:43-59`)

Columns: `id` , `brand_id` (FK `brands.id` CASCADE, indexed, NN), `platform` String(20) NN, `handle` String(100) NN (bare handle, no `@`), `handle_key` String(100) NN (lower-case duplicate key), `url` String(300) NN, `status` String(20) NN default `"unreachable"` (`verified` / `unreachable`), `check_message` String(300) null, `last_checked_at` DateTime(tz) null, `source` String(10) NN default `"live"` , `created_at` DateTime(tz) NN default `utcnow` . Constraint: `UNIQUE (brand_id, platform, handle_key)` (`uq_official_accounts_brand_id`). Handle columns were widened from 30 to 100 by 0008.

`official_apps` (`brand.py:62-78`)

Columns: `id`, `brand_id` (FK CASCADE, indexed), `name` String(120), `package_id` String(155), `package_key` String(155) (lower-case), `publisher` String(120), `store_id` String(20) null (numeric App Store id; migration 0005), `stores` JSON NN default `[]`, `status` String(20) NN default `"verified"`, `source` String(10) default `"live"`, `created_at`. Constraint: `UNIQUE (brand_id, package_key)`.

`brand_logos` (`brand.py:81-97`) - extra official logos (the first stays on `brands.logo_*`)

Columns: `id`, `brand_id` (FK CASCADE, indexed), `label` String(120) default `""`, `path` String(300), `format` String(10), `width`, `height`, `size_bytes` (Integer, NN), `sha256` String(64), `created_at`. Constraint: `UNIQUE (brand_id, sha256)`. At most 10 per brand is enforced by the API (`backend/app/api/brands.py:460-462`), not by the database.

`brand_publishers` (`brand.py:100-112`)

Columns: `id`, `brand_id` (FK CASCADE, indexed), `store` String(20) (`google_play` / `app_store` / `any`), `name` String(120) default `""`, `publisher_id` String(120) null, `publisher_key` String(120) (lower-case id, or name when no id), `created_at`. Constraint: `UNIQUE (brand_id, store, publisher_key)`.

`asset_verifications` (`brand.py:115-131`)

Columns: `id`, `brand_id` (FK CASCADE, indexed), `asset_type` String(12) (`domain|account|app|publisher|logo`), `asset_key` String(300), `status` String(12) default `"unverified"`, `method` String(40) null, `note` String(300) default `""`, `verified_at` DateTime(tz) null, `verified_by_id` Integer null (no FK), `verified_by` String(120) null. Constraint: `UNIQUE (brand_id, asset_type, asset_key)`. This is a person's attestation; the app does not check DNS or stores itself (`brand.py:116-117`).

2.3 Detections (`backend/app/models/detection.py`)

`detections` (`detection.py:10-53`)

Column	Type	Null	Default	Notes
id	Integer	NN	auto	PK
brand_id	Integer	NN		FK `brands.id` CASCADE, indexed
type	String(20)	NN		`fake_profile`, `fake_brand_page`, `scam_account`, `fake_app`, `lookalike_domain`
category	String(10)	NN		`social` / `apps` / `web`
platform	String(20)	NN		indexed
name	String(200)	NN	`""`	
handle, package_id, publisher	String(200)	null		
profile_url	String(500)	null		
score	Integer	NN		0-100
band	String(10)	NN		`high` / `medium` / `low`
severity	String(10)	NN	`"LOW"` Python and **server default**	`LOW` / `MEDIUM` / `HIGH` / `CRITICAL` ; migration 0005
algorithm_version	String(10)	null		`"1.0"` social/app, `"2.0"` web
scored_at	DateTime(tz)	null		
status	String(20)	NN	`"new"`	indexed; `new`, `reviewing`, `confirmed`, `dismissed`, `marked_official`, `cleared`
resolution	String(20)	null		`true_positive`, `false_positive`, `resolved`, `monitoring` (migration 0007)
canonical_key	String(400)	NN	`""`	`<platform>:<lower-case handle or package or url>`
first_seen_at, last_seen_at	DateTime(tz)	null		
last_scan_id	Integer	null		FK `scans.id` ON DELETE SET NULL
score_history	JSON	NN	`[]`	`[{at, score, scan_id}]`
notes	JSON	NN	`[]`	`[{at, actor, text, kind}]`
reasons	JSON	NN	`[]`	`[{signal, detail, points}]`
evidence	JSON	NN	`{}`	`{page_url, captured_at, sha256, snapshot_path, screenshot_path}`
details	JSON	NN	`{}`	per-type extras
stale	Boolean	NN	`False`	
source	String(10)	NN	`"live"`	`live` / `sample`
origin	String(12)	NN	`"scan"` Python and **server default**	`scan`, `manual`, `import`, `synthetic` ; migration 0006
found_at, created_at	DateTime(tz)	NN	`utcnow`	
updated_at	DateTime(tz)	NN	`utcnow` , on update	

Constraint: `UNIQUE (brand_id, canonical_key)` (`uq_detections_brand_id` , migration 0003) - this is what lets a re-scan update rather than duplicate. Indexes: `ix_detections_brand_id`, `ix_detections_platform`, `ix_detections_status`.

`official_matches` (`detection.py:56-81`) - items skipped because they matched the allowlist

Columns: `id`, `brand_id` (FK CASCADE, indexed), `scan_id` (FK `scans.id` SET NULL, null), `kind` String(10) (`account` / `app`), `platform` String(20) null, `label` String(300), `reason` String(40) (`exact_handle|exact_url|package_id|apple_id|bundle_id|marked_official`), `matched_rule` String(300) default `""`, `match_key` String(400) default `""` (`<kind>:<platform>:<key>`), `url` String(500) null, `last_seen_at` null, `times_seen` Integer default 1, `official_account_id` (FK `official_accounts.id` SET NULL, null), `official_app_id` (FK `official_apps.id` SET NULL, null), `source` String(10) default `"live"`, `created_at`. Constraint: `UNIQUE (brand_id, match_key)`.

`detection_events` (`detection.py:84-97`) - append-only history of one detection

Columns: `id`, `detection_id` (FK `detections.id` CASCADE, indexed), `at` DateTime(tz) default `utcnow`, `actor` String(120) default `"system"`, `actor_id` Integer null (no FK), `kind` String(20) default `"status"` (`status|scan|note|evidence`), `from_status` String(20) null, `to_status` String(20) null, `note` Text default `""`.

`candidate_images` (`detection.py:100-118`)

Columns: `id`, `brand_id` (FK CASCADE, indexed), `purpose` String(20) default `"candidate_image"` (`candidate_image|app_icon`), `path` String(300), `format` String(10), `width`, `height`, `size_bytes`, `sha256` String(64), `hashes` JSON default `{}` (`{phash, dhash, whash, colorhash}`), `created_by` Integer null (no FK), `created_at`. Constraint: `UNIQUE (brand_id, sha256)`.

2.4 Operations (`backend/app/models/ops.py`)

`sources` (`ops.py:10-25`) - one row per monitored platform

Columns: `id`, `platform` String(20) NN **unique**, `name` String(80), `kind` String(10) (`social` / `store`), `description` String(200) default `""`, `status` String(15) default `"online"` (`online|slow|unavailable`), `monitored` Boolean default `True`, `message` String(300) default `""`, `detail` String(200) default `""`, `last_sync` DateTime(tz) null, `retry_in_minutes` Integer null, `stale_since` DateTime(tz) null, `source` String(10) default `"live"`.

`collector_state` (`ops.py:28-46`) - health, back-off and last good result per collector (migration 0002)

Columns: `id`, `collector` String(40) **unique**, `status` String(15) default `"online"`, `reason` String(40) null, `message` String(400) default `""`, `latency_ms` Integer null, `last_attempt_at`, `last_success_at` (DateTime(tz) null), `consecutive_failures` Integer default 0, `next_retry_at` null, `cache_brand_id` Integer null (no FK), `cache` JSON null (last good normalised candidates), `cached_at` null, `source` String(10) default `"live"`.

`scans` (`ops.py:53-79`)

Column	Type	Null	Default	Notes
id	Integer	NN	auto	PK
job_id	String(40)	NN		unique
brand_id	Integer	NN		FK `brands.id` CASCADE, indexed
scope	String(10)	NN	`"all"`	`all
state	String(10)	NN	`"queued"`	`queued
progress	Integer	NN	0	
message	String(300)	NN	`""`	
steps	JSON	NN	`[]`	
result	JSON	null		
data_mode	String(10)	NN	`"live"`	
created_by	Integer	null		FK `users.id` SET NULL
started_by	String(120)	NN	`"scheduler"` (Python and server default)	user name or `scheduler`
counts, timings	JSON	NN	`{}`	server defaults `{}` (0003)
source_report, errors	JSON	NN	`[]`	server defaults `[]`
started_at, finished_at	DateTime(tz)	null		
created_at	DateTime(tz)	NN	`utcnow`	

Partial unique index `uq_scans_one_active_per_brand` on `(brand_id)` WHERE `state IN ('queued','running')` (`ops.py:49, 55-58`; migration 0004): the database itself guarantees one active scan per brand, on both SQLite and PostgreSQL.

`settings` (`ops.py:82-91`) - org-level key/value

Columns: `id`, `key` String(80) **unique**, `value` JSON null, `updated_at` DateTime(tz) default/on update `utcnow`, `updated_by` Integer null (FK `users.id` SET NULL). Keys seen in code include `monitoring`, `notifications`, `text_provider` (`backend/app/core/pipeline/text_analysis.py:28`) and the data-mode setting (`backend/app/services/data_mode.py`).

`reports` (`ops.py:94-113`)

Columns: `id`, `brand_id` (FK CASCADE, indexed), `filename` String(200), `format` String(5) (`csv|html`), `title` String(200) default `"Brand Protection Report"`, `status` String(10) default `"draft"` (`draft|approved`), `detection_ids` JSON default `[]`, `snapshot` JSON default `[]` (rows frozen at creation), `approved_by` (FK `users.id` SET NULL, null), `approved_at` null, `size_bytes` Integer default 0, `params` JSON default `{}`, `file_path` String(300) null, `includes_sample` Boolean default `False`, `source` String(10) default `"live"`, `created_by` (FK `users.id` SET NULL, null), `created_at`.

Needs confirmation: `reports.file_path` is never read or written by `backend/app/api/reports.py` or `backend/app/services/reports.py` (reports are rendered on demand from `snapshot`); it appears to be an unused column kept from migration 0001.

`api_usage` (`ops.py:116-126`) - credit / quota counters

Columns: `id` , `provider` String(60), `month` String(7) (`YYYY-MM`), `credits` BigInteger default 0, `updated_at` . Constraint: `UNIQUE (provider, month)` . Used for the monthly Tavily credit ledger (`backend/app/core/collectors/credits.py`) and, with keys like `studio:text:u<user>:<YYYYMMDD>` , for Studio AI daily quotas (`backend/app/services/studio_ai.py:283`).

`audit_log` (`ops.py:129-141`) - append-only

Columns: `id` , `ts` DateTime(tz) default `utcnow` , indexed (`ix_audit_log_ts`), `actor_id` Integer null (no FK), `actor` String(120) default `"system"` , `action` String(60) indexed (`ix_audit_log_action`), `target` String(300) default `""` , `detail` Text default `""` , `source` String(10) default `"live"` .

`lookalike_cases` (`ops.py:144-160`)

Columns: `id` , `brand_id` Integer null (FK `brands.id` CASCADE, indexed; migration 0010), `name` String(200), `expected` String(15) (`flagged|not_flagged|official`), `builtin` Boolean default `False` , `source` String(10) default `"live"` , `created_by` (FK `users.id` SET NULL, null), `created_at` . Partial unique index `uq_lookalike_cases_builtin_name` on `(brand_id, name)` WHERE `builtin` (`ops.py:50, 149-151`): built-in cases are seeded once per brand.

`text_analysis_cache` (`ops.py:163-171`)

Columns: `key` String(64) **PK** (hash of model + prompt version + context + text), `provider` String(20), `value` JSON default `{}` , `created_at` .

`stored_files` (`ops.py:174-184`) - file content in the database

Columns: `path` String(300) **PK** (same relative path the local store would use), `content` LargeBinary NN, `content_type` String(100) default `"application/octet-stream"` , `size_bytes` Integer default 0, `created_at` .

2.5 Brand Studio (`backend/app/models/studio.py`)

`studio_items` (`studio.py:11-22`)

Column	Type	Null	Default	Notes
id	Integer	NN	auto	PK
user_id	Integer	NN		FK `users.id` CASCADE, indexed (`ix_studio_items_user_id`)
type	String(20)	NN		one of `STUDIO_TYPES = logo, palette, brand_name, content, audit, sentiment` (`studio.py:9`); enforced by the API, not by a CHECK constraint
label	String(200)	NN	`""`	
data	JSON	null		up to 64 KB, API-enforced (`backend/app/api/studio.py:32`)
image_path	String(300)	null		relative file-store path `studio/user_<id>/<sha>.<ext>` (`studio.py:94`)
created_at	DateTime(tz)	NN	`utcnow`	

3. Constraint and index summary

Kind	Name / columns	Table	Where defined
unique	`users.email` (`ix_users_email`)	users	`user.py:15`
unique	`refresh_tokens.jti`	refresh_tokens	`user.py:30`
unique	`(brand_id, platform, handle_key)`	official_accounts	`brand.py:45`
unique	`(brand_id, package_key)`	official_apps	`brand.py:64`
unique	`(brand_id, sha256)`	brand_logos, candidate_images	`brand.py:86` , `detection.py:105`
unique	`(brand_id, store, publisher_key)`	brand_publishers	`brand.py:104`
unique	`(brand_id, asset_type, asset_key)`	asset_verifications	`brand.py:120`
unique	`(brand_id, canonical_key)`	detections	`detection.py:14`
unique	`(brand_id, match_key)`	official_matches	`detection.py:60`
unique	`platform` / `collector` / `key` / `job_id`	sources / collector_state / settings / scans	`ops.py:14, 34, 88, 61`
unique	`(provider, month)`	api_usage	`ops.py:121`
partial unique	`uq_scans_one_active_per_brand`	scans	`ops.py:56-58`
partial unique	`uq_lookalike_cases_builtin_name`	lookalike_cases	`ops.py:149-151`
index	`brand_id` on every brand child table; `owner_id` ; `user_id` (refresh_tokens, studio_items); `detection_id`	various	each model
index	`detections.platform` , `detections.status`	detections	`detection.py:20, 33`
index	`audit_log.ts` , `audit_log.action`	audit_log	`ops.py:135, 138`

Foreign-key delete behaviour: deleting a brand cascades to all its child rows; deleting a user cascades to `refresh_tokens` and `studio_items` but only nulls `brands.owner_id` , `scans.created_by` , `reports.created_by/approved_by` , `settings.updated_by` , `lookalike_cases.created_by` . In practice brands are soft-deleted (`archived_at`), never erased (`backend/app/api/brands.py:419-430`).

4. Migration history (Alembic, `backend/alembic/versions/`)

Linear chain, each file's `down_revision` is the previous number. `render_as_batch=True` is set in `backend/alembic/env.py:16, 25, 29` so column changes work on SQLite.

Rev	File	Date (file header)	What it does
0001	`0001_initial_schema.py`	2026-10-13	Creates the first 13 tables: `api_usage`, `audit_log`, `brands`, `sources`, `users`, `detections`, `official_accounts`, `official_apps`, `refresh_tokens`, `reports`, `scans`, `settings`, `official_matches`, with FKs, unique constraints and indexes
0002	`0002_collector_state.py`	2026-10-13	Adds `collector_state` (collector health, back-off, cached last good result)
0003	`0003_scan_pipeline.py`	2026-10-13	Scan reports (`started_by`, `counts`, `source_report`, `timings`, `errors`) and maps old scan states `done` -> `completed`, `error` -> `failed`; detection `canonical_key`, `first_seen_at`, `last_seen_at`, `last_scan_id`, `score_history`, `notes` + `UNIQUE(brand_id, canonical_key)` (existing rows get `legacy:<id>`); new `detection_events`; `official_matches` match keys (`matched_rule`, `match_key`, `url`, `last_seen_at`, `times_seen`) + unique; report draft fields (`title`, `status`, `detection_ids`, `snapshot`, `approved_by`, `approved_at`); new `lookalike_cases`
0004	`0004_integrity_indexes.py`	2026-10-13	Partial unique indexes: one queued/running scan per brand; built-in look-alike cases seeded once. First cleans duplicates (fails older active scans, deletes duplicate built-in cases)
0005	`0005_risk_engine_and_web.py`	2026-10-14	`detections.severity` (server default `LOW`), `algorithm_version`, `scored_at` with back-fill from score (85/70/40 cut-offs, version `1.0`); `official_apps.store_id`
0006	`0006_registry_v2_and_providers.py`	2026-10-14	`brands.description`, `brands.archived_at`; `detections.origin` (sample rows back-filled `synthetic`); new `brand_logos`, `brand_publishers`, `asset_verifications`, `candidate_images`, `text_analysis_cache`
0007	`0007_resolution.py`	2026-10-15	`detections.resolution` (nullable)
0008	`0008_wider_account_handles.py`	2026-10-15	`official_accounts.handle` / `handle_key` from 30 to 100 chars (matters on PostgreSQL only, SQLite ignores VARCHAR length)
0009	`0009_stored_files.py`	2026-10-16	New `stored_files`; on PostgreSQL also `ENABLE ROW LEVEL SECURITY` with no policy
0010	`0010_brand_owner.py`	2026-10-17	`brands.owner_id` (FK users SET NULL, indexed) and `lookalike_cases.brand_id` (FK brands CASCADE, indexed); existing brands go to the lowest-id admin, existing cases to the first brand; built-in unique index becomes `(brand_id, name)`. On SQLite, foreign keys are switched off around the `brands` rebuild so it does not cascade-delete children (`0010_brand_owner.py:30-37`)
0011	`0011_studio_items.py`	2026-10-17	New `studio_items`; on PostgreSQL `ENABLE ROW LEVEL SECURITY` with no policy
0012	`0012_active_brand.py`	2026-10-20	`users.active_brand_id` (nullable, no FK) - backs `POST /api/brands/{id}/select`

Every migration has a working `downgrade()` (0010's downgrade de-duplicates built-in cases first so the older single-column unique index can return, `0010_brand_owner.py:55-79`). `alembic_version` holds the current revision; the health check reads it (`backend/app/services/health.py:66`, `check_migration`).

How to run migrations

Run from `backend/`:

```
alembic upgrade head                # apply everything
alembic current                     # show the applied revision
alembic downgrade -1                # undo the last one
python -m scripts.migrate           # same as upgrade head, plus a Postgres advisory lock (safe with many replicas)
```

- `alembic.ini` points `script_location` at `alembic` (`backend/alembic.ini:2`). `env.py` takes the URL from `sqlalchemy.url` if set, else from `DatabaseSettings().migration_url` (`backend/alembic/env.py:11-12`). That reads only `MIGRATION_DATABASE_URL` or `DATABASE_URL`, so no `JWT_SECRET` etc. is needed to migrate (`backend/app/config/config.py:210-222`).
- **`MIGRATION_DATABASE_URL`** (optional): a session-mode or direct connection (port 5432) used only by Alembic and `scripts/migrate.py` (`config.py:35-37`). Use it with Supabase: set `DATABASE_URL` to the **transaction pooler** (port 6543) for the running app, and `MIGRATION_DATABASE_URL` to the session/direct URL (5432) for schema changes, because DDL and advisory locks need a session-level connection. When it is unset, migrations use `DATABASE_URL`. (Never put real URLs or passwords in docs or commits.)
- On port 6543 the engine disables server-side prepared statements (`prepare_threshold=None`, `backend/app/db/session.py:30-31`).
- `scripts/migrate.py` takes `pg_advisory_lock(727274 0001)` on PostgreSQL for the whole upgrade so concurrent starters wait, then find head already applied (`backend/scripts/migrate.py:21, 32-42`).
- Docker: the `api` role runs `python -m scripts.migrate` before uvicorn unless `RUN_MIGRATIONS=false`; the `migrate` role runs it once and exits (`backend/docker-entrypoint.sh:3-22`).
- If the app starts before migrations ran, seeding is skipped with a warning and `/health` reports the problem (`backend/app/main.py:76-83`).
- Tests check that migrations produce exactly the models' schema (`compare_metadata` in `backend/tests/db/test_migration.py:33-36`) and, on PostgreSQL only, inside a throw-away schema when `TEST_DATABASE_URL` is set (`backend/tests/db/test_postgres.py:1-30`).

---

5. Row-level security (PostgreSQL)

- Only two tables enable RLS, both with **no policy** (so non-owner roles can read nothing): `stored_files` (migration 0009) and `studio_items` (migration 0011). The `ALTER TABLE ... ENABLE ROW LEVEL SECURITY` runs only when the dialect is `postgresql` (`0009_stored_files.py:30-31`, `0011_studio_items.py:33-34`).

- Reason, from the migration docstrings: Supabase exposes the `public` schema through its REST API, and the API roles (`anon` , `authenticated`) must not be able to read uploaded files or saved Studio items. The backend connects as the table owner / `postgres` role, which is not subject to RLS, so the application is unaffected.
- Migration 0010 notes that it adds no new tables, so no new RLS was needed (`0010_brand_owner.py` docstring).
- **Needs confirmation:** no other table (for example `users` , `refresh_tokens` , `audit_log` , `brands`) has RLS enabled by any migration in `backend/alembic/versions/` . If the database is on Supabase with the REST API reachable, the project owner should check whether those tables also need RLS or whether the public schema is not exposed.
- Application-level tenant isolation is separate: it is done in code with `owner_id` checks (404 for another account's data), see `docs/api-reference.md` section 1.3 and `backend/tests/api/test_tenant_isolation.py` .

---

6. SQLite vs PostgreSQL

Topic	SQLite (default local)	PostgreSQL (production, e.g. Supabase)	Source
Default URL	`sqlite:///.../backend/data/scammind.db`	set `DATABASE_URL`	`config.py:28`
Foreign keys	switched on per connection with `PRAGMA foreign_keys=ON`	native	`session.py:36-43`
Concurrency	WAL journal mode, 30 s busy timeout, `check_same_thread=False` ; in-memory DBs use a `StaticPool`	pooled connections: 5 + 2 overflow (1 + 1 on Vercel), recycle 1800 s (300 on Vercel), TCP keepalives; `pool_pre_ping` only if `DB_PRE_PING` is set	`session.py:10-33`
Datetimes	returned **naive**; code treats them as UTC via `as_utc()`	timezone-aware	`config/time.py:11-13`
VARCHAR length	not enforced (the 0008 handle-length bug only showed on PostgreSQL)	enforced	`0008_wider_account_handles.py` docstring
Schema changes	Alembic batch mode (table rebuild); 0010 turns FKs off during the `brands` rebuild	normal `ALTER TABLE`	`alembic/env.py` , `0010_brand_owner.py:30-37`
Partial unique indexes	supported (`sqlite_where`)	supported (`postgresql_where`)	`ops.py:56-58, 149-151`
Migration locking	none needed (one process)	advisory lock in `scripts/migrate.py`	`migrate.py:32-42`
Row-level security	not applicable	enabled on `stored_files` , `studio_items`	section 5
JSON / binary	`JSON` stored as text; `LargeBinary` as BLOB	`JSON` ; `LargeBinary` as BYTEA	models
Transaction pooler	n/a	no prepared statements when URL has `:6543/`	`session.py:30-31`

Date-sensitive code paths use `as_utc` / `iso_z` , and API output always ends with `Z` (`config/time.py:16-20`).

---

7. Where files are stored

Uploaded and generated files never go inside normal table columns. Rows keep a **relative path** (`logo_path` , `candidate_images.path` , `studio_items.image_path` , `detections.evidence.snapshot_path`) and the bytes live in a "file store" chosen by `STORAGE_BACKEND` (`backend/app/core/storage.py:169-176` , setting at `config.py:34`):

`STORAGE_BACKEND`	Implementation	Where bytes live	When used
`local` (default)	`LocalFileStore` (`storage.py:55-93`)	files under `DATA_DIR` (default `backend/data`)	laptops, Docker with a volume
`database`	`DatabaseFileStore` (`storage.py:96-166`)	rows of the **`stored_files`** table (`path` PK, `content` blob)	serverless (Vercel), where the disk is read-only and not shared; default when `VERCEL` is set (`config.py:185`)

Any other value raises `ValueError` at start-up (`storage.py:174-175`). Both stores refuse unsafe paths (absolute, `..` , backslashes, NUL, longer than 300 characters) (`storage.py:45-52`), so a path in the database cannot escape the store.

Path layout (identical in both stores, content-addressed by SHA-256, so uploads de-duplicate):

Content	Relative path	Written by
Official brand logos	`logos/brand_/<sha256[:24]>.<png	jpg
Candidate images / app icons	`candidate_images/brand_<brand_id>/<sha256[:24]>.<ext>`	`backend/app/core/pipeline/candidate_images.py:67-70`
Evidence snapshots	`evidence/<sha[:2]>/<sha>.json`	`backend/app/core/pipeline/evidence.py:35`
Evidence screenshots (off by default)	`evidence/screenshots/<sha[:2]>/<sha>.png`	`evidence.py:42`
Brand Studio images	`studio/user_<user_id>/<sha256[:24]>.<ext>`	`backend/app/api/studio.py:94`

Notes:

- With the `database` backend, `DATA_DIR` is only scratch space; the health check then reports the database store instead of probing the disk (`backend/app/services/health.py:120-122` , `backend/app/main.py:72-75`).

- The database store joins the caller's transaction when a session is passed (so a rollback also drops the file row), otherwise it uses its own short session (`storage.py:102-125`).
- Reports are **not** stored as files: they are re-rendered from `reports.snapshot` on every download (see `reports.file_path` note in section 2.4).
- Moving from `local` to `database` (or back) does not migrate existing files. **Needs confirmation:** whether a copy script exists; none was found in `backend/scripts/` (`canary.py` , `e2e_server.py` , `evaluate.py` , `export_schema.py` , `live_check.py` , `migrate.py` , `seed_test_brand.py` , `studio_ai_check.py`).

8. Seed data

At start-up the app (inside `lifespan` , `backend/app/main.py:76-83`) seeds, when the tables exist: the real admin from `ADMIN_EMAIL` / `ADMIN_PASSWORD` , the optional shared demo analyst (`DEMO_EMAIL` / `DEMO_PASSWORD`) and the optional guarded demo administrator (`DEMO_ADMIN_EMAIL` / `DEMO_ADMIN_PASSWORD`) (`backend/app/services/seed.py`). `sources` rows are created on demand by `ensure_sources` (`backend/app/core/collectors/orchestrator.py`); built-in look-alike cases are created per brand by `lookalike.ensure_default_cases` (`backend/app/core/pipeline/lookalike.py:78`). `scripts/export_schema.py` generates `docs/DATABASE_SCHEMA.md` and `docs/schema.sql` .

DOCS/AUTH-AND-SECURITY.MD

Authentication and security

Plain-language summary

- People sign in with an email and password. The server hands back a short-lived "access" pass (15 minutes) and a longer "refresh" pass (7 days) that is swapped for a new pair each time it is used.
- Every person's brands, scans and reports belong to them; other accounts cannot see them. Only a small set of settings is shared by everyone.
- There are two public demo accounts whose passwords are shown on the login page. The demo administrator is "guarded": it is blocked from the few actions that would affect real users.
- Outside services are called only by the server, which keeps the API keys; the browser never sees them. Downloads of images and links are protected against being tricked into reaching internal networks.
- Known gaps are listed at the end. The most important: anyone can sign up without email verification, rate limits are per server process, and some "admin-only" screens are hidden in the interface but their read endpoints are open to any signed-in user.

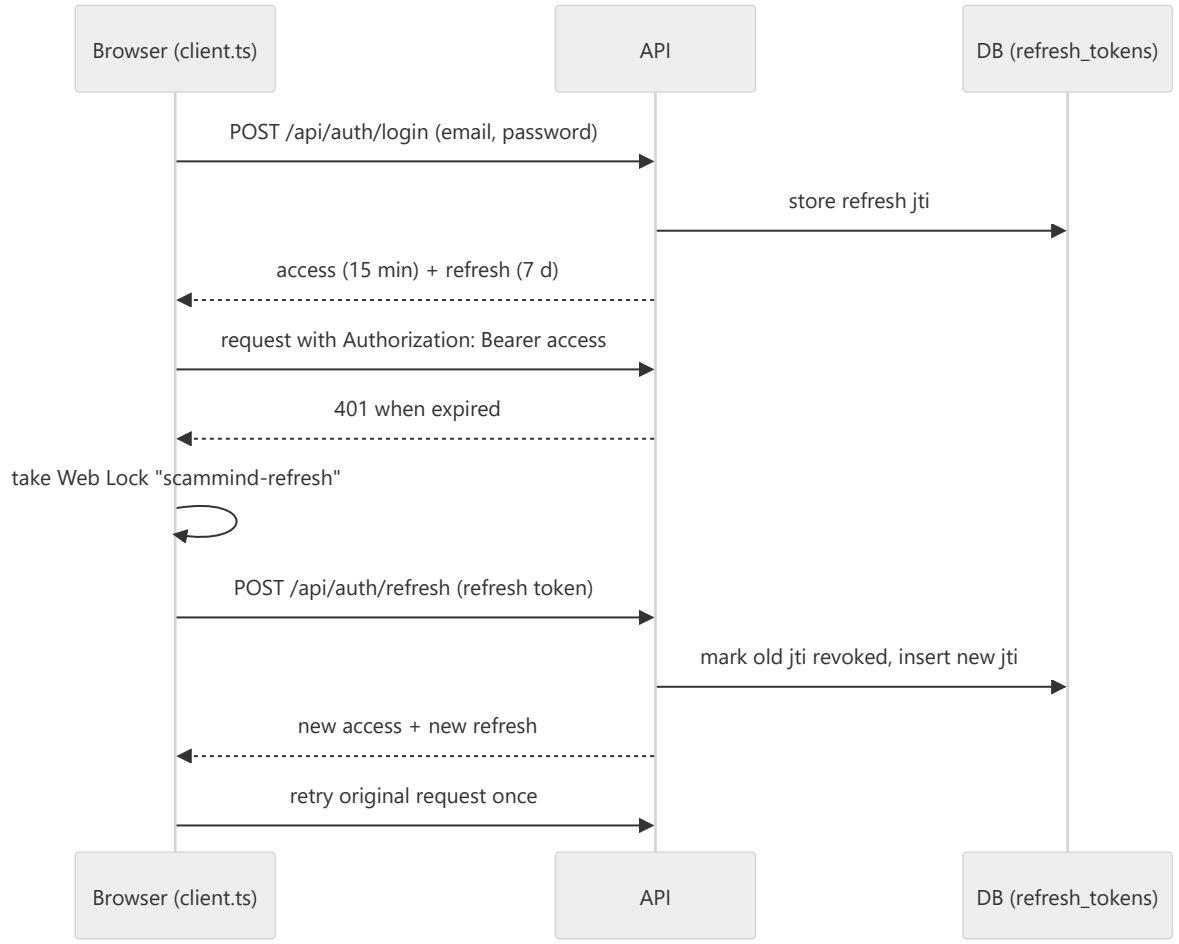
This page was written from the code. `docs/SECURITY.md` predates multi-tenant ownership and the demo accounts, so where the two differ, this page reflects the code.

1. Authentication

1.1 Passwords

Item	Behaviour	Source
Hashing	bcrypt, 12 rounds (tests lower it)	`backend/app/config/security.py:11,15-16`
Length limit	Only the first 72 bytes are used; longer passwords are rejected at the schema level	`security.py:12,15,21` , `backend/app/schemas/auth.py:10-15`
Sign-up policy	At least 10 characters, a denylist of trivial passwords, not all-digits, at least 5 distinct characters, not the same as name or email	`backend/app/schemas/auth.py:18-42,66-71`
Timing	Unknown emails are verified against a dummy hash so response time does not reveal whether an account exists	`backend/app/api/auth.py:26,69`
Login errors	Same message for wrong password, unknown email and inactive account: "Incorrect email or password"	`auth.py:70-74`
Failed logins	Counted per `email	client IP `and blocked after` LOGIN_MAX_FAILURES `(5) in` LOGIN_WINDOW_SECONDS` (900); failures are written to the audit log

1.2 Tokens (access + rotating refresh)



Item	Behaviour	Source
Signing	PyJWT, `HS256` by default, key `JWT_SECRET` (minimum 16 characters, no default)	`security.py:26-35`, `backend/app/config/config.py:40-41`
Claims	`sub`, `typ` (`access` or `refresh`), `iat`, `exp`, `jti`; decoding requires `exp`, `sub`, `typ`, `jti` and checks the type	`security.py:26-56`
Lifetimes	`ACCESS_TOKEN_MINUTES` = 15, `REFRESH_TOKEN_DAYS` = 7	`config.py:42-43`
Access check	Bearer token decoded, user loaded from the DB on every request; a missing or inactive user gives 401	`backend/app/config/deps.py:28-43`
Refresh rotation	Each refresh revokes the presented token and issues a new pair; only the `jti` is stored	`backend/app/api/auth.py:39-46,148-156`
Theft detection	Presenting an already-revoked refresh token revokes all of that user's refresh tokens	`auth.py:136-141`
Logout	Revokes the presented refresh token and writes an audit row; an invalid token is silently accepted	`auth.py:159-170`
Access tokens	Not individually revocable; they expire after at most 15 minutes. An account set inactive stops working at once because the user row is checked per request	`deps.py:41-42`

1.3 Token storage in the browser

Item	Behaviour	Source
Keys	`scammind.auth` (tokens) and `scammind.user` (name, email, role, default brand)	`frontend/src/api/client.ts:59-60`
Where	`localStorage` when "keep me signed in" is on (default `persist = true`), otherwise `sessionStorage`; reads check both	`client.ts:77-98`, `frontend/src/api/auth.ts:11-15`
Not cookies	Tokens travel in the `Authorization` header; CORS credentials are off	`client.ts:297-299` (`send`), `backend/app/main.py:160-167`
Sign-out	Clears both stores, legacy keys and the in-memory query cache so the next user never sees the previous session's pages	`client.ts:120-126`
Single-flight refresh	One refresh per tab; concurrent 401s share it; a refresh rejected with 4xx ends the session, a network or 5xx error keeps it	`client.ts:221-264,315-345`
Web Locks	Refresh runs inside `navigator.locks.request('scammind-refresh', ...)` so tabs and both apps (BrandShield and Brand Studio) never spend the same rotated refresh token twice; the tokens are re-read inside the lock; without the API it falls back to the in-tab single flight	`client.ts:227-262`, `studio/src/lib/api.ts:7,93-99`
Shared with Studio	Studio reads the same keys from the same origin	`studio/src/lib/session.ts:1-4`

Because tokens live in web storage, any script running on the page could read them (see "Known risks").

2. Authorization

2.1 Roles

Roles are `viewer`, `analyst`, `admin` (`frontend/src/api/client.ts:68-75`). Backend dependencies:

Dependency	Allows	Source
`get_current_user`	Any signed-in, active user (all roles)	`backend/app/config/deps.py:28-43`
`require_writer`	`admin` or `analyst`	`deps.py:46-52,56`
`require_admin`	`admin` only	`deps.py:46-57`

How accounts get roles:

- Self-service sign-up always creates an `analyst` (`backend/app/api/auth.py:114`).
- The real `admin` is created at startup from `ADMIN_EMAIL` / `ADMIN_PASSWORD` if absent; it never overwrites an existing password (`backend/app/services/seed.py:14-24`).
- Demo accounts are described in section 4.
- No API endpoint creates `viewer` accounts or changes roles; `viewer` is used by the e2e server's seed (`backend/scripts/e2e_server.py:54-56`) and tests. **Needs confirmation:** how production viewer accounts are meant to be created (no user-management route exists in `backend/app/api/`).

2.2 What each endpoint group requires

From the router decorators in `backend/app/api/*.py`:

Area	Read	Write	Source
Brands, accounts, apps, logos, publishers	signed-in, owner only	`require_writer` and owner	`backend/app/api/brands.py:124-163,165-217,262-420,449-547`
Brand archive / restore		`require_admin` and owner	`brands.py:419-446`
Scans, detections, reports, candidates, evaluation	signed-in, owner only	`require_writer` and owner	`backend/app/api/scans.py:25-106`, `detections.py:59-195`, `reports.py:81-196`, `candidates.py:36-115`, `evaluation.py:94-102`
Audit log and CSV export	`require_writer` (viewers cannot read)		`backend/app/api/audit.py:52-84`
Settings	any signed-in user (GET)	`require_admin` (PUT)	`backend/app/api/settings.py:125-139`
Data mode	any signed-in user (GET)	`require_admin` (PUT)	`backend/app/api/sources.py:233-250`
Source health	any signed-in user (GET)	`require_writer` (PATCH monitored, POST retry)	`sources.py:125,160,179`
Active AI provider	any signed-in user (GET)	`require_admin` (PUT)	`backend/app/api/providers.py:45-57`
Look-alike test cases	signed-in, owner	add/delete `require_admin`; run `require_writer`	`backend/app/api/lookalike.py:125-160`
Detailed health, canary		`require_admin`	`backend/app/api/health.py:36-49`
Studio saved items	signed-in, `user_id` scoped	signed-in, `user_id` scoped	`backend/app/api/studio.py:103-190`
Studio AI, voice	any signed-in user, per-user rate limits and quotas		`backend/app/api/studio_ai.py:63-110`, `backend/app/api/voice.py:55-60`
Public (no auth)	`/health`, `/api/health`, `/health/live`, `/api/auth/demo`, login, register, refresh, logout, FastAPI `/docs` and `/openapi.json`		`backend/app/api/health.py:22-33`, `auth.py:56-170`, `backend/app/main.py:94`
Cron	`GET /api/cron/scan` with `Authorization: Bearer <CRON_SECRET>`, compared with `hmac.compare_digest`; 503 when the secret is unset		`backend/app/api/cron.py:27-38`

Needs confirmation: `/docs` and `/openapi.json` are not disabled in `create_app` (`FastAPI(title="BrandShield", version="0.1.0", lifespan=lifespan)`, `backend/app/main.py:94`), so they are public wherever the API is reachable. Confirm whether Vercel routing exposes them (`vercel.json` routes only `/api/*`, `/health`, `/health/live` to the backend, `vercel.json:44-62`, so `/docs` would not reach it on Vercel, but would on the Docker API port).

2.3 Frontend route guards

Guard	Behaviour	Source
`RequireAuth`	Without stored tokens redirects to `/login`; an `scammind:unauthorized` event (refresh rejected) sends the user to `/login`	`frontend/src/App.tsx:56-78`
`RequireAdmin`	Wraps `/sources` (Source Health), `/settings`, `/evaluation` (Model Evaluation); reads the role from the stored `scammind.user` and redirects non-admins to `/dashboard`	`App.tsx:38-54,175-179`
Sidebar	Hides the three admin links unless `session.user()?.role === 'admin'`	`frontend/src/components/layout/Sidebar.tsx:33`

These are convenience checks. The role is read from `localStorage`, which a user can edit, so the real enforcement is the backend (section 2.2).

2.4 The three admin-only pages: does the backend also restrict non-admins?

Exactly what the code does, per page:

Page	Endpoint	Non-admin result	Source
Settings	`GET /api/settings`	**Allowed** for any signed-in user (viewer, analyst). The stored notification webhook URL is blanked unless the caller is a real (non-demo) admin; `webhook_configured` is still shown	`backend/app/api/settings.py:118,125-127`
Settings	`PUT /api/settings`	403 for non-admin	`settings.py:130-132`
Source Health	`GET /api/sources`	**Allowed** for any signed-in user; includes the global monthly Tavily credit snapshot and Safe Browsing reputation status (never the key)	`backend/app/api/sources.py:125-158`
Source Health	`PATCH /api/sources/{id}` (turn monitoring on/off) and `POST /api/sources/{id}/retry`	**Allowed** for `analyst` (so for any self-registered user). The `Source` rows are global (not tied to a brand), so a toggle affects every account. Retry runs collectors against the caller's own brand and may spend shared paid credits; it needs `confirm=true` when credits would be spent	`sources.py:160-231`
Source Health	`GET/PUT /api/settings/data-mode`	GET any signed-in user; PUT `require_admin`	`sources.py:233-250`
Model Evaluation	`GET /api/evaluation`, `POST /api/evaluation/run`	GET any signed-in user for their **own** brand; run needs `require_writer`. Scoped to the caller's brand through `resolve_brand`, so no other tenant's data is returned	`backend/app/api/evaluation.py:94-110`

Conclusion: only the **write** side of Settings and the data-mode switch are admin-enforced on the server. Reading Settings, Source Health and Evaluation is not admin-restricted on the server, and an analyst can change the global monitored flag of a source. Hiding the three pages is a UI choice, not an access control.

3. Tenant isolation

Each brand has `owner_id`; every brand-scoped route resolves the brand through helpers that compare it to the signed-in user:

- `get_brand_or_404`: a brand owned by someone else is a 404, never 403, so existence is not revealed (`backend/app/api/common.py:22-27`).
- `resolve_brand` and `own_brand`: `brand_id` is optional and defaults to the caller's selected or oldest non-archived brand (`common.py:30-50`).
- `owns_brand` for objects hanging off a brand (candidate images, detections, reports) (`common.py:53-58`, `backend/app/api/candidates.py:109-115`).
- Starting a scan for a brand the user does not own raises `NoBrand` (`backend/app/core/pipeline/scan_pipeline.py:152-155`).
- Studio items and Studio quotas are keyed by `user_id` (`backend/app/api/studio.py:103-119`, `backend/app/services/studio_ai.py:286-300`).
- The audit log shows non-admins only the rows they performed themselves; the shared demo administrator is also restricted to its own rows (`backend/app/api/audit.py:36-49,64`).
- The ownership helpers (`owns_brand`, `resolve_brand`, `get_brand_or_404`) appear on 65 lines across `backend/app/api/` (including imports), i.e. they are used throughout the brand-scoped routers.

Still shared by all accounts (by design, per `docs/DECISIONS.md:208`, confirmed in code): the `settings` table (monitoring, notifications, data mode, active text provider; `backend/app/services/app_settings.py:59-66`, `backend/app/services/data_mode.py:7-34`), source rows and collector state (`backend/app/models/ops.py:10-50`), Tavily and Safe Browsing budget counters (`api_usage`), the response cache (keyed per Authorization header, `backend/app/config/response_cache.py:15-18,43`), and the per-IP/per-token rate limiter.

The response cache is keyed by a hash of the `Authorization` header, so one user's cached page is not served to another (`backend/app/config/response_cache.py:5-6,24-25`).

4. Demo accounts and the guarded demo administrator

Item	Behaviour	Source
Public credentials	`GET /api/auth/demo` is unauthenticated and returns the demo account's email and password (and the demo admin's) when configured; the response is `Cache-Control: no-store`	`backend/app/api/auth.py:82-96`
Demo user	`DEMO_EMAIL` / `DEMO_PASSWORD`, role `analyst`; disabled if equal to `ADMIN_EMAIL`; an existing account with another role is never touched	`backend/app/services/seed.py:27-60`
Demo admin	`DEMO_ADMIN_EMAIL` / `DEMO_ADMIN_PASSWORD`, role `admin`; email must differ from `ADMIN_EMAIL` and `DEMO_EMAIL`; an existing non-admin account is never promoted	`seed.py:63-97`
Advertised only if real	Credentials are returned only if the account exists, is an active admin, and the password verifies	`seed.py:100-105`
Password resync	A changed env password is re-applied at the next start so the advertised password keeps working	`seed.py:52-54,89-91`
Identification	The demo admin is recognised by email: `is_demo_admin`	`seed.py:108-110`

What the guard (`forbid_demo_admin`, 403) blocks (`seed.py:113-116`):

Blocked action	Source
Choosing a data mode other than `sample` (in Settings or the data-mode endpoint)	`backend/app/api/settings.py:136-137`, `backend/app/api/sources.py:247-248`
Setting the notifications webhook	`settings.py:138-139`
Changing the active AI provider	`backend/app/api/providers.py:53`
Seeing other accounts' audit rows (restricted to own rows)	`backend/app/api/audit.py:64,80`
Seeing the stored webhook URL (blanked); an empty webhook from its form does not clear the real one	`settings.py:118,171-172`

What the demo administrator can still do, because it is a real `admin` and the credentials are public (so, effectively, anyone): change global monitoring settings (schedule, sensitivity, rules, impersonation words, which sources are monitored; `settings.py:140-166`), archive and restore its own brands, add or delete look-alike test cases for its own brand, call `GET /api/health/details` and `POST /api/health/canary` (`backend/app/api/health.py:36-49`), and use the Studio AI quotas shared per user. Note that the demo account and the demo admin are each one shared user, so everyone using them shares one workspace and one AI quota. **Needs confirmation:** whether changes to the global monitoring settings by the public demo admin are acceptable for the intended deployments; the guard list above does not cover them.

Frontend: the login page shows demo buttons from the server response, with an optional local fallback from `VITE_DEMO_EMAIL` / `VITE_DEMO_PASSWORD` that should never be set on a public deployment (`frontend/src/pages/LoginPage.tsx:21-23`, `frontend/.env.example:4-6`).

5. Sign-up limits

Control	Behaviour	Source
Switch	`SIGNUP_ENABLED` (default true); false returns 403	`backend/app/api/auth.py:107-108`
Limits	5 per client IP and 3 per email per hour, in memory per process	`auth.py:109-111`, `backend/app/main.py:100-101`, `config.py:58-59`
Duplicate email	409; a race on the unique email is also caught	`auth.py:112-120`
Result	New `analyst`, signed in immediately, audited as `auth.registered`	`auth.py:114-124`
Email verification	**None** (an address proves nothing)	`docs/DECISIONS.md:202`; no mail code in `backend/app/`
Password reset	The frontend client defines `forgot` and `reset` calls (`frontend/src/api/auth.ts:41-46`) but the backend has no `/auth/forgot` or `/auth/reset` route (search of `backend/app` finds none) and the UI does not call them	`backend/app/api/` (no match for `forgot` or `reset`)

6. Rate limiters

Limiter	Key	Default	Scope	Source
General API	client IP and a SHA-256 prefix of the Authorization header (both must pass); `/api/*` only, preflights exempt	600/min (`API_RATE_LIMIT_PER_MINUTE`, 0 = off)	in memory, per process	`backend/app/main.py:119-135`, `backend/app/config/ratelimit.py:35-61`, `config.py:163`
Login failures	`email	IP`	5 failures / 900 s	in memory, per process	
Sign-up	IP; email	5/h; 3/h	in memory, per process	`main.py:100-101`
Studio AI	`user:<id>`	20/min	in memory, per process	`main.py:102-103`, `backend/app/api/studio_ai.py:63-67`
Studio AI daily quota	per user per UTC day, stored in `api_usage` (survives restarts)	200 text, 50 images	database	`backend/app/services/studio_ai.py:279-300`, `config.py:151-152`
Voice	`user:<id>`	30/min	in memory, per process	`main.py:104`, `backend/app/api/voice.py:55-58`

`RequestLimiter` documents its own limit: "Single process only: with several workers each one counts separately" (`ratelimit.py:35-37`).

The client IP comes from `request.client.host`. In the container, uvicorn trusts `X-Forwarded-For` only from `FORWARDED_ALLOW_IPS` (default `127.0.0.1`) (`backend/docker-entrypoint.sh:20-21`), while the frontend nginx proxy sends the header from another container (`frontend/nginx/default.conf.template:60-70`). Unless `FORWARDED_ALLOW_IPS` is set, all users coming through nginx would appear as one IP, so per-IP limits (including sign-up) would be shared by everyone. **Needs confirmation:** how the client IP is derived on Vercel (no proxy-header handling exists in `backend/api/index.py` or `main.py`); if it is the platform's proxy address, per-IP limits are effectively global there too.

7. CORS

`CORSMiddleware` with an explicit origin list from `CORS_ORIGINS` (default `http://localhost:5173`), optional `CORS_ORIGIN_REGEX`, `allow_credentials=False`, all methods and headers allowed (`backend/app/main.py:160-167`, `backend/app/config/config.py:30-31,192-194`). On the single-origin deployments (Vercel rewrites, nginx proxy) the browser makes same-origin calls and CORS is not exercised (`backend/.env.example:93-94`).

8. Security headers and CSP

Headers sent on every path on Vercel (`vercel.json:88-117`) and by both nginx servers (`frontend/nginx/default.conf.template:14-20` , `studio/nginx/default.conf:13-18`):

Header	Value
`X-Content-Type-Options`	`nosniff`
`X-Frame-Options`	`DENY`
`Referrer-Policy`	`strict-origin-when-cross-origin`
`Permissions-Policy`	`camera=(), microphone=(self), geolocation=(), payment=()` on Vercel and in the frontend nginx (`vercel.json:106` , `frontend/nginx/default.conf.template:17`); `microphone=()` in the standalone Studio nginx (`studio/nginx/default.conf:16`)
`Strict-Transport-Security`	`max-age=31536000; includeSubDomains`
`Content-Security-Policy`	see below

Deployment	CSP differences	Source
Vercel	`default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; font-src 'self' data: https://fonts.gstatic.com; img-src 'self' data: blob: https:; connect-src 'self'; object-src 'none'; base-uri 'self'; form-action 'self'; frame-ancestors 'none'`	`vercel.json:112-115`
Frontend nginx	Same, but `img-src 'self' data: blob: ${API_URL}` and `connect-src 'self' ${API_URL}` (API address substituted at container start)	`frontend/nginx/default.conf.template:20`
Studio nginx	Same as Vercel (`img-src ... https:` , `connect-src 'self'`)	`studio/nginx/default.conf:18`

Notes:

- `'unsafe-inline'` is allowed for styles only; scripts are `'self'` only.
- The microphone is allowed for the page's own origin (`microphone=(self)`) because the voice assistant uses it; the standalone Studio container still sends `microphone=()` (Studio has no voice feature; in Compose the frontend's headers apply to proxied Studio pages).
- In Compose, frontend nginx proxies `/studio/` and hides Studio's own security headers so they are not duplicated (`frontend/nginx/default.conf.template:38-56`); the frontend's CSP (which restricts `img-src` to `API_URL`) therefore applies to proxied Studio pages. **Needs confirmation:** whether Studio shows any image from an `https:` host other than `API_URL` that this stricter CSP would block in Compose.
- Static assets are cached for a year, `/config.js` is `no-cache` (`vercel.json:118-144`).
- The API process itself adds no security headers; they come from Vercel or nginx. The API port in Compose is published only on `127.0.0.1` (`docker-compose.yml:85-86`).

9. SSRF protection (outbound requests)

Both fetchers share `validate_public_url` (`backend/app/core/pipeline/url_safety.py:63-92`):

Rule	Source
Only `http` and `https`	`url_safety.py:66-67`
No credentials in the URL	`url_safety.py:68-69`
Only ports 80 and 443	`url_safety.py:19,77-78`
Rejects `localhost` , `*.localhost` , `*.local` , `*.internal`	`url_safety.py:80-81`
IP literals are checked; hostnames are resolved and **all** addresses must be public (global, not private, loopback, link-local, multicast, reserved, unspecified; IPv4-mapped IPv6 unwrapped)	`url_safety.py:45-60,82-91`
Redirects are followed manually (limit `URL_CHECK_MAX_REDIRECTS` , 3) and every hop is re-validated	`url_safety.py:130-155`
The connection is pinned to the validated IP (Host header and SNI keep the hostname) so DNS cannot change between check and connect	`url_safety.py:134-146`
Environment proxies ignored (`trust_env=False` , `follow_redirects=False`)	`url_safety.py:95-96`
Response read capped (`URL_CHECK_MAX_BYTES` , 65536)	`url_safety.py:147-150`

`UrlChecker` never raises; every failure becomes an "unreachable" result (`url_safety.py:117-128`).

`SafeFetcher` (candidate logos and icons) adds a content-type allow-list (png, jpeg, webp, gif), `Content-Length` and streaming size caps (default 2 MB), a magic-byte check, and a 25-million-pixel decompression-bomb guard; it only raises `FetchError` (`backend/app/core/pipeline/image_fetch.py:20-106`). It is limited per scan by `SCAN_MAX_LOGO_FETCHES` (25) and only runs for live data (`backend/app/config/config.py:116,120`).

Other outbound URLs: the SearXNG base URL goes through `validate_public_url` unless the operator sets `SEARXNG_ALLOW_PRIVATE_HOSTS` (then only scheme and no-credentials checks apply) (`backend/app/core/collectors/searxng.py:79-93`). Candidate URLs themselves are not visited: Safe Browsing receives the URL string only (`backend/app/core/providers/safe_browsing.py:1-3` , `docs/DECISIONS.md:140`).

10. Upload and import validation

Input	Validation	Source
Brand logo, extra logos	Read at most cap + 1 byte (`LOGO_MAX_BYTES` , 2 MB); PNG/JPEG/WebP by magic bytes; Pillow `verify` then load; at least 256x256; roughly square (0.8 to 1.25); 25 MP guard; **re-encoded** (drops metadata and trailing bytes); stored under `logos/brand_<id>/<hash>.<ext>` , never the user's filename	`backend/app/api/brands.py:210-228` , `backend/app/core/pipeline/logo.py:11-13,44-80`
Candidate images and app icons	Same approach, plus a maximum side; hashed and re-encoded	`backend/app/api/candidates.py:75-95` , `backend/app/core/pipeline/candidate_images.py:36-60`
Studio item images	Same image processor; files stored under `studio/user_<id>/` ; JSON `data` limited in size; at most 500 items per user	`backend/app/api/studio.py:34,60-100`
CSV / JSON candidate import	Size cap `IMPORT_MAX_BYTES` (1 MB), row cap `IMPORT_MAX_ROWS` (500), strict schema, per-row errors, `dry_run`	`backend/app/api/candidates.py:51-72` , `backend/app/core/pipeline/candidate_intake.py:170`
Served files	Fixed media type from the stored format, `X-Content-Type-Options: nosniff` , SHA-256 ETag	`backend/app/api/brands.py:242-259` , `backend/app/api/candidates.py:109-115` , `backend/app/api/common.py:13-15`
Audit CSV export	Cells starting with `= + - @` or tab/CR get a leading `'` (formula-injection guard)	`backend/app/api/audit.py:70-72`
File paths in the store	Absolute paths, `..` , backslashes, NUL bytes rejected; resolved path must stay under the root	`backend/app/core/storage.py:45-52,59-64`
Platform limit	Vercel request body limit 4.5 MB	`docs/DEPLOY_VERCEL.md` (platform facts table)

11. Secrets management

Secrets are read only from the environment (or an untracked `.env`), never stored in the database, and never returned by an endpoint. The names, where each is read:

Secret (name only)	Read in
`JWT_SECRET`	`backend/app/config/config.py:40` -> `security.py:35,48`
`ADMIN_PASSWORD` , `DEMO_PASSWORD` , `DEMO_ADMIN_PASSWORD`	`config.py:46,48,50` -> `backend/app/services/seed.py`
`DATABASE_URL` , `MIGRATION_DATABASE_URL` (contain a password)	`config.py:28,37` -> `backend/app/db/session.py`
`REDIS_URL`	`config.py:110` -> `backend/app/services/jobs.py:63-64` , `backend/app/worker.py:95`
`CRON_SECRET`	`config.py:38` -> `backend/app/api/cron.py:34`
`TAVILY_API_KEY`	`config.py:91` -> `backend/app/core/collectors/selection.py:83`
`GOOGLE_SAFE_BROWSING_API_KEY`	`config.py:132` -> `backend/app/core/providers/safe_browsing.py:177` (header `x-goog-api-key` , never in the URL)
`GEMINI_API_KEY` , `VOICE_GEMINI_API_KEY`	`config.py:71,145` -> `backend/app/core/providers/text/factory.py:31` , `backend/app/services/studio_ai.py:208` , `backend/app/services/voice.py`
`HUGGINGFACE_API_TOKEN`	`config.py:74` -> `factory.py:33`
`GROQ_API_KEY` , `POLLINATIONS_API_KEY`	`config.py:139-140` -> `backend/app/services/studio_ai.py:94,180,230` (sent as `Authorization: Bearer`)
`SENTRY_DSN`	`config.py:171` -> `backend/app/config/observability.py:79-102`

Handling:

- Typed as `SecretStr` for the API keys, `REDIS_URL` , `SENTRY_DSN` , `CRON_SECRET` (`config.py:38,71,74,91,110,132,139,140,145,171`). `JWT_SECRET` , `ADMIN_PASSWORD` , demo passwords, `DATABASE_URL` are plain strings.
- **Log masking.** `configure_logging(level, settings.secret_values)` installs a filter that replaces exact secret values with `[REDACTED]` , then also masks `password/secret/token/api_key/authorization=...` pairs, `Bearer ...` strings and JWT-shaped strings (`backend/app/config/logging.py:8-28,58-64` , `backend/app/main.py:60`). Secrets shorter than 4 characters are ignored (`logging.py:21`). Exceptions are redacted too (`logging.py:36-41`).
- **Gaps in the exact-value list.** `Settings.secret_values` includes `JWT_SECRET` , admin and both demo passwords, Tavily, `REDIS_URL` , Sentry DSN, Safe Browsing, Gemini, Hugging Face and the voice Gemini key (`config.py:196-202`). It does **not** list `GROQ_API_KEY` , `POLLINATIONS_API_KEY` , `CRON_SECRET` or `DATABASE_URL` . Those are still caught when they appear after a recognised key name or as `Bearer ...` , but not when printed bare. The same list feeds Sentry scrubbing (`observability.py:90-98`).
- **Sentry scrubbing.** Before sending: request body and cookies dropped, headers and query string redacted, user reduced to id, local variables removed, server name removed, `send_default_pii=False` , `include_local_variables=False` (`observability.py:46-76,90-98`).
- **Provider status endpoints** report only whether a key is configured (`backend/app/api/providers.py:30-43` , `backend/app/api/sources.py:155-156`).
- **Browser bundle.** Nothing from the environment is injected into the Studio bundle (`studio/vite.config.ts:7-8`); the only `VITE_*` values are the API URL, the optional demo fallback and dev redirect targets (`docs/configuration.md` , sections 3 and 4).
- **Tests** pop real keys from the environment so they cannot reach a test run (`backend/tests/conftest.py:17-19`); the e2e server ignores `.env` files (`backend/scripts/e2e_server.py:4-6`).
- Secret files are git-ignored: `.env` , `.env.*` , `*.env` except the `.env.example` templates (`.gitignore`).
- `backend/.env` was deliberately not read to produce this page.

12. AI provider key handling

- All AI keys live on the server; Brand Studio and the frontend call BrandShield endpoints only (`backend/app/api/studio_ai.py:1-3` , `studio/src/services/gemini.ts:1-6` , `studio/vite.config.ts:7-8`).
- Studio AI endpoints require a signed-in user, a per-user rate limit (20/min) and a daily quota; timeouts and bounded retries (timeouts, connection errors and 5xx only) apply to every outbound call (`backend/app/api/studio_ai.py:63-67` , `backend/app/services/studio_ai.py:1-12` , `config.py:143,151-156`).
- Chat uses Groq first and Gemini as a fallback; images use Pollinations (`backend/app/services/studio_ai.py:39-41,152`).
- Hosted text analysis of scan candidates is **off** by default (`SCAM_LLM_ENABLED=false`), is never run for sample data, and only the configured provider is used (`config.py:66-69` , `backend/app/core/pipeline/text_analysis.py:2-3,77`).
- Prompt-injection controls: candidate text is truncated, control characters and delimiter strings removed, placed between explicit markers declared as untrusted data; emails, phone numbers and UPI ids are replaced by placeholders before sending; no tools or URL fetching are offered; the answer must match a strict JSON schema and can only add capped points, never lower a score or touch the allowlist (`backend/app/core/providers/text/prompt.py:1-40`). Regression tests are named in `docs/SECURITY.md` .

13. Voice agent safety rules

The voice endpoint is `POST /api/voice` , signed-in users only, rate limited to 30/min per user (`backend/app/api/voice.py:55-58`). Rules enforced in `backend/app/services/voice.py` (docstring `voice.py:1-18` , constants `voice.py:34-50`):

Rule	Detail
Sensitive fields never reach the planner	The server drops snapshot elements whose label or name matches `pass(word
Navigation allow-list	Only fixed paths (`/dashboard` , `/brand` , `/social` , `/apps` , `/web` , `/investigations` , `/lookalike` , `/sources` , `/reports` , `/settings` , `/evaluation` , `/setup` , `/protect-brand` , `/studio/`) (`voice.py:34-37`)
Element ids validated	Actions must reference elements in the snapshot that was sent; option names must exist on that element
Destructive clicks need a spoken yes	Words such as delete, remove, block, pay, purchase, archive, discard, revoke, erase, deactivate (`voice.py:39`)
Submits need a request or a yes	submit, register, create, finish, publish, launch, send, save, confirm (`voice.py:40`)
Value size	Values capped (`MAX_VALUE = 500` , `voice.py:50`); request fields bounded (text 1000 characters, 200 snapshot elements, 50 routes, `backend/app/api/voice.py:40-52`)
No logging of content	Snapshots, values and the API key are never logged (`voice.py:17-18`)
Fast path without network	Simple commands ("open dashboard") use regex and rapidfuzz, no key needed (`voice.py:7-14`)
Model	Gemini with function calling only after the fast path fails; `VOICE_GEMINI_MODEL` , fallback model, 10 s timeout (`config.py:145-150`)

14. Postgres Row Level Security (Supabase)

Supabase exposes the `public` schema through its REST API, so two tables that hold uploaded content have RLS enabled with **no policy**, which blocks the API roles (`anon` , `authenticated`) completely. The backend connects as the table owner, which RLS does not apply to:

Table	Migration
`stored_files` (logos, candidate images, evidence snapshots when `STORAGE_BACKEND=database`)	`backend/alembic/versions/0009_stored_files.py:1-10,31`
`studio_items` (saved Studio items)	`backend/alembic/versions/0011_studio_items.py:1-9,34`

Migration 0010 notes "No new tables, so nothing to do for row level security" (`0010_brand_owner.py:9`). A search of `backend/alembic` and `backend/app` finds no other `ROW LEVEL SECURITY` statement. **Needs confirmation:** whether Supabase's REST/Data API is disabled or the other tables (`users` , `refresh_tokens` , `brands` , `detections` , `audit_log` , `settings` , ...) are otherwise protected from the `anon` / `authenticated` roles; the repo contains no RLS or grants for them, so if the Data API is enabled with default grants they could be exposed. Authorization for those tables relies entirely on the FastAPI layer.

15. Other controls worth knowing

Control	Detail	Source
Audit trail	Every mutating action writes an `audit_log` row (actor, action, target, detail); logins, logout, registration, scans, settings and provider changes are recorded; detections also keep an event timeline	`backend/app/services/audit.py` , `backend/app/api/auth.py:72,77,122,168` , `backend/app/api/settings.py:178-179`
Error handling	Unhandled errors return a generic 500 and log only the exception type	`backend/app/config/errors.py:30-32`
Cron endpoint	Inert without `CRON_SECRET` ; constant-time compare	`backend/app/api/cron.py:27-38`
Containers	Non-root user (uid 10001); `no-new-privileges` ; `cap_drop: ALL` on api and worker; ports bound to `127.0.0.1` ; secrets with `${VAR:?}` so Compose refuses to start without them	`backend/Dockerfile:25-26,41` , `docker-compose.yml:8,100-103,127-130`
Redis	Password via environment variable, not command line	`docker-compose.yml:57-60`
Database pooling safety	Small pool and TCP keepalives for hosted Postgres	`backend/app/db/session.py:19-32`
Scan safety	One active scan per brand enforced by a partial unique index	`scan_pipeline.py:178-183`
Soft deletion	Archived brands return 404 unless explicitly included	`backend/app/api/common.py:25`

16. Known risks and limitations

Each item is based on what the code does today.

#	Risk or limitation	Evidence
1	**No email verification.** Anyone can register any address they can type; an address proves nothing. No CAPTCHA, no password-reset flow, no account deletion	`docs/DECISIONS.md:202` ; `backend/app/api/auth.py:99-124` ; no `/auth/forgot` route though `frontend/src/api/auth.ts:41-46` defines client calls
2	**Rate limiters are in memory per process.** Each worker or serverless instance counts separately and a restart resets counts; on Vercel every instance has its own. Login, sign-up, Studio AI, voice and the general limit are all affected. Daily AI quotas are the exception (database)	`backend/app/config/ratelimit.py:1-3,35-37` , `backend/app/main.py:99-106`
3	**Client IP may be the proxy's.** Without `FORWARDED_ALLOW_IPS` set to the proxy's address, per-IP limits (including sign-up and login keys) can be shared by all users behind nginx; Vercel behaviour not verified in code	`backend/docker-entrypoint.sh:20-21` ; section 6
4	**Tokens in web storage.** Access and refresh tokens are in `localStorage` (or `sessionStorage`), readable by any script on the origin; the CSP allows only same-origin scripts, which reduces but does not remove the risk. Access tokens cannot be revoked before they expire (15 min)	`frontend/src/api/client.ts:59,77-98` ; `vercel.json:112-115`
5	**Admin pages are hidden, not protected, for reads.** `GET /api/settings` , `GET /api/sources` , `GET /api/settings/data-mode` and `GET /api/providers` are open to any signed-in user; `PATCH /api/sources/{id}` and `POST /api/sources/{id}/retry` are open to any analyst (including self-registered users) and change or spend shared, global state	section 2.4; `backend/app/api/sources.py:125-231`
6	**Shared global state and budgets.** Settings, data mode, provider, source rows and Tavily / Safe Browsing counters are one pool for all accounts; one heavy account can use up the shared Tavily budget	`docs/DECISIONS.md:208` ; `backend/app/services/app_settings.py:59-66`
7	**Public demo admin is a real admin.** Its credentials are public; it is blocked from live data, the AI provider, the webhook and others' audit rows, but can still change global monitoring settings and read detailed health	section 4
8	**Google Safe Browsing terms.** The Lookup API v4 is for non-commercial use; a revenue-generating product should use Web Risk (paid). Candidate URLs are sent to Google. "Clean" means "not in Google's lists", not "safe"	`backend/app/core/providers/safe_browsing.py:15-17,3` , `docs/DECISIONS.md:139`
9	**Shared Supabase database. Needs confirmation:** the repo does not say whether development, e2e and production share one Supabase project. Evidence that local runs may use the hosted database: `docs/DECISIONS.md:166` ("Supabase in Tokyo costs 130-500 ms per query" while tuning local speed) and `backend/.env.example:81-86` pool settings. Tests use in-memory SQLite or a throwaway Postgres schema (`backend/tests/conftest.py:12` , `.github/workflows/ci.yml:90-93`)	
10	**RLS only on two tables.** Other tables rely on the API layer; see section 14	`0009_stored_files.py:31` , `0011_studio_items.py:34`
11	**Secret masking is not exhaustive.** Groq, Pollinations, `CRON_SECRET` and `DATABASE_URL` are missing from the exact-value list used for logs and Sentry	`backend/app/config/config.py:196-202`
12	**No MFA / SSO, no account lockout beyond login failures, no malware scanning of uploads, no WAF, no dependency scanning in CI, no independent security review**	`docs/SECURITY.md` "Not implemented"; `.github/workflows/ci.yml` has no such job
13	**Encryption at rest** is whatever the database or disk provides; evidence files may be stored as database rows	`backend/app/core/storage.py:96-130`
14	**Public API docs** (`/docs` , `/openapi.json`) are not disabled	`backend/app/main.py:94`
15	**Hosted-model and hash similarity are hints, not proof**; a person confirms every detection. Sample data is synthetic and is labelled as such	`backend/app/core/engine/risk_engine.py:19-22`
16	**Notifications are stored preferences only**; nothing is sent (delivery `not_implemented`), so a configured webhook URL is never called today	`backend/app/api/settings.py:117-119`
17	**Deployment unverified on Vercel.** Services are Beta and the setup has never been deployed per its own guide; some behaviours (IP handling, cron placement) are unconfirmed	`docs/DEPLOY_VERCEL.md:9-12`
18	**CI never executed.** The workflow header says it was written and YAML-checked locally but not run	`.github/workflows/ci.yml:3-5`

16.1 Differences from `docs/SECURITY.md`

`docs/SECURITY.md` says	Code says
"Tenant isolation ... NOT implemented" (single-tenant)	Implemented through `brands.owner_id` (section 3); settings and budgets remain global
Roles: admin also manages "users"	No user-management endpoint exists in `backend/app/api/`
No mention of demo accounts, sign-up, Studio AI, voice	Present (sections 4, 5, 12, 13)
"Fetches ... (official account/URL checks, image fetch, snapshots)" go through `url_safety` / `SafeFetcher`	Consistent for the fetchers found (`url_safety.py` , `image_fetch.py`); screenshots are off by default (`config.py:121`)

DOCS/FRONTEND.MD

BrandShield Frontend

Plain-language summary

The frontend is the web application people use to protect a brand: a marketing page, sign-in and sign-up screens, a first-run setup wizard, and a set of monitoring screens (dashboard, social, apps, web, investigations, look-alike lab, reports) plus administrator-only screens (source health, settings, model evaluation). It is a single-page React app: after the first load, moving between screens does not reload the page. It talks to the backend through one small API layer that attaches the signed-in user's token, refreshes it when it expires, and remembers the last answer of each screen so screens reappear instantly. A separate app, Brand Studio, lives in `studio/` and shares the same login.

1. Stack and folder map

Item	Detail	Source
Framework	React 19, React Router 7, TypeScript 6, Vite 8, lucide-react icons, d3-geo (map)	`frontend/package.json:16-39`
Entry	`main.tsx` applies the saved theme, then renders `<App />` in `StrictMode`	`frontend/src/main.tsx:1-13`
Folders	`src/pages` (screens + per-page CSS), `src/components/{layout,common,charts,monitor,voice}`, `src/hooks`, `src/api`, `src/utils`, `src/styles`, `src/landing`, `src/test`	`frontend/src/`
Not in this app	Brand Studio is in `studio/` (see `docs/features/brand-studio.md`)	

2. Pages and screens

Route table from `frontend/src/App.tsx:80-188`. "Layout" means wrapped in `AppLayout` (sidebar plus main panel).

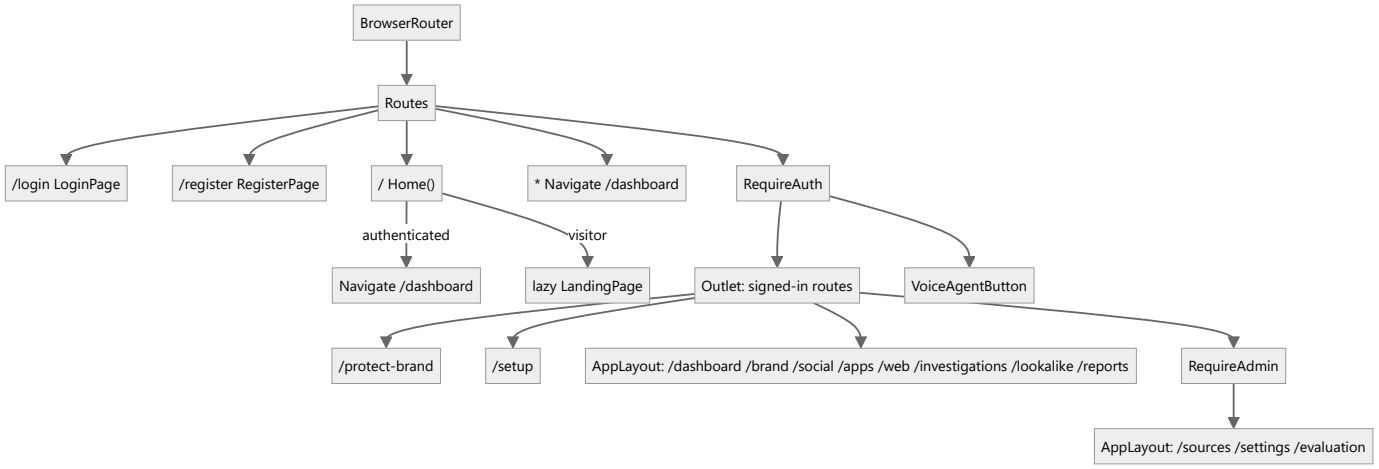
Route	Component	Layout	Who can open it	Notes
`/`	`Home()` then `LandingPage` (lazy)	none	Public; signed-in users are redirected to `/dashboard`	`App.tsx:26-36,183`; see `docs/features/landing-page.md`
`/login`	`LoginPage`	none	Public	Handles `?next=/studio/` via `safeStudioNext` (`pages/LoginPage.tsx:51-52`); demo credentials button (`e2e/demo-login.spec.ts`)
`/register`	`RegisterPage`	none	Public	Signs the new user in and goes to `/protect-brand` (`App.tsx:86`)
`/protect-brand`	`ProtectBrandPage`	none	Any signed-in user	Two cards: "Create a Brand" opens Studio (`openStudio()`), "Protect a Brand" goes to `/setup` (`/setup?new=1` when adding another brand) (`pages/ProtectBrandPage.tsx:8-18`). `/protect-your-brand` redirects here (`App.tsx:92`)
`/setup`	`SetupPage`	none	Any signed-in user at the route level; actions need analyst/admin on the API	3-step first-run wizard ("Set up brand protection in three steps"), "Skip setup for now" (`pages/SetupPage.tsx:312-319`)
`/dashboard`	`DashboardPage`	yes	Any signed-in user	"Dashboard"; starts scans (button disabled without write role)
`/brand`	`BrandProfilePage`	yes	Any signed-in user; editing needs `canWrite`	"Brand Profile": identity, accounts, apps, logos (`pages/BrandProfilePage.tsx:37,411-691`)
`/social`	`SocialMonitoringPage`	yes	Any signed-in user	"Social Media Monitoring" (shares `useMonitor` and `MonitorParts`)
`/apps`	`AppStoreMonitoringPage`	yes	Any signed-in user; scans need `canWrite`	"App Store Monitoring" (`pages/AppStoreMonitoringPage.tsx:24,86-87`)
`/web`	`WebThreatsPage`	yes	Any signed-in user	"Web Threats"
`/investigations`	`InvestigationsPage`	yes	Any signed-in user	"Investigations": case workflow
`/lookalike`	`LookalikeLabPage`	yes	Any signed-in user	"Look-alike Lab" live matcher (`useLookalike`, `useUrlTest`)
`/reports`	`ReportsPage`	yes	Any signed-in user	"Reports": draft, preview, approve, CSV/HTML export
`/sources`	`SourceHealthPage`	yes	**Administrator only** (`RequireAdmin`)	"Source Health"
`/settings`	`SettingsPage`	yes	**Administrator only**	"Settings" (sensitivity, schedule, webhook, data mode)
`/evaluation`	`EvaluationPage`	yes	**Administrator only**	"Evaluation" / "Model Evaluation"
`*`	redirect	none	any	`Navigate to="/dashboard"` (`App.tsx:184`)

Roles come from the account saved at sign-in: `admin`, `analyst`, `viewer` (`frontend/src/api/client.ts:68-75`). `useMe()` exposes `isAdmin` (`role === 'admin'`) and `canWrite` (`admin` or `analyst`); viewers can read but buttons that change data are disabled with a title such as "Your role can view but not start scans"

(`hooks/useMe.ts:11-34` , `pages/AppStoreMonitoringPage.tsx:86-87`). The real enforcement is on the API (`require_writer` , `require_admin`).

Sidebar menu labels differ from page titles: Overview, Brand Registry, Social Threats, App Threats, Web Threats, Investigations, Look-alike Lab, Source Health (admin), Reports, Model Evaluation (admin), then Brand Studio, then bottom items Settings (admin) and Support (`components/layout/Sidebar.tsx:60-190`).

3. Routing



- **`BrowserRouter` with `Routes`** in `App()` (`App.tsx:80-188`).
- **`RequireAuth`** : if `session.isAuthenticated()` is false it redirects to `/login` with `state.from` (`App.tsx:69`). It subscribes to the `scammind:unauthorized` event (fired when a refresh is rejected) and then calls `resetBrand()` and navigates to `/login` (`App.tsx:59-66`). It renders `<Outlet />` plus a single `<VoiceAgentButton />` so the voice agent keeps its state across pages (`App.tsx:71-77`). There is deliberately no offline or sample session: "without a token the user must sign in" (`App.tsx:68`).
- **`RequireAdmin`** : reads the role saved at sign-in; only if missing does it call `GET /api/me` once; non-admins are redirected to `/dashboard` ; while the role is unknown it renders nothing (`App.tsx:39-54`).
- **Lazy landing**: `lazy(() => import('./landing/LandingPage'))` inside `<Suspense fallback={null}>` (`App.tsx:26-36`). All other pages are imported eagerly (`App.tsx:10-24`).
- **Full-page jumps**: Brand Studio is not part of this router. The sidebar button is a normal `<a href="/studio/">` and the voice agent uses `window.location.assign` for paths beginning `/studio` (`Sidebar.tsx:156` , `hooks/useVoiceAgent.ts:199`).
- **Production rewrites**: SPA fallback for every non-asset path in `vercel.json:15-24` (frontend service) and nginx for Docker (`frontend/nginx/default.conf.template`).

4. Layout

`AppLayout` (`components/layout/AppLayout.tsx:11-39`) renders `.app-container` with `<Sidebar />` and `<main className="main-panel">` . It clones each child and injects `onOpenMobileMenu` so a page's `Header` can open the mobile sidebar. `main` has `key={epoch}` : pages are remounted (so they reload) only when the person switches to **another** brand, not when the first brand finishes loading (`AppLayout.tsx:13-17,24`).

`Sidebar` (`components/layout/Sidebar.tsx`):

1. Logo (`BrandMark` , `Wordmark`) linking to `/dashboard` (`:41-44`).
2. Search box with `data-voice="search"` (`:49`).
3. `BrandSwitcher` (`:55`).
4. Main menu `NavLink` s; admin-only links hidden unless `session.user()?.role === 'admin'` (`:32,65-91`).
5. **Studio CTA button**: `<a href={STUDIO_URL} className="studio-cta" data-voice-row>` "Brand Studio / Names, logos & palettes" (`:155-165`). `STUDIO_URL` is `VITE_STUDIO_URL` or `/studio/` (`utils/studio.ts:3`).
6. Bottom: Settings (admin) and Support (`alert('Support desk: support@brandshield.ai')`) (`:170-189`).

`BrandSwitcher` (`components/layout/BrandSwitcher.tsx`): shows the active brand, lists all brands from `GET /api/brands` when opened, and offers "Add another brand" (navigates to setup). Choosing a brand calls `POST /api/brands/{id}/select` , then `clearQueryCache()` (every cached page answer belonged to the previous brand), then `refreshBrand()` (`BrandSwitcher.tsx:17-61`).

`Header` (`components/layout/Header.tsx`): each page renders its own `<Header title subtitle actions onOpenMobileMenu />` . It shows a "Sample data" badge when the API said the data is sample (`useIsSample`), the `ThemeToggle` , a notifications bell, and a user chip with sign-out (`:55-90`). Sign-out calls `sm.logout()` (revokes the refresh token), `resetBrand()` , `resetSampleFlag()` and navigates to `/login` (`:26-33`).

Pages without `AppLayout` : `/login` , `/register` , `/protect-brand` , `/setup` .

5. State management

There is **no global store library** (no Redux, Zustand or React Query). State comes from five mechanisms:

Mechanism	What it holds	Source
Session in `api/client.ts`	Access and refresh tokens and the user, in `localStorage` (or `sessionStorage` when "keep me signed in" is off) under `scammind.auth` and `scammind.user`	`api/client.ts:59-136`
`useBrand` external store	The workspace's active brand, shared by sidebar, setup wizard and pages	`hooks/useBrand.ts`
`useApi` + `queryCache`	Per-request data with stale-while-revalidate	`hooks/useApi.ts`, `api/queryCache.ts`
`useMe`	Current user and role flags	`hooks/useMe.ts`
Local component state and small hooks	Forms, tabs, filters; `usePref` for harmless UI prefs under `scammind.ui.*`	`hooks/usePref.ts`, `hooks/useScan.ts`, `useMonitor.ts`, `useLookalike.ts`, `useUrlTest.ts`

`useBrand` is a module-level store used with `useSyncExternalStore`. Loading: `GET /api/me` (saves the user, reads `default_brand_id`) then `GET /api/brands/{id}`. `missing: true` when there is no brand or a 404. Concurrent calls share one request (`inflight`). `resetBrand()` clears it (sign-out, tests); `setBrandCache(brand)` replaces it after a mutation; `refreshBrand()` reloads (`hooks/useBrand.ts:21-66`).

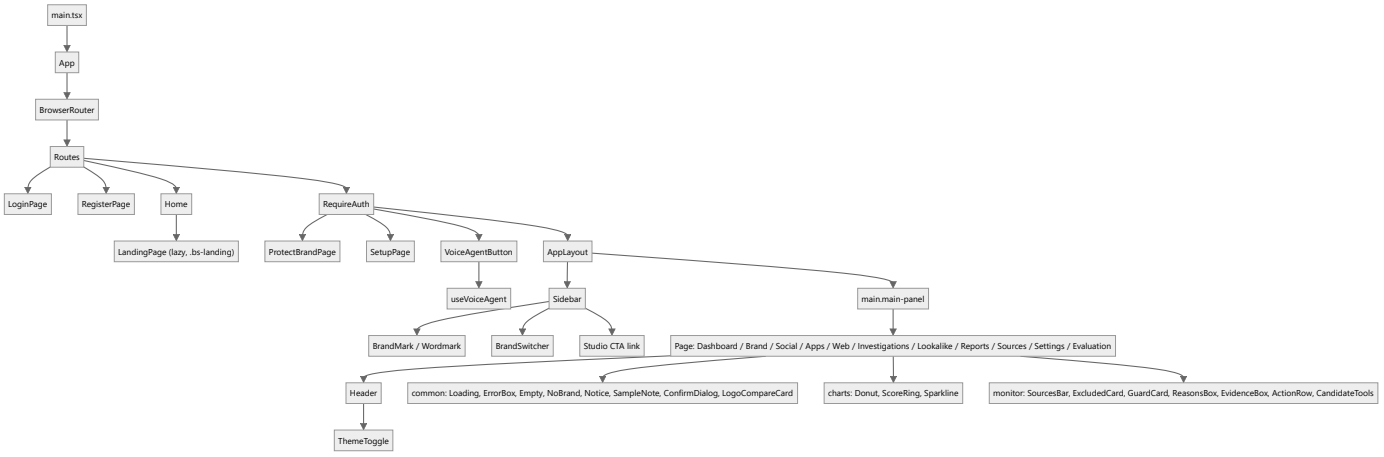
`useApi(fn, deps, enabled)`: the cache key is `userId|fn.toString()|JSON(deps)`. If the key is cached, the data appears immediately and a background request refreshes it (`refreshing: true`, no loading screen). The request is cancelled with `AbortController` when the component unmounts, deps change or a newer load starts, so a slow answer never overwrites a newer one. `setData` updates state and the cache (optimistic updates). Returns `{data, loading, refreshing, error, reload, setData}` (`hooks/useApi.ts:5-92`).

`queryCache`: a `Map` with at most 150 entries (oldest evicted); a `null` value deletes the entry (`api/queryCache.ts:5-25`). It is cleared by `session.clear()` (so the next person never sees another user's pages), by the brand switcher and by the test setup (`api/client.ts:120-123`, `components/layout/BrandSwitcher.tsx:48`, `test/setup.ts:8`).

`useMe`: starts from the saved user, then confirms with `GET /api/me` and saves it back (`hooks/useMe.ts:19-34`).

Other hooks: `useScan` starts a scan and polls `GET /api/jobs/{id}` (409 attaches to the running job; gives up after a maximum wait) (`hooks/useScan.ts:1-4`); `useMonitor` shares logic of Social and App pages with optimistic review actions and rollback (`hooks/useMonitor.ts:1-2`); `useLookalike` and `useUrlTest` debounce typing and cancel the previous call (`hooks/useLookalike.ts:1-3`). `api/sample.ts` derives a global "this is sample data" flag from API answers (`is_sample`, `data_mode`) through `onResponse` taps, never guessed (`api/sample.ts:1-3`).

6. Component hierarchy



Shared components (`frontend/src/components/`): `common/States.tsx` (`Loading`, `ErrorBox`, `Empty`, `NoBrand`, `Notice`, `SampleNote`), `common/ConfirmDialog.tsx` (`ConfirmDialog`, `RetryCostDialog`), `common/LogoCompareCard.tsx`, `common/BrandMark.tsx`, `charts/Donut|ScoreRing|Sparkline.tsx`, `monitor/MonitorParts.tsx` and `CandidateTools.tsx` (used by Social and App pages), `layout/*`, `voice/VoiceAgentButton.tsx`.

7. Styling

Piece	Detail	Source
Tokens	CSS variables on `:root` in `styles/tokens.css` (`--bg-page`, `--bg-shell`, `--bg-card`, `--text-main`, `--accent`, `--btn-primary-start/end`, `--red`, radii, ...); `styles/index.css` imports `tokens.css` then `light-mode-patches.css`	`styles/index.css:1-2`, `styles/tokens.css:1-30`
Per-page CSS	One `.css` file next to each page (for example `DashboardPage.css`), imported by the component (`ProtectBrandPage.tsx:6`) plus `components/layout/Layout.css`, `common/States.css`, `voice/VoiceAgentButton.css`	`frontend/src/pages/*.css`
Theme toggle	`utils/theme.ts`: preference stored in `localStorage` key `scammind.ui.theme`, default dark; `applyTheme` sets `data-theme` on `<html>`; applied before first render in `main.tsx`; toggled by `ThemeToggle` (in `Header`)	`utils/theme.ts:1-20`, `main.tsx:7`, `components/layout/ThemeToggle.tsx:6-27`
Light-mode patches	Page CSS uses hard-coded `rgba(255,255,255,...)`; `light-mode-patches.css` overrides those rules, all scoped to `:root[data-theme='light']`, so dark mode is untouched	`styles/light-mode-patches.css:1-20`
Landing page	Separate scoped CSS under `.bs-landing` with its own tokens	`landing/landing.css`

8. API integration

`api/client.ts` (HTTP core):

- Base URL precedence: runtime `window.__SCAMMIND_CONFIG__.API_URL` (from `/config.js`, written at container start, no rebuild needed) then `VITE_API_URL` then `''` (same origin) in production or `http://127.0.0.1:8000` in development (`api/client.ts:16-27`; `public/config.js`).
- `request<T>(path, {method, json, form, signal, text, blob, anonymous, fallback})` adds `Authorization: Bearer <access>` unless `anonymous` (`api/client.ts:271-287,296-347`).
- 401 handling: refresh **once** (single flight per tab and a Web Lock `scammind-refresh` across tabs and across BrandShield and Studio), store the rotated pair and retry the original request once. A rotated refresh token must never be sent twice. 4xx on refresh clears the session and fires `scammind:unauthorized`; a network failure or 5xx keeps the session (`api/client.ts:227-262,314-347`).
- Errors become `ApiError(status, message)`; `message` is the server's `detail` (or the 422 `msg` items joined by "; "); `fields` maps `loc[-1]` to message; `retryAfter` from the `Retry-After` header; network failures have `status 0`, `network: true` (`api/client.ts:31-51,187-224`).
- `downloadFile` and `checkHealth` helpers (`api/client.ts:370-382`).

`api/sm.ts`: the typed endpoint catalogue (`export const sm = {...}`): auth (`login`, `register`, `demo`, `logout` from `api/auth.ts`), `me`, `voice` (`POST /api/voice`), brands (`brands`, `selectBrand`, `brand`, `createBrand`, `updateBrand`, logos, accounts, apps), `dashboard`, scans and jobs, detections and review actions, candidates and imports, providers, sources, data mode, look-alike tests and logo compare, settings, reports (create, approve, preview, download), evaluation (`api/sm.ts:66-181`). Types live in `api/types.ts`; formatting helpers in `api/format.ts`.

Vite proxy (development): `frontend/vite.config.ts:8-14` proxies `/studio` (WebSocket enabled, default target `http://127.0.0.1:3000`, override `STUDIO_DEV_URL`) and `/api` (default `http://127.0.0.1:8000`, override `STUDIO_API_PROXY`). BrandShield itself uses `VITE_API_URL`; the `/api` proxy exists so Studio can use relative URLs on this origin.

9. Build tooling and tests

Task	Command (from `frontend/package.json:6-14`)	Detail
Dev server	`npm run dev` (`vite`)	port 5173 by default
Build	`npm run build` (`tsc -b && vite build`)	type-checks first; output in `dist/`
Typecheck	`npm run typecheck`	`tsconfig.app.json`: ES2023, `noUnusedLocals`, `noUnusedParameters`, `erasableSyntaxOnly`, `verbatimModuleSyntax` (`tsconfig.app.json:1-27`)
Lint	`npm run lint` (`oxlint`)	`.oxlintrc.json`: plugins react, typescript, oxc; `react/rules-of-hooks` error, `react/only-export-components` warn
Unit and component tests	`npm test` (`vitest run`)	jsdom, `testTimeout: 20_000` (form tests type passwords key by key), `css: false`, `restoreMocks`, setup in `src/test/setup.ts` (`vite.config.ts:15-22`)
End-to-end	`npm run test:e2e` (`playwright test`), `npm run e2e:install`	Chromium

Unit tests are next to the code: `App.test.tsx`, `vercelConfig.test.ts`, `api/*.test.ts` (base URL, client, format), `hooks/*.test.tsx` (`useApi` cache, `useLookalike`, `useScan`), `utils/*.test.ts` (brand examples, password policy, studio, theme, voice), `pages/*.test.tsx` (Investigations, Login, Login demo, ProtectBrand, Register, Setup, SourceHealth, WebThreats), `components/common/LogoCompareCard.test.tsx`, `landing/LandingRoute.test.tsx`. Helpers: `test/helpers.ts` (`jsonResponse`, `mockFetch`, `signIn`).

Playwright (`frontend/playwright.config.ts`): starts three servers: the REAL backend via `backend/scripts/e2e_server.py` (sample collector, temp SQLite database, no Tavily key, no network) on port 8765 (`E2E_API_PORT`), the BrandShield Vite dev server on 5174 (`E2E_WEB_PORT`), and Brand Studio's dev server on 3100 (`E2E_STUDIO_PORT`). One worker, serial, no retries; secrets are generated per run in memory (`playwright.config.ts:12-33,35-95`).

Spec (`frontend/e2e/`)	What it proves
`smoke.spec.ts`	Wrong-password login and guarded routes; setup (create brand, logo, first scan); brand profile; dashboard scan in sample mode; social review actions with rollback; apps; look-alike lab; source health; reports with downloads (`smoke.spec.ts:56-325`)
`visual.spec.ts`	Captures every page at 1440 px into `e2e/screenshots/` (git-ignored); asserts each renders with data (`visual.spec.ts:10-25`)
`demo-login.spec.ts`	"Use demo credentials" fills the form; demo account signs in as non-admin; guarded demo administrator
`register.spec.ts`	Create account, password policy messages, duplicate email, lands on `/protect-brand` ; "Create a Brand" goes to `/studio/` (Studio intercepted), "Protect a Brand" goes to `/setup`
`protect-hover.spec.ts`	Hover styling of the "Create a Brand" card
`studio-flow.spec.ts`	Whole cross-app flow: sign up, open Studio on the same login, create a brand there, see it in BrandShield, and a second account cannot see it; sign-out in Studio returns to `/studio/` after login
`multi-brand.spec.ts`	Several brands per account, Studio remembers inputs, saving a large picture to Collections, "Back to BrandShield", "Add another brand" opens an empty setup form
`voice.spec.ts`	Fake speech recognition: off until pressed, navigate and fill by voice, stays on across pages, releases the mic when off; friendly message where recognition is missing

Helper `e2e/png.ts` generates test pictures (`makePng`).

---

10. Deployment pointers

- Vercel: the `frontend` service is a Vite app with an SPA rewrite that skips `assets/` and file extensions (`vercel.json:15-24`); security headers (including CSP `connect-src 'self'` , so the SPA must use the same origin for the API, hence the empty `BASE_URL` default in production) apply to all paths (`vercel.json:96-115`).
- Docker: `frontend/Dockerfile` and `frontend/nginx/` (runtime config script writes `config.js` from `API_URL` ; `location /studio/` proxies to the studio container) (`api/client.ts:17` , `frontend/nginx/default.conf.template:35-60`).
- Existing docs `docs/FRONTEND_CONTRACT.md` and `docs/STUDIO.md` were not used as a source for this page; verify them against the code above before relying on them.

Needs confirmation: the exact set of props and layout (Header actions) of each individual page was summarised from titles and imports rather than from a full read of every page component; the route and role behaviour above is from `App.tsx` , `Sidebar.tsx` and `useMe.ts` and is verified, but per-page content descriptions are brief.

DOCS/BACKGROUND-JOBS-AND-INTEGRATIONS.MD

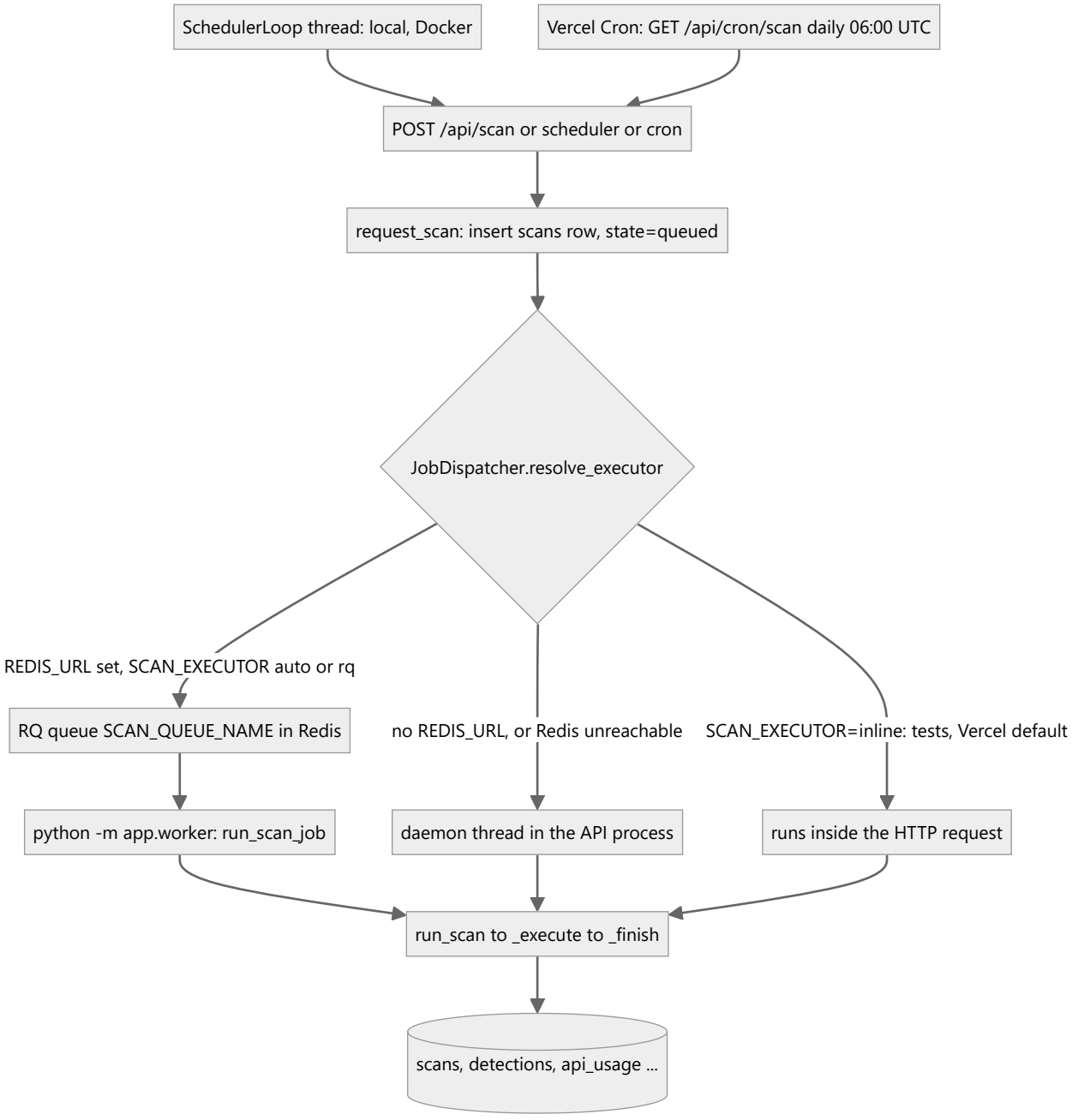
Background jobs and third-party integrations

Plain-language summary

BrandShield does some work in the background (running scans, running them on a timer, a daily cron call on Vercel) and talks to a handful of outside services (web search, app stores, Google, AI models, error reporting, databases). This page lists every one of them: what it is for, where it lives in the code, which environment variable names switch it on, what happens when it is down, and how its cost or quota is kept under control. Values of keys are never written here: only variable names. The golden rule of the design is that **an outside service failing must never break a scan or a page**: the scan ends "partial", the signal is shown as "unavailable" (never as "clean"), and everything else still works.

All paths are relative to the repository root (`ScamMind AI/`). Config is declared in `backend/app/config/config.py` (class `Settings` , env names are the upper-case field names, `case_sensitive=False` , a blank value counts as unset: `config.py:20-24`).

1. Execution modes at a glance



Mode	Selected when	Code	Notes
RQ worker	`REDIS_URL` set and `SCAN_EXECUTOR` is `auto` (default) or `rq`	`backend/app/services/jobs.py:23-30,59-67` ; worker `backend/app/worker.py`	Docker compose runs a separate `worker` service with `SCAN_EXECUTOR: rq` (`docker-compose.yml` , x-server-env and service `worker`)
Thread	no Redis, or Redis unreachable when queuing (logged, then falls back)	`jobs.py:46-52`	local dev works without Redis. The thread dies with the API process; a scan stuck `running` is failed after `SCAN_STALE_MINUTES` (30) by `fail_stale_scans` (`backend/app/core/pipeline/scan_pipeline.py:117-125`)
Inline	tests; on Vercel unless `SCAN_EXECUTOR` is set explicitly	`jobs.py:40-42` ; Vercel defaults in `config.py:175-190`	the HTTP response contains the finished scan; bounded by the function duration (`maxDuration: 300` in `vercel.json`)

2. Scheduler, cron and worker

2.1 SchedulerLoop (local and Docker)

- `SchedulerLoop` (`backend/app/services/scheduler.py:192-218`) is a daemon thread started in the FastAPI lifespan only when `SCHEDULER_ENABLED` is true (default true) and `APP_ENV != "test"` (`backend/app/main.py:84-89`); it is stopped on shutdown (91-92). It sleeps `SCHEDULER_POLL_SECONDS` (default 60, minimum 5, line 197) between passes.
- Every pass runs `scheduler_tick` (scheduler.py:147-189): for each non-archived brand (oldest first) it reads the schedule inputs, asks the pure function `decide_schedule` (52-89) and, if due, calls `request_scan(... scope="all", user=None)` which records `started_by = "scheduler"` . It never raises (a failure is logged and returns "Scheduler error (see log)", 187-189). At most `SCHEDULER_MAX_BRANDS_PER_RUN` (default 5) scans are started per pass (178-179).
- The interval comes from `SCAN_INTERVAL_MINUTES` (env wins, `0` = off) or the Settings page (`off | 5m | 15m | 1h | 1d` , default `1d`) (`backend/app/services/app_settings.py:15-16,19-24,79-83`).
- Brands with no official account or app are skipped to avoid spending credits ("Waiting for the first official account or app", scheduler.py:92-95,168-169).
- **Credit-aware stretching and pausing** (scheduler.py:1-13,68-84): when Tavily is part of the live chain, `cost = TAVILY_MAX_QUERIES_PER_SCAN x credit_cost(depth)` , `reserve = one scan` , `usable = remaining - reserve` ; if `usable < cost` scheduled scans pause until the month resets (manual scans still work), else the interval is stretched to `days_left x 24h / affordable_scans` . The interval is never shortened.
- Status for the UI: `schedule_status` (124-144), exposed in `GET /api/scan/status` and `GET /api/sources` (`backend/app/api/scans.py:56-80` , `backend/app/api/sources.py:139,148-149`).

2.2 Vercel Cron (`/api/cron/scan`)

Aspect	Detail
Schedule	`{"path": "/api/cron/scan", "schedule": "0 6 * * *"}` in the root `vercel.json` (`crons`), daily 06:00 UTC. Vercel Hobby: at most once a day, fired anywhere within the hour (`docs/DEPLOY_VERCEL.md:43-44`)
Route	`GET /api/cron/scan` (`backend/app/api/cron.py:32-46`), rewritten to the backend by the `/api/(.*)` rewrite in `vercel.json`
Auth	`Authorization: Bearer <CRON_SECRET>` , compared with `hmac.compare_digest` (cron.py:27-29). `CRON_SECRET` unset: **503** "Scheduled scans via cron are not configured"; wrong or missing header: **401** with `WWW-Authenticate: Bearer` (cron.py:34-37). `CRON_SECRET` is declared `SecretStr` (`config.py:38`)
Logic	`scheduler_tick(ctx, now + 60 min)` (cron.py:24,40): the same decision as the loop (interval, credit stretching, running check, official-assets check), looking ahead by `CRON_SLACK` = 60 min so a cron that fires a few minutes early is not skipped for a whole extra day
Response	`{"ran": bool, "skipped": bool, "reason": str, "scan_id": int
Execution	the scan runs inline in this request (Vercel default executor) up to 300 s
Retries	none by Vercel (`docs/DEPLOY_VERCEL.md:43`); delivery is best effort

Needs confirmation: `docs/DEPLOY_VERCEL.md:55-57` says Vercel's documentation lists `crons` only at the top level and does not describe cron in `services` mode, and that this was not verified live. Check in the Vercel dashboard (Project Settings, Cron Jobs) that the job exists after a deploy.

2.3 RQ worker (`backend/app/worker.py`)

- Entry: `python -m app.worker` (`worker.py:45-60`), or the container role `worker` (`backend/docker-entrypoint.sh` , `worker)` case). Requires `REDIS_URL` ; otherwise logs an error and exits with code 2.
- The API enqueues the function path `app.worker.run_scan_job` with the scan id, `job_id = "scan-<id>"` , timeout `SCAN_JOB_TIMEOUT_SECONDS` (900) on queue `SCAN_QUEUE_NAME` (`scans`) (`jobs.py:59-67`).
- The worker builds one `ScanContext` per process (engine, `CollectorRuntime` , fetcher; `worker.py:28-37`) and calls `run_scan` , which never raises: failures are written on the scan row (`worker.py:40-42`). It uses `SimpleWorker` on Windows because the default RQ worker forks (line 58). Sentry is initialised in `main()` (line 47).
- Health: `GET /health` reports `redis` (PING with 1 s timeouts) and `worker` (registered RQ workers, last heartbeat age, queue length; `backend/app/services/health.py:89-117`). Compose marks the worker healthy by the same check. No worker registered means "queued scans will not run" (health.py:116).
- Not covered by automated tests against a real Redis server (docstring, `worker.py:3-5` ; `docs/DEPENDENCIES.md`).

2.4 Database migrations and startup on each platform

`backend/docker-entrypoint.sh` : role `api` runs `python -m scripts.migrate` (PostgreSQL advisory lock so replicas do not collide) unless `RUN_MIGRATIONS=false` , then starts uvicorn; role `migrate` runs it once. On Vercel the entrypoint is `backend/api/index.py` (`app = create_app()`); it never starts the scheduler thread and seeds the admin lazily (`cold_start`).

3. Integration catalogue

#	Integration	Used for	Real or mock
1	Tavily	live social and store search	real, paid credits
2	SearXNG	optional free social search fallback	real, self-hosted
3	Apple iTunes Search	App Store listings	real, free, rate limited
4	google-play-scraper	Google Play listings	real (scraping), optional package
5	Google Safe Browsing	URL reputation of web candidates	real, only when a key is set
6	Google Gemini	scan text analysis, voice agent, Studio chat fallback	real, optional
7	Hugging Face	alternative scan text analysis	real, optional
8	Groq	Brand Studio chat (primary)	real, optional
9	Pollinations	Brand Studio image generation	real, optional
10	Sentry	error reporting	real, optional
11	Redis	RQ job queue, health	real, optional
12	Supabase / PostgreSQL	production database	real; SQLite is the local default
-	Sample collector	fake candidates for demos and tests	MOCK (no network)

3.1 Tavily

- **Purpose:** primary live search for social profiles, brand pages and store listings, restricted to `instagram.com`, `facebook.com`, `x.com` / `twitter.com`, `t.me`, `linkedin.com`, `youtube.com`, `play.google.com`, `apps.apple.com` and six APK mirror sites (`backend/app/core/collectors/tavily.py:41-51`). It does **not** search general websites.
- **Where:** `backend/app/core/collectors/tavily.py`, ledger `backend/app/core/collectors/credits.py`, wiring `backend/app/core/collectors/selection.py:163-169`. Endpoint `POST https://api.tavily.com/search` (tavily.py:38) with `Authorization: Bearer <key>`.
- **Config:** `TAVILY_API_KEY`, `TAVILY_MONTHLY_BUDGET` (900), `TAVILY_MAX_QUERIES_PER_SCAN` (12), `TAVILY_SEARCH_DEPTH` (`basic` =1 credit, `advanced` =2), `TAVILY_MAX_RESULTS` (10, capped at 20) (`config.py:91-95`).
- **Credit ledger:** table `api_usage`, provider `tavily`, one row per UTC month (`YYYY-MM`). `reserve(cost)` is one atomic `UPDATE ... WHERE credits + cost <= budget`, so parallel scans cannot overspend (`credits.py:83-98`); it also refuses when Tavily's self-reported usage (provider `tavily:reported`, from `include_usage`) plus the cost would exceed the budget (line 90). The guard uses the larger of the two counters (`_snapshot`, 57-69). `refund()` returns the cost when the call provably did not run (4xx, connection error); a timeout or 5xx keeps it because it may have been billed (`tavily.py:165-176`). `GET /api/sources` returns `tavily_credits {used, budget, remaining, reported_used, month}` (`backend/app/api/sources.py:154`).
- **Failure behavior:** missing key -> source `unavailable (no_key)`, retried next scan with no waiting; 401 `invalid_key`; 432/433 `quota`; 429 `rate_limited` (honours `Retry-After`, up to 24 h); budget reached -> `budget` with no call made; 5xx/timeout `unreachable` / `timeout`. The orchestrator then applies back-off 1 min, 5 min, 15 min, 1 h, 6 h and serves cached results marked stale (`backend/app/core/collectors/orchestrator.py:39-68,258-331`). If it fails after at least one successful query, the partial results are kept with a warning (tavily.py:129-133).
- **Cost control summary:** per-scan query cap, monthly budget, credit-aware scheduler, retry confirmation dialog for manual source retries that can spend credits (`backend/app/api/sources.py:44-53,207-214`).

3.2 SearXNG

- **Purpose:** optional self-hosted search fallback for the five social networks (`backend/app/core/collectors/searxng.py:57`). Active only when `SEARXNG_BASE_URL` is set (`selection.py:104-107`).
- **Config:** `SEARXNG_BASE_URL`, `SEARXNG_ALLOW_PRIVATE_HOSTS` (false), `SEARXNG_MAX_QUERIES_PER_SCAN` (12) (`config.py:96-98`).
- **Safety:** the base URL must resolve to a public address (SSRF guard `validate_public_url`, `backend/app/core/pipeline/url_safety.py:63`) unless private hosts are explicitly allowed; then only http(s) without credentials is accepted (`searxng.py:78-93`). Errors: 403 means JSON is disabled on the instance (`json_disabled`), 429 `rate_limited`, 5xx `unreachable`.
- **Cost:** none from us. **Failure:** same orchestrator back-off and stale cache.

3.3 Apple iTunes Search

- **Purpose:** App Store listings, no key (`backend/app/core/collectors/itunes.py`, `GET https://itunes.apple.com/search?term&country&entity=software&limit`).
- **Config:** `ITUNES_COUNTRY` (in), `ITUNES_CALLS_PER_MINUTE` (20), `ITUNES_CACHE_TTL_SECONDS` (3600), `ITUNES_MAX_QUERIES_PER_SCAN` (8) (`config.py:99-102`).
- **Quota handling:** `SpacingLimiter` waits between calls (one every 3 s at 20/min) and a `TTLCache` (512 entries) avoids repeating the same term within the TTL (`backend/app/core/collectors/util.py:13-56`; `selection.py:134-135`). Alone it does not flip `auto` data mode to live (`selection.py:77-80`).
- **Failure:** timeouts and HTTP errors become `SourceUnavailable`; back-off and cache as above.

3.4 google-play-scraper

- **Purpose:** Google Play search (Google has no public Play search API). The package is imported lazily and is **not** in `backend/requirements.txt`, only in `backend/requirements-extra.txt` (`backend/app/core/collectors/google_play.py:1-9,37-56`). Licence and terms risk are marked unverified in the file header.
- **Config:** `GOOGLE_PLAY_COUNTRY` (in), `GOOGLE_PLAY_MAX_QUERIES_PER_SCAN` (6) (`config.py:103-104`). A fixed 30 calls/min spacing limiter (`selection.py:136`).
- **Failure:** package missing -> `not_installed` (a local-configuration reason: no waiting period, `orchestrator.py:42`); any exception from the library -> `unreachable`. If the package is installed `auto` data mode counts as live (`selection.py:85,92-97`).

3.5 Google Safe Browsing

- **Purpose:** reputation of **web candidates that already look brand related**, one batched request per scan (up to 500 URLs; our cap 100). URLs are sent to Google and never opened. Details in `docs/features/web-threats.md` section 4.
- **Where:** `backend/app/core/providers/safe_browsing.py`; called only from `_reputation_pass` (`backend/app/core/pipeline/scan_pipeline.py:445-484`), which scans and manual/imported candidates use. Not called by the URL tester.
- **Config:** `GOOGLE_SAFE_BROWSING_API_KEY` (sent in the `x-goog-api-key` header), `SAFE_BROWSING_TIMEOUT_SECONDS` (5), `SAFE_BROWSING_DAILY_BUDGET` (5000 requests per UTC day), `SAFE_BROWSING_MAX_URLS_PER_SCAN` (100), `SAFE_BROWSING_CACHE_TTL_SECONDS` (3600) (`config.py:132-136`).
- **Quota:** `DailyLedger` in `api_usage` (provider `safe_browsing:YYYY-MM-DD`), atomic reserve; a 429 blocks further requests until `Retry-After` (or 1 h); TTL cache of `match` / `clean` answers.
- **Failure:** no key, budget used, quota -> `unknown`; timeout, 5xx, bad key -> `failed`. Both reach the engine as `unavailable` (zero points and never "clean"). Status for Source Health: `runtime.safe_browsing.status()` (key never included). Terms: non-commercial use; Web Risk is the paid alternative (file header, lines 14-15).

3.6 Google Gemini

Three separate uses, all through the official `google-genai` SDK (lazy imports; `backend/requirements-extra.txt` and `backend/requirements.txt` both list it):

Use	Code	Key and model	When it runs	Failure behavior
Scan text analysis (bio, app description)	`backend/app/core/providers/text/gemini.py:16-52`, wired by `build_provider` (`factory.py:24-31`)	`GEMINI_API_KEY`, `GEMINI_MODEL` (default `gemini-3.5-flash-lite`, `config.py:71-72`)	only if `SCAM_LLM_ENABLED=true` and the active provider is `gemini` (admin setting, else `SCAM_LLM_PROVIDER`), not for sample or stale candidates; structured JSON output, temperature 0, 2048 output tokens	never raises: `unavailable` finding; cached answers 30 days; 10 s timeout (`TEXT_PROVIDER_TIMEOUT_SECONDS`), 2 retries on 5xx/timeout (`TEXT_PROVIDER_MAX_RETRIES`); quota cool-down 600 s, bad key 900 s (`providers/text/base.py:22-23,141-159`)
Voice agent (`POST /api/voice`)	`backend/app/services/voice.py:492-615`, `backend/app/api/voice.py:54-66`	`VOICE_GEMINI_API_KEY`, else `GEMINI_API_KEY`; `VOICE_GEMINI_MODEL` (`gemini-3-flash-preview`), `VOICE_GEMINI_FALLBACK_MODEL` (`gemini-flash-latest`, tried once if the first refuses within 4 s), `VOICE_THINKING_LEVEL`, `VOICE_TIMEOUT_SECONDS` (10) (`config.py:145-150`)	simple commands (open page, scroll, fill) are handled locally with no model (`fast_command`, `fast_fill`); Gemini function calling only when those do not match; sensitive fields are stripped from the page snapshot before use (`api/voice.py:60`)	one attempt, hard timeout; any error returns a friendly sentence ("The AI service is busy..." for 429/503/504, otherwise "not understood"), never an error page (voice.py:531-540). No key: only the simple commands work. Per-user rate limit `VOICE_RATE_PER_MINUTE` (30, in memory, single process; `api/voice.py:56`)
Brand Studio chat fallback	`backend/app/services/studio_ai.py:150-218` (`_gemini`)	`GEMINI_API_KEY`, `GEMINI_MODEL`	when `GROQ_API_KEY` is unset or Groq fails; REST `generateContent` with header `x-goog-api-key`	see 3.8

Optional embeddings (`TEXT_EMBEDDINGS_ENABLED`, `GEMINI_EMBEDDING_MODEL`, no default model) are built only to show status in `GET /api/providers` (`backend/app/api/providers.py:38`; `factory.py:34-39`); nothing in scoring calls them.

3.7 Hugging Face

- **Purpose:** alternative hosted text provider for scan bios and descriptions (`backend/app/core/providers/text/huggingface.py`), through `huggingface_hub.InferenceClient.chat_completion` (lazy import, in `requirements-extra.txt`).
- **Config:** `HUGGINGFACE_API_TOKEN`, `HUGGINGFACE_MODEL` (no default: unusable until a model is chosen), plus `SCAM_LLM_ENABLED=true` and the active provider set to `huggingface` (`config.py:74-75`; `PUT /api/providers/active`, admin only, not allowed for the shared demo administrator, `backend/app/api/providers.py:50-57`).
- **Behavior:** requests JSON-schema output, repeats once without `response_format` if the provider rejects it (huggingface.py:54-65); `max_tokens` 400, temperature 0. Same cache, retry, cool-down and "never fails a scan" rules as Gemini.
- The same package is also pulled in by `fastembed` for the CLIP model download (see `docs/features/lookalike-lab.md`).

3.8 Groq (Brand Studio chat)

- **Purpose:** primary chat model of Brand Studio's AI helper, `POST https://api.groq.com/openai/v1/chat/completions` (`backend/app/services/studio_ai.py:39,171-189`), served at `POST /api/studio/ai/chat` (`backend/app/api/studio_ai.py:88-106`); the Studio frontend calls it through `studio/src/lib/ai.ts:66`. Keys stay on the server.
- **Config:** `GROQ_API_KEY`, `STUDIO_GROQ_MODEL` (`openai/gpt-oss-120b`), `STUDIO_AI_TIMEOUT_SECONDS` (45), `STUDIO_AI_MAX_RETRIES` (2), `STUDIO_AI_RETRY_BACKOFF_SECONDS`, `STUDIO_MAX_MESSAGES` (40), `STUDIO_MAX_CHARS`, `STUDIO_MAX_TOKENS`, `STUDIO_AI_RATE_PER_MINUTE` (20 per user, in memory) (`config.py:139-156`).
- **Order:** Groq first, then Gemini when Groq has no key or fails; if neither key is set: HTTP 503 "AI chat is not set up on the server" (`studio_ai.py:150-169`).
- **Quota and cost:** per-user daily quota stored in `api_usage` under provider `studio:text:u<user>:<YYYYMMDD>` (`STUDIO_DAILY_TEXT_QUOTA`, 200, 0 = unlimited; `studio_ai.py:282-316`), reserved before the call and refunded on provider failure (`api/studio_ai.py:101-105`).
- **Failure mapping** (`studio_ai.py:70-80`): 401/403 -> 503 "not set up correctly", 402 -> 503 "run out of credits", 429 -> 429 with `Retry-After`, 4xx request errors -> 422, 5xx/timeout -> 502. Only timeouts, connection errors and 5xx are retried; responses are size capped (512 KB) and prompts/answers/keys are never logged (docstring, lines 1-10).

3.9 Pollinations (Brand Studio images)

- **Purpose:** AI image generation for Brand Studio: `GET https://gen.pollinations.ai/image/<prompt>?model&width&height&seed=-1` with `Authorization: Bearer <key>` (`studio_ai.py:41,222-236`), served at `POST /api/studio/ai/image`.

- **Config:** `POLLINATIONS_API_KEY` (no key: image generation is not configured, `image_configured()`), `STUDIO_IMAGE_MODEL` (`black-forest-labs/flux.1-schnell`), `STUDIO_IMAGE_TIMEOUT_SECONDS` (120), `STUDIO_DAILY_IMAGE_QUOTA` (50), `STUDIO_MAX_PROMPT_CHARS` (2000) (`config.py:140-160`).
- **Safety:** response capped at 12 MB, decoded, side and pixel limits (4096 px, 16 million), only PNG/JPEG/WebP accepted, then **re-encoded as PNG** before it reaches the browser (`studio_ai.py:42-46,257+`). httpx request logging is turned down because the prompt is in the URL path (lines 35-37).
- **Failure and cost:** same error mapping and daily quota refund as Groq (provider `studio:image:u<user>:<day>`).

3.10 Sentry

- **Purpose:** optional error reporting. Off unless `SENTRY_DSN` is set; if set but `sentry-sdk` is missing, one warning and the app continues (`backend/app/config/observability.py:79-87`). The SDK is in `backend/requirements-extra.txt` (Docker build arg `INSTALL_EXTRAS`).
- **Config:** `SENTRY_DSN` (secret), `SENTRY_ENVIRONMENT` (default `APP_ENV`), `SENTRY_TRACES_SAMPLE_RATE` (0.0) (`config.py:171-173`). Initialised in `create_app` (`backend/app/main.py:61`) and in the worker (`worker.py:47`).
- **Privacy:** `send_default_pii=False`, no request bodies, no local variables; `before_send` blanks auth headers, cookies, query strings, user identity except the id, `server_name`, and redacts every configured secret value found in messages (observability.py:46-76,89-98). A bad DSN is logged and ignored (99-101).

3.11 Redis

- **Purpose:** the RQ job queue and the worker heartbeat. Nothing else uses it: rate limiters, response cache and the Tavily/Safe Browsing ledgers are in memory or in the database (`backend/app/config/ratelimit.py`, `response_cache.py`).
- **Config:** `REDIS_URL` (secret, may contain a password), `SCAN_EXECUTOR`, `SCAN_QUEUE_NAME`, `SCAN_JOB_TIMEOUT_SECONDS` (`config.py:110-113`). Compose provides a password-protected `redis:7.2-alpine` with AOF persistence (`docker-compose.yml`, service `redis`).
- **Failure:** if Redis is down when a scan is queued the scan runs in a thread instead (`jobs.py:46-52`); `/health` reports `redis: down` and `worker: degraded`, but only a database failure makes `/health` return 503 (`backend/app/api/health.py:22-27`). `redis` and `rq` are imported lazily so the app starts and the tests run without a server.
- **On Vercel:** the default executor is `inline`, so Redis is not used for scans.

3.12 Supabase / PostgreSQL

- **Purpose:** production database (SQLAlchemy 2, psycopg 3; migrations with Alembic `backend/alembic/versions/0001...0012`). Local default is a SQLite file `backend/data/scammind.db` (`config.py:28`), with WAL mode, foreign keys on and a 30 s busy timeout (`backend/app/db/session.py:10-45`). Docker compose runs `postgres:17-alpine`.
- **Config:** `DATABASE_URL` (runtime), `MIGRATION_DATABASE_URL` (used only by Alembic and `scripts/migrate.py`; falls back to `DATABASE_URL`; `config.py:37,214-221`), `DB_POOL_SIZE`, `DB_MAX_OVERFLOW`, `DB_POOL_RECYCLE`, `DB_PRE_PING` (read from the environment in `session.py:24-27`), `STORAGE_BACKEND` (`local` or `database`).
- **Supabase specifics:** use the transaction pooler (port 6543) for runtime and a direct or session connection (5432) for migrations (`docs/DEPLOY_VERCEL.md:46,132-160`). The engine sets, for any `postgresql` URL: small pool (5 + 2; on Vercel 1 + 1), recycle 1800 s (300 s on Vercel), TCP keepalives, and for URLs containing `:6543/` it disables server-side prepared statements (`prepare_threshold=None`) (`session.py:18-32`).
- **Files in the database:** with `STORAGE_BACKEND=database` (default on Vercel, `config.py:181-187`) uploaded logos, candidate images and evidence snapshots live in table `stored_files` (`backend/app/core/storage.py:96-166`), because the Vercel function file system is read-only except `/tmp`.
- **What else lives here that behaves like an integration:** the counters `api_usage` (Tavily month, Safe Browsing day, Studio user/day), `collector_state` (per-collector status, back-off and cached results), `text_analysis_cache`, `settings` (data mode, monitoring, text provider).
- **Failure:** `/health` returns 503 only when the database is down; migration drift is reported as degraded (`backend/app/services/health.py:66-79`). The lifespan skips seeding with a warning when tables do not exist yet (`main.py:81-83`).

3.13 Sample collector (mock)

`backend/app/core/collectors/sample.py`: deterministic fictional candidates built from your own brand name (look-alikes in three tiers, hard negatives, echoes of your official assets, `.test` web hosts), generated logos derived from your uploaded logo. No network, no cost. Used when the data mode is `sample`, or `auto` with no live source configured (`selection.py:88-100`); every detection it creates has `source="sample"` and every signal is labelled `synthetic`. The in-memory **canary** (`POST /api/health/canary`, admin only) also uses it against a private database (`backend/app/services/canary.py:1-12`).

4. Failure and cost matrix

Integration	If the key or service is missing	If it errors	Spend control
Tavily	`unavailable (no_key)`; scan still runs with the other collectors; auto mode needs it, SearXNG, or google-play to go live	back-off 1m/5m/15m/1h/6h, stale cache, scan `partial`	monthly budget ledger, per-scan query cap, credit-aware scheduler, confirmation on manual retry
SearXNG	skipped (inactive)	back-off + cache	none needed
iTunes	always on in live mode	back-off + cache	20 calls/min spacing, 1 h cache, 8 queries per scan
google-play-scraper	`unavailable (not_installed)`	`unreachable`, back-off	6 queries per scan, 30 calls/min
Safe Browsing	reputation `unavailable`	`unavailable`, 429 block	5000 requests/UTC day, 100 URLs per scan, 1 h cache
Gemini / Hugging Face (scan)	skipped; local baseline only	`unavailable` finding, cool-down 10-15 min	answer cache 30 days, `SCAM_LLM_ENABLED` master switch (off by default)
Gemini (voice)	simple commands only	friendly message	per-user rate limit 30/min
Groq / Gemini / Pollinations (Studio)	503 "not set up"	mapped 4xx/5xx messages	per-user rate limit and daily quotas (text 200, image 50)
Sentry	no reporting	ignored	sampling `SENTRY_TRACES_SAMPLE_RATE` (0.0)
Redis	thread fallback	thread fallback	n/a
Database	`/health` 503, API errors	n/a	n/a

5. Environment variable names by integration (no values)

Area	Names
Jobs and timer	`SCAN_EXECUTOR`, `REDIS_URL`, `SCAN_QUEUE_NAME`, `SCAN_JOB_TIMEOUT_SECONDS`, `SCAN_STALE_MINUTES`, `SCHEDULER_ENABLED`, `SCAN_INTERVAL_MINUTES`, `SCHEDULER_POLL_SECONDS`, `SCHEDULER_MAX_BRANDS_PER_RUN`, `CRON_SECRET`, `VERCEL` (set by the platform)
Tavily	`TAVILY_API_KEY`, `TAVILY_MONTHLY_BUDGET`, `TAVILY_MAX_QUERIES_PER_SCAN`, `TAVILY_SEARCH_DEPTH`, `TAVILY_MAX_RESULTS`
SearXNG	`SEARXNG_BASE_URL`, `SEARXNG_ALLOW_PRIVATE_HOSTS`, `SEARXNG_MAX_QUERIES_PER_SCAN`
iTunes / Play	`ITUNES_COUNTRY`, `ITUNES_CALLS_PER_MINUTE`, `ITUNES_CACHE_TTL_SECONDS`, `ITUNES_MAX_QUERIES_PER_SCAN`, `GOOGLE_PLAY_COUNTRY`, `GOOGLE_PLAY_MAX_QUERIES_PER_SCAN`, `SOURCE_SLOW_THRESHOLD_MS`, `COLLECTOR_TIMEOUT_SECONDS`
Safe Browsing	`GOOGLE_SAFE_BROWSING_API_KEY`, `SAFE_BROWSING_TIMEOUT_SECONDS`, `SAFE_BROWSING_DAILY_BUDGET`, `SAFE_BROWSING_MAX_URLS_PER_SCAN`, `SAFE_BROWSING_CACHE_TTL_SECONDS`
Text analysis	`SCAM_LLM_ENABLED`, `SCAM_LLM_PROVIDER`, `GEMINI_API_KEY`, `GEMINI_MODEL`, `HUGGINGFACE_API_TOKEN`, `HUGGINGFACE_MODEL`, `TEXT_PROVIDER_TIMEOUT_SECONDS`, `TEXT_PROVIDER_MAX_RETRIES`, `TEXT_EMBEDDINGS_ENABLED`, `GEMINI_EMBEDDING_MODEL`
Voice	`VOICE_GEMINI_API_KEY`, `VOICE_GEMINI_MODEL`, `VOICE_GEMINI_FALLBACK_MODEL`, `VOICE_THINKING_LEVEL`, `VOICE_TIMEOUT_SECONDS`, `VOICE_RATE_PER_MINUTE`
Studio AI	`GROQ_API_KEY`, `POLLINATIONS_API_KEY`, `STUDIO_GROQ_MODEL`, `STUDIO_IMAGE_MODEL`, `STUDIO_AI_RATE_PER_MINUTE`, `STUDIO_DAILY_TEXT_QUOTA`, `STUDIO_DAILY_IMAGE_QUOTA`, `STUDIO_AI_TIMEOUT_SECONDS`, `STUDIO_IMAGE_TIMEOUT_SECONDS`, `STUDIO_AI_MAX_RETRIES`, `STUDIO_AI_RETRY_BACKOFF_SECONDS`, `STUDIO_MAX_MESSAGES`, `STUDIO_MAX_CHARS`, `STUDIO_MAX_TOKENS`, `STUDIO_MAX_PROMPT_CHARS`
Sentry	`SENTRY_DSN`, `SENTRY_ENVIRONMENT`, `SENTRY_TRACES_SAMPLE_RATE`
Database and storage	`DATABASE_URL`, `MIGRATION_DATABASE_URL`, `DB_POOL_SIZE`, `DB_MAX_OVERFLOW`, `DB_POOL_RECYCLE`, `DB_PRE_PING`, `STORAGE_BACKEND`, `DATA_DIR`, `RUN_MIGRATIONS`
Logo / images	`IMAGE_SIMILARITY_BACKEND`, `LOGO_COMPARE_CLIP`, `SCAN_LOGO_FETCH`, `SCREENSHOTS_ENABLED`

Real values live only in `backend/.env` (never committed, never documented) or the platform's environment settings. The template with names only is `backend/.env.example`. `Settings.secret_values` lists the secrets that log redaction and Sentry scrubbing hide (`config.py:197-202`).

6. Open questions

Needs confirmation: `REACHABILITY_CHECKS_PER_SCAN` and `SCHEDULER_MAX_QUERIES_PER_SCAN` are declared in `config.py:107,127` but no scan or scheduler code reads them (details in `docs/features/scanning-and-detection.md` section 12).

Needs confirmation: `fastembed` and `google-play-scraper` are only in `backend/requirements-extra.txt`; the Docker build installs extras only when `INSTALL_EXTRAS=true` (`docker-compose.yml`, `api.build.args`). Whether the Vercel deployment installs them (and therefore whether CLIP, Google Play search and Hugging Face are ever available in production) is not visible in the repository files read for this document.

Needs confirmation: the shared in-memory rate limiters (`RequestLimiter`, `FailureLimiter`) and the response cache are per process (comments in `config.py:149,163`); on Vercel each function instance has its own, so the effective limits are looser than the numbers suggest. Confirm whether that is acceptable for the public demo.

DOCS/TESTING.MD

Testing

Plain-language summary

- There are four layers of tests: backend (Python, `pytest`), BrandShield frontend (`vitest`), Brand Studio (`vitest`) and browser tests (Playwright) that drive the real app in sample mode.
- No test talks to the internet or uses real keys: the backend suite blocks network access and strips real keys from the environment.
- The backend suite runs on SQLite by default. CI also runs it on PostgreSQL by setting one variable.
- Counts below come from reading the test directories (files and `def test_` / `it(` occurrences), not from running them. Parametrized tests expand to more cases at run time.

1. Overview and commands

Layer	Tool	Files / tests (counted by reading)	Run from	Command
Backend	pytest	63 test files; `pytest --collect-only` reports 1263 test cases (parametrized tests expand the ~750 `def test_` functions)	`backend/`	`python -m pytest -q`
BrandShield frontend	vitest + jsdom + Testing Library	22 files, about 137 `it(`	`frontend/`	`npm test`
Brand Studio	vitest + jsdom	6 files, 41 `it(`	`studio/`	`npm test` (CI uses `npx vitest run`)
Browser (e2e)	Playwright, Chromium	8 spec files, 31 tests	`frontend/`	`npm run e2e:install` once, then `npm run test:e2e`
Backend lint	ruff	n/a	`backend/`	`python -m ruff check .`
Frontend lint / types	oxlint / `tsc -b`	n/a	`frontend/`	`npm run lint` / `npm run typecheck`
Studio types	`tsc --noEmit`	n/a	`studio/`	`npm run lint`

An earlier README quoted "about 1,240 tests" for the backend; that figure includes parametrized expansion and was not re-verified here (`python -m pytest --collect-only -q` reported 1263 collected tests on 2026-10-23; the last full run passed 1262 with 5 skipped).

Use the project virtual environment on Windows: `.\.venv\Scripts\python -m pytest -q` (created by `npm run backend`, see `docs/setup-and-deployment.md`). `requirements-dev.txt` adds `pytest>=8,<9` and `ruff>=0.5` to `requirements.txt` (`backend/requirements-dev.txt`).

2. Backend (pytest)

Configuration: `backend/pyproject.toml:19-22` (`testpaths = ["tests"]`, `pythonpath = ["."]`, DeprecationWarning ignored). There are no custom markers; the only markers are `skipif` ones (`backend/tests/api/test_logo_clip.py:214` skips without `fastembed`; `backend/tests/db/test_postgres.py:24` skips without `TEST_DATABASE_URL`).

Files by directory

Directory	Test files	`def test_`	Covers
`tests/api/`	23	311	HTTP endpoints: auth, register, brands, accounts, tenant isolation, scans, screens, Studio AI and items, voice, demo login, health, response cache, Vercel behaviour (`test_vercel.py`), logo CLIP/lookalike
`tests/config/`	2	14	`test_logging.py` (redaction, JSON), `test_sentry.py`
`tests/core/engine/`	9	part of 371	name/domain matching, distance and normalization, URLs, risk engine, scoring pipeline, logo similarity, LLM text, synthetic evaluation
`tests/core/collectors/`	9	part of 371	Tavily, iTunes, SearXNG, Google Play, selection and orchestrator, contract, sample collector, live check, race and SQLite lock cases
`tests/core/pipeline/`	9	part of 371	scan flow, scan pipeline, web candidates, URL safety, registry intake, logos and images
`tests/core/providers/`	2	part of 371	Safe Browsing, text providers
`tests/core/` (top)	1	part of 371	`test_storage_backends.py` (local and database file stores)
`tests/db/`	4	21	migrations (`test_migration.py`, `_0004`, `_0010`) and `test_postgres.py`
`tests/services/`	4	33	audit, canary, profile health, scheduler per brand

`tests/core/` totals 30 files and 371 `def test_` (9 + 9 + 9 + 2 + 1).

Support files: `tests/conftest.py`, `tests/pg_support.py`, `tests/scan_helpers.py`, `tests/fixtures/` (hand-written Tavily, iTunes and SearXNG responses and `golden_scores_v1.json`; see `tests/fixtures/README.md`: they are written from public API docs, not captured live).

What `conftest.py` guarantees

Behaviour	Source
Sets throwaway environment before the app is imported: `APP_ENV=test`, a test-only `JWT_SECRET`, `DATABASE_URL=sqlite://`, `SCAN_EXECUTOR=inline`, `SCHEDULER_ENABLED=false`	`backend/tests/conftest.py:4-15`
Removes real keys from the shell: `VERCEL`, `STORAGE_BACKEND`, `MIGRATION_DATABASE_URL`, `CORS_ORIGIN_REGEX`, `CRON_SECRET`, `GOOGLE_SAFE_BROWSING_API_KEY`, `TAVILY_API_KEY`, `SEARXNG_BASE_URL`, `GEMINI_API_KEY`, `GROQ_API_KEY`, `POLLINATIONS_API_KEY`, `HUGGINGFACE_API_TOKEN`, `REDIS_URL`, `SENTRY_DSN`, …	`conftest.py:18-19`
Fast bcrypt (`BCRYPT_ROUNDS = 4`)	`conftest.py:42`
Network guard (autouse): `httpx.HTTPTransport.handle_request` raises `AssertionError("live network access attempted in a test")`; `socket.getaddrinfo` is blocked except `localhost`, `127.0.0.1`, `::1` (needed for the PostgreSQL job). `httpx.MockTransport` is unaffected	`conftest.py:65-82`
Optional packages treated as absent (Google Play scraper, fastembed/CLIP) so local extras do not change results	`conftest.py:85-94`
`Settings(_env_file=None)` so a developer's `backend/.env` never leaks in	`conftest.py:60-62`

Fixtures

Fixture	Gives you	Source
`settings`	`Settings` with temp `data_dir`, in-memory DB, response cache off	`conftest.py:59-62`
`checker`	`FakeChecker` replacing the network URL checker (records calls, configurable result)	`conftest.py:47-56,97-99`
`app`	`create_app(...)` on a fresh in-memory SQLite engine with tables from `Base.metadata.create_all`	`conftest.py:102-106`
`client`	`TestClient` (runs lifespan, so the admin is seeded)	`conftest.py:109-112`
`db`	a SQLAlchemy session from the app	`conftest.py:115-118`
`admin_h`, `analyst_h`, `viewer_h`	`Authorization` headers for admin (`admin@example.com`), analyst and viewer users	`conftest.py:148-162`
`brand`	creates "Acme Bank" via `POST /api/brands`	`conftest.py:165-169`
helpers `login`, `headers_for`, `make_user`, `hand_brand_to`	create users and give legacy tests a brand owner	`conftest.py:121-145`

SQLite default and the PostgreSQL job

- Default: every engine is in-memory SQLite (`DATABASE_URL=sqlite://`, `conftest.py:11`).
- Set `TEST_DATABASE_URL` to run the same suite on PostgreSQL. `tests/pg_support.py:17-47` replaces `make_engine("sqlite://")` so each test engine gets its own random schema `t_<12 hex>` in that database, dropped at interpreter exit. File-based SQLite engines (migration tests, the seed race test) stay SQLite on purpose (`conftest.py:21-23`). **Use a throwaway database only.**
- CI: `test-server-postgres` starts `postgres:17-alpine`, runs `python -m scripts.migrate` twice (second must do nothing), then `pytest -q` with `TEST_DATABASE_URL` set (`.github/workflows/ci.yml:73-112`).

```
cd backend
$env:TEST_DATABASE_URL = "postgresql+psycopg://user:password@localhost:5432/throwaway"
.\.venv\Scripts\python -m pytest -q
```

Useful pytest invocations

Goal	Command
One file	`python -m pytest tests/api/test_auth.py -q`
One test	`python -m pytest tests/api/test_auth.py -k login -q`
Only engine tests	`python -m pytest tests/core/engine -q`
List tests without running	`python -m pytest --collect-only -q`
Pipeline self-check (not pytest)	`python -m scripts.canary` (also run in CI: `ci.yml:69-71`)

3. BrandShield frontend (vitest)

Configuration: `frontend/vite.config.ts:188-195` : jsdom, 20 s timeout (form tests type long passwords key by key), `src/**/*.test.{ts,tsx}`, `restoreMocks: true`, setup file `src/test/setup.ts`. The setup runs Testing Library `cleanup`, clears the page query cache, `localStorage` and `sessionStorage` after every test (`frontend/src/test/setup.ts:6-15`). Helpers: `src/test/helpers.ts` (`jsonResponse`, `mockFetch`, `authHeader`).

Test files (22): `App.test.tsx`, `vercelConfig.test.ts`, `api/` (`baseUrl`, `client`, `format`), `components/common/LogoCompareCard`, `hooks/` (`hooks`, `useApi.cache`), `landing/LandingRoute`, `pages/` (`InvestigationsPage`, `LoginPage`, `LoginPage.demo`, `ProtectBrandPage`, `RegisterPage`, `SetupPage`, `SourceHealthPage`, `WebThreatsPage`), `utils/` (`brandExamples`, `passwordPolicy`, `studio`, `theme`, `voice`).

`npm run build` is also a check: it runs `tsc -b` then `vite build` (`frontend/package.json:8`).

4. Brand Studio (vitest)

Configuration: `studio/vite.config.ts:22-27` (jsdom, globals, `src/test/setup.ts`). Six files: `contexts/AuthContext.test.tsx`, `lib/api.test.ts`, `lib/brandshield.test.ts`, `lib/devRedirect.test.ts`, `lib/items.test.ts`, `lib/session.test.ts`. `npm run lint` is `tsc --noEmit` (`studio/package.json:51`).

5. Playwright end-to-end

Config: `frontend/playwright.config.ts`. One worker, no retries, 90 s test timeout (`:108-114`). It starts three servers itself (`:123-171`):

Server	Port	Command	Notes
Backend	8765 (`E2E_API_PORT`)	`backend/scripts/e2e_server.py` using `backend/.venv` python (or `E2E_PYTHON`)	sample collector, temp SQLite, no Tavily/Gemini/Redis, stubbed URL checker, scheduler off (`e2e_server.py:1-26,45-51`)
BrandShield dev server	5174 (`E2E_WEB_PORT`)	`npx vite --port 5174 --strictPort`	proxies `/studio` and `/api`
Brand Studio dev server	3100 (`E2E_STUDIO_PORT`)	`npx vite --port 3100 --strictPort` in `studio/`	not 3000, so a developer's Studio can keep running

Secrets: admin and viewer passwords are random per run, passed through environment only (`playwright.config.ts:99-105`). Demo accounts use fixed throwaway values defined in the config (`:138-141`).

Specs (`frontend/e2e/`):

Spec	Tests	Covers
`smoke.spec.ts`	13	login, setup wizard, brand profile, dashboard scan, social, apps, look-alike lab, source health, reports, settings, viewer role, token refresh, full page walk with no console errors
`register.spec.ts`	6	sign-up page and policy, duplicate email
`studio-flow.spec.ts`	3	Studio under the shared login, back to BrandShield, tenant separation
`multi-brand.spec.ts`	3	two brands from Studio, switch, "Add another brand"
`demo-login.spec.ts`	2	demo and guarded admin demo buttons
`voice.spec.ts`	2	voice agent with a fake speech recognizer, missing-recognition message
`protect-hover.spec.ts`	1	Protect Your Brand hover
`visual.spec.ts`	1	captures every page screenshot
`png.ts`	n/a	helper that builds test images

Reports: in CI, list + GitHub annotations + HTML report, trace kept on failure (`playwright.config.ts:115,121`); the e2e job uploads `frontend/test-results/` and `frontend/playwright-report/` for 7 days (`ci.yml:190-199`).

6. CI mapping

CI job (`.github/workflows/ci.yml`)	Local equivalent
`lint` (`:19-47`)	`ruff check .` in `backend/`; `npm run lint` and `npm run typecheck` in `frontend/`
`test-server` (`:49-71`)	`python -m pytest -q`; `python -m scripts.canary` (matrix Python 3.11 and 3.12)
`test-server-postgres` (`:73-112`)	`TEST_DATABASE_URL=... python -m pytest -q` after `python -m scripts.migrate`
`test-frontend` (`:114-132`)	`npm test`; `npm run build`
`test-studio` (`:134-155`)	`npx tsc --noEmit`; `npx vitest run`; `npm run build`
`e2e` (`:157-199`)	`npm run test:e2e` in `frontend/` (needs `studio/` and `frontend/` installed, Chromium, backend packages)
`build` (`:201-231`)	`docker build` of `backend`, `studio`, `frontend`

7. How to add a test

Backend endpoint test

1. Put it in `backend/tests/api/test_<topic>.py` (or the matching `core/`, `services/`, `db/` folder).
2. Use `client` plus `admin_h` / `analyst_h` / `viewer_h`; create data with the `brand` fixture or `client.post(...)`.
3. For scans use `tests/scan_helpers.py` (fake collectors; `SCAN_EXECUTOR=inline` makes scans synchronous).
4. Never reach the network: use `httpx.MockTransport` or the fakes. Real calls raise `AssertionError` (`conftest.py:72-79`).
5. Brand data is per account: another account's records must answer 404 (`tests/api/test_tenant_isolation.py`).
6. Run `python -m pytest tests/api/test_<topic>.py -q`, then `python -m ruff check .`.

Frontend unit test: add `Name.test.tsx` next to the file; mock `fetch` with `mockFetch` from `src/test/helpers.ts`; run `npm test`.

Studio unit test: add `*.test.ts(x)` under `studio/src`; run `npm test` in `studio/`.

Browser test: add `frontend/e2e/<name>.spec.ts` ; read credentials from `process.env.E2E_ADMIN_EMAIL` / `E2E_ADMIN_PASSWORD` (as `voice.spec.ts` does); keep it sample-mode and offline.

Collector fixtures: hand-written files live in `backend/tests/fixtures/{tavily,itunes,searxng}` ; after a real live check replace them (see `tests/fixtures/README.md`).

DOCS/TROUBLESHOOTING.MD

Troubleshooting

Plain-language summary

- Most problems on a laptop are one of three things: another program already uses a port, the API is not running, or the database has not been migrated.
- Problems with AI features (Studio logos, the voice assistant) are usually a missing key, an exhausted free quota or a model name that the provider has retired. The messages shown in the app say which.
- If a page looks empty or fictional, check the data mode (sample or live) first.
- Each entry below gives the symptom, what the code does, and the fix. Line numbers refer to the repo as read for this document.

Quick index

Symptom	Section
"Port 5173, 3000 is already in use"	1
`ECONNREFUSED 127.0.0.1:8000` , "Cannot reach the server"	2
Database not ready, 500 on first request	3
Supabase "max clients" error	4
Gemini 429 / 503 / 504, "AI service is busy", retired model	5
"Logo generation is unavailable", Pollinations credits	6
"Too many requests"	7
Safe Browsing "not checked"	8
Page shows sample data / no live results	9
Sign-in loops, signed out unexpectedly	10
Microphone blocked, voice needs Chrome/Edge	11
Studio on port 3000 sends me elsewhere	12
Windows / CRLF problems	13

1. Port already in use

Message	`Port 8000 is already in use. Close the old server (or the other terminal running this app) and run the command again.` (ports listed: 8000 for backend; 5173, 3000 for frontend)
Cause	`scripts/dev.mjs` connects to the port on `127.0.0.1` and `::1` ; any listener counts as busy (`scripts/dev.mjs:57-78`). Backend checks 8000 (`:89`), frontend checks 5173 and 3000 (`:80`). Studio and e2e also use `strictPort` , so they refuse to pick another port (`studio/vite.config.ts:16` , `frontend/playwright.config.ts:151,162`).
Fix	Close the earlier terminal running `npm run backend` / `npm run frontend` , or find the process: `netstat -ano`

2. `ECONNREFUSED 127.0.0.1:8000` / "Cannot reach the server"

Cause	The Vite dev server proxies `/api` to `127.0.0.1:8000` (`frontend/vite.config.ts:13`) and the app itself calls `http://127.0.0.1:8000` in development (`frontend/src/api/client.ts:27`). If the API is not running, the proxy logs `ECONNREFUSED` and the browser shows "Cannot reach the server. Check your connection and try again." (`client.ts:29`).
Fix	Start the API with `npm run backend` in another terminal and wait for `API http://localhost:8000` . Check http://localhost:8000/health/live. If the API exited right after start, read its output: typically a missing `JWT_SECRET` (validation error, `backend/app/config/config.py:40`) or a failed migration (section 3).
Also	If `npm run frontend` is run from a different working directory than the repo root, use the root or the parent folder (both have the scripts: `package.json:5-8`).

3. Database not ready / `alembic upgrade head`

Symptoms	`npm run backend` prints `Database migration failed: check DATABASE_URL in backend/.env` (`scripts/dev.mjs:102-106`); the API logs `admin seed skipped: database not ready (run alembic upgrade head)` (`backend/app/main.py:81-83`); `/health` shows `migration: degraded` with note `run: alembic upgrade head` (`backend/app/services/health.py:66-80`); first request returns 500.
Cause	Tables are created only by migrations. In `APP_ENV=production` a missing `alembic_version` table is `degraded` ; in other environments it is `unknown` (`health.py:76`).
Fix	From `backend/` : `.\.venv\Scripts\python -m scripts.migrate` (or `python -m alembic upgrade head`). Verify `DATABASE_URL` points at the intended database. On Vercel, run it from your computer with `MIGRATION_DATABASE_URL` (see `docs/setup-and-deployment.md` section 6). Migrations on PostgreSQL take an advisory lock so concurrent runs wait (`backend/scripts/migrate.py:32-42`).
SQLite locked	Writers wait up to 30 s before "database is locked" (`backend/app/db/session.py:13-14` , `PRAGMA busy_timeout=30000` at `:40`). Close other tools holding `backend/data/scammind.db` open.
`ModuleNotFoundError: psycopg2`	`DATABASE_URL` must start with `postgresql+psycopg://` , not `postgresql://` (`docs/DEPLOY_VERCEL.md` section 6).

4. Supabase `EMAXCONNSESSION` / too many connections

Cause	Supabase **session mode** pooler allows only 15 clients in total; each warm serverless instance holds its own pool (`backend/app/db/session.py:19-22`). If `DATABASE_URL` points at the session pooler (port 5432) or direct connection on Vercel, connections run out. The repo's guide describes the symptom as `FATAL: max clients reached / too many connections` (`docs/DEPLOY_VERCEL.md` section 6).
Fix	Use the **transaction pooler, port 6543** for `DATABASE_URL` ; keep the session/direct string only in `MIGRATION_DATABASE_URL` (used by migrations, `backend/app/config/config.py:35-37`). On Vercel the code already shrinks the pool to 1 + 1 connections with 300 s recycle (`session.py:23-26`); `DB_POOL_SIZE` , `DB_MAX_OVERFLOW` , `DB_POOL_RECYCLE` override. For `:6543/` URLs prepared statements are disabled automatically (`session.py:30-31`). A `prepared statement ... does not exist` error means port 6543 is not being used.
Note	**Needs confirmation:** the literal string `EMAXCONNSESSION` does not appear anywhere in the repo; it is the code Supabase returns for this condition, taken from Supabase's behaviour, not from this code.

5. Gemini 429 / 503 / 504 and retired model names

The voice agent and Studio chat use Gemini, Gemini is also the optional text-analysis provider.

Symptom	Cause in code	Fix
Voice reply "The AI service is busy right now. Please try again in a moment."	Gemini call raised with status 429, 503 or 504; the service returns that friendly message and logs `voice gemini call failed: <type> <code>` (`backend/app/services/voice.py:534-539`)	Wait and retry. Quota is per Google key; check `VOICE_GEMINI_API_KEY` / `GEMINI_API_KEY` .
Voice reply "Sorry, I didn't understand that. Try "open reports"..." for every sentence that is not a simple command	Any other failure (timeout, 404 for a retired model, unexpected answer) maps to this message (`voice.py:47,539`)	Read the API log line `voice gemini call failed:` ; a code of 404 means the model name is not served any more. Set `VOICE_GEMINI_MODEL` (default `gemini-3-flash-preview` , `config.py:146`) and `VOICE_GEMINI_FALLBACK_MODEL` (default `gemini-flash-latest` , `config.py:147`) to a current Flash model that supports function calling. The fallback is tried once if the first model fails quickly (`voice.py:606-612`).
Voice says "I can open pages, scroll and fill fields with simple commands" only	No Gemini key configured: simple commands work, others do not (`voice.py:530-531`)	Set `VOICE_GEMINI_API_KEY` or `GEMINI_API_KEY` in `backend/.env` .
Voice timeout	Deadline is `VOICE_TIMEOUT_SECONDS` , default 10 and "Gemini refuses a deadline under 10 s" (`config.py:148`)	Do not set it lower than 10.
Studio chat "The AI service is busy right now. Wait a minute and try again."	Provider 429 maps to HTTP 429 and is never retried (`backend/app/services/studio_ai.py:76-77` , `config.py:155`)	Wait; or switch key. Studio chat uses `GROQ_API_KEY` first, else `GEMINI_API_KEY` (`studio_ai.py:154`).
Studio chat "...its key was rejected"	Provider 401/403 maps to 503 (`studio_ai.py:72-73`)	Correct the key on the server.
Studio chat "could not use this request"	Provider 400/404/413/422 maps to 422 (`studio_ai.py:78-79`); a retired `STUDIO_GROQ_MODEL` or `GEMINI_MODEL` can produce 404	Update `STUDIO_GROQ_MODEL` (default `openai/gpt-oss-120b`) or `GEMINI_MODEL` (default `gemini-3.5-flash-lite` , `config.py:72,141`).

6. Pollinations out of credits / "Logo Generator unavailable"

Symptom	In Brand Studio's Logo Generator: "Logo generation is unavailable" with "Use your own logo:" (`studio/src/components/LogoGenerator.tsx:315-324`).
Cause	`POST /api/studio/ai/image` fails with 402 (provider out of credits, mapped to 503 "The AI service has run out of credits. Tell the administrator.", `backend/app/services/studio_ai.py:74-75`), 503 because `POLLINATIONS_API_KEY` is not set (`studio_ai.py:225-226`), or 502 (image service not answering). The browser maps these to `imageUnavailable` and a message, "...the image service (Pollinations) has run out of credits. You can upload your own logo instead" (`studio/src/lib/ai.ts:35-60`).
Fix	Add credit or set a valid `POLLINATIONS_API_KEY` ; check `STUDIO_IMAGE_MODEL` is a model your account may use (default `black-forest-labs/flux.1-schnell` , `config.py:142`). Meanwhile upload your own logo (PNG, JPEG or WebP, max `LOGO_MAX_BYTES` 2 MB, `config.py:61`). Daily per-user quotas also apply: `STUDIO_DAILY_IMAGE_QUOTA` 50 and `STUDIO_DAILY_TEXT_QUOTA` 200 (`config.py:151-152`).

7. "Too many requests"

Message	Source and limit	Fix
`Too many requests. Slow down and retry shortly.` (HTTP 429, `Retry-After: 60`)	General limiter: `API_RATE_LIMIT_PER_MINUTE` , default 600, per client IP and per bearer token, in memory (`backend/app/main.py:119-135` , `config.py:163`)	Wait a minute. `0` turns it off. Behind a proxy, set `FORWARDED_ALLOW_IPS` so IPs are real (`backend/docker-entrypoint.sh:20`), otherwise all users share one IP bucket.
`Too many failed attempts. Try again later.`	Login: 5 failures per email+IP per 900 s (`backend/app/api/auth.py:62-70` , `config.py:53-54`)	Wait 15 minutes or restart the API (in-memory limiter).
`Too many sign-up attempts. Try again later.`	5 per IP and 3 per email per hour (`auth.py:111` , `config.py:58-59`)	Wait; `SIGNUP_ENABLED=false` disables sign-up.
`Too many AI requests. Wait a moment and try again.`	Studio AI: 20 per user per minute (`backend/app/api/studio_ai.py:66` , `config.py:143`)	Wait 30 s.
`Too many voice commands. Wait a moment and try again.`	30 per user per minute (`backend/app/api/voice.py:57` , `config.py:149`)	Wait 20 s.

8. Safe Browsing is not called ("not checked")

By design Safe Browsing is skipped when:

Case	Evidence
Data is **sample**: the note is "Sample data: the URL is not sent to Google."	`backend/app/core/pipeline/scan_pipeline.py:457-460`
The candidate is a cached copy (`stale`)	`scan_pipeline.py:461-463`
The domain does not resemble the brand ("so it was not sent for a reputation check")	`scan_pipeline.py:465-467`
More than `SAFE_BROWSING_MAX_URLS_PER_SCAN` (100) URLs in one scan	`scan_pipeline.py:468-470` , `config.py:135`
No `GOOGLE_SAFE_BROWSING_API_KEY` : the adapter makes no request and the verdict is `unknown` / `unavailable` , which the engine never treats as clean	`backend/app/core/providers/safe_browsing.py:1-14` , `scan_pipeline.py:632`
Daily budget used (`SAFE_BROWSING_DAILY_BUDGET` , 5000 per UTC day)	`safe_browsing.py:16-17` , `config.py:134`

Fix: set the key in `backend/.env` , switch to live data (section 9), and scan a brand with look-alike domains. Terms: Safe Browsing is for non-commercial use (`safe_browsing.py:14-15`).

9. Data mode: sample vs live

Fact	Source
Modes are `live` , `sample` , `auto` ; default `auto`	`backend/app/services/data_mode.py:9-11`
`auto` is live only if `TAVILY_API_KEY` , `SEARXNG_BASE_URL` or the google-play package is configured; iTunes alone does not switch it	`backend/app/core/collectors/selection.py:77-97`
In sample mode the pages show a "Sample data" badge and nothing contacts a real platform	`selection.py:99-100`
Only admins change the mode (Settings > Account); the guarded demo admin can only select `sample`	`backend/app/api/settings.py:130-136,181-182`
Sample-mode candidate URLs are never sent to Safe Browsing	section 8
Source health reasons such as `no_key` , `invalid_key` , `rate_limited` are shown on the Source Health page (admin)	`backend/app/core/collectors/tavily.py:114,209,216`

If you expected live data: add `TAVILY_API_KEY` , restart the API, open Settings and choose Live or Auto, check Source Health for reasons, then run `python -m scripts.live_check` from `backend/` (costs one credit, see `docs/COLLECTORS.md`).

10. Sign-in loops and unexpected sign-out

Cause	Detail	Fix
`JWT_SECRET` changed or differs between instances	existing tokens fail to decode, refresh returns 401, the client clears the session and goes to `/login` (`backend/app/api/auth.py:127-134` , `frontend/src/api/client.ts:255,293-294`)	Keep one stable `JWT_SECRET` ; sign in again.
Two tabs refresh at the same moment	refresh tokens rotate; presenting a rotated token again revokes all of that user's refresh tokens, so both tabs are signed out (`auth.py:136-141`)	Sign in again; avoid hammering one session from several tabs after the access token (15 min) expires.
Account disabled	`user.active` false is refused on login and refresh (`auth.py:70,143`)	Re-enable the user (admin).
Visiting Studio signed out	Studio redirects to `/login?next=/studio/` , and after sign-in the `next` value is honoured only if it starts with `/studio/` (`studio/src/lib/session.ts:71-76` , `frontend/src/utils/studio.ts:14-17` , `frontend/src/pages/LoginPage.tsx:51-52`)	Sign in on the BrandShield origin.
Different origin for Studio	the session lives in `localStorage` keys `scammind.auth` and `scammind.user` which are per origin (`client.ts:57-58` , `studio/src/lib/session.ts:2-3`)	Serve both apps on one origin (the Vite proxy, nginx or `vercel.json` do this).
Browser blocks storage	session cannot persist (`studio/src/lib/session.ts:35-38`)	Allow site storage / leave private mode.
Admin-only page redirects to the dashboard	`RequireAdmin` sends non-admins away (`frontend/src/App.tsx:39-54`)	Sign in as an admin; analysts and viewers cannot open `/sources` , `/settings` , `/evaluation` .

11. Microphone blocked / voice needs Chrome or Edge

Symptom	Source	Fix
"Microphone access was blocked. Allow it in the browser address bar, then try again."	`frontend/src/hooks/useVoiceAgent.ts:326-327` (`not-allowed` / `service-not-allowed`)	Click the lock icon in the address bar, allow Microphone for this site, press the voice button again. The microphone is requested only after you switch the agent on (`:342`).
"Voice commands need Chrome or Edge on a computer or Android phone."	`useVoiceAgent.ts:42-43,376` : the browser must provide `SpeechRecognition` or `webkitSpeechRecognition`	Use Chrome or Edge.
"The microphone stopped unexpectedly. Press the button to try again."	`useVoiceAgent.ts:328`	Press the button again.
Microphone blocked on a deployed site even after allowing	An older `Permissions-Policy: microphone=()` header disables the microphone for the page. It is now `microphone=(self)` in `vercel.json:106` and `frontend/nginx/default.conf.template:17` ; the standalone Studio nginx (`studio/`) still sends `microphone=()`	Redeploy so the new header is served; check the response header in the browser dev tools. Not verified on a deployed build.

12. Studio redirects when opened on port 3000

Brand Studio's dev server on port 3000 has no BrandShield login; the session lives on port 5173's origin. Opening `http://localhost:3000` now redirects to `http://localhost:5173/studio/` (`studio/src/lib/devRedirect.ts:16-21`). Always use **http://localhost:5173** (Studio is at `/studio/`). If the redirect does not happen, check you are on `localhost` or `127.0.0.1` and port exactly 3000. In Docker, port 3000 is only published by the dev overlay (`docker-compose.dev.yml:18-19`); use `http://localhost:5173/studio/` for the shared login. If 5173 shows "Bad gateway" for `/studio/` the Studio dev server is not running: `npm run frontend` starts both (`scripts/dev.mjs:87-88`).

13. Windows and CRLF issues

Symptom	Cause	Fix
Container fails with `exec ... no such file or directory` for `docker-entrypoint.sh`	CRLF line endings on a shell script	`.gitattributes` forces LF for `*.sh` , `Dockerfile` , `*.conf` , `*.template` (`.gitattributes:2-5`) and the Dockerfile also strips `\r` (`backend/Dockerfile:34`). Re-checkout after changing `core.autocrlf` ; keep `.gitattributes` .
`python` not found by `npm run backend`	the script uses `python` on Windows and `python3` elsewhere when no `.venv` exists (`scripts/dev.mjs:92`)	Install Python 3.11+ and add it to PATH, or create `backend/.venv` yourself.
Child process lingers after Ctrl+C	dev script kills the process tree with `taskkill /T /F` on Windows (`scripts/dev.mjs:39`)	Check Task Manager for leftover `python.exe` / `node.exe` if a port stays busy.
`npm run frontend` slow on first start	installs packages in `frontend/` and `studio/` (`scripts/dev.mjs:50-56`)	Wait.
`copy backend\.env.example backend\.env` complains	PowerShell vs bash syntax	In bash use `cp backend/.env.example backend/.env` .
Case-insensitive file systems	only a documentation concern: `docs/` already has `USER_GUIDE.md` and similar names in capitals	Do not create files that differ only by case.

14. Docker specifics

Symptom	Cause	Fix
`docker compose up` stops with "Set POSTGRES_PASSWORD in .env"	required variables use `${VAR:?...}` (`docker-compose.yml:20-23,43,79-80`)	Fill the five required values in the root `.env` .
Worker shows unhealthy	The healthcheck used to import a module that does not exist (`app.core.config`); it now imports `app.config.config` (`docker-compose.yml:122`). Otherwise Redis is down or the worker is not registered (`backend/app/services/health.py:99`)	Run `docker compose ps` and `docker compose logs worker` ; check `REDIS_URL` . Not verified by running Docker.
Password with special characters breaks the database URL	compose embeds passwords in URLs (`docker-compose.yml:19-21`)	Use hex passwords, e.g. `python -c "import secrets; print(secrets.token_hex(24))"` .
Frontend container exits: `API_URL must start with http:// or https://`	`frontend/nginx/40-runtime-config.sh:10-17`	Set `PUBLIC_API_URL` to a full origin without credentials or quotes.

DOCS/CONTRIBUTING.MD

Contributing

Plain-language summary

- Before you push, run the same checks CI runs: `ruff` for Python, `oxlint` and `tsc` for the web app, plus the test suites.
- The repo enforces style through CI only. It has no written branch, review or release policy, so those points are marked **Needs confirmation** below instead of being invented.
- The sections at the end are step-by-step recipes for the most common changes: a new API endpoint, a database migration, a new page, a new data source (collector) and a new scoring signal.
- Project rule: commits and pull requests must not carry AI or Claude co-author or "generated with" trailers.

1. Code style and checks

Area	Tool and rules	Source
Python lint	ruff, line length 140, target Python 3.11, rule sets `E F I UP B`, `B008` ignored (FastAPI `Depends()` defaults), `alembic/versions` excluded	`backend/pyproject.toml:24-31`
Python long lines	`E501` is ignored for tests, scripts, engine, collectors, providers, models, `app/api/*`, and a list of named modules	`backend/pyproject.toml:33-55`
Run	`python -m ruff check .` in `backend/`	`.github/workflows/ci.yml:31-33`
Python imports	sorted by ruff `I`; `# noqa: E402` is used where environment must be set before import (`backend/tests/conftest.py`, `backend/api/index.py`)	same
Frontend lint	oxlint with plugins `react`, `typescript`, `oxc`; `react/rules-of-hooks` is an error, `react/only-export-components` a warning	`frontend/.oxlintrc.json`, `frontend/package.json:9`
Frontend types	`tsc -b` (`npm run typecheck`); `noUnusedLocals`, `noUnusedParameters`, `noFallthroughCasesInSwitch`, `erasableSyntaxOnly`, `verbatimModuleSyntax`	`frontend/tsconfig.app.json:19-25`
Studio types	`tsc --noEmit` (`npm run lint`); `isolatedModules`, no strict flags	`studio/package.json:51`, `studio/tsconfig.json`
Line endings	LF for `*.sh`, `Dockerfile`, `*.conf`, `*.template`	`.gitattributes:2-5`
Secrets	never commit `.env`; only `.env.example` files are tracked	`.gitignore` (`.env`, `.env.*`, `!.env.example`, `!backend/.env.example`)

TypeScript strictness: **neither** `frontend/tsconfig.app.json` nor `studio/tsconfig.json` sets `"strict": true`. The frontend enables the unused and fallthrough checks listed above; Studio enables none of them. **Needs confirmation:** whether turning on `strict` is intended; it is not configured today.

Conventions visible in the code (not enforced by a tool): every API route that changes data records an audit row (`audit.record(...)`, for example `backend/app/api/health.py:47`); secrets are `SecretStr` in `Settings` and never logged (`backend/app/config/config.py:71,91,110`); module docstrings explain the "why" and cite the decision record (`docs/DECISIONS.md`).

2. Branches, pull requests and review

What the repo enforces:

Item	Fact	Source
CI triggers	pushes to `main` and every pull request	`.github/workflows/ci.yml:6-9`
Required checks	the jobs `lint`, `test-server` (3.11 and 3.12), `test-server-postgres`, `test-frontend`, `test-studio`, `e2e`, `build`	`ci.yml:19-231`
Pipeline order	`e2e` needs the three test jobs; `build` needs lint and the three test jobs	`ci.yml:160,204`
Stale runs	a newer push on the same ref cancels the older run	`ci.yml:14-16`
PR template, CODEOWNERS, branch-protection rules	none in the repo (`.github/` contains only `workflows/ci.yml`)	`.github/`

Needs confirmation: branch naming, whether `main` is protected, number of required reviewers, merge style (squash or merge commit) and release/tagging process are not defined anywhere in the repo. The git history shows direct work on `main` with remotes `origin`, `newrepo` and `brandshield-old`.

3. Commit messages

Read-only inspection of `git log` shows: a single imperative or descriptive sentence in sentence case, no issue numbers, no body, sometimes a short prefix naming the area. Prefixes seen: `Docs:`, `Fix ...`, `Studio:`, `Frontend:`, `Tests:`, `e2e:`, `Look-alike Lab:`, `Phase N:`, `Vercel ...`, `Brand Studio ...`, `Add ...`.

Examples from history:

- `Tests: 20 s default timeout for form tests (Register page flaked on Node 22)`
- `Studio: opening its dev server directly (port 3000) now continues on BrandShield's origin, where the shared login lives`

- `Docs: submission guide (project description, test access and evaluation walk-through)`
- `Fix CI: Node 22-safe logo test mock, browser tests follow the new Protect Your Brand sign-in flow and Setup wizard`

Rules:

- Describe what changed and why in one line; prefix with the area when it helps.
- **No AI or Claude co-author trailers.** Do not add `Co-Authored-By: Claude ...` or "Generated with Claude Code" lines to commits or pull request descriptions (project rule recorded in the maintainers' memory file `no-claude-coauthor-trailer.md`; it overrides any default).
- **Needs confirmation:** whether a formal convention (Conventional Commits) is wanted; none is configured (no commitlint, no hook).

4. Add a backend endpoint

1. **Router.** Create or extend a file in `backend/app/api/` with `router = APIRouter(prefix="/<name>")`. Pattern: `backend/app/api/providers.py:1-18`.
2. **Auth.** Use dependencies from `backend/app/config/deps.py`: `get_current_user`, `require_roles(...)`, `require_admin`, `get_settings_dep`, `get_store`; DB session via `Depends(get_db)` (`backend/app/db/session.py:52-57`). Brand data must be scoped to the caller's account (owner checks in `backend/app/api/brands.py`; isolation tests in `backend/tests/api/test_tenant_isolation.py`).
3. **Schema.** Request and response models are Pydantic; shared ones live in `backend/app/schemas/` (`auth.py`, `brand.py`), small ones are declared in the router file (see `ActiveIn` in `providers.py:23`).
4. **Register it.** Import the module in `backend/app/main.py:11-29` and add `api.include_router(<module>.router)` in the block at `main.py:170-181` (everything is mounted under `/api`; health is the only router mounted without it, `main.py:182`).
5. **Rate limits, cache.** `/api` calls count against the global limiter automatically (`main.py:119-135`). Read endpoints can opt into the short response cache by being listed in `backend/app/config/response_cache.py:15` (`CACHEABLE`); any write busts it (`main.py:155-158`).
6. **Audit.** Record changes with `audit.record(db, user, "<area>.<action>", target, detail)` then commit.
7. **Tests.** Add `backend/tests/api/test_<name>.py` using `client`, `admin_h` / `analyst_h` / `viewer_h`; test 401, 403 for the wrong role, 404 for another account's data (see `docs/testing.md`).
8. **Docs.** Update `docs/API.md` and, if the user sees it, the matching page in `docs/features/`.
9. **Frontend.** Add the call in `frontend/src/api/sm.ts` (object `sm`, line 64) and types in `frontend/src/api/types.ts`.
10. Run `python -m ruff check .` and `python -m pytest -q`.

5. Add a database migration (Alembic)

Migrations live in `backend/alembic/versions/` and are numbered by hand (`0001` to `0012` today). Template: `0012_active_brand.py`.

1. Change the SQLAlchemy model in `backend/app/models/` (and export it in `backend/app/models/__init__.py` if it is a new class, because `alembic/env.py:1` imports `app.models` to register tables).
2. Create `backend/alembic/versions/0013_<short_name>.py` with `revision = '0013'`, `down_revision = '0012'`, and both `upgrade()` and `downgrade()`. Every existing migration defines `downgrade()`.
3. Keep it additive and nullable where possible (see the note in `0012`), and SQLite-safe: Alembic runs with `render_as_batch=True` (`backend/alembic/env.py:16,25`).
4. Apply: `python -m scripts.migrate` from `backend/` (safe with several replicas on PostgreSQL: `backend/scripts/migrate.py:32-42`). The dev command `npm run backend` also applies pending migrations.
5. Test on SQLite and PostgreSQL: add to `backend/tests/db/` (patterns: `test_migration.py`, `test_migration_0010.py`); CI runs the suite on PostgreSQL (`ci.yml:73-112`) and runs the migration twice.
6. Update `docs/DATABASE_SCHEMA.md` / `docs/schema.sql` (generated by `backend/scripts/export_schema.py`).
7. On Vercel the migration is a manual step (`docs/setup-and-deployment.md` section 6).

6. Add a frontend page

1. Create `frontend/src/pages/<Name>Page.tsx` and a `.css` file next to it (pattern: `InvestigationsPage.tsx`).
2. **Route.** In `frontend/src/App.tsx` import it and add a `<Route path="/<name>" element={<AppLayout><NamePage /></AppLayout>} />` inside `<Route element={<RequireAuth />}>` (`App.tsx:89`). Put it inside `<Route element={<RequireAdmin />}>` (`App.tsx:175-179`) for admin-only pages.
3. **Layout.** Wrap with `AppLayout` (`frontend/src/components/layout/AppLayout.tsx`): it renders the `Sidebar` and remounts the page when the active brand changes.
4. **Sidebar.** Add a `NavLink` with an icon in `frontend/src/components/layout/Sidebar.tsx` (items start at the `sidebar-nav` block, around line 60); hide it for non-admins using `isAdmin` (`Sidebar.tsx:28`) when needed.
5. **Voice agent.** Add `{ path, spoken_names }` to `ALL_ROUTES` in `frontend/src/utils/voice.ts:74-89` (and to `ADMIN_ONLY`, `:72`, if admin only), so "open " works. Give form fields a `data-voice="<field>"` attribute so the agent can fill them (examples: `Sidebar.tsx:49`, `BrandProfilePage.tsx:265`, `CandidateTools.tsx:120`); `data-voice-row` marks list rows (`Sidebar.tsx:156`). Password-like fields are never filled by voice (`voice.ts:96`, `backend/app/services/voice.py:38`).
6. **Data.** Fetch through `useApi` / the `sm` client (`frontend/src/api/sm.ts`) so the page cache and 401 refresh apply.
7. **Tests.** Add `<Name>Page.test.tsx` (pattern `InvestigationsPage.test.tsx`) and, for user flows, a spec in `frontend/e2e/`. Add the page to the full-walk test in `frontend/e2e/smoke.spec.ts` if it should be checked for console errors.
8. Run `npm run lint`, `npm run typecheck`, `npm test`, `npm run build` in `frontend/`.

7. Add a collector (data source)

1. Subclass `Collector` in a new file under `backend/app/core/collectors/` (`backend/app/core/collectors/base.py:179-196`): set `name`, `kind`, `platforms`, optionally `is_configured()`, and implement `collect(brand, ctx)` returning `Candidate` objects. It must raise only `SourceUnavailable(reason, message)`; the orchestrator catches it and carries on (`base.py:1-6`, `:36-45`).
2. Use a short `reason` code (for example `no_key`, `rate_limited`, `timeout`). Reasons in `PERMANENT_REASONS` are not retried automatically (`base.py:31`).
3. Use the shared SSRF-safe HTTP helpers (`default_client_factory`, `base.py:203`) and a per-scan query cap setting.
4. Add settings to `backend/app/config/config.py` (keys as `SecretStr`) and names to `backend/.env.example`.
5. Register it: add the name to `CHAIN_ORDER` (`backend/app/core/collectors/selection.py:27`), a `ChainEntry` in `select_collectors` (`selection.py:102-113`, which decides what counts as "live"), and construct it in `CollectorRuntime.build` (`selection.py:155` onward).

6. Credits or rate limits: see `backend/app/core/collectors/credits.py` (Tavily ledger) and the iTunes limiter in `CollectorRuntime`.
7. Fixtures and tests: hand-written responses under `backend/tests/fixtures/<name>/` and `backend/tests/core/collectors/test_<name>.py` using `httpx.MockTransport` (live calls fail the suite, `conftest.py:72-79`). Update `docs/COLLECTORS.md`.
8. Platforms shown on Source Health are listed in `base.py:28`; add new ones there.

8. Add a scoring signal

Scores explain themselves: every point is a `Reason(signal, points, max_points, note)` (`backend/app/core/engine/scoring.py:48-55`).

1. Decide the category: social (`SOCIAL_MAX`, `scoring.py:34`), app (`APP_MAX`, `:35`) or web (`WEB_MAX`, `backend/app/core/engine/domain_match.py:39`, plus `url_reputation` in `WEIGHTS_V2_WEB`, `backend/app/core/engine/risk_engine.py:43`).
2. Add the key with its maximum points to the table, then emit a `Reason` in the scoring function for that candidate type (for social, the function around `scoring.py:141-184`; web signals through `assess_web`, `risk_engine.py:199`).
3. The risk engine wraps reasons into `Signal` objects with provenance and status `present | clean | unavailable` (`risk_engine.py:121-148`); a signal that could not be checked must be `unavailable`, never "clean".
4. Weights are versioned: `ALGORITHM_V1 = "1.0"` (social, app) and `ALGORITHM_V2 = "2.0"` (web) (`risk_engine.py:36-37`). Changing existing weights moves real scores, and the repo pins them with `backend/tests/fixtures/golden_scores_v1.json` and `backend/tests/core/engine/test_risk_engine.py`. **Needs confirmation:** the approval process for changing weights; `risk_engine.py:41-42` notes new web weights "need the owner's approval, see docs/DECISIONS.md" but no procedure is written.
5. Severity cutoffs (`LOW` below 40, `MEDIUM` 40, `HIGH` 70, `CRITICAL` 85) come from `RiskConfig` and `SEVERITY_*_MIN` settings (`risk_engine.py:47-77`, `config.py:166-168`); the older band cutoffs (high 85, medium 70) are in `backend/app/core/pipeline/bands.py:1-11`.
6. Tests: extend `backend/tests/core/engine/test_scoring_pipeline.py` or `test_risk_engine.py`; keep the explanation text meaningful because it is shown to users and in reports.
7. Update the matching guide in `docs/features/` and `docs/glossary.md` if you introduce a new term.

DOCS/GLOSSARY.MD

Glossary

Plain-language summary

- This page defines the words BrandShield uses, in the sense the code uses them, with a pointer to where each one lives.
- Terms are grouped: what the product finds, how it matches, how it scores, where data comes from, who owns what.
- If a word here differs from everyday usage (for example "official" or "clean"), the code's meaning wins.

Detecting and reviewing

Term	Meaning in BrandShield	Where
Detection	A suspected impersonation stored for a person to review. Types: `fake_profile`, `fake_brand_page`, `scam_account`, `fake_app`, `lookalike_domain`; categories `social`, `apps`, `web`. Starts with status `new`; "a human must confirm every one".	`backend/app/models/detection.py:10-33`
Candidate	Raw material before scoring: something a collector found (a social profile, an app, a web page) or a person typed or imported. Becomes a detection if it scores high enough. Kinds: social, app, web.	`backend/app/core/collectors/base.py:1-6`, `backend/app/core/engine/models.py:34-60`
Candidate origin	Where a candidate came from: `scan` (a collector), `manual`, `import` (CSV/JSON) or `synthetic` (sample data).	`backend/app/models/detection.py:51-52`
Detection status	`new`, `reviewing`, `confirmed`, `dismissed`, `marked_official`, `cleared`. People can set `reviewing`, `confirmed`, `dismissed`; `cleared` (below threshold but near-name) and `marked_official` never appear in the lists.	`backend/app/models/detection.py:30-31`, `backend/app/services/detections.py:30-34`
Resolution	The outcome of an investigation, separate from status: `true_positive`, `false_positive`, `resolved`, `monitoring` (none recorded = null). Setting it writes a history event and an audit row.	`backend/app/models/detection.py:35`, `backend/app/services/detections.py:329-343`
Investigation	The review workflow around a detection: status, resolution, notes, event history, score history.	`backend/app/models/detection.py:84-98`, `frontend/src/pages/InvestigationsPage.tsx`
Stale	A cached copy of a candidate shown because the source was unavailable; it is not re-checked (for example by Safe Browsing).	`backend/app/models/detection.py:49`, `backend/app/core/pipeline/scan_pipeline.py:461-462`
Evidence	Page URL, capture time and SHA-256 of a saved snapshot (and optional screenshot) attached to a detection.	`backend/app/models/detection.py:44-45`
Report	A CSV or HTML export of detections (no PDF), prepared as a draft and approved by a person; nothing is sent.	`backend/app/models/ops.py:94-100`, `README.md`
Canary	A self-check that plants a fictional fake brand in an in-memory database and runs the real pipeline to prove name matching, allowlist and scoring still work. It makes no network calls.	`backend/app/api/health.py:43-49`, `backend/scripts/canary.py`

Brand assets and exclusion

Term	Meaning	Where
Brand	What an account protects: name, aliases, keywords, domains, logo. An account may own several (max 10, archived ones excluded).	`backend/app/models/brand.py:10-40`, `backend/app/api/brands.py:46,145-146`
Official asset	Something that really belongs to the brand: an official social account (platform and handle), official app (package id, store ids, publisher), official domain, or logo. Verified assets record a verification state.	`backend/app/models/brand.py:43-125`
Allowlist	The exact-match set built from official assets. Matching candidates are dropped before scoring and listed as "Excluded as official". Never excludes by name alone. Handles are compared without `@` and in lower case.	`backend/app/core/engine/allowlist.py:1-30`, `backend/app/models/detection.py:56-62`
Marked official	A reviewer's decision that a detection is actually the brand's own asset (status `marked_official`).	`backend/app/services/detections.py:31`
Brand registry	The Brand Registry page and `/brand` route where official assets are managed.	`frontend/src/App.tsx:107-115`, `frontend/src/components/layout/Sidebar.tsx`
Active brand	Which of an account's brands the user is currently working on; stored on the user and revalidated on every read. Switch via `POST /api/brands/{id}/select`.	`backend/app/models/user.py:21`, `backend/app/api/brands.py:129-150`, `backend/alembic/versions/0012_active_brand.py:1-6`

Matching and look-alikes

Term	Meaning	Where
Look-alike	A name, handle or domain deliberately close to the brand's. Techniques named by the matcher: homoglyph, digit or symbol swap, character swap, missing or extra letter, transposition, spacing or separator, added word.	`backend/app/core/engine/name_match.py:56-75`, `backend/app/core/engine/synthetic.py:1-25`
Look-alike Lab	The page with a name tester, test cases per brand, and logo comparison.	`frontend/src/pages/LookalikeLabPage.tsx`, `backend/app/api/lookalike.py`
Homoglyph	A character that looks like another: Unicode look-alikes, fullwidth forms, accented letters, plus folding of `0/o`, `1/l`, `3/e`, `rn/m`, `vv/w`. Flag `F_HOMOGLYPH`.	`backend/app/core/engine/normalize.py:12`, `backend/app/core/engine/name_match.py:17`
Punycode	The ASCII form (`xn--...`) of an internationalized domain name. The code keeps both forms: ASCII for comparison and Unicode for display and homoglyph checks.	`backend/app/core/engine/domain_match.py:7,197-214`
IDN homograph	A domain mixing scripts or using look-alike letters from other alphabets; worth up to 15 points.	`backend/app/core/engine/domain_match.py:39,348`
Registrable domain	The part of a host you can register: `label.public-suffix`, found with `tldextract` and its bundled public suffix list (`shop.acme-bank.co.uk` gives `acme-bank.co.uk`). Official-domain exclusion and look-alike comparison use it, so a subdomain on an official domain is not a look-alike.	`backend/app/core/engine/domain_match.py:5-12,220`
Brand in subdomain	A deceptive host such as `acme.evil.com`; up to 20 points.	`backend/app/core/engine/domain_match.py:39`
Added words	Extra words next to the brand name. Strong scam words (`STRONG_SCAM_WORDS`) and impersonation words ("support", "kyc"; `IMPERSONATION_WORDS`) add points only when the name already matched.	`backend/app/core/engine/vocab.py:4,14,61`, `backend/app/core/engine/scoring.py:150-162`
pHash / dHash / wHash	Perceptual image hashes. Logo similarity is 0-100 from pHash and dHash by default (algorithm `image-1`); `multihash` adds wHash (`image-2`: 0.4 pHash, 0.3 dHash, 0.3 wHash). "Likely copy" starts at 80, "similar" at 55. A hint, not proof of reuse.	`backend/app/core/engine/logo_similarity.py:1-32`, `backend/app/core/engine/image_hashes.py:1-14`
CLIP	An optional AI vision model (via `fastembed`, model `Qdrant/clip-ViT-B-32-vision`, ~336 MB download) used by the Look-alike Lab logo tester when installed (`LOGO_COMPARE_CLIP=auto`). Scans use hashes unless `IMAGE_SIMILARITY_BACKEND=clip`. Not available on Vercel.	`backend/requirements-extra.txt`, `backend/app/config/config.py:79-83`
Safe Browsing	Google's URL reputation service (Lookup API v4). Candidate URLs are sent to Google, never opened. Verdicts: `match`, `clean` (not a safety guarantee), `unknown`, `failed`; the last two reach the engine as `unavailable`. Needs `GOOGLE_SAFE_BROWSING_API_KEY`; non-commercial terms.	`backend/app/core/providers/safe_browsing.py:1-17`, `backend/app/core/providers/base.py:19-22`
SSRF guard	Every outbound fetch using a URL from data resolves DNS and refuses private, loopback, link-local and metadata addresses, limits redirects, size and content type.	`backend/app/core/pipeline/url_safety.py`, `backend/app/core/pipeline/image_fetch.py`

Scoring

Term	Meaning	Where
Score	0-100 integer for a candidate; every point comes from a `Reason` with signal name, points, maximum and a note, so every score is explainable. Social maxima: name 34, logo 28, added words 12, bio 28, account 18. App maxima: name 24, icon 30, publisher 22, description 17, store 10.	`backend/app/core/engine/scoring.py:34-35,48-55`
Signal	A scoring input with status `present`, `clean` or `unavailable` and a provenance.	`backend/app/core/engine/risk_engine.py:70-90`
Score band	The older three-way label stored in `Detection.band`: `high` 85+, `medium` 70-84, `low` below 70.	`backend/app/core/pipeline/bands.py:1-11`, `backend/app/models/detection.py:25`
Severity	The newer four-way label `LOW`, `MEDIUM`, `HIGH`, `CRITICAL`, stored beside the band. Default cutoffs: MEDIUM 40, HIGH 70, CRITICAL 85 (`SEVERITY_MEDIUM_MIN`, `SEVERITY_HIGH_MIN`, `SEVERITY_CRITICAL_MIN`).	`backend/app/core/engine/risk_engine.py:47-68`, `backend/app/config/config.py:166-168`
Algorithm version	`1.0` for social and app scoring, `2.0` for web scoring.	`backend/app/core/engine/risk_engine.py:36-37`, `backend/app/models/detection.py:28`
Provenance	A label on each signal telling where it came from: `real_api` (live external answer), `local_algorithm` (deterministic code here), `pretrained_model`, `synthetic` (sample data, "not real evidence").	`backend/app/core/providers/base.py:10-15`
Threshold / sensitivity	The score above which a candidate becomes a detection; adjustable in Settings and previewed with "would review" counts.	`backend/app/api/settings.py:206`
Prefilter	Skips logo and AI-text steps for candidates with no name signal and no keyword hit (`SCAN_PREFILTER`).	`backend/app/config/config.py:115`
Model Evaluation	Admin page that measures the name matcher on synthetic mutations and hard negatives; "NOT ground truth for real-world attackers".	`backend/app/core/engine/synthetic.py:1-8`, `backend/app/api/evaluation.py`

Data sources and modes

Term	Meaning	Where
Collector	A class that fetches raw results from one provider and returns candidates; raises only `SourceUnavailable`. Live chain: Tavily, SearXNG (optional), iTunes, Google Play (optional); plus a sample collector.	`backend/app/core/collectors/base.py:179-196`, `backend/app/core/collectors/selection.py:27`
Source / Source Health	A platform row (instagram, facebook, x, telegram, linkedin, google_play, app_store, third_party_apk) with state and reasons such as `no_key`, `rate_limited`, `timeout`. Shown to admins on Source Health.	`backend/app/core/collectors/base.py:28`, `frontend/src/pages/SourceHealthPage.tsx`
Data mode	Setting with values `live`, `sample`, `auto` (default). `sample` uses fictional data and the "Sample data" badge; `live` queries real sources; `auto` is live only when a Tavily key, SearXNG URL or the google-play package is configured (iTunes alone does not count). Admin only.	`backend/app/services/data_mode.py:9-11`, `backend/app/core/collectors/selection.py:77-97`
Effective mode	What the data mode resolves to right now (`live` or `sample`).	`backend/app/services/data_mode.py:34-38`
Sample data	Fictional detections for a demo; flagged `source = "sample"`; never sent to Safe Browsing.	`backend/app/models/detection.py:50`, `backend/app/core/pipeline/scan_pipeline.py:457-460`
Credits	Tavily's monthly budget (`TAVILY_MONTHLY_BUDGET`, default 900 of the free 1000) counted in the ledger; one scan costs up to `TAVILY_MAX_QUERIES_PER_SCAN` (12). Studio and Safe Browsing have separate usage counters in `api_usage`.	`backend/app/config/config.py:92-93`, `backend/app/core/collectors/credits.py`
Scan	One run: collect, de-duplicate, allowlist, name match, logo and text signals, score, save. One queued or running scan per brand is enforced by the database.	`backend/app/models/ops.py:53-58`, `backend/app/core/pipeline/scan_pipeline.py`
Scan executor	How scans run: `auto` (RQ if `REDIS_URL`, else thread), `rq`, `thread`, `inline` (tests, Vercel).	`backend/app/config/config.py:111`
Scheduler / cron	Automatic scans: an in-process loop (one process only) or Vercel Cron calling `GET /api/cron/scan`.	`backend/app/services/scheduler.py:147`, `backend/app/api/cron.py`
Text provider	Optional hosted AI text analysis of bios and descriptions (`gemini`, `huggingface`, `local`, `off`); off by default so no candidate text leaves the server.	`backend/app/config/config.py:66-69`, `backend/app/api/providers.py`

Accounts, ownership and access

Term	Meaning	Where
Tenant / owner	Each account owns its own brands; the brand's `owner_id` links to the user. Another account's brand, detections, reports, logos and evidence answer 404. (Settings, data mode, source health and credit counters are shared.)	`backend/app/models/brand.py:14`, `backend/tests/api/test_tenant_isolation.py`
Role	`admin` (settings, data mode, health details, canary, Source Health, Evaluation), `analyst` (review and scan; self-registered and demo accounts), `viewer` (read-only).	`backend/app/config/deps.py:46-57`, `backend/app/api/auth.py:114`
Demo account	A public shared analyst account from `DEMO_EMAIL` / `DEMO_PASSWORD`, advertised by a button on the login page. Both must be set.	`backend/app/config/config.py:47-48`, `backend/app/services/seed.py`
Demo administrator	A second public, guarded admin from `DEMO_ADMIN_EMAIL` / `DEMO_ADMIN_PASSWORD`: cannot select live or auto data, change the AI provider, see other accounts' audit rows or set the webhook (403 "Disabled for the shared demo administrator").	`backend/app/services/seed.py:113-116`, `backend/app/api/settings.py:130-140`
Access / refresh token	JWT pair: access lasts 15 minutes, refresh 7 days; refresh tokens are stored server-side and rotated on each use; reusing a rotated one revokes all of that user's refresh tokens. Stored in browser `localStorage` under `scammind.auth` and `scammind.user`.	`backend/app/config/config.py:42-43`, `backend/app/api/auth.py:127-158`, `frontend/src/api/client.ts:57-58`
Audit log	Record of who did what (for example `detection.resolution_set`, `settings.data_mode_changed`, `health.canary`).	`backend/app/services/audit.py`, `backend/app/api/audit.py`

Apps and surfaces

Term	Meaning	Where
Brand Studio	A separate React app at `/studio/` (naming, logos, palettes, copy) sharing the BrandShield login and registering brands in BrandShield.	`studio/vite.config.ts:11`, `docs/STUDIO.md`
Studio AI proxy	Server endpoints (`/api/studio/ai/*`) that call Groq or Gemini (chat) and Pollinations (images) so no provider key reaches the browser.	`backend/app/services/studio_ai.py:1-8`, `backend/app/api/studio_ai.py`
Voice agent	A signed-in microphone assistant: simple commands run locally, other sentences use Gemini function calling via `POST /api/voice`. Chrome or Edge only.	`backend/app/services/voice.py`, `frontend/src/hooks/useVoiceAgent.ts`
`data-voice`	HTML attribute naming a field the voice agent may fill.	`frontend/src/components/layout/Sidebar.tsx:49`
Migration	An Alembic script that changes the database schema; numbered `0001` to `0012`.	`backend/alembic/versions/`
Worker	The RQ process that runs scans when Redis is configured (`python -m app.worker`).	`backend/app/worker.py`

DOCS/OPEN-QUESTIONS.MD

Open questions and technical debt

Plain-language summary. While the documentation was written from the code, a number of things were found that are either unclear (a decision only the owner can make) or look like defects. Nothing here was changed in the code unless stated. Each item points to the document that has the details and the file references.

Items marked **Needs confirmation** are questions. Items under "Defects and inconsistencies" are things the code does that contradict its own comments, its UI or its docs.

1. Security and access (decide first)

#	Item	Where
1	**Admin-only pages are admin-only in the browser only.** Settings, Source Health and Model Evaluation are hidden and guarded in the frontend (`frontend/src/App.tsx` `RequireAdmin` , `Sidebar.tsx`), but the backend lets any signed-in user read `GET /api/settings` , `/api/sources` , `/api/settings/data-mode` , `/api/providers` . Any analyst (including self-registered ones) can `PATCH /api/sources/{id}` and `POST /api/sources/{id}/retry` , which change shared state and can spend search credits. `GET /api/evaluation` is open to any role and analysts can re-run it. **Needs confirmation:** should these routes require an administrator?	auth-and-security.md, features/source-health-and-providers.md, features/model-evaluation.md, features/settings-and-audit.md
2	**The public demo admin can still change global monitoring settings** (schedule, sensitivity, rules, impersonation words, source toggles); `POST /api/sources/{id}/retry` has no demo-admin guard. **Needs confirmation:** acceptable?	features/authentication-and-accounts.md
3	`GET /api/settings` shows sensitivity, rules and impersonation words to every role. **Needs confirmation.**	features/settings-and-audit.md
4	A report's creator can approve their own report; reports do not re-verify the evidence hash. (From the unpublished reports draft, see section 5.)	section 5
5	Log and Sentry masking omits `GROQ_API_KEY` , `POLLINATIONS_API_KEY` , `CRON_SECRET` and `DATABASE_URL` (`Settings.secret_values` , `backend/app/config/config.py`).	auth-and-security.md
6	`/docs` and `/openapi.json` are not disabled (`backend/app/main.py:94`). **Needs confirmation:** public on Vercel/Docker?	auth-and-security.md
7	No endpoint creates `viewer` accounts or changes roles. **Needs confirmation:** how should production viewers be created?	auth-and-security.md
8	Client IP for per-IP limits is `request.client.host` with no proxy-header handling on Vercel; in-memory rate limiters and the response cache are per instance there. **Needs confirmation.**	auth-and-security.md, background-jobs-and-integrations.md
9	Row-level security exists only on `stored_files` and `studio_items` . **Needs confirmation:** is the Supabase Data API disabled for the other tables? Also: do dev, e2e and production share one Supabase project (the repo does not say; local runs here use the hosted database)?	data-model.md, auth-and-security.md
10	The standalone Studio nginx still sends `Permissions-Policy: microphone=()` .	auth-and-security.md
11	The frontend CSP restricts `img-src` to `API_URL` . **Needs confirmation:** does it block any Studio images when Compose proxies `/studio/` ?	auth-and-security.md
12	Google Safe Browsing: manually added web URLs are sent to Google when a key is set. **Needs confirmation:** intended? The API's terms are non-commercial; Google suggests Web Risk for commercial use.	features/web-threats.md
13	`/api/health` is not exempt from the rate limiter, although a comment in `main.py` says health routes are.	api-reference.md

2. Defects and inconsistencies found in the code

#	Item	Where
1	The Setup wizard offers values the backend rejects with 422: platforms `tiktok` and `telegram`, store `apple` (backend expects `app_store`).	features/brand-registry.md
2	The wizard's scan-depth and alert-email fields are collected but never sent.	features/brand-registry.md
3	Removing an official account, app, logo or publisher does not reopen detections that were closed as `marked_official`.	features/brand-registry.md
4	No endpoint lists archived brands, so restoring needs the brand id.	features/brand-registry.md
5	The Investigations "Marked official" status filter always returns an empty list.	features/investigations.md
6	`frontend/src/api/auth.ts` defines `forgot()` and `reset()` but `/api/auth/forgot` and `/api/auth/reset` do not exist.	features/authentication-and-accounts.md
7	Dashboard "Needs review" links web look-alike cases to `/social` instead of `/web`.	features/dashboard.md
8	Manual and imported candidates are saved with `source="live"`, so the default list filter hides them in sample mode. The import report's `accepted` count includes official and not-flagged rows.	features/social-and-app-threats.md, features/candidate-import.md
9	`REACHABILITY_CHECKS_PER_SCAN` and `SCHEDULER_MAX_QUERIES_PER_SCAN` are declared but nothing reads them; `TEXT_EMBEDDINGS_ENABLED` only feeds `/api/providers` status.	features/scanning-and-detection.md
10	UI wording that no code supports: Web Threats empty state mentions "web checks enabled"; the App page footer says store screenshots are saved as evidence but screenshots are off by default. `TestBody.platform` is never sent by the UI.	features/web-threats.md, features/lookalike-lab.md
11	`reports.file_path` is never read or written.	data-model.md
12	`google-genai` has different version ranges in `requirements.txt` (`>=1.20,<3`) and `requirements-extra.txt` (`>=2.30,<3`).	tech-stack.md
13	CLIP install hints differ between the UI (`pip install fastembed`) and the backend (mentions `requirements-extra.txt`).	features/lookalike-lab.md
14	Neither `frontend` nor `studio` enables TypeScript `strict`.	contributing.md
15	The comment at `docker-compose.yml:11` says `backend/` belongs to an older product, while the images build from `./backend`.	configuration.md

Fixed while this documentation was written: `Permissions-Policy` was `microphone=()` in `vercel.json` and the frontend nginx (now `microphone=(self)`), and the worker healthcheck in `docker-compose.yml` imported a non-existent module (now `app.config.config`). Neither fix was verified on a deployed or Docker build.

3. Operations and process

#	Question	Where
1	How are Vercel deploys triggered, and is there a documented rollback? There is no deploy job in CI and no staging environment in the repo.	setup-and-deployment.md
2	Is the Vercel cron in `services` mode working? It was not verified live (`docs/DEPLOY_VERCEL.md:55-57`).	background-jobs-and-integrations.md
3	Are the optional extras (fastembed, google-play-scraper, huggingface_hub) installed on Vercel?	background-jobs-and-integrations.md
4	Branch naming, branch protection, reviewers, merge style, release process, commit convention, and who approves scoring-weight changes. None is defined in the repo.	contributing.md
5	No script moves files between `STORAGE_BACKEND=local` and `database`.	data-model.md
6	The standalone landing-page HTML the React page was generated from is not in the repo.	features/landing-page.md

4. Documentation gaps

- Exact result keys of `/api/candidates/*`, the `/api/lookalike/test` verdict, non-database health statuses and the voice fast-path `reply` / `finish` values are not enumerated in api-reference.md.
- Per-page prop and layout detail in frontend.md was summarised from titles and imports, not a full read of every page.
- `EMAXCONNSESSION` is not a string in this repo; it is the code Supabase returns (troubleshooting.md).

5. Missing document

`docs/features/reports.md` was not written: the agent drafting it was refused by its Write tool and left a draft outside the repo. Its findings (from the draft's own list): reports do not re-verify the evidence hash, and the creator can approve their own report. Reports are described briefly in api-reference.md and the older `docs/USER_GUIDE.md`.

DOCS/FEATURES/AUTHENTICATION-AND-ACCOUNTS.MD

Authentication and accounts

Plain-language summary

People sign in with an email and a password. The server hands back a short-lived "access pass" (15 minutes by default) and a longer "renewal pass" (7 days). The browser quietly swaps the renewal pass for a fresh pair whenever the access pass runs out, so users are not thrown out mid-work. Anyone can create their own account on the Register page; it becomes an `analyst` that owns its own brand workspace. The login page can also show two shared demo buttons: a normal demo analyst and a demo administrator whose dangerous powers are switched off. There are three roles: `viewer` (read only), `analyst` (can work with detections, brands, scans) and `admin` (also settings, sources, evaluation).

Everything below is real code; there is no mock authentication. The demo accounts are real database users created from environment variables, but they are a convenience for demos, not a different code path.

Purpose and user-facing behaviour

Screen / action	What the user sees	Code
`/login`	Email, password, "Keep me signed in" checkbox, "Use demo credentials" / "Use admin demo credentials" buttons (only when the server advertises them), link to Register.	`frontend/src/pages/LoginPage.tsx:12` , `:188-229`
"Forgot password?"	Only shows an `alert()` saying reset by email is not available.	`frontend/src/pages/LoginPage.tsx:154-160`
`/register`	Name, email, password, confirm, live password checklist. Signs the user in at once and goes to `/protect-brand` .	`frontend/src/pages/RegisterPage.tsx:15-75`
Sign out (header menu)	Revokes the refresh token on the server, clears local storage, goes to `/login` .	`frontend/src/components/layout/Header.tsx:32-38` , `frontend/src/api/auth.ts:27-35`
Admin-only menu items	Source Health, Settings and Model Evaluation are hidden for non-admins.	`frontend/src/components/layout/Sidebar.tsx:33` , `:124` , `:143` , `:171`

Needs confirmation: `frontend/src/api/auth.ts:41-46` defines `forgot()` and `reset()` that call `POST /api/auth/forgot` and `POST /api/auth/reset` , but no such backend route exists (searched `backend/app`). They look unused (the Forgot password button only shows an alert). Confirm they can be deleted.

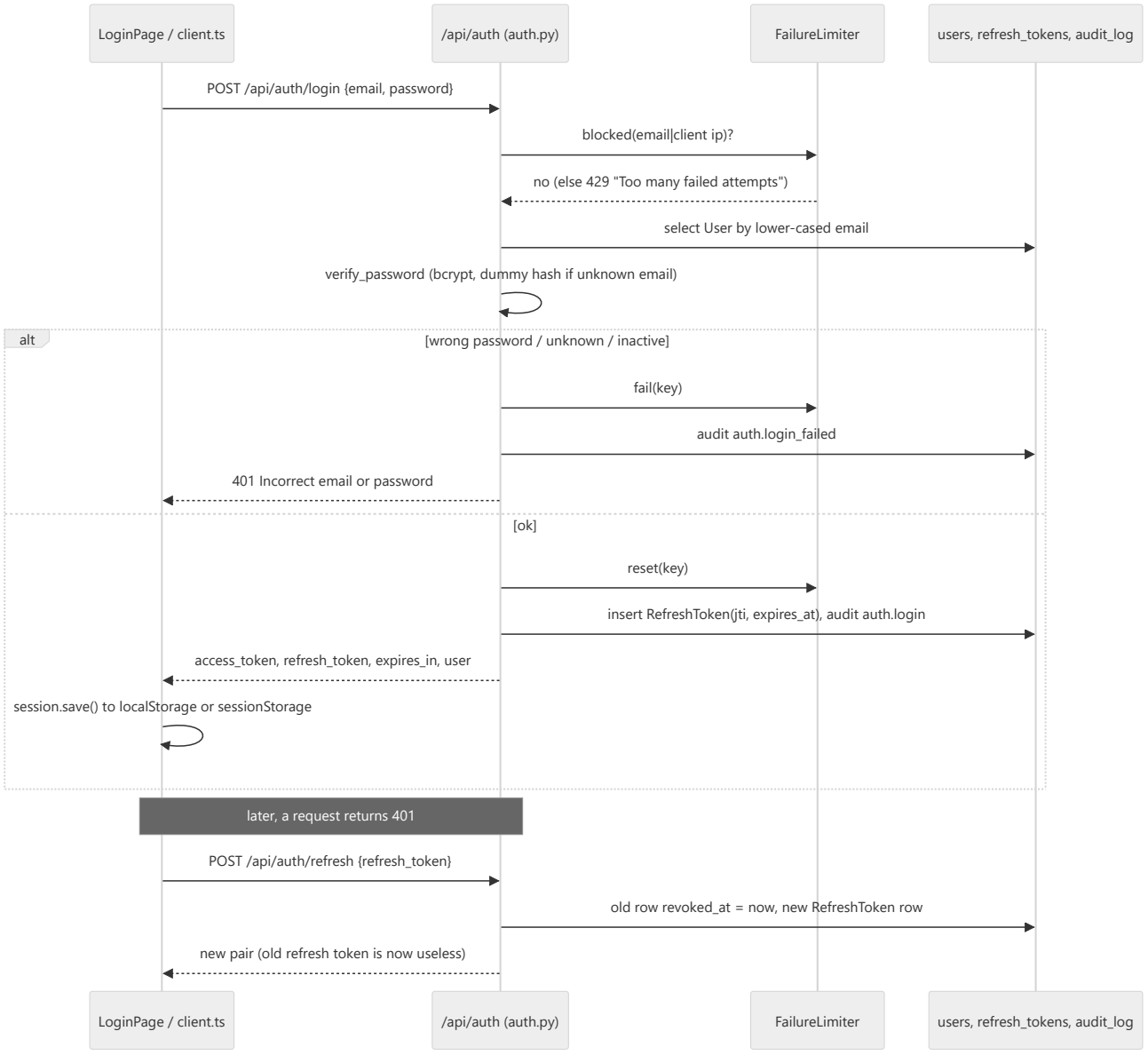
Roles

Role	Set how	May do	Enforced by
`viewer`	Default value of the column (`backend/app/models/user.py:17`); no self-service path creates one	`GET` endpoints only. Cannot read the audit log.	`require_writer` rejects with 403 (`backend/app/config/deps.py:46-52`)
`analyst`	Every self-registered account (`backend/app/api/auth.py:114`); the demo analyst (`backend/app/services/seed.py:44`)	Everything writer-level: scans, reviews, brand edits, reports, source monitoring toggle and retry	`require_writer = require_roles("admin", "analyst")` (`backend/app/config/deps.py:56`)
`admin`	Seeded from `ADMIN_EMAIL` / `ADMIN_PASSWORD` (`seed.py:14-24`) and the demo admin (`seed.py:71-97`)	Writer powers plus settings, data mode, provider choice, archive/restore a brand, Look-alike Lab cases, health details, canary	`require_admin = require_roles("admin")` (`backend/app/config/deps.py:57`)

Admin-only endpoints (from `Depends(require_admin)`): `PUT /api/settings` (`backend/app/api/settings.py:132`), `PUT /api/settings/data-mode` (`backend/app/api/sources.py:243`), `PUT /api/providers/active` (`backend/app/api/providers.py:51`), `DELETE /api/brands/{id}` archive and `POST .../restore` (`backend/app/api/brands.py:421` , `:434`), look-alike cases add/delete (`backend/app/api/lookalike.py:134` , `:148`), `GET /api/health/details` and canary (`backend/app/api/health.py:37` , `:44`).

Step-by-step flows

Login and token rotation

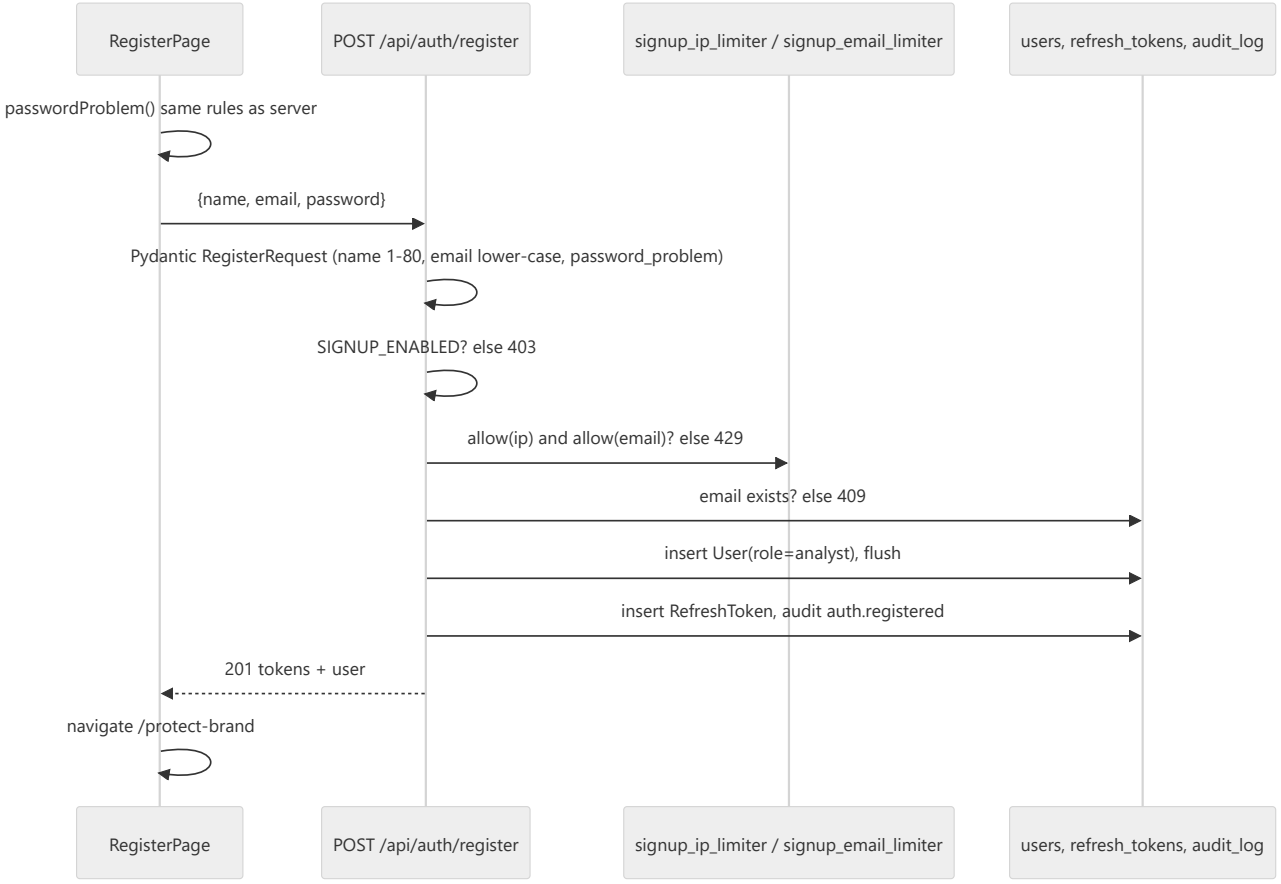


1. `POST /api/auth/login` (`backend/app/api/auth.py:56`). The limiter key is `"<email>|<client ip>"` (`:65`). After `LOGIN_MAX_FAILURES` (5) failures inside `LOGIN_WINDOW_SECONDS` (900) the answer is 429 (`:66-67` ; defaults at `backend/app/config/config.py:53-54`). The limiter is an in-memory sliding window per process (`backend/app/config/ratelimit.py:6-32`).
2. Timing equalisation: an unknown email is checked against `_DUMMY_HASH` so that response time does not reveal which emails exist (`auth.py:26` , `:69`).
3. A wrong password, an unknown email and an inactive user all give the same 401 text (`:70-74`) and an audit row `auth.login_failed` with `actor_label=email` .
4. Success issues tokens via `_issue` (`:39-53`): a JWT access token (`typ=access` , `ACCESS_TOKEN_MINUTES` =15) and a JWT refresh token (`typ=refresh` , `REFRESH_TOKEN_DAYS` =7). Each carries `sub` , `typ` , `iat` , `exp` , `jti` (`backend/app/config/security.py:26-35`), algorithm `JWT_ALGORITHM` (HS256) signed with `JWT_SECRET` (min length 16, `config.py:40`). The refresh token's `jti` is stored in `refresh_tokens` (`backend/app/models/user.py:24-34`).
5. `POST /api/auth/refresh` (`auth.py:148-156`) validates the token (`_active_refresh_row` , `:127-145`), marks the old row `revoked_at` , issues a new pair. **Reuse detection:** presenting an already-revoked refresh token revokes every refresh token of that user (`:136-141`) and returns 401. That is treated as a stolen token.
6. `POST /api/auth/logout` (`:159-170`) revokes the presented refresh token and writes `auth.logout` ; an already-invalid token returns `ok` anyway.
7. Passwords: bcrypt with 12 rounds (`security.py:11`), input cut at 72 bytes (`:12` , `:16`). `LoginRequest` rejects passwords over 72 bytes (`backend/app/schemas/auth.py:10-15`).

Browser session handling (`frontend/src/api/client.ts`)

- Tokens live under the single key `scammind.auth` , user under `scammind.user` (`client.ts:59-60`). "Keep me signed in" ticked means `localStorage` ; unticked means `sessionStorage` (tab only) (`:89-98` , `LoginPage.tsx:18`).
- Every request adds `Authorization: Bearer <access>` unless `anonymous` (`client.ts:297-307`).
- On a 401 the client refreshes once, single-flight, under a Web Lock named `scammind-refresh` shared across tabs and with Brand Studio (`:227-269`), then retries the original request once (`:310-348`). A 4xx on refresh clears the session and fires `scammind:unauthorized` ; a network error or 5xx keeps the session (`:254-263`).
- `session.clear()` also clears the query cache so the next user never sees the previous user's pages (`:121-126`).

Register (self-service)



- Endpoint `backend/app/api/auth.py:99-124`. Disabled with `SIGNUP_ENABLED=false` gives 403 "Sign-up is disabled" (`:107-108`).
- Rate limits: `SIGNUP_MAX_PER_IP_PER_HOUR` (5) and `SIGNUP_MAX_PER_EMAIL_PER_HOUR` (3), one-hour in-memory windows (`config.py:58-59`, `backend/app/main.py:100-101`). Both are counted by `allow()`, so each attempt, successful or not, uses a slot. A limit of 0 disables the check (`backend/app/config/ratelimit.py:46-48`).
- Validation (`backend/app/schemas/auth.py:45-71`): name cleaned of non-printable characters and 1-80 chars long; email lower-cased, at most 254; password through `password_problem` (`:30-42`): at least 10 characters, at most 72 bytes, not on the trivial list (`TRIVIAL_PASSWORDS`, `:19-26`), not all digits, at least 5 distinct characters, not equal to name, email or the email local part (or the local part doubled when it is 6+ chars).
- The frontend mirrors these rules in `frontend/src/utils/passwordPolicy.ts:24-54` (the server stays the authority) and maps statuses 403 / 409 / 422 / 429 to messages (`RegisterPage.tsx:56-71`).
- Duplicate email: pre-check (`auth.py:112-113`) plus `IntegrityError` catch for simultaneous sign-ups (`:116-120`), both 409.
- There is no email verification (`auth.py:106` docstring; `docs/DECISIONS.md`). A new account has no brand yet; the next screen `/protect-brand` creates the first one (see `docs/features/brand-registry.md`).

`/api/me`

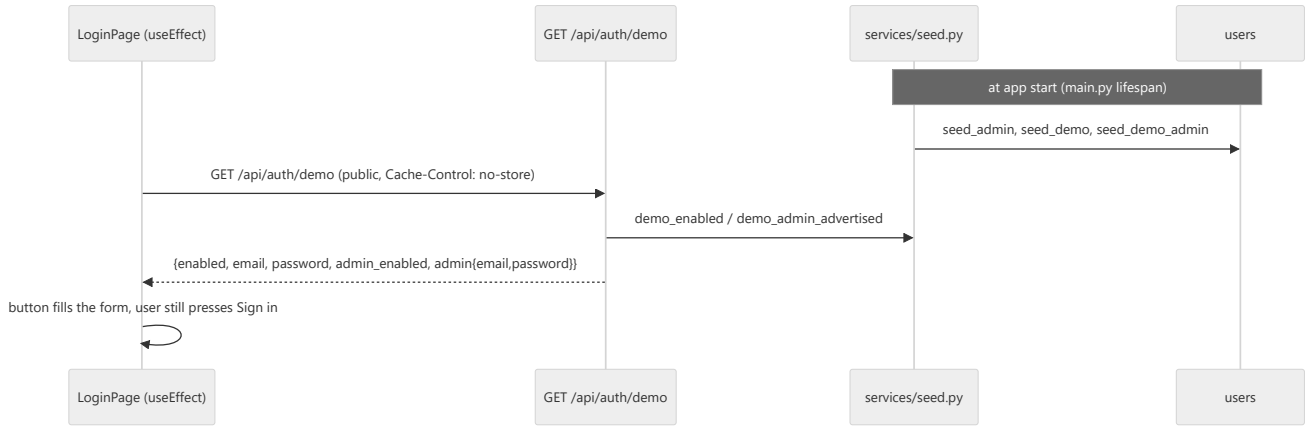
`GET /api/me` and `GET /api/auth/me` return `UserOut`: `id, name, email, role, active, default_brand_id` (`auth.py:173-176`, `schemas/auth.py:78-85`). `default_brand_id` is the account's current brand from `own_brand()` (`auth.py:29-36`). The frontend `useMe` shows the stored copy first, then confirms with the server and exposes `isAdmin` and `canWrite` (`frontend/src/hooks/useMe.ts:15-34`).

Route guards in the SPA

Guard	Behaviour	Code
`RequireAuth`	No stored token: redirect to `/login` (remembering `from`). Also listens for `scammind:unauthorized` to reset the brand cache and go to `/login`. Mounts the voice assistant button once above all signed-in routes.	`frontend/src/App.tsx:56-78`
`RequireAdmin`	Uses the role saved at sign-in; if missing, asks `GET /api/me` once. Non-admin gets `<Navigate to="/dashboard">`. Wraps `/sources`, `/settings`, `/evaluation`.	`App.tsx:39-54`, `:174-179`
`Home` at `/`	Signed in: go to `/dashboard`; else the landing page.	`App.tsx:29-36`

These guards are a convenience only. The real protection is the server-side `require_admin` / `require_writer` listed above.

Demo analyst and guarded demo administrator



- **Seeding** happens in the app lifespan: `seed_admin`, `seed_demo`, `seed_demo_admin` (`backend/app/main.py:76-80`). If the database is not migrated yet the seed is skipped with a warning (`:81-83`).
- **Real admin** is created only if absent and a password is never overwritten (`seed.py:14-24`).
- **Demo analyst** (`DEMO_EMAIL`, `DEMO_PASSWORD`): advertised only when both are set and the email differs from `ADMIN_EMAIL` (`seed.py:27-31`). The account is created as `Demo User`, role `analyst` (`:44`). A changed `DEMO_PASSWORD` is re-applied at next start, an inactive account is re-activated (`:51-59`), and an existing account with another role is left untouched (`:48-50`).
- **Demo administrator** (`DEMO_ADMIN_EMAIL`, `DEMO_ADMIN_PASSWORD`): enabled only when the email differs from both the real admin's and the demo analyst's (`seed.py:63-68`). Created as `Demo Admin`, role `admin` (`:81`); a non-admin account with that email is never promoted (`:85-87`). It is advertised only if the stored account really is an active admin with that password (`demo_admin_advertised`, `:100-105`).
- **`GET /api/auth/demo`** (`auth.py:82-96`) is deliberately public: it returns the demo credentials when enabled, `{"enabled": false}` otherwise, plus `admin_enabled` and `admin`. The real admin is never advertised. Because the password of a shared demo account is public by design, never put real data in the demo accounts.
- The login page also supports local build-time `VITE_DEMO_EMAIL` / `VITE_DEMO_PASSWORD` (`LoginPage.tsx:22-23`, `:38-39`); the server value wins.

What the demo administrator cannot do

The demo admin is recognised by email (`is_demo_admin`, `seed.py:108-110`). `forbid_demo_admin` raises 403 `Disabled for the shared demo administrator: <what>` (`seed.py:113-116`).

Guard	Rule	Code
Data mode	Only `sample` may be chosen; `live` and `auto` are refused (they spend search credits and change the demo for everyone). Applies to both `PUT /api/settings/data-mode` and `PUT /api/settings` with `data_mode`.	`backend/app/api/sources.py:246-247`; `backend/app/api/settings.py:136-137`
AI provider	`PUT /api/providers/active` is refused entirely.	`backend/app/api/providers.py:53`
Webhook	Setting a non-empty `notifications.webhook_url` is refused; an empty value from its form is silently skipped so it cannot clear the real URL; the stored URL is never shown back to it.	`settings.py:138-139`, `:171-172`, `:118`
Audit log	Sees only rows it created itself (`own_only`), both in the list and the CSV export. The real admin sees every row.	`backend/app/api/audit.py:36-40`, `:64`, `:80`

Other admin actions (monitoring schedule, sensitivity, rules, impersonation words, source toggles) are allowed for the demo admin. **Needs confirmation:** `POST /api/sources/{id}/retry` (`sources.py:179`) has no demo-admin guard; it is a writer endpoint, and credits are only spent after `confirm=true` when Tavily is active. Confirm that this is acceptable for the shared demo (the login page note at `LoginPage.tsx:225-229` does not mention it).

Files, tables, services

Item	Path
Auth routes	`backend/app/api/auth.py`
Token and password helpers	`backend/app/config/security.py`
Current-user dependency, role checks	`backend/app/config/deps.py:28-57` (`get_current_user`, `require_roles`)
Limiters	`backend/app/config/ratelimit.py` (`FailureLimiter`, `RequestLimiter`), wired in `backend/app/main.py:99-106`
Request schemas, password policy	`backend/app/schemas/auth.py`
Seeding and demo guards	`backend/app/services/seed.py`
Audit helper	`backend/app/services/audit.py:6-26`
Models	`backend/app/models/user.py` (`User`, `RefreshToken`)
Frontend	`frontend/src/api/auth.ts`, `frontend/src/api/client.ts`, `frontend/src/App.tsx`, `frontend/src/hooks/useMe.ts`, `frontend/src/pages/LoginPage.tsx`, `RegisterPage.tsx`

Tables: `users` (`id, name, email unique lower-case, password_hash, role, active, created_at, active_brand_id`), `refresh_tokens` (`jti, user_id, expires_at, revoked_at`), `audit_log` (rows `auth.login`, `auth.login_failed`, `auth.registered`, `auth.logout`). **External services:** none (no email, no OAuth).

Validation, errors and edge cases

Situation	Result	Code
Wrong password, unknown or inactive account	401 "Incorrect email or password"	`auth.py:70-74`
5 failures in 15 minutes for the same email and IP	429	`auth.py:66-67`
No / bad / expired access token	401 `Not authenticated` / `Invalid or expired token`, with `WWW-Authenticate: Bearer`	`backend/app/config/deps.py:33-42`
Deactivated user with a valid token	401	`backend/app/config/deps.py:41-42`
Wrong role	403 "You do not have permission to do this"	`backend/app/config/deps.py:46-50`
Refresh token reused after rotation	401 and all that user's refresh tokens revoked	`auth.py:136-141`
Whole API: more than `API_RATE_LIMIT_PER_MINUTE` (600) requests per minute per IP or per token	429 with `Retry-After: 60`	`backend/app/main.py:119-135`
Validation failure	422 with `detail` list of `{loc,msg,type}`	`backend/app/config/errors.py:22-27`

Notes: the in-memory limiters count per process, so several workers each keep their own counters (`ratelimit.py` docstring). Access tokens cannot be revoked individually; they simply expire (default 15 minutes).

Related configuration (names only)

`JWT_SECRET`, `JWT_ALGORITHM`, `ACCESS_TOKEN_MINUTES`, `REFRESH_TOKEN_DAYS`, `ADMIN_EMAIL`, `ADMIN_PASSWORD`, `ADMIN_NAME`, `DEMO_EMAIL`, `DEMO_PASSWORD`, `DEMO_ADMIN_EMAIL`, `DEMO_ADMIN_PASSWORD`, `LOGIN_MAX_FAILURES`, `LOGIN_WINDOW_SECONDS`, `SIGNUP_ENABLED`, `SIGNUP_MAX_PER_IP_PER_HOUR`, `SIGNUP_MAX_PER_EMAIL_PER_HOUR`, `API_RATE_LIMIT_PER_MINUTE`; frontend: `VITE_DEMO_EMAIL`, `VITE_DEMO_PASSWORD`, `VITE_API_URL`. Defined in `backend/app/config/config.py:40-59`, `:163`; templates in `backend/.env.example` (the demo and sign-up names are at `.env.example:99-129`).

DOCS/FEATURES/BRAND-REGISTRY.MD

Brand registry

Plain-language summary

The brand registry is where a company tells BrandShield "this is who we are": the brand name, spelling variants, keywords, official websites, official social accounts, official mobile apps and official logos. Everything the scanner later finds is compared against this list, and anything on it is never flagged as fake. One account can protect up to 10 brands and switch between them from the sidebar. Brands belong to one account only; another account can never see them. Changing the brand's name, keywords or domains makes older findings obsolete, so they are hidden ("retired") rather than mixed with the new identity.

Everything here is real code. The only sample data is the optional sample-mode scan that re-runs after an identity change when the data mode resolves to sample (`backend/app/api/brands.py:195-201`).

Purpose and user-facing behaviour

Screen	What it does	Code
`/protect-brand` (Protect Your Brand)	Two cards: "Create a Brand" (opens Brand Studio, a separate app, via `openStudio()`) and "Protect a Brand" (goes to the setup wizard). With `?add=1` it says "Add another brand to protect." and shows "Back to dashboard"; the wizard link then becomes `/setup?new=1` .	`frontend/src/pages/ProtectBrandPage.tsx:8-19` , `:41-45`
`/setup` (Setup wizard, 3 steps)	Step 1 brand identity and logo; step 2 official accounts and apps; step 3 first scan. "Skip setup for now" returns to the dashboard.	`frontend/src/pages/SetupPage.tsx:46` , `:311-313`
`/brand` (Brand Profile)	Edit identity, logo, accounts, apps; see profile health; record verification of official identifiers.	`frontend/src/pages/BrandProfilePage.tsx` (`:114` , `:147` , `:163` , `:193` , `:813-857`)
Sidebar `BrandSwitcher`	Card showing the active brand and its official asset count; opens a menu listing every brand of the account, "Brand profile", and "Add another brand". With no brand it links to `/setup` .	`frontend/src/components/layout/BrandSwitcher.tsx:13-103`

Setup wizard in detail (`SetupPage.tsx`)

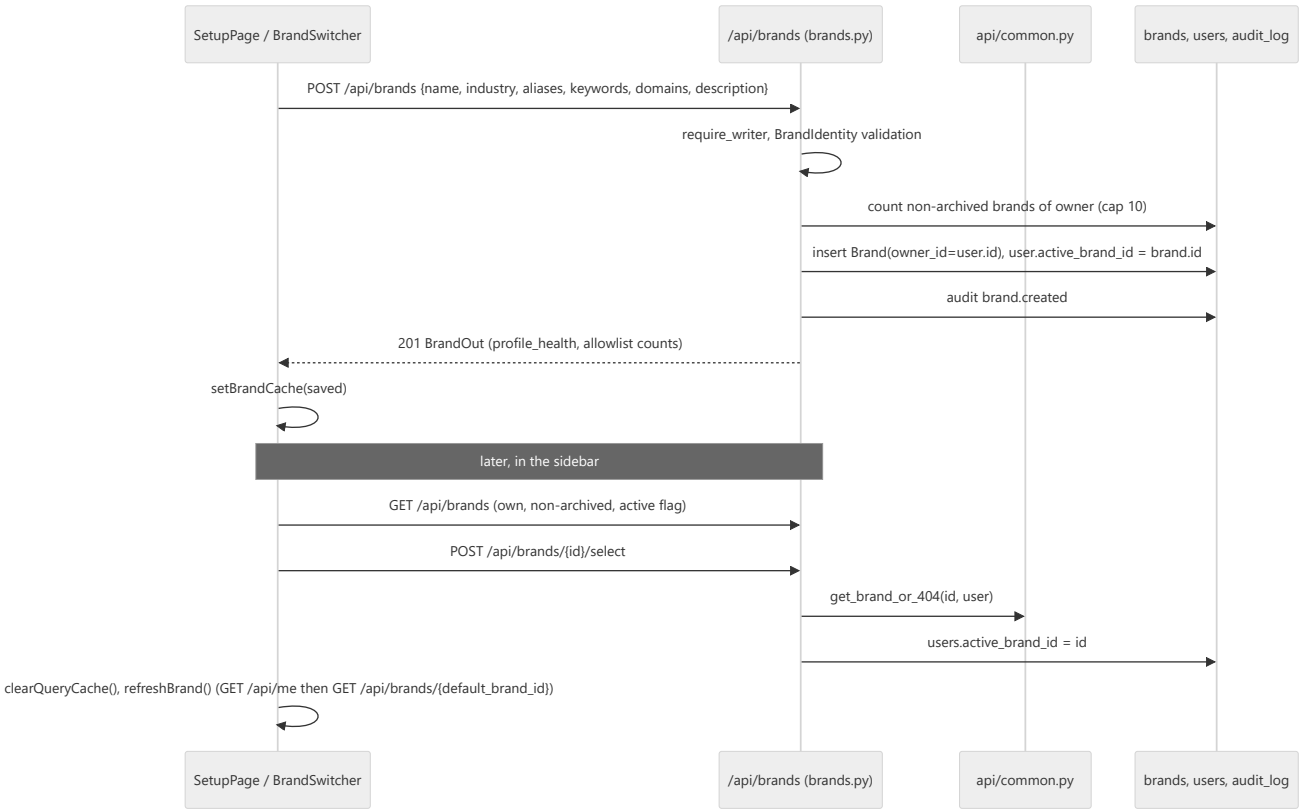
1. **Step 1, identity** (`saveBrand` , `:124-185`): company name (required), industry, description, aliases, keywords, domains (each comma separated; `https://` and any path are stripped from domains client-side, `:141-144`), optional logo (pre-checked by `checkLogoFile` , `:111-121`). Saves with `sm.updateBrand` when a brand exists, else `sm.createBrand` (`:149-165`); then uploads the logo with `sm.uploadLogo` (`:167-171`).
2. **Step 2, official assets** (`handleAddAccount` `:188-207` , `handleAddApp` `:220-247`): add accounts (platform and handle) and apps (name, package id, publisher, store). Removal via `handleRemoveAccount` / `handleRemoveApp` .
3. **Step 3, first scan** (`launch` , `:260-277`): if "run immediately" is on, calls `sm.startScan(scanScope)` (`POST /api/scan`); a 409 (a scan already running) is ignored; any other error is shown and the user is told to start it from the dashboard. Then it goes to `/dashboard` .

`?new=1` makes the wizard start from an empty form and create a new brand instead of editing the active one (`SetupPage.tsx:49-53`).

Needs confirmation: the wizard's UI and the backend disagree in four places. (a) The platform dropdown offers `tiktok` and `telegram` (`SetupPage.tsx:25-33`), but the backend only accepts `instagram, x, facebook, linkedin, youtube` (`backend/app/services/validators.py:4` , enforced at `backend/app/schemas/brand.py:45-51`), so those two return 422. (b) The store option "Apple App Store Only" sends `apple` and "Google Play & App Store" sends `['google_play','apple']` (`SetupPage.tsx:232` , `:700-702`), but the backend accepts `google_play` and `app_store` only (`validators.py:5` , `schemas/brand.py:75-85`), so both give 422; the Brand Profile page uses the correct `app_store` value (`BrandProfilePage.tsx:626-627`). (c) The step 3 fields "Scan Analysis Mode" (`scanDepth`) and "Alert recipient email" (`alertEmail`) are collected (`SetupPage.tsx:84-85` , `:828-847`) but `launch()` sends only the scope (`:265`). (d) `sm.addApp` has no `apple_app_id` support in the wizard. Please confirm whether these are known gaps.

Step-by-step flows

Create a brand and switch brands

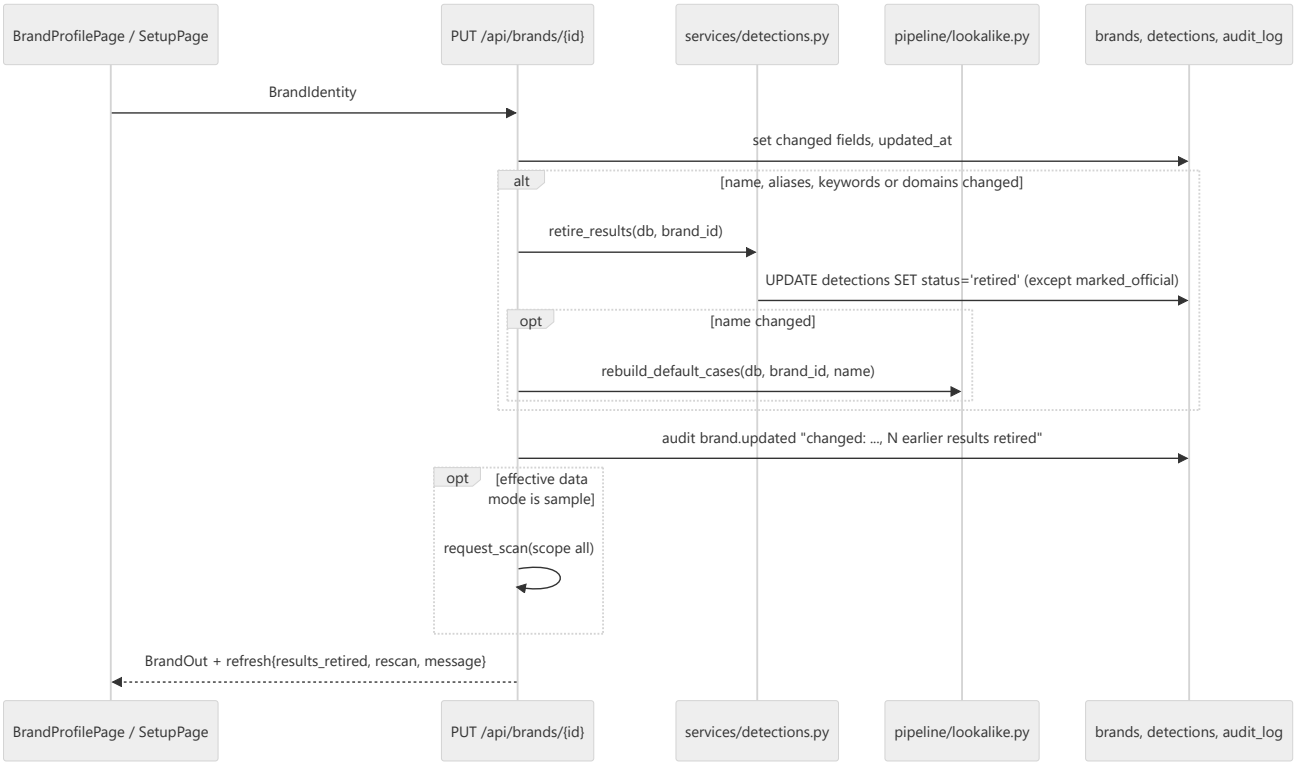


- Create: `create_brand` (`backend/app/api/brands.py:142-154`). Cap `MAX_BRANDS_PER_ACCOUNT = 10` (`:46`), archived brands do not count; at the cap the answer is 409 "You can protect up to 10 brands. Archive one first." (`:145-146`). A newly created brand becomes the active one (`:150`).
- List: `GET /api/brands` returns only `owner_id == user.id` and non-archived brands, ordered by id, with `active` marked (`:124-129`). Not even an admin sees another account's brands.
- Select: `POST /api/brands/{id}/select` sets `users.active_brand_id` (`:132-139`). Column added by migration `backend/alembic/versions/0012_active_brand.py:20` ; model `backend/app/models/user.py:21` .
- Every page that does not receive a `brand_id` works on `own_brand(db, user)` : the selected brand if still the user's and not archived, otherwise the oldest non-archived one, otherwise `None` (`backend/app/api/common.py:30-40`). `resolve_brand` uses it as the default when `brand_id` is omitted and answers 404 "No brand profile yet. Create one first." when none exists (`:43-50`). `GET /api/me` exposes the result as `default_brand_id` (`backend/app/api/auth.py:29-36`), which the frontend `useBrand` hook then loads (`frontend/src/hooks/useBrand.ts:36-60`).
- A per-request memo avoids repeated brand queries inside one request (`memo(...)` in `common.py:24` , `:40` ; see `docs/features/dashboard.md`).

Tenant isolation

`get_brand_or_404` loads the brand and raises a plain `404 Brand not found` if it does not exist, belongs to another `owner_id` , or is archived (unless `include_archived`) (`common.py:22-27`). It is deliberately 404 and never 403 so another account's brand ids cannot be probed. `owns_brand` does the same check for objects that carry a `brand_id` (detections, scans, reports, images) (`common.py:53-58`). Every brand sub-route (`accounts` , `apps` , `logo` , `logos` , `publishers` , `verification`) starts with `get_brand_or_404` . Migration `0010_brand_owner.py` added `brands.owner_id` and assigned previously owner-less brands to the first admin (`backend/alembic/versions/0010_brand_owner.py:44`).

Update identity and retire old detections



- `update_brand` (`brands.py:165-204`). `IDENTITY_FIELDS = ("name", "aliases", "keywords", "domains")` (`:162`). `description` is only changed when the client sends it (`:176-177`), so an older client cannot wipe it.
- `retire_results` sets `Detection.status = "retired"` for every detection of the brand except `retired` and `marked_official` rows; nothing is deleted, but retired rows leave every list, count and report (`backend/app/services/detections.py:499-505` , constant `RETIRED` at `:32`).
- The Look-alike Lab's built-in cases are rebuilt when the name changes (`brands.py:188-189`).
- For live data a new scan is not started automatically because scans spend search credits; the response says `rescan: "needed"` (`:191`). For sample data a scan starts at once (`:195-201` ; `ScanAlreadyRunning` and `NoBrand` are ignored). The profile page shows `refresh.message` (`BrandProfilePage.tsx:127` , `:696`).

Official accounts, apps, domains, publishers

Asset	Endpoint(s)	Rules	Code
Domains	part of the brand (`domains` array on `POST/PUT /api/brands`)	Lower-case hostname only: no scheme, path, port, `@` , IP literal; valid labels and TLD; max 253 chars; max 50 domains, deduplicated	`backend/app/services/validators.py:51-61` , `backend/app/schemas/brand.py:26-35`
Aliases / keywords	same	trimmed, blanks dropped, case-insensitive dedupe, each at most 80 chars, at most 50 items each	`validators.py:64-79` ; `schemas/brand.py:24-25` (list field cap 200 at `:10-12`)
Official accounts	`GET/POST /api/brands/{id}/accounts` , `DELETE .../accounts/{account_id}`	Platforms `instagram, x, facebook, linkedin, youtube` ; handle 3-30 chars of letters, digits, dot, underscore (hyphen also for linkedin and youtube; linkedin up to 100); leading `@` stripped; duplicate per brand/platform returns 409; the profile URL is built (`instagram.com/<h>` , `linkedin.com/company/<h>` , `youtube.com/@<h>`) and probed once with `UrlChecker` ; failure is stored as `status=unreachable` with a warning, not an error	`brands.py:268-325` ; `validators.py:7-48` ; `schemas/brand.py:41-56`
Official apps	`GET/POST /api/brands/{id}/apps` , `PATCH .../apps/{app_id}` (App Store id), `DELETE`	Stores `google_play` , `app_store` ; Google package must look like `com.example.app` , Apple bundle id letters/digits/hyphen/dot; same package on the other store adds the store instead of 409; optional numeric App Store id (number, `id123` , or an `apps.apple.com/.../id123` URL), only for apps on the App Store; duplicate returns 409	`brands.py:337-413` ; `schemas/brand.py:59-105` ; `validators.py:82-109`
Publishers / developers	`POST /api/brands/{id}/publishers` , `DELETE .../publishers/{publisher_id}`	`store` is `google_play` , `app_store` or `any` ; needs a name or an id; duplicate returns 409; new publishers start unverified and a publisher id only counts for the allowlist after it is verified	`brands.py:502-529` ; `schemas/brand.py:207-226`
Extra logos	`POST /api/brands/{id}/logos?label=` , `GET .../logos/{logo_id}/file` , `DELETE .../logos/{logo_id}`	Same image rules as the main logo; duplicate hash returns 409; at most 10 extra logos (422); label max 120 chars	`brands.py:449-496`

After adding an account or app, or changing an App Store id or a verification, `sync_with_allowlist` closes any existing detection that exactly matches the new official asset by setting it to `marked_official` , writing a `DetectionEvent` and an audit row `detection.marked_official` (`brands.py:304` , `:355` , `:372` , `:396` , `:568` ; `backend/app/services/detections.py:430-445`). Removing an account, app, logo or publisher does not reopen detections (**Needs confirmation:** no code path reverts `marked_official` on removal in `brands.py:313-325` , `:401-413` ; confirm this is intended).

There is no UI for publishers or extra-logo upload on the Brand Profile page; extra logos are added from the Look-alike Lab (`frontend/src/components/common/LogoCompareCard.tsx:221`) and `sm.addExtraLogo` (`frontend/src/api/sm.ts:88`). No frontend `sm` function exists for publishers, archive

or restore (searched `frontend/src`): these exist only as API endpoints.

Logo upload rules

`process_logo` in `backend/app/core/pipeline/logo.py:44-73`:

Rule	Value	Error
Max size	`LOGO_MAX_BYTES` = 2 MB (`config.py:61`); the endpoint reads at most cap + 1 bytes (`brands.py:220`)	413 "Logo is larger than 2 MB"
Format	decided from the bytes (PNG, JPEG, WebP), never the filename (`logo.py:34-41`)	415 "Logo must be a PNG, JPEG or WebP image"
Minimum side	256 px (`MIN_SIDE`, `:12`)	422
Aspect ratio	0.8 to 1.25, "roughly square" (`:13`)	422
Decompression-bomb guard	25,000,000 pixels (`:11`)	415 invalid image
Re-encode	image is re-saved, dropping metadata and appended bytes (`:63-66`)	
Stored as	`logos/brand_<id>/<sha256[:24]>.<ext>` via the file store (`save_logo`, `:76-80`)	

On success the brand row stores format, size, dimensions and `logo_sha256`, the old file is deleted, and `logo.replaced` is audited (`brands.py:225-239`). The browser pre-checks with the same limits (`frontend/src/utils/logoCheck.ts:3-66`) but the server is the authority.

Verification

`PUT /api/brands/{id}/verification` (`brands.py:547-571`) records that a person checked an identifier: `asset_type` in `domain, account, app, publisher, logo`; `status` `unverified` or `verified`; a verified record needs a `method` from `dns_txt, website_link, store_listing, platform_badge, manual_review, other` (`schemas/brand.py:229-247`). It stores who and when (`verified_by`, `verified_at`) and audits `asset.verification_changed`. The key must name an asset that exists (else 404, `_asset_exists` `:535-544`). The docstring is explicit that the application does not check DNS or stores itself; it is an analyst attestation (`:549-550`, model `backend/app/models/brand.py:115-131`). The UI is the verification card (`BrandProfilePage.tsx:813-857`).

Archive and restore

- `DELETE /api/brands/{id}?confirm=<exact name>` (admin only) sets `archived_at`; a wrong `confirm` returns 409 (`brands.py:419-430`). Nothing is erased.
- `POST /api/brands/{id}/restore` (admin only) clears it, or 409 when the account already has 10 active brands (`:433-443`).
- Archived brands disappear from lists and scans, and from `own_brand`. There is no archived-brand list endpoint (**Needs confirmation:** restoring requires knowing the brand id; no `GET` returns archived brands in `brands.py`).

Profile health

`compute_profile_health` gives a deterministic 0-100 score with a reason per missing item (`backend/app/services/profile_health.py:5-53`): brand name and aliases 20, keywords 10, logo present 20 (the code only checks `logo_path is not None`, `:32`, although the label says "meets requirements"), domains 15, accounts 15, apps with publisher names 10, all account URLs reachable 10. It is returned as `profile_health` inside `BrandOut` (`brands.py:109`).

Roles and permissions

Action	Roles
List/get brand, accounts, apps, logos, select a brand	any signed-in user (`get_current_user`)
Create, update, upload logo, add/remove accounts, apps, logos, publishers, set verification	`admin`, `analyst` (`require_writer`)
Archive, restore	`admin` only (`require_admin`)
See another account's brand	nobody (404)

Self-registered accounts are analysts, so they can create and edit brands but cannot archive them (see `docs/features/authentication-and-accounts.md`).

Tables and external services

`brands` (`owner_id`, `archived_at`, `source`), `official_accounts`, `official_apps`, `brand_logos`, `brand_publishers`, `asset_verifications`, `users.active_brand_id`, `detections` and `detection_events` (retire/sync), `audit_log`, `stored_files` when `STORAGE_BACKEND=database` (`backend/app/models/brand.py`). External services: the official-account URL probe through `UrlChecker` (SSRF-safe fetch, `backend/app/config/deps.py:24-25`; limits `URL_CHECK_TIMEOUT_SECONDS`, `URL_CHECK_MAX_REDIRECTS`, `URL_CHECK_MAX_BYTES`). Nothing else leaves the server.

Audit actions written

`brand.created`, `brand.updated`, `brand.archived`, `brand.restored`, `logo.replaced`, `logo.added`, `logo.removed`, `account.added`, `account.removed`, `app.added`, `app.updated`, `app.removed`, `publisher.added`, `publisher.removed`, `asset.verification_changed`, `detection.marked_official` (all in `backend/app/api/brands.py`).

Related configuration (names only)

`LOGO_MAX_BYTES`, `URL_CHECK_TIMEOUT_SECONDS`, `URL_CHECK_MAX_REDIRECTS`, `URL_CHECK_MAX_BYTES`, `STORAGE_BACKEND`, `DATA_DIR`, `SCAN_EXECUTOR` (for the sample re-scan), `IMAGE_SIMILARITY_BACKEND` (how logos are compared later). The brand cap of 10 is a code constant, not an environment variable (`brands.py:46`).

DOCS/FEATURES/SCANNING-AND-DETECTION.MD

Scanning and detection

Plain-language summary

A "scan" is BrandShield's way of looking for people who pretend to be your brand. It searches social networks and app stores for names that look like yours, checks each result against the official accounts and apps you registered (those are skipped), scores everything that is left (name similarity, logo similarity, suspicious wording, account age ...) and saves the suspicious ones as "detections" for a human to review. Nothing is reported or taken down automatically. A scan can be started by hand, on a timer, or by a daily cron call. When no real search service is configured the app uses clearly fake "sample" data instead.

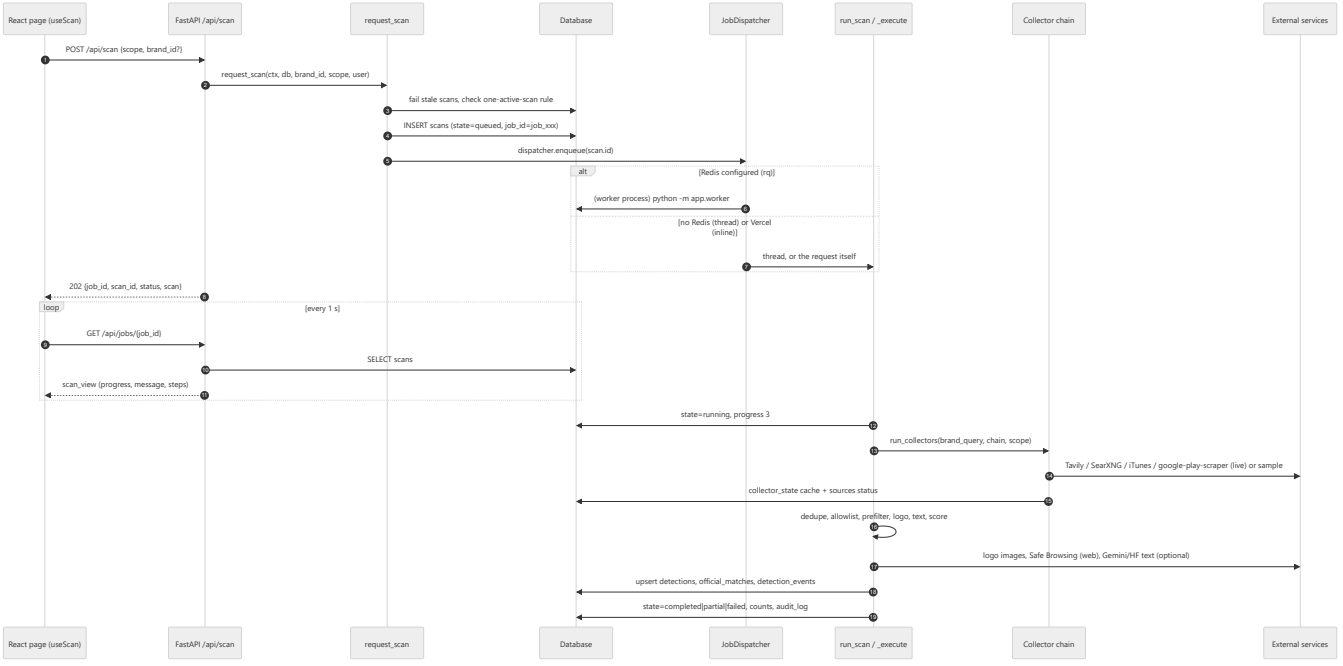
All paths below are relative to the repository root (`ScamMind AI/`). Line numbers refer to the code at the time of writing.

1. Purpose and user-facing behavior

What the user sees	Where it comes from
"Scan social platforms now" / "Scan app stores now" buttons (scope `social` / `apps`)	`frontend/src/pages/SocialMonitoringPage.tsx:80-89` , `frontend/src/pages/AppStoreMonitoringPage.tsx:82-90` , hook `frontend/src/hooks/useMonitor.ts:64`
Dashboard scan button (scope `all` , which also covers web candidates)	`frontend/src/pages/DashboardPage.tsx:70` (`useScan('all', ...)`)
Progress percentage and message while scanning	`frontend/src/hooks/useScan.ts:44-65` polls `GET /api/jobs/{job_id}` every 1 s (`POLL_MS` , line 10) for at most 10 min (`MAX_WAIT_MS` , line 11)
"A scan is already running" is shown as a running scan, not an error	HTTP 409 handling in `frontend/src/hooks/useScan.ts:89-102` ; a scan started elsewhere (scheduler, other tab) is attached to on page load (`useScan.ts:105-115`)
Sources bar with online / slow / unavailable pills	`frontend/src/components/monitor/MonitorParts.tsx:51-138` , fed by `GET /api/sources` (`backend/app/api/sources.py:125-157`)
Only scans for the signed-in user's own brand	`backend/app/api/scans.py:34-35` (someone else's `brand_id` is a 404 "Brand not found") and `backend/app/core/pipeline/scan_pipeline.py:146-155` (no own brand: `NoBrand` , HTTP 409)

Roles: starting a scan needs role `admin` or `analyst` (`require_writer` , `backend/app/config/deps.py:56`). Viewers can read scans and jobs (`get_current_user`). The UI disables the buttons for viewers (`SocialMonitoringPage.tsx:84-85`).

2. End-to-end flow



Step by step:

1. **Queue.** `POST /api/scan` (`backend/app/api/scans.py:25-45`) accepts `{"scope": "all"|"social"|"apps", "brand_id": int|null}` (`ScanBody` , lines 20-22) and returns HTTP 202. It calls `request_scan` (`backend/app/core/pipeline/scan_pipeline.py:146-191`), which:
   - resolves the brand (explicit id, or the user's oldest non-archived brand; the scheduler passes `user=None` , lines 149-155);
   - fails any queued/running scan older than `SCAN_STALE_MINUTES` (30) via `fail_stale_scans` (lines 117-125);
   - refuses with `ScanAlreadyRunning` (HTTP 409 and the running job in the body, `scans.py:40-43`) if one is active. A partial unique index `uq_scans_one_active_per_brand` also enforces this in the database (`backend/app/models/ops.py:56-58` , scan_pipeline.py:176-185);

- freezes the data mode on the row (`Scan.data_mode` , line 170) and writes a `scan.started` audit row (lines 186-188);
- generates `job_id = job_<ms-hex><brand-hex>` (line 164).

2. **Dispatch.** `JobDispatcher.enqueue` (`backend/app/services/jobs.py:39-52`) picks the executor, see section 3.
3. **Run.** `run_scan` (scan_pipeline.py:770-792) only starts a scan whose state is still `queued` , sets `running` /progress 3, calls `_execute` (803-871) and converts any crash into state `failed` with a redacted message (781-792).
4. **Collect.** `_execute` builds the collector chain (`select_collectors` then `CollectorRuntime.build` , scan_pipeline.py:809-815) and calls `run_collectors` (810-823). Progress is set to 40.
5. **Process** every unique candidate: allowlist, prefilter, logo, text analysis, scoring, persistence (sections 5-9). Progress moves from 40 to 95 and is committed every 25 candidates (scan_pipeline.py:865-867).
6. **Finish.** `_finish` (873-919) decides the final state, stores `counts` , `source_report` , `steps` , `errors` , `timings` , `result` and writes the `scan.finished` audit row.

Scan states and what the API returns

`scan_view` (scan_pipeline.py:926-955) exposes two vocabularies: `status` = the real state (`queued` | `running` | `completed` | `partial` | `failed`) and `state` = the older job vocabulary (`queued` | `running` | `done` | `error` ; `completed` and `partial` both map to `done` , `failed` to `error`).

Final state	Rule (scan_pipeline.py:873-887)
`failed`	every active (non-skipped) collector was unavailable AND no fresh (non-cached) candidate exists
`partial`	some collector unavailable, or a slow collector reported warnings
`completed`	none of the above

A failing source never blocks a scan: its cached results (flagged `stale`) are used instead (see section 4).

3. Job queue vs inline execution (Vercel), scheduler and cron

Executor	When	Code
`rq` (Redis + RQ worker)	`SCAN_EXECUTOR=auto` (default) and `REDIS_URL` set, or `SCAN_EXECUTOR=rq`	`backend/app/services/jobs.py:23-30` ; enqueue `jobs.py:59-67` (`app.worker.run_scan_job` , `job_id=f"scan-{id}"` , timeout `SCAN_JOB_TIMEOUT_SECONDS` =900)
`thread` (in-process daemon thread)	no `REDIS_URL` , or Redis could not be reached when queuing (falls back and logs a warning, `jobs.py:46-51`)	`jobs.py:51-52`
`inline` (runs inside the HTTP request)	tests, and **Vercel by default** (`SCAN_EXECUTOR` defaults to `inline` when `VERCEL` is set unless you set it yourself)	`jobs.py:40-42` , `backend/app/config/config.py:175-190` (defaults at 181-187)

- Worker process: `python -m app.worker` (`backend/app/worker.py:45-60`). It builds its own `ScanContext` (`worker.py:28-37`), listens on queue `SCAN_QUEUE_NAME` (default `scans`) and uses `SimpleWorker` on Windows because RQ's forking worker is not available there (line 58). It exits with code 2 when `REDIS_URL` is missing (lines 49-51).
- On Vercel the request that started the scan also runs it, so the `POST /api/scan` response already contains the finished scan (`scans.py:44` , comment "an inline run has already finished by now"). `vercel.json` gives the backend function `maxDuration: 300` seconds (`vercel.json` , `services.backend.functions`).
- **Scheduler loop** (local / Docker): `SchedulerLoop` (`backend/app/services/scheduler.py:192-218`) is a daemon thread that calls `scheduler_tick` every `SCHEDULER_POLL_SECONDS` (60, minimum 5). It is started in the app lifespan only if `SCHEDULER_ENABLED` and `APP_ENV != test` (`backend/app/main.py:84-89`). On Vercel `SCHEDULER_ENABLED` defaults to false (config.py:182).
- **Cron** (Vercel): `GET /api/cron/scan` (`backend/app/api/cron.py:32-46`) is scheduled `0 6 * * *` in `vercel.json` (`crons`). Requires `Authorization: Bearer <CRON_SECRET>` (503 when the secret is unset, 401 when wrong, `cron.py:34-37`). It calls the same `scheduler_tick` with `now + 60 min` slack (`CRON_SLACK` , cron.py:24) and returns `{ran, skipped, reason, scan_id}` . Full detail in `docs/background-jobs-and-integrations.md` .
- **Schedule decision** (`decide_schedule` , scheduler.py:52-89, pure function): interval from `SCAN_INTERVAL_MINUTES` env (wins) or the Settings page value `off|5m|15m|1h|1d` (`backend/app/services/app_settings.py:15-16,79-83` , default `1d` , line 20). When Tavily is part of the live chain the interval is **stretched** so the monthly credit budget lasts (scheduler.py:1-13, 68-84), and scheduled scans are **paused** (manual scans still work) when fewer than two scans' worth of credits remain (lines 70-77). A brand with no official account or app is not scanned by the scheduler ("Waiting for the first official account or app", scheduler.py:92-95, 168-169). At most `SCHEDULER_MAX_BRANDS_PER_RUN` (5) brands are started per pass (line 178).
- **Status endpoints**: `GET /api/scan/status` (`scans.py:56-80` : last scan, next scan, schedule, running job, data mode), `GET /api/jobs/{job_id}` (48-53), `GET /api/scans` (83-95, `limit` 1-100), `GET /api/scans/latest` (98-102), `GET /api/scans/{scan_id}` (105-110). A scan or job of someone else's brand is a plain 404 (`owns_brand` , `backend/app/api/common.py:53-58`).

4. Data modes (live / sample / auto) and the collector chain

Data mode is stored in the `settings` table under key `data_mode` (`backend/app/services/data_mode.py:9-11` ; default `auto`). `select_collectors` (`backend/app/core/collectors/selection.py:68-114`) resolves it to an effective mode:

Requested mode	Effective mode
`sample`	always sample (one collector: `sample`)
`live`	always live
`auto`	live if at least one of: `TAVILY_API_KEY` , `SEARXNG_BASE_URL` , the `google_play_scraper` package is present; otherwise sample. iTunes needs no key, so on its own it does **not** switch auto to live (selection.py:77-80, 92-97)

Who can change it: `PUT /api/settings/data-mode` (admin only; the shared demo administrator may only choose `sample`), `backend/app/api/sources.py:239-249` ; also via `PUT /api/settings` (`backend/app/api/settings.py:130-182`).

Chain in live mode, in order (`CHAIN_ORDER` , selection.py:27, entries 102-113):

#	Collector	Platforms covered	Needs	Real or mock
1	`tavily` (`backend/app/core/collectors/tavily.py`)	instagram, facebook, x, telegram, linkedin, google_play, app_store, third_party_apk (line 84)	`TAVILY_API_KEY` ; spends credits	REAL external call to `https://api.tavily.com/search` (line 38)
2	`searxng` (`searxng.py`)	instagram, facebook, x, telegram, linkedin (line 57)	`SEARXNG_BASE_URL` (optional, only active when set)	REAL call to your own SearXNG instance
3	`itunes` (`itunes.py`)	app_store	nothing	REAL call to `https://itunes.apple.com/search` (line 34), ~20 calls/min, cached
4	`google_play` (`google_play.py`)	google_play	optional package `google-play-scraper` (in `backend/requirements-extra.txt` , not in `requirements.txt`)	REAL scraping via the third-party package; reports `not_installed` if absent (lines 44-56)
-	`sample` (`sample.py`)	all, including `web` (line 73)	nothing	MOCK: deterministic fake candidates, every one has `source="sample"`

Collector details:

- **Tavily**: queries are quoted brand terms combined with lure words (`support official helpline rewards`), aliases and generated look-alike spellings (`build_queries` , tavily.py:55-78, `generate_variants` in `variants.py:22-46`). The request is restricted with `include_domains` to social and store domains only (`PLATFORM_DOMAINS` , tavily.py:41-51; body at 150-160), uses `exact_match` and `include_usage` . Each call first **reserves credits** in the ledger (tavily.py:143-149; `CreditLedger.reserve` , `credits.py:83-98` , an atomic `UPDATE ... WHERE credits + cost <= budget`). 4xx answers are refunded, a timeout or 5xx keeps the reservation because it may have been billed (tavily.py:165-176). Error mapping at 204-223: 401 `invalid_key` , 432/433 `quota` , 429 `rate_limited` , 5xx `unreachable` . Cost: `basic` = 1 credit, `advanced` = 2 (`credits.py:25-29`). Caps: `TAVILY_MAX_QUERIES_PER_SCAN` (12), `TAVILY_MAX_RESULTS` (10, max 20).
- **SearXNG**: `GET {base}/search?q=...&format=json` . The base URL must resolve to a public address unless `SEARXNG_ALLOW_PRIVATE_HOSTS=true` (`_check_base` , searxng.py:78-93). HTTP 403 means JSON output is disabled on the instance (`json_disabled` , lines 129-130).
- **iTunes**: `term` , `country` (`ITUNES_COUNTRY` , default `in`), `entity=software` ; calls spaced by `SpacingLimiter` and cached by `TTLCache` (`backend/app/core/collectors/util.py:13-56` , TTL `ITUNES_CACHE_TTL_SECONDS` =3600).
- **Google Play**: `search(term, lang, country, n_hits)` of `google_play_scraper` ; listings without a package id are dropped (google_play.py:123-127). Licence and terms risk is noted as unverified in the file header (lines 1-9).
- **No live collector discovers websites.** Verified: every live collector only builds candidates through `candidate_from_url` , which returns social profiles and store listings only (`backend/app/core/collectors/base.py:329-378`), Tavily's `include_domains` excludes ordinary websites, and `candidate_from_web_url` (base.py:381-400) is called only by the sample collector (`sample.py:291,296`) and by manual / imported rows (`backend/app/core/pipeline/candidate_intake.py:143`). See `docs/features/web-threats.md` .

Orchestrator

`run_collectors` (`backend/app/core/collectors/orchestrator.py:211-255`):

- builds the platform set from the scope and from the `sources.monitored` flags (lines 227-234); for scope `all` it adds the pseudo platform `web` (line 234), which has no Source Health row;
- runs each collector in `_run_one` (258-331): a collector with no covered platform is `skipped` ; one that raises `SourceUnavailable` (or crashes, lines 290-292) is marked `unavailable` , its failure counter rises and the next retry is delayed **1 min, 5 min, 15 min, 1 h, then 6 h** (`BACKOFF_SCHEDULE_SECONDS` , orchestrator.py:39). Problems fixed by local configuration (`no_key` , `not_installed` , `not_configured` , `unsafe_base_url`) are retried on the next scan without waiting (line 42, 62-68);
- while in back-off or after a failure it returns the **cached** previous result, every candidate marked `stale=True` (`_cached` , 191-204);
- after success it stores the new result in `collector_state.cache` (lines 299-314) and writes per-platform status to `sources` (`_write_sources` , line 334).

5. Candidate de-duplication, allowlist and prefilter

- **Dedupe** (`dedupe_candidates` , scan_pipeline.py:213-233): the key is `platform:handle` (lower-cased, `@` stripped) or the canonical URL (`canonical_key` , 198-203). Later duplicates only fill empty fields of the kept one. The collectors also dedupe internally (`base.py:403`). Counter `duplicates_removed` .
- **Cap**: more than `SCAN_MAX_CANDIDATES` (500) unique candidates are truncated, counter `truncated` (scan_pipeline.py:833-835).
- **Allowlist first** (`BrandAllowlist.check` , `backend/app/core/pipeline/matching.py:168-191`): exact official handle, exact official profile URL, Google Play package id, numeric App Store id or bundle id, a VERIFIED publisher id, or (web) an official registrable domain (`_check_web` , matching.py:194-199). **The brand name alone never excludes anything.** Matches are dropped before scoring and recorded in `official_matches` (`_record_official` , scan_pipeline.py:727-749). An existing detection for such an asset is closed as `marked_official` with an event (lines 855-861). Apple numeric ids are learned from iTunes bundle ids in memory per scan (`learn_apple_ids` , matching.py:154-167).
- **Prefilter** (`SCAN_PREFILTER` , default true): a social/app candidate whose name is `safe` with name match below 25% (`NEAR_NAME_PCT`) and no brand keyword/domain in its text is `ignored` before logo/AI steps. Manual and imported rows are never prefiltered (scan_pipeline.py:526-528, `_kw_hit` 236-243).
- **Look-alike rules** from Settings (character swaps, digits and symbols, added words, spacing, transposed/missing letters) can downgrade a flagged name to `review` when all its techniques are switched off (`apply_rules` , matching.py:94-103; defaults 35-41). **Impersonation words** default to `support helpline official refund rewards kyc` (matching.py:34).

6. Logo fetch and similarity

`_logo_step` (scan_pipeline.py:383-442) never fails a candidate: any problem becomes `{"status": "unavailable", "reason": "Logo unavailable: ..."}` and counter `logo_unavailable` . Three origins:

Candidate	What is compared	Network?
sample	generated image bytes carried by the sample candidate (variants of the brand's own uploaded logo, `sample_images.py`); `origin: generated_sample` (lines 391-407)	none
manual / import	only an image the analyst uploaded and linked by `image_id` (`_intake_image`, 364-377); its URLs are never fetched (408-421)	none
live (tavily, searxng, google_play, itunes)	the `icon_url` downloaded by the SSRF-safe `SafeFetcher` (`backend/app/core/pipeline/image_fetch.py`: public hosts only, redirect hops re-validated, size cap, content-type allow-list, magic-bytes check, decompression-bomb guard). Skipped when `SCAN_LOGO_FETCH=false`, no official logo is uploaded, no icon URL, the listing is a cached copy, or `SCAN_MAX_LOGO_FETCHES` (25) is reached (422-442)	yes, to the candidate's own image host

Similarity backend is chosen by `IMAGE_SIMILARITY_BACKEND` (`_image_impl`, scan_pipeline.py:332-347):

Value	Algorithm label	Method
`hash` (default)	`image-1`	0.6 x pHash + 0.4 x dHash, each rescaled `(1 - hamming/64 - 0.5)/0.5` to 0-100 (`backend/app/core/engine/logo_similarity.py:76-124`)
`multihash`	`image-2`	0.4 pHash + 0.3 dHash + 0.3 wHash (`image_hashes.py:22,76-83`)
`imagehash`	`image-2-imagehash`	same weights with the optional ImageHash library
`clip`	`image-3-clip`	CLIP ViT-B/32 via fastembed ONNX, calibrated to 0-100 (`logo_similarity.py:185-211,319-326`); falls back to `image-1` with one warning if not installed

Thresholds: `SIMILAR_MIN = 55` and `LIKELY_COPY_MIN = 80` (logo_similarity.py:32-33). Below 55 a logo scores 0 points; from 55 it earns a third of the maximum, rising linearly to the maximum at 90 (`image_points`, scoring.py:102-107). Hash similarity is only a hint, never proof (note text in `_logo_step`). More in `docs/features/lookalike-lab.md`.

7. Text analysis (local / Gemini / Hugging Face)

`TextAnalysisService.analyze` (`backend/app/core/pipeline/text_analysis.py:66-88`) is called for a social bio (`kind="bio"`, scan_pipeline.py:537) and an app description (`kind="app_description"`, line 544).

1. The **local baseline always runs**: `LocalTextProvider` (`backend/app/core/providers/text/local.py:18-45`) applies phrase rules and RapidFuzz against the brand description. It is informational and adds **no points** (no double counting, local.py:1-5).
2. A **hosted provider** runs only when ALL are true: `SCAM_LLM_ENABLED=true` (default false), the active provider is `gemini` or `huggingface` (stored in `settings` key `text_provider`, else `SCAM_LLM_PROVIDER`, `active_choice` lines 32-35), the provider is configured (key; Hugging Face also needs `HUGGINGFACE_MODEL`), the candidate is **not sample** data and **not a cached (stale) copy** (line 74), and the text has at least 12 characters (`MIN_TEXT_CHARS`, `providers/text/base.py:21`).
3. Providers: Gemini (`gemini.py`, default model `gemini-3.5-flash-lite`, JSON schema output, temperature 0), Hugging Face (`huggingface.py`, no default model). Both go through `ApiTextProvider.analyze` (`base.py:106-138`): sanitised text, answer cache (SHA-256 key, in-memory plus `text_analysis_cache` table, 30 days, `cache.py:18-26`), bounded retries with backoff for 5xx/timeout only (`base.py:141-151`), quota cool-down 600 s and bad-key cool-down 900 s (`base.py:22-23,153-159`). A failure becomes an `unavailable` finding, never an exception (`base.py:3-4`).
4. Effect on the score: only a finding with `scam_likelihood >= 0.6` on a **social bio of at least 20 characters** reaches the scorer, adding `round(10 x likelihood)` points inside the bio cap of 28 (scan_pipeline.py:538; `score_social`, scoring.py:168-171). For apps the text result is informational only (scan_pipeline.py:543-545). An AI answer can only add points: it cannot lower a score or change the allowlist.
5. A scan-level warning when the AI check is switched on but unusable (`note`, text_analysis.py:56-64) is appended to `scans.errors` only (scan_pipeline.py:879-880); the final state is decided by the sources alone (881-887), so it does not make a scan `partial`.

8. Scoring and risk engine

`RiskEngine` (`backend/app/core/engine/risk_engine.py:150-222`) wraps the scorers and adds severity, algorithm version and per-signal availability.

Weights (frozen as `WEIGHTS_V1`, risk_engine.py:41; maxima in `scoring.py:34-35`):

Social (algorithm 1.0)	max	App (algorithm 1.0)	max
name	34	name	24
logo	28	icon	30
added_words	12	publisher	22
bio	28	description	17
account (age 6, few posts 3, follower oddity 4, unverified 2, empty bio 1, foreign link 2)	18	store (third-party mirror 4, few installs 3, new listing 2, few ratings 1)	10

Details: `name_points` (scoring.py:92-99) gives `max x pct/100` for a flagged name, 90% of max for the exact brand name that is not on the allowlist, half weight for `review`. Account signals: `_account_reasons` (187-218). App publisher points count only if another brand signal (name, icon or description) already scored (`_publisher_reason`, 265-280). `score = min(100, raw_total)` (`_finish`, 79-82); each category is capped when produced; maxima sum above 100 on purpose.

Web (algorithm 2.0, `WEIGHTS_V2_WEB`, risk_engine.py:43): `domain_similarity` 40, `rapidfuzz_similarity` 12, `deceptive_subdomain` 20, `idn_homograph` 15, `added_words` 10 (`WEB_MAX`, `backend/app/core/engine/domain_match.py:39`) plus `url_reputation` 45 only on a Safe Browsing match (`assess_web`, risk_engine.py:199-222). See `docs/features/web-threats.md`.

Band (unchanged since v1): `>= 85` high, `70-84` medium, `< 70` low (`backend/app/core/pipeline/bands.py:1-11`, `scoring.py:39-44`).

Severity (beside band): `LOW < 40`, `MEDIUM 40-69`, `HIGH 70-84`, `CRITICAL >= 85`; cutoffs from `SEVERITY_MEDIUM_MIN=40`, `SEVERITY_HIGH_MIN=70`, `SEVERITY_CRITICAL_MIN=85` and validated `0 < medium < high < critical <= 100` (`RiskConfig`, risk_engine.py:46-55; `severity_for`, 61-68; config.py:166-168).

Signals (`Signal`, risk_engine.py:71-90) have status `present | clean | unavailable`, `value` = null when unavailable (a missing answer is never treated as clean) and provenance `real_api | local_algorithm | pretrained_model | synthetic` (sample data is labelled `synthetic` on every signal, `providers/base.py:10-15`).

What becomes a detection (`is_hit`, matching.py:244-250): the candidate needs a reason tied to THIS brand (name, logo/icon, description, domain signals; `BRAND_SIGNALS`, matching.py:83-84) AND (score >= threshold OR the name matcher flagged it outright OR, for web, verdict is look-alike). Threshold = `100 - sensitivity`; sensitivity is 50-95, default 75, so default threshold is 25 (`threshold_for`, app_settings.py:74-76; defaults 17-24). Outcomes (`evaluate`, scan_pipeline.py:515-554):

Outcome	Meaning	Stored as
`hit`	passes `is_hit`	detection with status `new`
`cleared`	below threshold but its name still resembles the brand (verdict not `safe` or name match >= 25%)	status `cleared`: the false-positive guard list ("similar names of other businesses")
`ignored`	nothing ties it to the brand	not stored

9. Detection persistence

`upsert_detection` (scan_pipeline.py:678-724) keyed by `(brand_id, canonical_key)` (unique constraint, `backend/app/models/detection.py:14`):

- **New**: inserts the row with score, band, severity, `algorithm_version`, `scored_at`, `reasons`, `details` (name match, logo, signals, weights, text analysis, domain, reputation; `_details`, 604-635), `evidence` (SHA-256 snapshot saved in the file store, `_evidence_for`, 574-591 and `backend/app/core/pipeline/evidence.py:31-37`), first score-history point and a `scan` event. A `cleared` candidate whose name does not resemble the brand is not stored (686-687).
- **Existing, undecided**: fields are re-applied; a change across the threshold writes a `status` event (`new <-> cleared`) (718-724).
- **Existing, decided by a human** (`HUMAN_STATES` = reviewing, confirmed, dismissed, marked_official, line 72): status, score and reasons are **never reset**; only `last_seen_at`, evidence history and score history are updated, plus a note when the score moved by at least 10 points (`MATERIAL_DELTA`, 75; lines 710-716).
- `detection_type`: `lookalike_domain` (web), `fake_app` (app), `scam_account` if a bio signal gives >= 6 points, else `fake_brand_page` (Facebook/LinkedIn) or `fake_profile` (`_detection_type`, 249-256).
- `origin`: `synthetic` (sample), `manual`, `import`, or `scan` (`origin_of`, 80-81). `source` column is `sample` or `live` (`Candidate.data_source`, `base.py:75-78`).
- Screenshots: only if `SCREENSHOTS_ENABLED=true` and Playwright is installed (default off, `backend/app/core/pipeline/screenshot.py:43-46`).

Tables touched by a scan: `scans`, `sources`, `collector_state`, `api_usage` (Tavily and Safe Browsing counters), `detections`, `detection_events`, `official_matches`, `stored_files` (when `STORAGE_BACKEND=database`), `text_analysis_cache`, `audit_log`, `settings` (read).

10. Counters and result shape

`scans.counts` keys (scan_pipeline.py:825-830): `candidates_found, duplicates_removed, unique_candidates, official_excluded, prefiltered, evaluated, detections_new, detections_updated, detections_protected, cleared_new, logo_compared, logo_unavailable, llm_checked, llm_errors, official_detections_closed, truncated, web_candidates, reputation_checked, reputation_not_checked, reputation_match, reputation_clean, reputation_unavailable, reputation_skipped_sample, reputation_over_limit`. `scans.result` (lines 900-909) feeds the UI sentence "Social scan finished: N candidates checked, M new detections". `scans.timings` has `collect_ms`, `process_ms`, `total_ms`.

11. Config and feature flags (names only)

Name	Default	Purpose	Declared
`SCAN_EXECUTOR`, `REDIS_URL`, `SCAN_QUEUE_NAME`, `SCAN_JOB_TIMEOUT_SECONDS`, `SCAN_STALE_MINUTES`	auto, unset, scans, 900, 30	job execution	`backend/app/config/config.py:110-114`
`SCAN_PREFILTER`, `SCAN_LOGO_FETCH`, `SCAN_LOGO_MAX_BYTES`, `SCAN_LOGO_TIMEOUT_SECONDS`, `SCAN_MAX_CANDIDATES`, `SCAN_MAX_LOGO_FETCHES`	true, true, 2 MB, 5 s, 500, 25	pipeline limits	config.py:115-120
`SCREENSHOTS_ENABLED`	false	Playwright evidence hook	config.py:121
`SCHEDULER_ENABLED`, `SCAN_INTERVAL_MINUTES`, `SCHEDULER_POLL_SECONDS`, `SCHEDULER_MAX_BRANDS_PER_RUN`	true, unset, 60, 5	timer	config.py:122-126
`CRON_SECRET`	unset	enables `/api/cron/scan`	config.py:38
`TAVILY_API_KEY`, `TAVILY_MONTHLY_BUDGET`, `TAVILY_MAX_QUERIES_PER_SCAN`, `TAVILY_SEARCH_DEPTH`, `TAVILY_MAX_RESULTS`	-, 900, 12, basic, 10	Tavily	config.py:91-95
`SEARXNG_BASE_URL`, `SEARXNG_ALLOW_PRIVATE_HOSTS`, `SEARXNG_MAX_QUERIES_PER_SCAN`	-, false, 12	SearXNG	config.py:96-98
`ITUNES_COUNTRY`, `ITUNES_CALLS_PER_MINUTE`, `ITUNES_CACHE_TTL_SECONDS`, `ITUNES_MAX_QUERIES_PER_SCAN`	in, 20, 3600, 8	iTunes	config.py:99-102
`GOOGLE_PLAY_COUNTRY`, `GOOGLE_PLAY_MAX_QUERIES_PER_SCAN`	in, 6	Google Play	config.py:103-104
`SOURCE_SLOW_THRESHOLD_MS`, `COLLECTOR_TIMEOUT_SECONDS`	4000, 15	collector timing	config.py:105-106
`IMAGE_SIMILARITY_BACKEND`, `LOGO_COMPARE_CLIP`	hash, auto	logo scoring	config.py:81,83
`SCAM_LLM_ENABLED`, `SCAM_LLM_PROVIDER`, `GEMINI_API_KEY`, `GEMINI_MODEL`, `HUGGINGFACE_API_TOKEN`, `HUGGINGFACE_MODEL`, `TEXT_PROVIDER_TIMEOUT_SECONDS`, `TEXT_PROVIDER_MAX_RETRIES`	false, gemini, ...	text analysis	config.py:68-88
`GOOGLE_SAFE_BROWSING_API_KEY`, `SAFE_BROWSING_*`	-	web reputation	config.py:132-136
`SEVERITY_MEDIUM_MIN`, `SEVERITY_HIGH_MIN`, `SEVERITY_CRITICAL_MIN`	40, 70, 85	severity cutoffs	config.py:166-168
`RESPONSE_CACHE_SECONDS`	15 (0 on Vercel)	cached dashboard/list reads, busted when a scan is queued and when it ends (`_busts_cache`, scan_pipeline.py:132-142)	config.py:123

Runtime (database) settings, admin only: data mode, monitoring `schedule`, `sensitivity` (50-95), look-alike `rules`, `impersonation_words` (`backend/app/services/app_settings.py:19-24`, `backend/app/api/settings.py:130-182`); active text provider (`backend/app/api/providers.py:50-57`).

12. Error handling and edge cases

- No brand: `POST /api/scan` returns 409 "Create the brand profile before scanning." (scans.py:38-39).
- Second scan for the same brand: 409 with `job_id`, `scan_id` and the running scan (scans.py:40-43).
- Dead worker: a queued/running scan older than 30 minutes is failed on the next request or tick (scan_pipeline.py:117-125).
- Crash inside a scan: state `failed`, message "The scan stopped because of an internal error.", error text redacted of configured secrets (`_redact`, 755-757; handler 781-792).
- All sources down and nothing cached: `failed`, "No data source was available, so nothing could be checked." (`_finish`, 910-914).
- Cached results are labelled stale; stale candidates skip logo fetch, hosted text analysis and Safe Browsing (scan_pipeline.py:428-429, 461-463; text_analysis.py:74).
- Scan counts are not decreased by human decisions; `detections_protected` counts decided detections that were seen again.
- Brand renamed or its domains changed: earlier detections are set to status `retired` and vanish from every list (`retire_results`, `backend/app/services/detections.py:499-505`).

Needs confirmation: `REACHABILITY_CHECKS_PER_SCAN` (config.py:107, `.env.example`) is documented as an SSRF-safe reachability check of result URLs, but `_execute` passes `url_checker=None, reachability_checks=0` to `run_collectors` (scan_pipeline.py:820-821) and no other code reads the setting (only `backend/app/services/canary.py:111` sets it to 0). It looks like a dead setting for real scans; confirm with the maintainers.

Needs confirmation: `SCHEDULER_MAX_QUERIES_PER_SCAN` (config.py:127) is declared as a credit estimate override but `schedule_inputs` uses `TAVILY_MAX_QUERIES_PER_SCAN` directly (scheduler.py:116); the setting appears unused.

Needs confirmation: the text-analysis embeddings (`TEXT_EMBEDDINGS_ENABLED`, `GEMINI_EMBEDDING_MODEL`) are built only to report status in `GET /api/providers` (`backend/app/api/providers.py:38`); no scoring path calls `GeminiEmbedder.embed`. Treated here as not used by scans.

13. Key files

Area	File
API	`backend/app/api/scans.py`, `backend/app/api/cron.py`, `backend/app/api/sources.py`, `backend/app/api/settings.py`
Pipeline	`backend/app/core/pipeline/scan_pipeline.py`, `matching.py`, `evidence.py`, `image_fetch.py`, `url_safety.py`, `text_analysis.py`
Collectors	`backend/app/core/collectors/{selection,orchestrator,base,credits,tavily,searxng,itunes,google_play,sample,sample_images,variants,util}.py`
Engine	`backend/app/core/engine/{scoring,risk_engine,name_match,domain_match,logo_similarity,image_hashes,text_signals,allowlist}.py`
Providers	`backend/app/core/providers/{safe_browsing,base}.py`, `backend/app/core/providers/text/*.py`
Jobs and timer	`backend/app/services/{jobs,scheduler,context,data_mode,app_settings}.py`, `backend/app/worker.py`, `backend/api/index.py`, `vercel.json`
Models	`backend/app/models/ops.py` (Scan, Source, CollectorState, ApiUsage, Setting), `backend/app/models/detection.py`
Frontend	`frontend/src/hooks/useScan.ts`, `useMonitor.ts`, `frontend/src/api/sm.ts:102-105`

DOCS/FEATURES/SOCIAL-AND-APP-THREATS.MD

Social Threats and App Threats

Plain-language summary

These two screens ("Social Media Monitoring" and "App Store Monitoring") are the review queue. After a scan, every suspicious fake profile, fake brand page, scam account or fake app appears in a list sorted by risk. You click one, see why it scored the way it did (name, logo, bio, publisher ...), look at the evidence that was saved, and then make a decision: confirm it is an impersonation, keep reviewing, dismiss it as a false positive, or mark it as one of your own official accounts or apps. A person always decides: the system never reports or takes anything down by itself.

All paths are relative to the repository root (`ScamMind AI/`).

1. Purpose and user-facing behavior

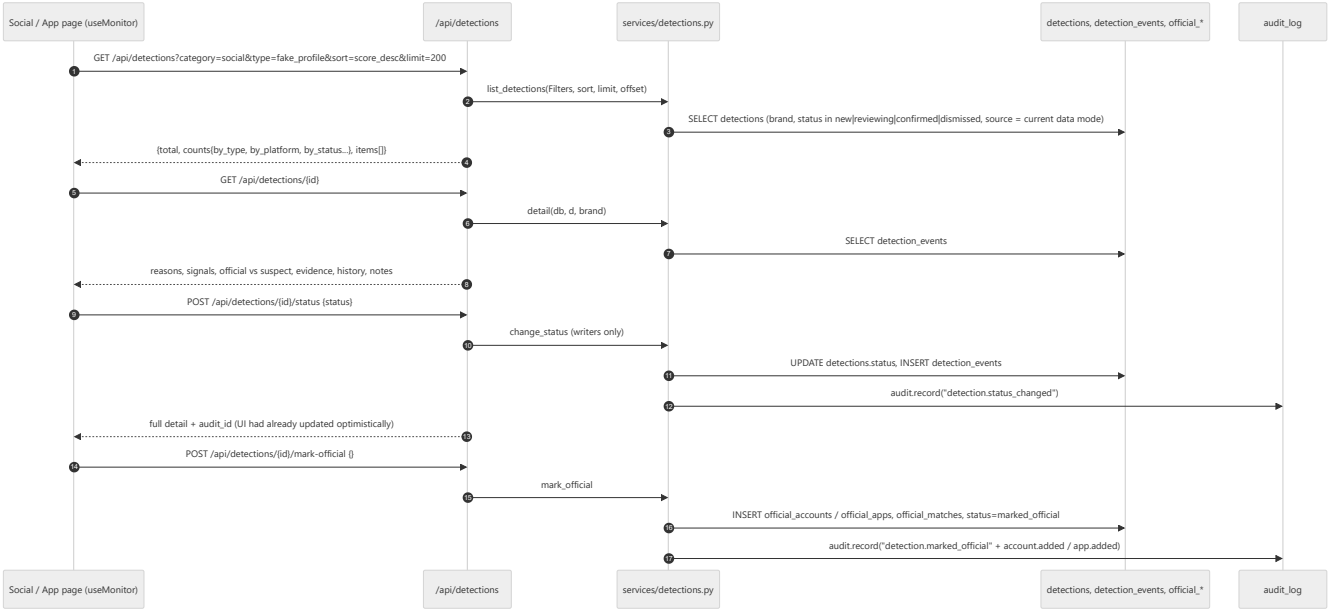
Screen	Page component	Route data
Social Media Monitoring	`frontend/src/pages/SocialMonitoringPage.tsx`	`useMonitor('social')` (`frontend/src/hooks/useMonitor.ts:16`)
App Store Monitoring	`frontend/src/pages/AppStoreMonitoringPage.tsx`	`useMonitor('apps')`

What each page shows (both share `useMonitor` and the building blocks in `frontend/src/components/monitor/MonitorParts.tsx`):

1. **Sources bar** with the status of each platform (online / slow / unavailable) and, on the App page, an inline Retry for a failing source (`MonitorParts.tsx:51-138`).
2. **Filter tabs with counts.** Social: All, Fake profiles, Fake brand pages, Scam accounts (filter is the detection `type` ; `SocialMonitoringPage.tsx:14-19`). Apps: All stores, Google Play, App Store, Third-party APK (filter is the `platform` ; `AppStoreMonitoringPage.tsx:14-19`). The App page opens on Google Play unless a `?detection=` deep link is present (`useMonitor.ts:18-21`).
3. **Scan button** ("Scan social platforms now" / "Scan app stores now") and **Add candidate / Import CSV / JSON** tools (`CandidateTools` , see `docs/features/candidate-import.md`).
4. **Detections list**, sorted by risk (`sort=score_desc` , `frontend/src/api/sm.ts:108-109`), each row showing name, handle or package id, platform, type, score, severity chip and status chip (`SocialMonitoringPage.tsx:118-143`).
5. **Detail panel** for the selected row (the first row by default, or the one in `?detection=`):
   - official-versus-suspect comparison cards. Social shows bio, followers, joined date, link and "Logo N% match" (`SocialMonitoringPage.tsx:181-229`). Apps shows publisher, store, a red "Publisher differs from the official one" banner (`AppStoreMonitoringPage.tsx:180-189`), and "Icon N% match";
   - "Why it scored N": every reason with its points plus a per-signal availability list (`ReasonsBox` , `MonitorParts.tsx:238-283`);
   - "Evidence kept": link to the profile URL, capture time, SHA-256 of the saved snapshot, and a screenshot link only if one exists (`EvidenceBox` , 285-323);
   - the action row: **Confirm impersonation / Confirm fake app**, **Keep reviewing**, **Dismiss as false positive**, **Mark as official** (`ActionRow` , 325-344).
6. **Two bottom panels**: "Excluded as official" (items matched on the allowlist, never scored) and the false-positive guard list ("cleared automatically": similar-sounding names of other businesses) (`ExcludedCard` 153, `GuardCard` 184; used at `SocialMonitoringPage.tsx:247-248` , `AppStoreMonitoringPage.tsx:272-273`).
7. A banner "Sample data" on a detail whose `source` is `sample` , and a footer note when the app is in sample mode (`SocialMonitoringPage.tsx:159-161` , `SampleNote` in `frontend/src/components/common/States.tsx:53-61`).

Roles: every GET needs a signed-in user. Every action (status change, resolution, mark official, scan, add or import candidates) needs role `admin` or `analyst` (`require_writer` , `backend/app/config/deps.py:56`). A `viewer` can read and sees disabled buttons (`disabled={!canWrite || ...}` , `SocialMonitoringPage.tsx:235`). Detections of another account's brand are a 404 "Detection not found" (`backend/app/api/detections.py:38-42` , `owns_brand`).

2. Flow



Listing and filtering

`GET /api/detections` (`backend/app/api/detections.py:59-97`). Query parameters (all optional):

Parameter	Values	Notes
`category`	`social` , `apps` , `web`	the pages pass `social` / `apps`
`type`	`fake_profile` , `fake_brand_page` , `scam_account` , `fake_app` , `lookalike_domain` , or the words `social` , `app` , `apps` , `web` , `website(s)`	`normalize_type` , `backend/app/services/detections.py:63-76` ; anything else is 422
`platform`	e.g. `instagram` , `google_play` , `app_store` , `third_party_apk`	exact match
`band`	`high` , `medium` , `low`	
`severity`	`LOW` , `MEDIUM` , `HIGH` , `CRITICAL`	
`resolution`	`true_positive` , `false_positive` , `resolved` , `monitoring`	
`status` (repeatable)	`new` , `reviewing` , `confirmed` , `dismissed` , `marked_official` ; aliases `confirm` , `dismiss` , `keep-reviewing`	other values 422 (detections.py:84-89). `marked_official` is always stripped (`_base` , services/detections.py:82)
`source`	`live` , `sample` , `all`	default = the current effective data mode (`_source_filter` , detections.py:52-56). So in sample mode only sample detections are listed and vice versa
`q`	up to 200 chars	case-insensitive contains on name, handle, package id, publisher (`_base` , 104-109)
`min_score`	0-100	
`sort`	`score_desc` (default), `score_asc` , `found_desc` , `found_asc` , `last_seen_desc`	
`from` , `to`	ISO datetimes on `found_at`	
`limit` / `offset`	1-200 (default 50) / >= 0	the UI asks for 200
`brand_id`	int	defaults to the user's own brand (`resolve_brand`)

Default visible statuses are `new, reviewing, confirmed, dismissed` (`LISTED_STATUSES` , services/detections.py:31). `cleared` , `marked_official` and `retired` never appear in these lists (cleared ones surface only in the guard list, official ones in the excluded list). The response (`detections.py:94-97`) holds `total` , `data_mode` , `is_sample` , `includes_sample` , `counts` and `items` (shape: `summary_row` , services/detections.py:178-202: `id, category, type, platform, name, handle, package_id, publisher, profile_url, score, risk_band, severity, status, resolution, summary, tags, found_at, last_seen_at, stale, source, origin`). Tab counts are computed in one round trip with `UNION ALL` and exclude the tab being counted so "All 6 / Fake profiles 3" add up (`list_detections` , 129-163).

Detail

`GET /api/detections/{id}` returns `detail()` (services/detections.py:229-283): everything in the summary plus `algorithm_version` , `scored_at` , `weights` (`weights_v1`), `signals` , `reasons` (each `{signal, label, detail, points, max_points}`), `name_match` , `raw_total` , `capped` , `threshold` , `official` (the registered side), `logo` (status `compared` or `unavailable` with a reason), `evidence` , `score_history` , `notes` , `history` (events), `text_analysis` . App detections add `publisher_differs` , `official_publisher` , `suspect{icon_match_percent, rating, ...}` and `description{snippet, overlap_points}` (265-274). Social adds `suspect{bio, followers, following, posts, joined, verified, logo_match_percent}` (279-282). `GET /api/detections/{id}/evidence` returns the saved snapshot and `hash_verified` (re-hash of the stored bytes vs the stored SHA-256, `evidence_bundle` , 286-293). `GET /api/detections/{id}/evidence/screenshot` returns a PNG, or 404 "No screenshot was captured (screenshots are off by default)" (detections.py:146-154).

Review status actions

UI button	Endpoint	Resulting status	Code
Confirm impersonation / Confirm fake app	`POST /api/detections/{id}/status {"status":"confirmed"}` (UI) or shortcut `POST .../confirm`	`confirmed`	detections.py:163-167, 186
Keep reviewing	same, `reviewing` , or `.../keep-reviewing`	`reviewing`	detections.py:187
Dismiss as false positive	same, `dismissed` , or `.../dismiss`	`dismissed`	detections.py:188
Mark as official	`POST /api/detections/{id}/mark-official`	`marked_official`	detections.py:191-200

`change_status` (services/detections.py:308-326): only `reviewing` | `confirmed` | `dismissed` can be set by a person (`new` , `cleared` , `marked_official` cannot, 422). A `marked_official` detection refuses status changes with 409 "Remove it from the allowlist first". Setting the same status with no note is a no-op. An optional `note` (max 1000 chars) is stored in `detections.notes` (keeps the last 50), a `detection_events` row is added with the actor, and `audit_log` gets `detection.status_changed` ("old -> new: note"). Note that the UI button "Dismiss as false positive" sets **status** `dismissed` ; it does not set the separate `resolution` field `false_positive` .

`POST /api/detections/{id}/resolution {"resolution": "true_positive|false_positive|resolved|monitoring", "note"}` records the investigation outcome without changing the status (`set_resolution` , 332-345; audit `detection.resolution_set`). On these two pages no button calls it; the typed client function `sm.setResolution` (`frontend/src/api/sm.ts:115`) is used by the Investigations page (`frontend/src/pages/InvestigationsPage.tsx:179` , "Save resolution").

Mark as official (`mark_official` , services/detections.py:348-414):

- social detection: needs a handle of at most 30 characters; creates an `official_accounts` row (status `verified` , "Marked official by a reviewer") unless one exists, builds its URL with `build_account_url` (375-394);
- app detection: only `google_play` or `app_store` listings (a third-party APK is never official, 358-359); needs a package id; creates an `official_apps` row with the given or detected publisher, `stores=[platform]` (357-374);

- web detections cannot be marked official: 422 "add its domain to the brand's official domains on the Brand Profile" (355-356);
- writes/updates an `official_matches` row with reason `marked_official` and the rule "Marked official by : " (395-402), sets the status to `marked_official`, writes an event, `detection.marked_official` audit row (+ `account.added` / `app.added`), and touches `brand.updated_at`;
- afterwards future scans drop that exact asset before scoring (`BrandAllowlist`, see `docs/features/scanning-and-detection.md` section 5). Adding an official account or app by hand also closes matching existing detections (`sync_with_allowlist`, services/detections.py:430-445).
- UI behaviour (`useMonitor.ts:95-126`): optimistic removal from the list, restored with an error message if the server refuses.

Optimistic updates for confirm/keep reviewing/dismiss (`useMonitor.ts:75-92`) roll back list and detail if the server answers with an error.

Excluded items and false positives

- `GET /api/detections/excluded?category=` (`detections.py:100-111`, `excluded_items`, services/detections.py:452-475): lists the brand's registered official accounts and apps with how many times scans matched them (`official_matches.times_seen`), reason `exact_handle` or `package_id`.
- `GET /api/detections/false-positives?category=&source=` (`detections.py:114-126`, `false_positive_items`, 478-496): detections with status `cleared`, ordered by score, each with `reason` text built by `_clear_reason` ("Name partly resembles the brand (N%); logo X% match; score S is below the review threshold of T; no other brand evidence", scan_pipeline.py:557-567), `name_match_percent`, and logo or icon match.

Detection lifecycle (statuses)

`new` -> `reviewing` / `confirmed` / `dismissed` (human) -> `marked_official` (allowlist). A scan creates `new` (above threshold) or `cleared` (near-name but below threshold). A re-scan never changes a human decision (`HUMAN_STATES`, `backend/app/core/pipeline/scan_pipeline.py:72,710-716`) but may append a note when the score moves by 10 or more. `retired` (brand identity changed) hides rows everywhere (`retire_results`, services/detections.py:499-505).

3. Data, APIs, services

Item	Detail
Tables	`detections` (`backend/app/models/detection.py:10-53`), `detection_events` (84-97), `official_matches` (56-81), `official_accounts`, `official_apps` (`backend/app/models/brand.py:43-80`), `audit_log`
Endpoints used by these pages	`GET /api/detections`, `GET /api/detections/{id}`, `POST /api/detections/{id}/status`, `POST /api/detections/{id}/mark-official`, `GET /api/detections/excluded`, `GET /api/detections/false-positives`, `GET /api/detections/{id}/evidence/screenshot`, `GET /api/sources?kind=social`
Frontend client	`frontend/src/api/sm.ts:102-122` (scan, detections, status, mark official, excluded, false positives)
Backend files	`backend/app/api/detections.py`, `backend/app/services/detections.py`, `backend/app/core/pipeline/scan_pipeline.py` (what creates them)
External services	none are called when a person reviews. Scans (separate feature) call Tavily, SearXNG, iTunes, google-play-scraper; the Sources retry button runs collectors (`backend/app/api/sources.py:179-230`) and for Tavily needs a confirmation because it can spend credits (409 `confirmation_required`, lines 207-214; `MonitorParts.tsx:64-86`)

Real versus mock: with data mode `sample` (or `auto` and no live source configured) every row on these pages is fabricated by the sample collector from your own brand name and carries a "Sample data" badge. Logo percentages in sample mode come from generated images derived from your uploaded logo, not from real profile pictures (`scan_pipeline.py:391-407`).

4. Validation, permissions, errors and edge cases

Case	Behavior
Not signed in	401 (`get_current_user`, deps.py:28-45)
Viewer role tries an action	403 "You do not have permission to do this" (deps.py:46-53); the UI also disables buttons
Detection id belongs to another account	404 "Detection not found"
Invalid status	422 "status must be one of: reviewing, confirmed, dismissed"
Note longer than 1000 characters	422 (pydantic `Field(max_length=1000)`, detections.py:19-35)
Change status of an official detection	409
Mark official twice	409 "already marked official"
Mark official: no handle / no package id / handle > 30 chars / non-Play non-App Store app / web	422 with a plain message (services/detections.py:355-380)
Cached (`stale`) detection	the App detail shows "cached " (`AppStoreMonitoringPage.tsx:157`); source data came from the previous good scan
Detection created by manual entry or import	`origin` is `manual` / `import` and `source` is `live` (see below)
No brand	the pages render a "create a brand profile first" state (`NoBrand`, `SocialMonitoringPage.tsx:44-53`)

Edge cases worth knowing:

- Screenshots are off by default (`SCREENSHOTS_ENABLED=false`), yet the App page footer says "Store screenshot and listing details are saved as evidence" (`AppStoreMonitoringPage.tsx:264`); in practice only the JSON snapshot plus SHA-256 is saved unless Playwright is enabled (`scan_pipeline.py:582-589`, `backend/app/core/pipeline/screenshot.py:43-46`).
- The App detail panel reads `suspect.downloads` and `official.downloads`, but the server always sends `downloads: null` for them (`services/detections.py:214,270`), so the Downloads line only appears when `released` exists.
- The official-side social card shows "The official page bio is not collected" because `bio`, `followers`, `joined` are `None` (`services/detections.py:219-220`).

Needs confirmation: manual and imported candidates are saved with `source = "live"` (`Candidate.data_source` returns `sample` only for the sample collector, `backend/app/core/collectors/base.py:75-78` ; assigned at `scan_pipeline.py:674`). The default list filter equals the current effective data mode (`detections.py:52-56`), so while the app is in `sample` mode a freshly added manual candidate would be hidden from the default list until the viewer asks for `source=all` (the pages never send `source`). Please confirm whether this is intended.

5. Related config (names only)

`SEVERITY_MEDIUM_MIN` , `SEVERITY_HIGH_MIN` , `SEVERITY_CRITICAL_MIN` (severity chip); Settings page values `sensitivity` (review threshold = 100 - sensitivity), look-alike `rules` , `impersonation_words` (`backend/app/services/app_settings.py:19-24`); `SCREENSHOTS_ENABLED` ; `STORAGE_BACKEND` (where the evidence snapshot is kept, `backend/app/core/storage.py:169-175`); `RESPONSE_CACHE_SECONDS` . Data mode (live / sample / auto) decides which rows are listed by default.

DOCS/FEATURES/WEB-THREATS.MD

Web Threats (look-alike domains and URLs)

Plain-language summary

The Web Threats screen answers one question: "does this website address imitate our official domains?" You can type any domain or URL into the tester and instantly see whether it is your official site, a look-alike (for example a swapped ending such as `.net` instead of `.com`, look-alike letters from another alphabet, or your brand placed in front of someone else's domain), or unrelated. Addresses that are saved as web threats are listed below the tester for review. **Important:** BrandShield does not go out and discover websites by itself. Web threats only exist when they come from the built-in sample data, or when a person adds a URL by hand or imports a file. The app never opens a website; the optional Google Safe Browsing check only sends the address to Google.

All paths are relative to the repository root (`ScamMind AI/`).

1. Purpose and user-facing behavior

Page: `frontend/src/pages/WebThreatsPage.tsx` (route "Web Threats", header at line 121). It has two parts.

A. Domain / URL tester (`WebThreatsPage.tsx:138-259`)

- Text box "Domain or URL (never opened)", max 2048 characters (line 159). The result is requested automatically 350 ms after typing stops (`useUrlTest` , `frontend/src/hooks/useUrlTest.ts:16-50` , `DEBOUNCE_MS` from `useLookalike.ts:8`); each keystroke cancels the previous request.
- Preset pills come from `GET /api/lookalike/test-urls` (example inputs generated from the registered brand and its official domains).
- Result: headline, explanation, verdict chip (Official domain / Look-alike domain / Related: review / No look-alike signal / Cannot be compared, lines 22-29), "Domain score N/100", name match %, technique chips, "How the address was read" (hostname, punycode vs unicode form, subdomain, registrable domain, TLD, which official domain it was compared with, port and `user@` credentials) and every scoring reason with its points (lines 174-225).
- A note always states that reputation was not checked in the tester (`r.reputation_note` , line 222).
- "Run examples" runs every preset through the tester and shows pass/fail against its expected verdict (lines 77-94, 227-258).
- If the brand has no official domain registered a notice links to Brand Registry (lines 147-151) and only the brand name is compared.

B. Web detections list (`WebThreatsPage.tsx:284-358`)

- Severity filter (All / Critical / High / Medium / Low), list "Suspicious domains" sorted by risk, detail panel with the shared `ReasonsBox` and `EvidenceBox` , and three actions: **Confirm threat**, **Keep reviewing**, **Dismiss as false positive** (lines 344-354) via `sm.setReview -> POST /api/detections/{id}/status` (`frontend/src/api/sm.ts:117-118`). There is no "Mark as official" for web detections.
- "Add candidate" and "Import CSV / JSON" tools for kind `web` (`CandidateTools` , line 296; see `docs/features/candidate-import.md`).
- Empty state text: "No web threats. Add a URL by hand or run a scan with web checks enabled." (line 310). See the note on this wording in section 4.

Roles: viewing and testing need any signed-in user (`get_current_user` ; `backend/app/api/lookalike.py:77-80`). Review actions and candidate tools need `admin` or `analyst` (`require_writer` , `backend/app/config/deps.py:56` ; buttons disabled otherwise, `WebThreatsPage.tsx:345-352`).

2. Where web candidates come from (verified)

Source	Produces web candidates?	Evidence
`tavily`	NO. Search is restricted to social and app-store domains, and results pass through `candidate_from_url` , which only yields profiles and store listings	`backend/app/core/collectors/tavily.py:41-51,116,154` ; `backend/app/core/collectors/base.py:329-378`
`searxng`	NO. Platforms are the five social networks only	`backend/app/core/collectors/searxng.py:57,146-153`
`itunes`	NO. App Store only	`backend/app/core/collectors/itunes.py:59`
`google_play`	NO	`backend/app/core/collectors/google_play.py:71`
`sample`	YES, fake `.test` hosts (RFC 2606 reserved, can never resolve), only in sample data mode	`backend/app/core/collectors/sample.py:73,271-299`
Manual entry (`POST /api/candidates/web`) and import (`POST /api/candidates/import?kind=web`)	YES, whatever the person types	`backend/app/api/candidates.py:46-48,51-72` ; `backend/app/core/pipeline/candidate_intake.py:143`

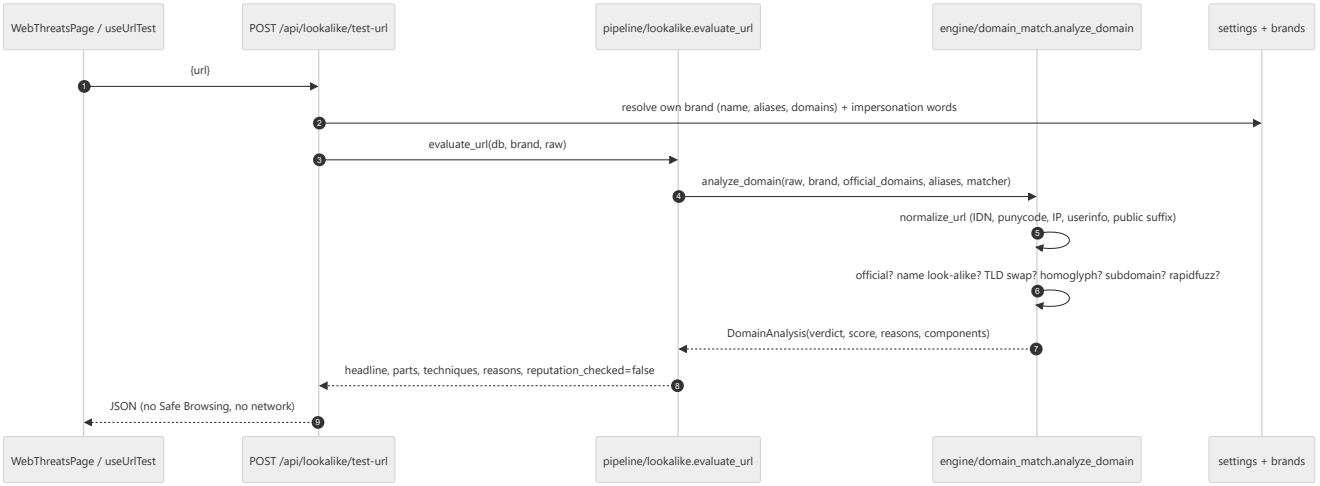
The only function that builds a web candidate is `candidate_from_web_url` (`backend/app/core/collectors/base.py:381-400`), and a repository-wide search finds exactly three callers: `sample.py:291` , `sample.py:296` , `candidate_intake.py:143` . Therefore **no live collector discovers websites**.

The orchestrator adds the pseudo platform `web` to a scan of scope `all` (`backend/app/core/collectors/orchestrator.py:233-234`), but with live collectors this changes nothing because none emits web candidates. Scopes `social` and `apps` never include `web` . The sample collector produces, for a brand name of at least 4 letters, six look-alike URLs and, if the brand has an official domain, an official echo to prove the allowlist works (sample.py:281-298):

Technique label	Example shape (for brand `acmebank`)	Tier
`tld_swap`	`https://acmebank.test/login`	strong
`combo`	`https://acmebank-support.test/kyc-update`	strong
`deceptive_subdomain`	`https://acmebank.<official>.verify-account.test/`	strong
`typo`	letters 2 and 3 swapped, `.test`	medium
`homoglyph`	first `a` (or `o`) replaced by a Cyrillic letter, `.test`	medium
`hard_negative`	`https://acme-<benign suffix>.test/` (another business)	weak

3. Domain look-alike analysis

All in `backend/app/core/engine/domain_match.py`. Pure code: no network, no database (`domain_match.py:1-20`).



Normalisation (`normalize_url`, domain_match.py:149-230)

Never raises and never touches the network:

1. Rejects empty input, input over 2048 characters (`MAX_URL_LENGTH`), and any control character or space (`_CONTROL`, line 45). Backslashes become slashes; a missing scheme is guessed as `http://`; only `http` and `https` are accepted.
2. Splits off port and `user@` userinfo (flagged, because `acmebank.com@evil.test` is a known trick).
3. Recognises IP hosts, including decimal/hex/octal forms such as `2130706433` or `0x7f000001` (`_integer_ipv4`, 101-126). IP hosts have no domain name, so the verdict is `not_comparable` (line 314-315).
4. **Punycode / IDN**: the host is encoded with `idna.encode(..., uts46=True)` to the ASCII form (`xn--...`) and decoded back for display (lines 197-214); `has_idn` is true if the two differ. A non-strict ASCII name (underscore) is tolerated with a warning.
5. **Registrable domain** via `tldextract` using its bundled Public Suffix List snapshot (`suffix_list_urls=()`, no download, `_extractor`, 48-51). An unknown TLD (`.test`, newer gTLDs) is treated as a one-label suffix so `acmebank.com.evil.test` resolves to `evil.test` (`_split_host`, 137-146).

Verdicts and scoring (`analyze_domain`, 302-417)

Verdict	Meaning	How it is reached
`not_comparable`	cannot be judged	unparseable input, IP host, or no public suffix (lines 311-317)
`official`	exact match	the registrable domain equals one of the brand's official registrable domains (never substring; lines 318-320, `official_registrables` 271-277)
`lookalike`	imitates the brand	name verdict `flagged` or `brand_name`, or a deceptive subdomain / userinfo trick, or an IDN homograph reason (415-416)
`related`	some brand signal but not enough	any positive reason, or a name verdict other than `safe`
`unrelated`	no signal	otherwise

Signals and maximum points (`WEB_MAX`, domain_match.py:39):

Signal	Max	Rule
`domain_similarity`	40	the domain label against brand name, aliases and official domain labels with the same `NameMatcher` used for social names, so decided hard negatives hold (`acmeplumbing.com` is not a look-alike of Acme Bank) (329-346). Same label under another TLD sets `tld_swap` (337-344)
`idn_homograph`	15	non-ASCII label that folds to the brand (homoglyph technique), or mixed alphabets while resembling the brand (348-360). Mixed scripts that do not resemble the brand are noted but not scored
`deceptive_subdomain`	20	brand (flagged or exact, never weak `review`) appears in a subdomain chunk, e.g. `acmebank.com.verify-account.test` ; or in the part before `@` (362-389)
`added_words`	10	only when the name matched: 10 for strong scam words, 6 for impersonation words (391-397)
`rapidfuzz_similarity`	12	extra signal only when the home-grown matcher said `safe` and RapidFuzz ratio is at least 90 for a label of at least 6 characters (`RAPIDFUZZ_MIN_RATIO` , `RAPIDFUZZ_MIN_LEN` , 399-410)
`url_reputation`	45	only from Google Safe Browsing, see section 4; added in `RiskEngine.assess_web` (`backend/app/core/engine/risk_engine.py:199-222`)

The domain score is `min(100, sum of points)` . When saved as a detection, the risk engine combines it with reputation (algorithm version `2.0` , weights `weights_v2_web` , `risk_engine.py:11,43`); band and severity use the same cutoffs as social and app detections (85 / 70 and 40 / 70 / 85).

`is_hit` for web (`backend/app/core/pipeline/matching.py:244-250`): a brand-related reason is required AND (score >= threshold OR name flagged OR verdict `lookalike`), so a look-alike always goes to review even with a low score. Detection type is `lookalike_domain` , category `web` (`scan_pipeline.py:249-252,659`). The row stores `handle` = ASCII host, `name` = unicode host, `profile_url` = canonical URL without credentials, query or fragment (`candidate_from_web_url` ; `_apply_fields` , `scan_pipeline.py:654-676`). Summary tags such as "TLD swap", "brand in the subdomain", "look-alike characters", "listed by Google Safe Browsing" come from `_evaluate_web` (scan_pipeline.py:487-512).

4. Google Safe Browsing: when it is and is NOT called

Adapter: `backend/app/core/providers/safe_browsing.py` (Lookup API v4 `threatMatches:find` , endpoint at line 42, key in header `x-goog-api-key` at line 177, never in the URL).

Called only from the scan pipeline's `_reputation_pass` (`backend/app/core/pipeline/scan_pipeline.py:445-484`), which is used by scans (line 845) and by manual / imported candidates (`candidate_intake.py:247`). A URL is sent only if ALL of these hold:

1. the candidate is a web candidate with a URL;
2. it is not an official match on the allowlist (line 455);
3. it is not sample data: sample URLs are never sent (lines 457-460, note "Sample data: the URL is not sent to Google.");
4. it is not a cached (`stale`) copy (461-463);
5. the domain analysis says it is brand related (`analysis.brand_related` , 464-467), so unrelated domains are never sent (quota and privacy);
6. fewer than `SAFE_BROWSING_MAX_URLS_PER_SCAN` (100) URLs have been collected (468-471);
7. inside the client: a key is configured, no 429 block is active, and the daily budget has room (below).

Never called by:

- `POST /api/lookalike/test-url` and the tester on the Web Threats page: `evaluate_url` is documented "no network, no Safe Browsing" (`backend/app/core/pipeline/lookalike.py:248-249,298` ; it returns `reputation_checked: false`);
- any social or app detection, the Look-alike Lab name tester, or detection review actions;
- the Sources page except to read its status: `GET /api/sources` includes `reputation: runtime.safe_browsing.status()` (`backend/app/api/sources.py:156`), which makes no request.

Verdicts (`backend/app/core/providers/base.py:18,22-44`):

Verdict	Meaning	Reaches the engine as
`match`	Google lists the URL (`MALWARE` , `SOCIAL_ENGINEERING` , `UNWANTED_SOFTWARE` , `POTENTIAL..._APPLICATION` ; safe_browsing.py:44)	`present` , +45 points
`clean`	request succeeded, no match. NOT a safety guarantee (`CLEAN_NOTE` , line 47)	`clean` , 0 points
`unknown`	not asked: no key (`no_key`), our daily budget used (`budget`), HTTP 429 or quota 403 (`quota`), unattributed match	`unavailable` , 0 points
`failed`	asked and it went wrong: timeout, network error, 5xx, 400/401/403, bad body	`unavailable` , 0 points

Quota ledger: `DailyLedger` (safe_browsing.py:50-87) counts **requests** per UTC day in table `api_usage` under provider `safe_browsing:YYYY-MM-DD` ; `reserve` is one atomic `UPDATE ... WHERE credits + n <= budget` . The limit is our own `SAFE_BROWSING_DAILY_BUDGET` (default 5000), not a Google figure. Up to 500 URLs go into one request (`MAX_URLS_PER_REQUEST` , line 43), so one scan normally costs one request. Cache: `TTLCache` keyed by URL with `SAFE_BROWSING_CACHE_TTL_SECONDS` (3600); only `match` and `clean` are cached (lines 139-142, 242-243). A 429 sets `blocked_until` (the `Retry-After` value or one hour) and later calls return `unknown/quota` without a request (158-164, 197-200). Without `GOOGLE_SAFE_BROWSING_API_KEY` no request is made at all (159-160). Terms note in the file header: non-commercial use only; Web Risk is the paid alternative.

What the detail panel shows: `details.reputation` (`ReputationResult.to_dict`) or `{"verdict": "unavailable", "provider": "google_safe_browsing", "checked_at": null}` when it was not checked (`scan_pipeline.py:630-633`), plus a per-URL note why it was skipped (`reputation_note` , lines 458-471). Counters in `scans.counts` : `web_candidates` , `reputation_checked` , `reputation_match` , `reputation_clean` , `reputation_unavailable` , `reputation_skipped_sample` , `reputation_over_limit` .

5. Step-by-step flows

5.1 Testing a typed URL

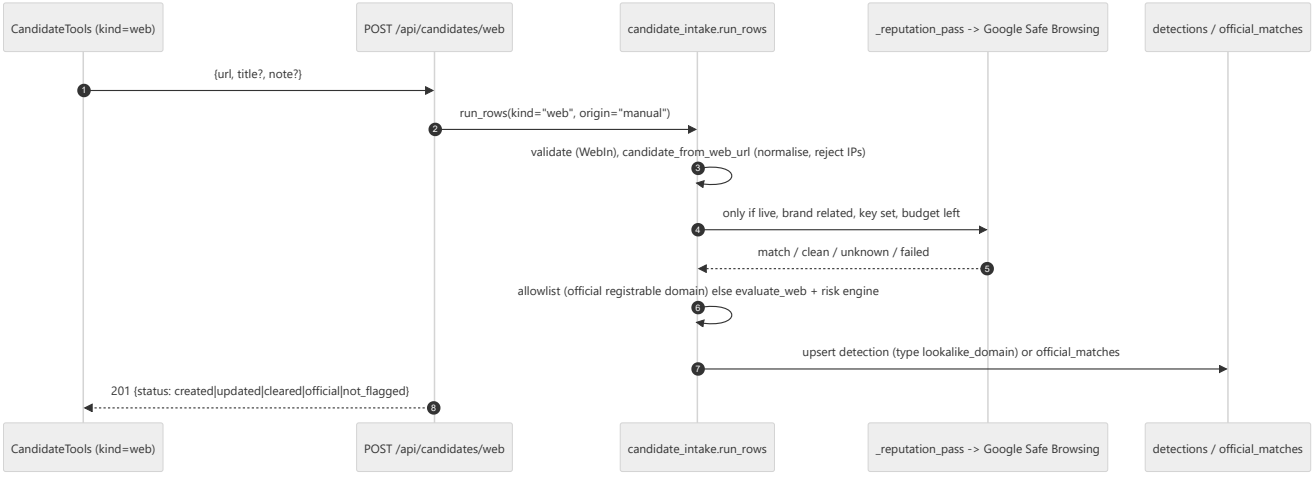
1. User types; after 350 ms `sm.lookalikeTestUrl(url)` sends `POST /api/lookalike/test-url {"url": "...", "brand_id"?: n}` (`frontend/src/api/sm.ts:152`).

2. `UrlBody` trims the text and rejects empty input or more than 2048 characters (`backend/app/api/lookalike.py:40-50` ; 422).
3. `evaluate_url` (`backend/app/core/pipeline/lookalike.py:248-299`) loads the Settings impersonation words, builds a matcher over brand name, aliases and official domain labels (`make_domain_matcher` , domain_match.py:280-287) and calls `analyze_domain` .
4. The response (lines 285-299) holds `verdict` , `verdict_label` , `headline` , `explanation` , `official_domains` , `compared_with` , `parts{hostname, hostname_unicode, subdomain_labels, registrable_domain, domain_label, tld, is_idn, punycode, port, has_credentials, is_ip, warnings}` , `techniques` , `score` , `name_match_percent` , `reasons` , `reputation_checked: false` , `reputation_note` .

5.2 Examples

`GET /api/lookalike/test-urls` returns labelled examples from `domain_examples` (lookalike.py:208-245): official domain, brand in subdomain (`<official>.verify-account.test`), added word (`<label>-support.<suffix>`), two TLD swaps, a homoglyph and its punycode form, a separator variant, and two unrelated business names. If the brand has no domain a plausible `<name>.com` is derived and `derived: true` with an explanatory note.

5.3 Saving a web threat



Details of validation, limits and statuses are in `docs/features/candidate-import.md` . The same pipeline runs for CSV/JSON imports and for a scan of scope `all` in sample mode.

6. Files, tables, APIs, services

Item	Detail
Frontend	`frontend/src/pages/WebThreatsPage.tsx` , `frontend/src/hooks/useUrlTest.ts` , `frontend/src/components/monitor/CandidateTools.tsx` , `MonitorParts.tsx` (`ReasonsBox` , `EvidenceBox` , `SeverityChip`)
API	`POST /api/lookalike/test-url` , `GET /api/lookalike/test-urls` (`backend/app/api/lookalike.py:77-80,119-122`); `GET /api/detections?category=web[&severity=]` ; `GET /api/detections/{id}` ; `POST /api/detections/{id}/status` ; `POST /api/candidates/web` ; `POST /api/candidates/import?kind=web&format=csv`
Engine	`backend/app/core/engine/domain_match.py` , `risk_engine.py` (`assess_web`), `name_match.py` , `urls.py`
Pipeline	`backend/app/core/pipeline/lookalike.py` (tester), `scan_pipeline.py:445-512` (`_reputation_pass` , `_evaluate_web`), `matching.py:194-199` (official domain exclusion)
Provider	`backend/app/core/providers/safe_browsing.py` , `providers/base.py`
Tables	`detections` , `detection_events` , `official_matches` , `api_usage` (Safe Browsing day counter), `collector_state` (sample cache), `audit_log`
External service	Google Safe Browsing `https://safebrowsing.googleapis.com/v4/threatMatches:find` (REAL, only when a key is set; URL is sent, never opened). Everything else on this feature is local code or sample data

Mock versus real: the tester and the domain scoring are real local algorithms (provenance `local_algorithm`). Detections in sample mode are fabricated (`.test` hosts) and every signal is labelled `synthetic` (`risk_engine.py:17-18,121-122`). The reputation signal is `real_api` only when Google answered.

7. Validation, errors, edge cases

Case	Behavior
Empty or over-long URL to the tester	422 (`lookalike.py:44-50`)
Input with spaces or control characters	verdict `not_comparable` , explanation "contains spaces or control characters" (`NOT_COMPARABLE_WHY` , lookalike.py:198-203)
IP address host (also integer/hex forms)	`not_comparable` ; also rejected as a candidate ("IP addresses are not compared", `candidate_intake.py:144-145`)
URL not http(s)	`not_comparable` in the tester; "url must be an http(s) address" for candidates (`_http_url` , candidate_intake.py:107-115); a missing scheme is added as `https://` there
Official domain registered with a different TLD	other TLD is a `tld_swap` look-alike
Official domain list empty	name-only comparison; UI notice
Unknown TLD (e.g. `.test`)	treated as one-label suffix with a warning (`domain_match.py:218-219`)
Brand token shorter than 5 characters	only matched exactly by the name matcher (`MIN_LEN_ONE_EDIT = 5` , `name_match.py:81`)
Safe Browsing key missing, quota, timeout	reputation is `unavailable` (never "clean"), scan continues
Review of a web detection	same rules as other detections: status change audited, re-scans never reset it (`docs/features/social-and-app-threats.md`)
Mark web detection official	422: add the domain under Brand Profile official domains instead (`backend/app/services/detections.py:355-356`)

Notes and wording issues found while verifying:

- The empty-state text "run a scan with web checks enabled" (`WebThreatsPage.tsx:310`) suggests a switch that does not exist. There is no web-checks setting in `backend/app/config/config.py` , and the only scans that produce web candidates are sample-mode scans of scope `all` . The Web Threats page itself has no scan button; scope `all` is started from the Dashboard (`DashboardPage.tsx:70`).
- Changing a brand's official `domains` (or name, aliases, keywords) through `PUT /api/brands/{id}` retires ALL earlier detections of that brand (status `retired` , hidden everywhere), rebuilds the Look-alike Lab built-in cases when the name changed, and tells the user to re-scan (`backend/app/api/brands.py:162,165-195` , `retire_results` , `backend/app/services/detections.py:499-505`). So an old web detection for a domain you later register as official disappears with that edit.

Needs confirmation: whether a manually added web candidate should be sent to Google Safe Browsing. The code does send it when a key is configured and the domain looks brand related (`candidate_intake.py:247` calls `_reputation_pass` ; `data_source` is `live` for `manual` / `import` , `base.py:75-78`), although the page text promises that URLs "are never opened" (true) and the tester note says reputation is not checked "here" (true for the tester only).

8. Config (names only)

`GOOGLE_SAFE_BROWSING_API_KEY` , `SAFE_BROWSING_TIMEOUT_SECONDS` (5), `SAFE_BROWSING_DAILY_BUDGET` (5000), `SAFE_BROWSING_MAX_URLS_PER_SCAN` (100), `SAFE_BROWSING_CACHE_TTL_SECONDS` (3600) (`backend/app/config/config.py:132-136`); `SEVERITY_*` cutoffs; Settings page `impersonation_words` and `sensitivity` (affects the `score >= threshold` branch of `is_hit`); `IMPORT_MAX_BYTES` (1 MB) and `IMPORT_MAX_ROWS` (500) for imports (config.py:85-86). Data mode `sample` is what makes sample `.test` web threats appear.

DOCS/FEATURES/LOOKALIKE-LAB.MD

Look-alike Lab

Plain-language summary

The Look-alike Lab is a safe playground for checking how BrandShield judges impersonation, without creating any detection. You can type a profile name, handle or app name and see instantly whether it would be flagged as a look-alike of your brand and why; upload a suspicious logo and see how closely it resembles your official logos; run a built-in checklist of example names (some that must be flagged, some that must not, some that are yours) to prove the matcher still behaves; and test domain names on the Web Threats page. Nothing you do in the Lab is stored as a finding, and no score or report is created.

All paths are relative to the repository root (`ScamMind AI/`).

1. Purpose and user-facing behavior

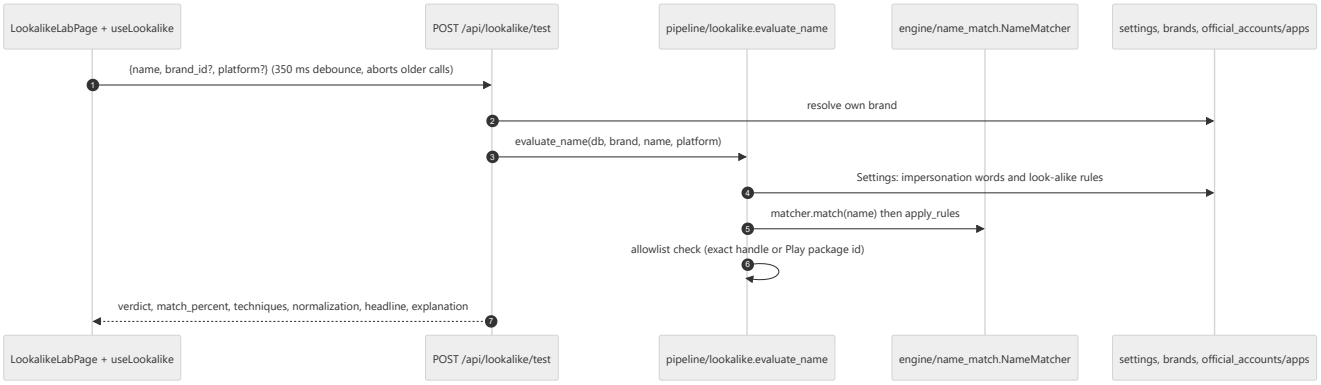
Page: `frontend/src/pages/LookalikeLabPage.tsx` (header "Look-alike Lab", lines 63-66). Components, top to bottom:

Block	What it does	Code
Compare a logo card	upload a candidate logo, compare with every official logo or with one manual reference logo	`frontend/src/components/common/LogoCompareCard.tsx:142-396` , mounted at `LookalikeLabPage.tsx:71`
Test a name	text box "Profile name, handle or app name"; verdict appears 350 ms after typing stops	`LookalikeLabPage.tsx:76-143` , hook `frontend/src/hooks/useLookalike.ts:8-45`
Preset pills	the first 8 stored test-case names of this brand	`LookalikeLabPage.tsx:50,93-104`
"How the name was normalized"	5-step timeline (typed, lower-cased, look-alike characters folded, spaces and symbols removed, brand + extra words)	`LookalikeLabPage.tsx:128-139` , built by `_normalization` (`backend/app/core/pipeline/lookalike.py:101-111`)
"Variations we catch" and "How we avoid over-flagging"	static explanations with examples derived from the brand name (`brandExamples`)	`LookalikeLabPage.tsx:148-211`
Test cases table	stored cases (expected verdict), button "Run test cases", pass/fail per row and "N of M pass"	`LookalikeLabPage.tsx:216-275`
URL tester	lives on the **Web Threats** page, not here (see `docs/features/web-threats.md`)	`frontend/src/pages/WebThreatsPage.tsx:138-259`

Roles: the name tester, the URL tester, logo comparison (`POST /api/lookalike/test-logo`), reading test cases and the logo-engine status need any signed-in user (`get_current_user` , `backend/app/api/lookalike.py:70-122`). **Run test cases** needs `admin` or `analyst` (`require_writer` , lookalike.py:158-164; the button is disabled for viewers, `LookalikeLabPage.tsx:232-233`). Adding or deleting a test case needs `admin` (`require_admin` , lookalike.py:133-155); the page has no UI for that, only the API. "Add this reference as an official logo" (logo card) needs `admin` / `analyst` because it calls `POST /api/brands/{id}/logos` (`backend/app/api/brands.py:449-453` , `require_writer` ; button shown only when `canWrite` , `LogoCompareCard.tsx:382`).

Nothing is saved by the testers: "Pure: no side effects, safe to call on every keystroke" (lookalike.py:72) and "Nothing is stored; no score or detection is created" (lookalike.py:94). The only writes in this feature are the seeding of built-in test cases, adding/deleting cases, an audit row when cases are run, and the optional "add as official logo".

2. Name tester



- Request body `TestBody` (`backend/app/api/lookalike.py:26-37`): `name` 1-200 characters, whitespace collapsed and control characters dropped (`clean_name` , lines 21-23); empty after cleaning is a 422 "Type a name to test". `brand_id` defaults to the user's own brand; someone else's brand id is a 404.
- Logic (`evaluate_name` , `backend/app/core/pipeline/lookalike.py:114-152`): uses the **same engine and the same Settings** as a real scan (`build_matcher` with the configured impersonation words; `apply_rules` with the Settings switches for character swaps, digits and symbols, added words, spacing, transposed or missing letters). Verdicts returned:

verdict	When	official_basis
official	the name (without @) is an exact official social handle (optionally for the given platform) or an exact Google Play package id on the allowlist	allowlist
official	the engine verdict is brand_name (the exact brand name). The text explains that in a real scan the name alone excludes nothing, only an exact handle, URL or package id does	name
flagged	engine verdict flagged; reasons lists techniques (character swap, digit or symbol swap, added word, spacing, transposed, missing/extra letter, homoglyph) and impersonation words	null
not_flagged	anything else, including review (half weight in a scan); mentions switched-off rules when relevant	null

- Response also contains match_percent, engine_verdict, techniques, technique_labels, squat_category, edit_distance, normalization[], on_allowlist, headline, explanation (lookalike.py:125-133).
- The matcher itself (backend/app/core/engine/name_match.py:1-25) decides in order: exact after folding, brand token plus extra words (flagged only for digits or impersonation words), one edit, window within one edit with only known leftover words, two edits for long brands (always review), first word only, otherwise safe. Folding handles lower case, @, leet digits, Unicode homoglyphs, accents, full-width, rn -> m, vv -> w, separators. Tokens shorter than 5 characters are matched exactly only (MIN_LEN_ONE_EDIT, line 81). This is why "Acme Bakery" is never a hit for "Acme Bank".
- All local code: provenance local_algorithm; no external service is called.

3. Test cases

- Built-in cases are generated **per brand** from its registered name by default_cases (backend/app/core/pipeline/lookalike.py:27-70): 12 cases, de-duplicated case-insensitively:

#	Expected	Construction (example for "Acme Bank")
1	flagged	glyph swap: m -> rn, w -> vv, d -> cl, or a Cyrillic look-alike letter: acrnebank
2	flagged	<name> Support: Acme Bank Support
3	flagged	separated: acme-bank
4	flagged	spaced letters: A C M E Bank
5	flagged	digit swap (e -> 3, a -> 4, o -> 0, i -> 1): acm3bank
6	flagged	transposed last word: Acme Bnak
7	flagged	<compact>_official: acmebank_official
8-10	not_flagged	<first word> Bakery, Hotels, Weather (other businesses)
11	official	the exact name: Acme Bank
12	official	@<compact>: @acmebank

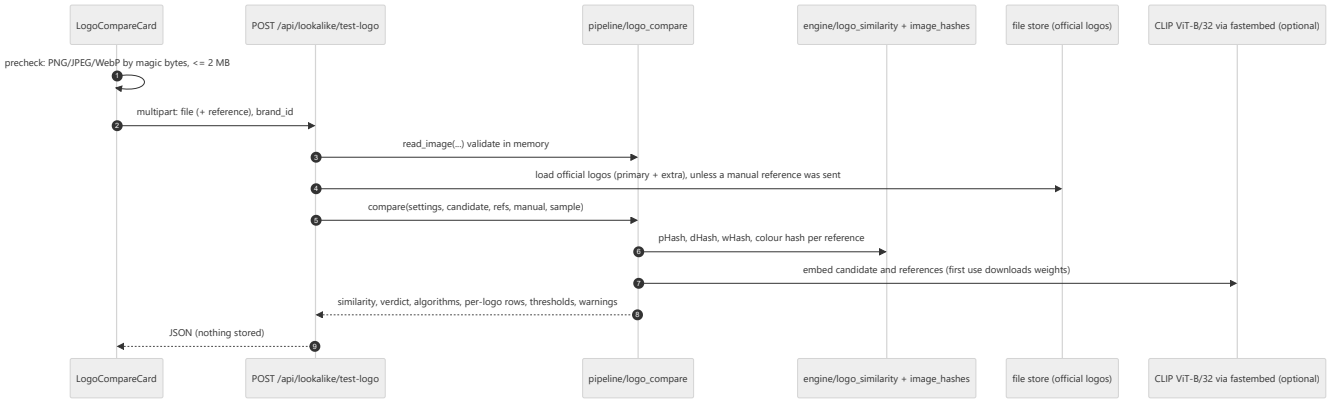
The docstring says they are not filtered by what the matcher says, "a failing case is information" (lookalike.py:30-31).

- Seeding: ensure_default_cases (78-98) inserts them once per brand when it has none (so an admin's deletions are respected), under a process lock plus the partial unique index uq_lookalike_cases_builtin_name to avoid duplicates from simultaneous requests (backend/app/models/ops.py:144-160). Rows get builtin=True, source="sample". It is also triggered by GET /api/lookalike/test-cases, so a read can write.
- When a brand is renamed, its built-in cases are replaced by cases for the new name; admin-added cases are kept (rebuild_default_cases, lookalike.py:186-190, called from backend/app/api/brands.py:188-189).
- GET /api/lookalike/test-cases returns {total, items:[{id, name, expected, builtin, source}]} (lookalike.py:125-130).
- POST /api/lookalike/test-cases (admin): body {name 1-200, expected: flagged|not_flagged|official}; duplicate name+expected is 409 "This test case already exists"; audit lookalike.case_added (133-144).
- DELETE /api/lookalike/test-cases/{id} (admin): 404 if the case is not in the user's brand; audit lookalike.case_removed (147-155).
- POST /api/lookalike/test-cases/run (writer): run_cases (lookalike.py:155-168) runs every stored case through evaluate_name and compares with expected; the result is **computed, never copied from the expected value**. Returns {total, passed, failed, summary: "N of M pass", items:[{id, name, expected, result, pass, match_percent, builtin}], ran_at} and writes audit lookalike.cases_run (lookalike.py:158-164).
- confusion(items) (lookalike.py:171-183) computes true/false positives and negatives, precision and recall of "flagged" over the cases. It is used by the Evaluation page (backend/app/api/evaluation.py:60,84), not by the Lab page.
- Results shown in the table exist only after a run; before that every row shows "Not run" (LookalikeLabPage.tsx:57-59,264).

4. URL tester (summary)

POST /api/lookalike/test-url and GET /api/lookalike/test-urls (lookalike.py:77-80,119-122) are surfaced on the Web Threats page; they compare a domain or URL with the official domains, never open it and never call Safe Browsing (backend/app/core/pipeline/lookalike.py:248-249,298). Full description: docs/features/web-threats.md.

5. Logo comparison



Endpoints

Endpoint	Role	Purpose
`POST /api/lookalike/test-logo` (multipart `file`, optional `reference`, query `brand_id`)	any signed-in user	compare; 422 "Add an official logo in Brand Registry or upload a reference logo." when there is nothing to compare against (`backend/app/api/lookalike.py:83-110`, `NO_LOGO`, `logo_compare.py:47`)
`GET /api/lookalike/logo-engine`	any signed-in user	is the AI (CLIP) engine usable? Checks the package and the weights on disk only: no model load, no network (lookalike.py:113-116, `clip_status`, `logo_compare.py:117-126`)

Validation (`read_image`, `backend/app/core/pipeline/logo_compare.py:60-78`)

Size cap `LOGO_MAX_BYTES` (2 MB; 413), type from magic bytes only (PNG, JPEG, WebP; SVG and everything else 415; the filename is never read), Pillow `verify()` plus decode, minimum 16 x 16 px and maximum 6000 x 6000 px (422), maximum 25 million pixels (decompression-bomb guard `MAX_PIXELS`, line 32). The browser pre-checks type and size (`precheckImage`, `LogoCompareCard.tsx:28-33`), but the server is authoritative. A very small image (< 64 px) or an almost single-colour one gets a warning (`_warnings`, 103-110).

Algorithms

Algorithm	Where	How
pHash	`backend/app/core/engine/logo_similarity.py:76-91`	32 x 32 grayscale (auto-contrast, LANCZOS), 2-D DCT, low 8 x 8 block compared with its median (DC term excluded) -> 64 bits
dHash	logo_similarity.py:94-105	9 x 8 grayscale, bit = left pixel brighter than right neighbour -> 64 bits
wHash	`backend/app/core/engine/image_hashes.py:25-35`	simplified Haar low-pass of a 64 x 64 image reduced to 8 x 8 (not bit-identical to the ImageHash library)
colour hash	image_hashes.py:38-57	14-bin hue/saturation histogram, reported separately
hash score	logo_similarity.py:112-114	`max(0, (1 - hamming/64 - 0.5) / 0.5) x 100` (unrelated images sit near 0.5 raw)
default hash similarity (`image-1`)	logo_similarity.py:117-124	0.6 x pHash + 0.4 x dHash
`multihash` (`image-2`)	image_hashes.py:22,76-83	0.4 pHash + 0.3 dHash + 0.3 wHash
CLIP (`Qdrant/clip-ViT-B-32-vision`)	logo_similarity.py:185,241-278	image embedding through fastembed (ONNX Runtime, no torch), L2 normalised; cosine similarity
CLIP calibration	logo_similarity.py:190-203	piecewise linear knots cosine -> score: (0.62,0), (0.80,55), (0.88,80), (0.97,90), (1.00,100)
combined	logo_similarity.py:206-211	CLIP leads; a hash score >= 90 only lifts an already-similar (>= 55) CLIP score, never rescues a low one

The Lab tester (`compare`, `logo_compare.py:140-188`) always returns the four fingerprint scores per reference (`algorithms`: `phash`, `dhash`, `whash`, `colorhash`), the headline `similarity`, and the CLIP score when available. `method` is `clip+hash` or `hash`. Which hash backend produces `hash_similarity` follows `IMAGE_SIMILARITY_BACKEND` (`_image_impl`, scan_pipeline.py:332-347), so the Lab and scans use the same engine. When CLIP is on, the model weights (about 336 MB) download into `DATA_DIR/models` on first use (a hint says up to half a minute; `FIRST_USE_NOTE`, `logo_compare.py:42`); a failure is remembered for 60 s so an offline machine does not retry each request (`get_clip_embedder`, logo_similarity.py:291-310, `_FAILURE_TTL` 284). `LOGO_COMPARE_CLIP=off` forces hashes only; `auto` (default) uses CLIP when installed.

Thresholds and verdicts (`verdict_for`, `logo_compare.py:50-57`; constants 29, `logo_similarity.py:32-33`)

Similarity	Verdict	Meaning in the UI
>= 90 (`SAME_MIN`)	`same_logo`	red
>= 80 (`LIKELY_COPY_MIN`)	`very_similar`	red; copy, resize, recolour, JPEG recompress measured >= 80
>= 55 (`SIMILAR_MIN`)	`somewhat_similar`	orange; slight crops or light edits, scores points in scans
< 55	`different`	green; no points in scans

(`VERDICT_TONE` , `LogoCompareCard.tsx:17` .) The same 55 / 80 cutoffs drive scan scoring (`image_points` , `backend/app/core/engine/scoring.py:102-107`): nothing below 55, a third of the maximum at 55, the maximum at 90.

Extra behaviours: the best result across all official logos is the headline (`best`); a candidate against hashes only that scores below 55 is re-tested colour-inverted and, if that matches, a warning says it "looks like a colour-inverted version" (`logo_compare.py:168-173` ; known gap documented in `logo_similarity.py:11`). The response carries `thresholds` , `explanation` ("a risk signal, not proof of reuse"), `warnings` , `is_sample` and `sample_note` (when the app is in sample data mode, `logo_compare.py:43-46`).

UI details (`LogoCompareCard.tsx`)

Drag-and-drop pickers (accept `image/png,image/jpeg,image/webp` , 2 MB hint, line 128-137); official logos shown as blob thumbnails because the files sit behind the bearer header (`useOfficialThumbs` , 57-93); optional manual reference mode (line 145, `manual = useManual && !!reference`); score ring, bar, per-algorithm rows with labels such as "pHash (overall shape)", per-logo table, a CLIP hint with the install command when the engine is not available (`showHint` , lines 173-175; `CLIP_HINT = 'pip install fastembed'`); each new comparison aborts the previous one (`abortRef` , 154,194-197). "Add this reference as an official logo" (writers, manual mode only) uploads it as an extra logo labelled "Added from Look-alike Lab" and refreshes the brand cache (221-223). That upload uses the stricter brand-logo rules (minimum 256 x 256 px, at most 10 extra logos, duplicates 409; `backend/app/core/pipeline/logo.py:12,59-60` , `backend/app/api/brands.py:455-463`).

6. Files, tables, APIs, external services

Item	Detail
Frontend	`frontend/src/pages/LookalikeLabPage.tsx` , `frontend/src/components/common/LogoCompareCard.tsx` , `frontend/src/hooks/useLookalike.ts` , `frontend/src/utils/logoCheck.ts` , `frontend/src/api/sm.ts:150-162`
API	`backend/app/api/lookalike.py` (`/api/lookalike/test` , `/test-url` , `/test-logo` , `/logo-engine` , `/test-urls` , `/test-cases` GET/POST/DELETE, `/test-cases/run`)
Pipeline and engine	`backend/app/core/pipeline/lookalike.py` , `logo_compare.py` , `backend/app/core/engine/name_match.py` , `logo_similarity.py` , `image_hashes.py` , `normalize.py` , `domain_match.py`
Tables	`lookalike_cases` (`backend/app/models/ops.py:144-162`), `audit_log` , read-only: `brands` , `brand_logos` , `official_accounts` , `official_apps` , `settings` ; files via `stored_files` or `DATA_DIR`
External services	none at request time. The only network use is the one-off download of the CLIP model weights from Hugging Face by fastembed (REAL, optional, only when `fastembed` is installed and `LOGO_COMPARE_CLIP` is not `off`)
Mock or sample	the built-in test-case rows carry `source="sample"` because they are generated; logic is real. With data mode `sample` the logo card shows the sample note, and sample scan candidates compare against generated images (`backend/app/core/collectors/sample_images.py`)

7. Validation, errors, edge cases

Case	Behavior
Name empty or only control characters	422 "Type a name to test"
Name over 200 characters	422 (pydantic `max_length`); the hook also truncates to 200 (`useLookalike.ts:32`)
No brand	name tester: 404 "No brand profile yet. Create one first."; the page shows `NoBrand` (`LookalikeLabPage.tsx:108`)
Logo larger than 2 MB / wrong type / corrupt / too small or large	413 / 415 / 415 / 422 with a plain message (`logo_compare.py:60-78`)
No official logo and no reference	422 `NO_LOGO` ; card shows a link to Brand Registry
CLIP not installed, weights missing, offline	falls back to hashes only; response `clip: {available: false, reason: not_installed
First CLIP comparison	slow (model load); the UI warns up to half a minute
Duplicate test case	409; non-admin add/delete 403
Case deleted by an admin	not re-seeded as long as the brand still has at least one case
Hash tester on inverted logos	scores about 0; a warning appears (known gap)

Needs confirmation: `TestBody.platform` (`backend/app/api/lookalike.py:29`) lets a caller restrict the allowlist check to one platform, but `LookalikeLabPage` and `sm.lookalikeTest` (`frontend/src/api/sm.ts:151`) never send it, so in the UI the check is across all platforms. Confirm whether a platform selector is planned.

Needs confirmation: the hint text in `LogoCompareCard.tsx:25` is `pip install fastembed` , while the backend hint also mentions `requirements-extra.txt` (`logo_similarity.py:186`); `fastembed` is listed in `backend/requirements-extra.txt` and not in `requirements.txt` . Confirm that the hosted deployment installs the extra requirements, otherwise CLIP is never active in production.

8. Config (names only)

`LOGO_COMPARE_CLIP` (`auto` | `off` , default `auto` ; config.py:83), `IMAGE_SIMILARITY_BACKEND` (`hash` | `multihash` | `imagehash` | `clip` ; config.py:81), `LOGO_MAX_BYTES` (2 MB; config.py:61), `DATA_DIR` (model cache `DATA_DIR/models` , `logo_compare.py:113-114`). Settings page values `impersonation_words` and the five look-alike `rules` change name-tester results because the tester reads them (`lookalike.py:116-124`). Dependencies: `fastembed` (optional, `backend/requirements-extra.txt`), `rapidfuzz` , `tldextract` , `idna` (`backend/requirements.txt`).

DOCS/FEATURES/CANDIDATE-IMPORT.MD

Candidate intake: manual entry and CSV / JSON import

Plain-language summary

Sometimes you already know about a suspicious account, app or website: a customer sent a screenshot, or a partner shared a list. Candidate intake lets an analyst type one in ("Add candidate") or upload a CSV or JSON file with many rows ("Import CSV / JSON"). Each row is checked for mistakes, then judged exactly like something a scan found: official assets are skipped, the rest are scored, and the suspicious ones appear in the Social, App or Web Threats list for review. Nothing in a row is ever opened or downloaded; you can also do a "dry run" first, which checks the file and saves nothing.

All paths are relative to the repository root (`ScamMind AI/`).

1. Purpose and user-facing behavior

The tools appear as two buttons, **Add candidate** and **Import CSV / JSON**, on three pages (`frontend/src/components/monitor/CandidateTools.tsx:45-87`):

Page	`kind`	Mounted at
Social Media Monitoring	`social`	`frontend/src/pages/SocialMonitoringPage.tsx:91`
App Store Monitoring	`app`	`frontend/src/pages/AppStoreMonitoringPage.tsx:93`
Web Threats	`web`	`frontend/src/pages/WebThreatsPage.tsx:296`

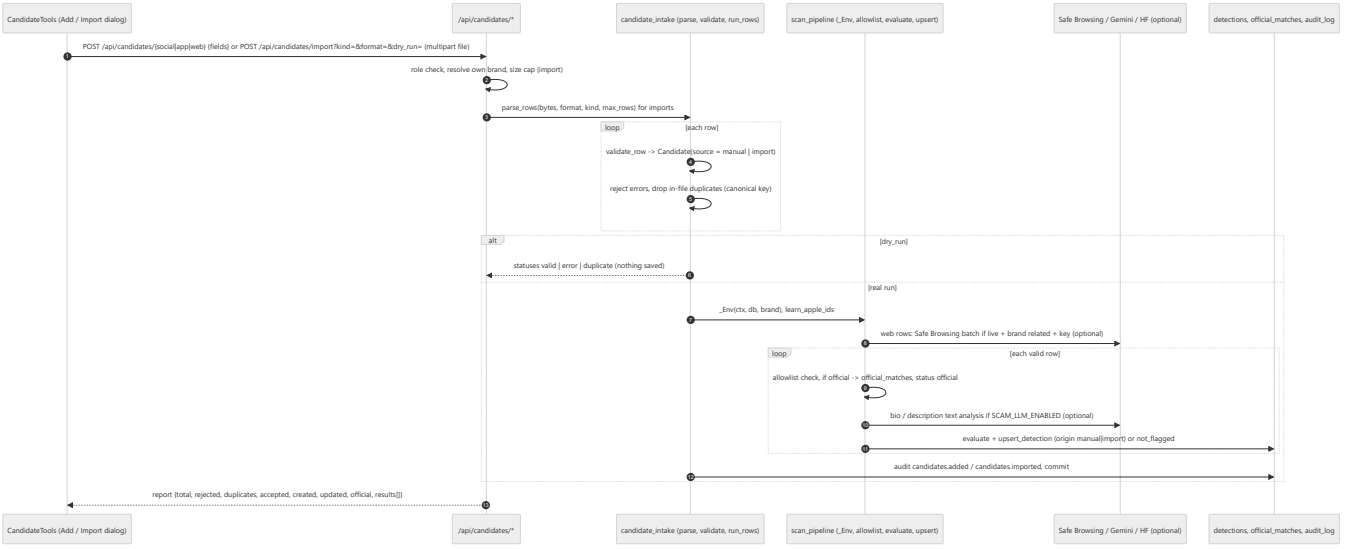
- Buttons are disabled for viewers with the tooltip "Your role can view but not add candidates" (`CandidateTools.tsx:48,52-57`). The API requires role `admin` or `analyst` (`require_writer` , `backend/app/config/deps.py:56` , used in `backend/app/api/candidates.py:37,42,47,58`).
- **Add dialog** ("Add a candidate by hand"): form fields per kind (`FIELDS` , `CandidateTools.tsx:12-32`). Only non-empty values are sent (line 100). Result text: the server `message` or "Candidate saved (status)." (line 102).

Kind	Fields in the form (required marked *)
social	platform * (instagram, facebook, x, telegram, linkedin, youtube), handle *, display name, profile URL, bio
app	store * (google_play, app_store, third_party_apk), package or bundle id *, app name *, publisher, description
web	URL * ("never opened by this app"), title, note

The API accepts more fields than the form shows (followers, following, posts, created_at, verified, image_id for social; publisher_id, installs, rating_count, image_id for apps), see section 3.

- **Import dialog** (`ImportDialog` , `CandidateTools.tsx:146-216`): choose a `.csv` or `.json` file (format picked from the file extension, line 151), press **Dry run** (validates and saves nothing; shows "N rows, N valid, N rejected, N duplicates" and every error as "Row N: message", lines 168-196), then **Import valid rows** (enabled only after a dry run with at least one valid row, line 208). After a real import: "Import finished: N created, N updated, N rejected." (line 160).
- After either action the page reloads its lists (`onChanged` , `useMonitor.reloadAll`).

2. Flow



Single entry

`POST /api/candidates/social|app|web?brand_id=` (`backend/app/api/candidates.py:36-48` , HTTP 201). The body is validated by the pydantic model (unknown fields refused, section 3), then `_single` (lines 27-33) calls `run_rows(..., origin="manual", dry_run=False)` with one row; an `error` result becomes HTTP 422 with the row's

message. Any other outcome is 201 with `{row, status, detection_id, score, severity, message}` where `status` is one of `created`, `updated`, `cleared`, `official`, `not_flagged` (`backend/app/core/pipeline/candidate_intake.py:253-267`). So a 201 does not mean a detection exists: `official` and `not_flagged` store no detection.

Import

`POST /api/candidates/import?kind=social|app|web&format=csv|json&dry_run=true|false` with multipart field `file` (`candidates.py:51-72`):

1. Reads at most `IMPORT_MAX_BYTES + 1` bytes (default 1 MB); larger is HTTP 413 "The file is larger than N KB" (lines 65-67).
2. `parse_rows` (`candidate_intake.py:170-204`):
   - decoding: UTF-8 (BOM allowed) or UTF-16 with a byte-order mark; anything else 422 "must be UTF-8" (`decode_bytes`, 159-167);
   - **CSV**: strict `csv.DictReader`; needs a header row; every header must be an allowed column of the kind (`COLUMNS`, lines 33-36) else 422 "Unknown column(s): ... Allowed: ..."; values are stripped; a row with more values than headers becomes an error row "Too many values in this row"; more than `IMPORT_MAX_ROWS` (500) rows is 413; malformed CSV 422;
   - **JSON**: an array of objects, or `{"rows": [...]}`; not JSON 422; not an array 422; more than 500 rows 413; a non-object element becomes an error row "Row is not an object".
3. `run_rows` (225-276) validates each row, then (unless `dry_run`) analyses it. Valid rows are saved even when others fail (partial success, `candidates.py:63`).

Allowed columns

Kind	Columns (`candidate_intake.py:33-35`)
social	`platform, handle, display_name, url, bio, followers, following, posts, created_at, verified, image_id`
app	`store, package_id, name, publisher, publisher_id, url, description, installs, rating_count, image_id`
web	`url, title, note`

Example JSON for web (shape from `WebIn`, lines 98-101):

```
[{"url": "https://acmebank-support.example/kyc-update", "title": "KYC page", "note": "reported by a customer"}]
```

Example CSV for social:

```
platform,handle,display_name,bio
instagram,acmebank_support,Acme Bank Support,Official helpline. DM to unblock your account
```

3. Validation rules

Every row model forbids unknown fields and strips whitespace (`_Row`, `candidate_intake.py:49-50`); empty strings become null (`_blank_to_none`, 207-208). Error messages are shortened to 300 characters.

Field	Rule	Code
`platform` (social)	one of instagram, x, facebook, linkedin, youtube, telegram; `twitter` is mapped to `x`	lines 66-74, `services/validators.py:4`
`handle`	1-100 characters, then `normalize_handle`: leading `@` removed, 3-30 of letters, digits, dot, underscore; LinkedIn and YouTube also allow hyphen (LinkedIn up to 100, YouTube 30)	`backend/app/services/validators.py:7-40`
`display_name` / `name`	max 200 (app `name` required, min 1)	lines 56, 80
`url`	max 500; must be http(s): a missing scheme gets `https://`; otherwise "url must be an http(s) address". Social without URL gets a built one (`instagram.com/<h>`, `linkedin.com/company/<h>`, `youtube.com/@<h>`, `t.me/<h>`)	`_http_url` 107-115, `to_candidate` 120-123
`bio`	max 2000; `description` max 4000	lines 58, 84
`followers`, `following`	integer 0 to 10^10; `posts` 0 to 10^9	lines 59-61
`created_at`	ISO date	line 62
`verified`	boolean	line 63
`store` (app)	google_play, app_store, third_party_apk	lines 89-95
`package_id` (app)	1-155 chars; for Google Play must look like `com.example.app` (dot separated parts starting with a letter); for App Store a reverse-DNS bundle id; not checked for `third_party_apk`	`validate_package_id`, `validators.py:82-92`
`publisher` / `publisher_id`	max 200 / 120. `publisher_id` is used by the allowlist only if that publisher is VERIFIED in Brand Registry (`matching.py:151-152,185-187`)	lines 81-82
`installs`	integer 0 to 10^12, stored as `installs_count` for the few-installs signal	lines 85,138-139
`rating_count`	0 to 10^10	line 86
`image_id`	integer >= 1: id returned by `POST /api/candidate-images`	lines 64, 87
web `url`	max 500; must normalise to a host with a registrable domain; IP addresses are refused ("IP addresses are not compared")	`to_candidate` 142-146, `base.py:381-400`
web `title` / `note`	max 300 / 1000	lines 100-101

Text cleaning (`clean_text` , lines 39-46): control characters become spaces, whitespace is collapsed, a leading `=` , `+` , `-` , `@` , tab or carriage return is defused with a leading `'` (spreadsheet formula injection), and the text is cut to its limit. The API never renders anything as HTML.

Duplicates inside one file: rows with the same canonical key (`platform:handle` , or the canonical URL) as an earlier row are reported as `duplicate` and skipped (lines 235-239; `canonical_key` , `backend/app/core/pipeline/scan_pipeline.py:198-203`). Duplicates of an existing detection are not rejected: they update it (below).

Per-row result statuses: `error` , `duplicate` , `valid` (dry run only), `official` , `not_flagged` , `created` , `updated` , `cleared` . Report fields (lines 273-276): `dry_run, kind, origin, total, rejected, duplicates, accepted (= total - rejected - duplicates, which includes official and not_flagged rows), created, updated, official, results[]` .

4. How rows become detections

`run_rows` (`candidate_intake.py:225-276`) deliberately reuses the scan pipeline functions, so a manual row is judged like a scan row:

1. `sp._Env(ctx, db, brand)` : the brand's matcher, allowlist, official logos, Settings (sensitivity threshold, rules, impersonation words), risk engine and text service (`scan_pipeline.py:287-315`).
2. `learn_apple_ids` and `_reputation_pass` (Safe Browsing for web rows, conditions in `docs/features/web-threats.md` section 4).
3. For each valid row: **allowlist first** (`BrandAllowlist.check`): exact official handle, URL, package id, bundle id, verified publisher id or official domain: recorded in `official_matches` , result `official` , "Excluded: ".
4. `evaluate` (`scan_pipeline.py:515-554`): the prefilter is **off** for these rows (`c.source not in INTAKE_SOURCES` , line 526); a candidate with nothing tying it to the brand is `ignored` , reported as `not_flagged` "Evaluated; nothing ties it to the brand, so it was not saved." (candidate_intake.py:258-260).
5. `upsert_detection(ctx, db, None, brand, oc, existing, counts)` (scan_pipeline.py:678-724) with no scan (`scan=None`): creates the detection with event "Added by manual entry." / "Added by import entry." (line 697) or updates an existing one; a decided detection (reviewing, confirmed, dismissed, marked_official) keeps its status (lines 710-716). A score below the review threshold but a similar name stores a `cleared` row ("Below the review threshold: kept in the cleared list").
6. `origin` is `manual` or `import` (`origin_of` , scan_pipeline.py:80-81); `source` is `live` (`Candidate.data_source` , `base.py:75-78`). Logo: URLs are never fetched; only an image uploaded through `POST /api/candidate-images` and linked with `image_id` is compared with the official logos (`_logo_step` , scan_pipeline.py:408-421; `_intake_image` 364-377; a row linked to another account's image gets "unavailable").
7. One audit row `candidates.added` or `candidates.imported` ("N analysed, M detections", lines 268-269), then a single `commit` .

Scoring, bands and severity are identical to scans: see `docs/features/scanning-and-detection.md` sections 8 and 9. The detection shows up in the matching page list (social, apps or web). A viewer of those lists sees the origin in the row (`origin` field of `summary_row` , `backend/app/services/detections.py:201`).

Candidate image upload

`POST /api/candidate-images?purpose=candidate_image|app_icon&brand_id=` (`backend/app/api/candidates.py:75-106`), writer only: reads at most `CANDIDATE_IMAGE_MAX_BYTES` (2 MB) + 1; `process_candidate_image` (`backend/app/core/pipeline/candidate_images.py:36-64`) takes the type from the bytes (PNG, JPEG, WebP), refuses SVG, enforces 32-6000 px per side and 25 million pixels, decodes and re-encodes (metadata dropped), computes pHash/dHash/wHash/colour hash, and stores it under a hash-derived name `candidate_images/brand_<id>/<sha24>.<ext>` . The same image for a brand is stored once (unique `(brand_id, sha256)`). Response: `{id, format, width, height, size_bytes, sha256, purpose, url, note}` . `GET /api/candidate-images/{id}/file` serves it only to the owner of the brand with `nosniff` . Errors: 413, 415, 422. **The web UI has no upload control for this** (no reference to `candidate-images` or `image_id` in `frontend/src`); it is API only today.

5. Files, tables, APIs, services

Item	Detail
Frontend	`frontend/src/components/monitor/CandidateTools.tsx` (+ `.css`), client calls `sm.addCandidate` , `sm.importCandidates` at `frontend/src/api/sm.ts:125-131` , report type `ImportReport` (`frontend/src/api/types.ts:621-631`)
API	`backend/app/api/candidates.py` (`/api/candidates/social` , `/app` , `/web` , `/import` , `/api/candidate-images` , `/api/candidate-images/{id}/file`)
Logic	`backend/app/core/pipeline/candidate_intake.py` , `candidate_images.py` , reuse of `scan_pipeline.py` , `matching.py` , `services/validators.py` , `core/collectors/base.py` (`candidate_from_web_url`)
Tables	`detections` , `detection_events` , `official_matches` , `candidate_images` , `audit_log` , `stored_files` (when `STORAGE_BACKEND=database`), `api_usage` (Safe Browsing day counter), `text_analysis_cache` ; reads `brands` , `official_accounts` , `official_apps` , `brand_publishers` , `asset_verifications` , `settings`
External services	none required. Optional, only when configured: Google Safe Browsing for web rows (the URL is sent, never opened); Gemini or Hugging Face for a social bio of at least 20 characters or an app description when `SCAM_LLM_ENABLED=true` (manual and imported rows are real, not sample, so they are eligible: `text_analysis.py:74`)
Not called	SearXNG, Tavily, iTunes, google-play-scraper (no discovery for manual rows); no logo or page download

6. Permissions, error handling, edge cases

Case	Behavior
Viewer role	403; UI buttons disabled
No brand profile	404 "No brand profile yet. Create one first." (`resolve_brand`); someone else's `brand_id` : 404
File over 1 MB	413
More than 500 rows	413 (CSV raises as soon as the 501st row is read; JSON counts the array)
Wrong encoding / not CSV / not JSON / unknown column	422 with a plain message
A single invalid row in an import	that row is `error` ; the others still import (partial success). The UI shows the row numbers
Single-add validation error	422 with the row message (`candidates.py:31-32`); unknown field names are also 422 from the model (`extra="forbid"`)
Candidate is your own official asset	`official` : nothing saved as a detection; an `official_matches` row is created or updated
Same candidate added twice	the second call is `updated` (same detection), not a second row (unique `(brand_id, canonical_key)` , `backend/app/models/detection.py:14`)
Web row that is an IP address, or has no registrable domain	`error` "url is not a usable website address (IP addresses are not compared)"
Dry run	nothing is written, not even audit; Safe Browsing and AI are not called (the `if dry_run` branch skips the whole analysis, `candidate_intake.py:241-242`)
Safe Browsing key missing or budget used	web row still analysed; reputation `unavailable` (never clean)
Hosted AI unavailable	local text analysis still runs; hosted finding marked `unavailable`
Existing detection already confirmed/dismissed	status and score unchanged (human decision survives)
Sample data mode	see the note below

Needs confirmation: rows added by hand or imported get `source = "live"` even when the app is running in sample data mode. The detection list defaults to the current data mode as its source filter (`backend/app/api/detections.py:52-56`), so in sample mode such a row would not appear in the default Social, App or Web list. Confirm whether intake is meant to be used only in live mode or whether the list filter should include `manual` and `import` origins.

Needs confirmation: `accepted` in the import report counts rows that were `official` or `not_flagged` as accepted (`candidate_intake.py:274`) while the dialog text calls them "valid" (`CandidateTools.tsx:186`); confirm whether the wording should distinguish "valid and saved" from "valid but not saved".

7. Config (names only)

`IMPORT_MAX_BYTES` (1 MB), `IMPORT_MAX_ROWS` (500), `CANDIDATE_IMAGE_MAX_BYTES` (2 MB) (`backend/app/config/config.py:84-86`); inherited from the pipeline: `SCAM_LLM_ENABLED` , `SCAM_LLM_PROVIDER` , `GEMINI_API_KEY` , `HUGGINGFACE_API_TOKEN` , `GOOGLE_SAFE_BROWSING_API_KEY` , `SAFE_BROWSING_DAILY_BUDGET` , `IMAGE_SIMILARITY_BACKEND` , `STORAGE_BACKEND` , `SEVERITY_*` , and the Settings page `sensitivity` , look-alike rules and impersonation words (`backend/app/services/app_settings.py:19-24`).

DOCS/FEATURES/INVESTIGATIONS.MD

Investigations

Plain-language summary

When a scan finds something that might be impersonating the brand (a fake profile, a fake app, a look-alike website), it becomes a "case" with a risk score. The Investigations page lets a person work each case to a decision. They record a status (keep reviewing, confirm as fake, or dismiss) and, separately, an outcome called a resolution (true positive, false positive, resolved, or still monitoring), with an optional note. Every decision is saved with who made it and when, and also written to the audit log. The system never reports or takes anything down by itself: it only records what a person decided.

This feature is real code against the database. The cases themselves come from scans; in sample data mode they are sample records (see `docs/features/source-health-and-providers.md` for data modes).

Purpose and user-facing behaviour

Page `/investigations` (`frontend/src/App.tsx:145-152` , component `InvestigationsPage` , `frontend/src/pages/InvestigationsPage.tsx:18`):

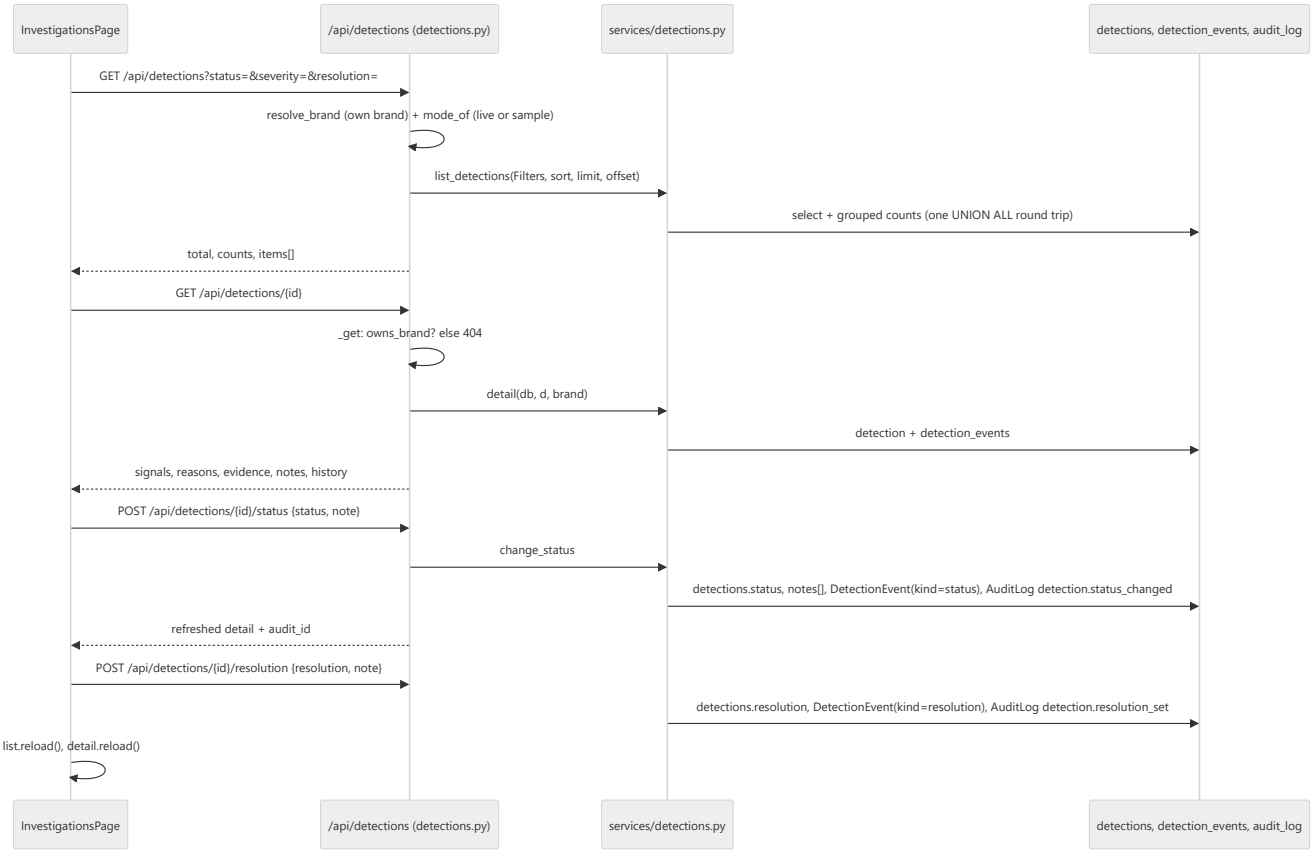
- Three filters (status, severity, resolution) and a list of cases for the active brand (`:21-33`). Each row shows name or handle, category, platform, resolution, score, severity chip and status chip (`:94-110`). The first case is selected by default (`:35`).
- The case panel (`CasePanel` , `:124-200`) shows status, severity and resolution chips, the score line, a note box (max 1000 characters, `:161`), three status buttons (Reviewing, Confirmed, Dismissed), a resolution dropdown with "Save resolution", and a timeline built from `history` (newest first, `:187-197`).
- A viewer sees "Read only: your role can view but not change cases." and all buttons are disabled (`:183` , `canWrite` from `frontend/src/hooks/useMe.ts:33`).
- Status buttons are disabled once a case is `marked_official` (`:167`).
- With no brand yet the page shows `NoBrand` (`:39-48`).

Vocabulary

Field	Values	Set by	Code
`status`	`new` (initial), `reviewing` , `confirmed` , `dismissed` , `marked_official` , `cleared` (below threshold but near-name; false-positive guard), `retired` (brand identity changed)	people can set only `reviewing` , `confirmed` , `dismissed` ; `marked_official` via the mark-official action; `new` , `cleared` , `retired` by the system	`backend/app/models/detection.py:32-33` ; `backend/app/services/detections.py:30-33` , `:308-312`
`resolution`	`true_positive` , `false_positive` , `resolved` , `monitoring` , or null	`POST /api/detections/{id}/resolution`	`models/detection.py:34-35` ; `services/detections.py:329-345`
`severity`	`LOW` , `MEDIUM` , `HIGH` , `CRITICAL`	risk engine at scan time (default cutoffs 40 / 70 / 85)	`models/detection.py:28-29` ; `backend/app/config/config.py:166-168`
`band`	`high` , `medium` , `low`	scan	`models/detection.py:27`

Status and resolution are independent: setting a resolution never changes the status (`services/detections.py:333` , docstring).

Step-by-step flow



1. **List.** `GET /api/detections` (`backend/app/api/detections.py:59-97`). Query filters: `category` (`social|apps|web`), `type` , `platform` , `band` , `severity` , `resolution` , `status` (repeatable), `source` (`live|sample|all`), `q` (max 200, substring match on name, handle, package id, publisher), `min_score` , `sort` , `from` , `to` , `limit` (1-200, default 50), `offset` , `brand_id` . The brand is `resolve_brand(db, brand_id, user)` , so it is the user's active brand unless an owned `brand_id` is passed (`:81`). By default only records of the current data mode are returned; `source=all` shows both (`_source_filter` , `:52-56`). The Investigations page sends none of `source` , `sort` , `limit` except the `sm.detections` defaults `sort=score_desc, limit=200` (`frontend/src/api/sm.ts:108-109`).
2. **Which statuses appear.** With no `status` filter the list shows `new, reviewing, confirmed, dismissed` (`LISTED_STATUSES` , `services/detections.py:31`). `cleared` , `retired` and `marked_official` never appear in a list; asking for `marked_official` explicitly is stripped out (`:81-83`), so the "Marked official" option in the status filter (`InvestigationsPage.tsx:14`) returns an empty list. **Needs confirmation:** confirm this is the intended behaviour for that filter option. Invalid status values return 422 (`detections.py:87-89`).
3. **Detail.** `GET /api/detections/{id}` returns the score breakdown (`signals` , `weights` , `reasons` , `name_match` , `domain` , `reputation` , `logo` , `text_analysis`), the evidence summary (page URL, capture time, snapshot hash, screenshot link), `score_history` , `notes` , the append-only `history` , and category-specific blocks (`suspect` , `official`) (`services/detections.py:229-283`).
4. **Status.** `POST /api/detections/{id}/status` with `{status, note}` (`note` max 1000). Shortcuts `POST .../confirm` , `.../keep-reviewing` , `.../dismiss` do the same with an optional note (`detections.py:177-188`). `change_status` (`services/detections.py:308-326`): aliases such as `confirm` or `keep-reviewing` are accepted (`:34-35`); anything other than `reviewing|confirmed|dismissed` returns 422; a `marked_official` detection returns 409 "This asset was marked official and is on the allowlist. Remove it from the allowlist first."; setting the same status with no note is a no-op (returns no audit id); otherwise it stores the note in `detections.notes` (keeps the last 50, `[-50:]`), adds a `DetectionEvent` and an audit row `detection.status_changed` , and commits.
5. **Resolution.** `POST /api/detections/{id}/resolution` with `{resolution, note}` (`detections.py:170-174`); `set_resolution` stores it, adds a `DetectionEvent(kind="resolution")` and an audit row `detection.resolution_set` with detail `"<old or none> -> <new>: <note>"` (`services/detections.py:332-345`).
6. **Mark official** (not on the Investigations page, but the same service). `POST /api/detections/{id}/mark-official` adds the asset to the brand allowlist (`official_accounts` or `official_apps`), records an `official_matches` row and closes the case, so future scans drop it before scoring (`services/detections.py:348-414`). Refused (422) for web look-alike domains (add the domain to the brand profile instead), for apps not listed on Google Play or the App Store, for detections with no package id or handle, and for handles over 30 characters; 409 if already marked official.

Files, functions, classes

Item	Path
Routes and request bodies (`StatusBody` , `ResolutionBody` , `NoteBody` , `MarkOfficialBody`)	`backend/app/api/detections.py`
Queries, serialisation, review actions, `ActionError`	`backend/app/services/detections.py`
Models `Detection` , `DetectionEvent` , `OfficialMatch`	`backend/app/models/detection.py`
Audit helper	`backend/app/services/audit.py:6-26`
Page and hooks	`frontend/src/pages/InvestigationsPage.tsx` , `frontend/src/hooks/useApi.ts` , `useBrand.ts` , `useMe.ts` ; API calls `sm.detections` , `sm.detection` , `sm.setReview` , `sm.setResolution` (`frontend/src/api/sm.ts:108-118`)

APIs, tables, external services

Endpoint	Method	Role	Notes
`/api/detections`	GET	any signed-in user	list with filters and counts
`/api/detections/{id}`	GET	any signed-in user	detail, 404 if not the user's brand
`/api/detections/{id}/evidence`, `/evidence/screenshot`	GET	any signed-in user	see `docs/features/reports.md`; screenshot 404 "No screenshot was captured (screenshots are off by default)"
`/api/detections/{id}/status`, `/confirm`, `/keep-reviewing`, `/dismiss`	POST	`admin`, `analyst`	writer only
`/api/detections/{id}/resolution`	POST	`admin`, `analyst`	writer only
`/api/detections/{id}/mark-official`	POST	`admin`, `analyst`	writer only
`/api/detections/excluded`, `/false-positives`	GET	any signed-in user	official items and `cleared` list

Tables touched: `detections` (`status`, `resolution`, `notes`, `updated_at`), `detection_events` (append-only history), `audit_log`, and for mark-official `official_accounts`, `official_apps`, `official_matches`. The `resolution` column came from migration `0007_resolution.py`. No external service is called: this feature only records human decisions (`services/detections.py:1-5`, docstring).

Validation, permissions and error handling

- **Tenant isolation:** `_get` returns 404 "Detection not found" unless `owns_brand` is true (`detections.py:38-42`); `owns_brand` is in `backend/app/api/common.py:53-58`.
- **Roles:** reads need any signed-in user; every change needs `require_writer` (`admin` or `analyst`); a `viewer` gets 403 "You do not have permission to do this" (`backend/app/config/deps.py:46-56`). The UI disables the buttons for viewers as well (`InvestigationsPage.tsx:167`, `:174`, `:179`).
- **Notes:** at most 1000 characters (Pydantic `Field(max_length=1000)`, `detections.py:21`, `:26`); trimmed; kept as `{at, actor, text, kind:"reviewer"}`, newest 50 only.
- **Errors:** `ActionError` is converted to `HTTPException` with the same status and message (`detections.py:45-49`). Validation errors are 422 with a `detail` list.
- **Audit:** `detection.status_changed`, `detection.resolution_set`, `detection.marked_official`, plus `app.added` / `account.added` when mark-official creates the allowlist entry. Rows carry the detection's `source` (`live` or `sample`). The audit log page is described in `docs/features/settings-and-audit.md`.
- **Edge cases:** a brand identity change retires all non-official detections (`status="retired"`) so they vanish from this page (see `docs/features/brand-registry.md`); `sync_with_allowlist` can move a case to `marked_official` automatically when an official account or app is added (`services/detections.py:430-445`); a status change to the same value with a note still records the note and event (`:317-318`).

Related configuration (names only)

`SEVERITY_MEDIUM_MIN`, `SEVERITY_HIGH_MIN`, `SEVERITY_CRITICAL_MIN` (severity cutoffs, `backend/app/config/config.py:166-168`), `SCREENSHOTS_ENABLED` (screenshots off by default, `:121`), `STORAGE_BACKEND`. There is no feature flag for the review workflow itself.

DOCS/FEATURES/SOURCE-HEALTH-AND-PROVIDERS.MD

Source Health and providers

Plain-language summary

BrandShield looks for fakes on several platforms (Instagram, Facebook, X, Telegram, LinkedIn, Google Play, the App Store and third-party APK sites) through "data sources" such as a search service and the Apple store search. The Source Health page shows whether each platform's source is working (online), slow (partial results) or unavailable, when it was last checked and when the system will try again by itself. An administrator can pause a platform, or press "Retry now"; if the retry could spend paid search credits the system first asks for confirmation and shows how many credits are left. The page is administrator-only. Where the data comes from (real, sample, or automatic) and which AI text-analysis provider is used are chosen on the Settings page, and this document explains both because they are one system.

Real versus sample: when the data mode resolves to `sample` no outside service is contacted and every source row says it will use sample data. All numbers on this page come from the database and live collector state.

Purpose and user-facing behaviour

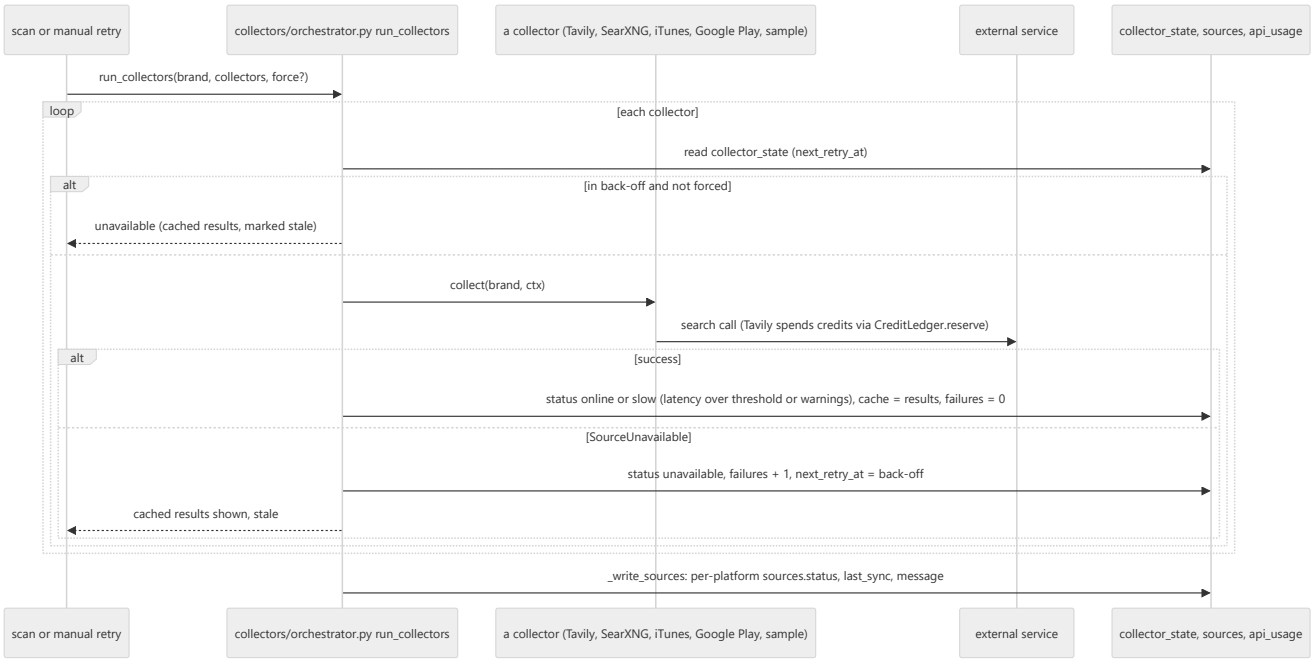
Page `/sources` (`frontend/src/pages/SourceHealthPage.tsx:62`), route wrapped in `RequireAdmin` (`frontend/src/App.tsx:174-179`) and menu item hidden for non-admins (`frontend/src/components/layout/Sidebar.tsx:33` , `:124`).

Part of the page	Behaviour	Code
Four tiles: Online, Slow, Unavailable, Next scan	counts from `summary` and the schedule	`SourceHealthPage.tsx:171-213`
Sources list	one row per platform: status chip, "Monitor" checkbox, message, "Last sync", cached age, next retry, "Retry now" (only for `slow` or `unavailable`)	`:229-298`
"When a source is unavailable"	four fixed explanation steps; step 3 is generated from `summary.retry_policy` by `describeRetry`	`:304-331` , `:29-40`
Recent source events	built in the browser from the source reports of the latest 5 scans (`GET /api/scans?limit=5`); there is no separate event feed	`:48-60` , `:334-352`
Search credits (Tavily) meter	`used of budget credits used in <month> · N left` ; amber at 80 percent or more	`:140-142` , `:355-372`

Viewers and analysts cannot open this page (admin only), but the API endpoints below still apply the roles listed in the table further down. The Dashboard shows a six-row "Source health" mini card for everyone and a "Manage" link only for admins (`frontend/src/pages/DashboardPage.tsx:399-434`).

The data-mode switch, the provider choice and the Safe Browsing status are not on this page. They are on Settings, tab "Account" (`frontend/src/pages/SettingsPage.tsx:494-577` for data mode, `:434-491` for `ProvidersPanel` and the Safe Browsing line). See `docs/features/settings-and-audit.md` .

How a source gets its status



- **Statuses.** `online` , `slow` , `unavailable` (`backend/app/models/ops.py:18` , `:35`). A call that works but takes longer than `SOURCE_SLOW_THRESHOLD_MS` (default 4000) or returns warnings is `slow` with the reason `slow` or `partial` and its results are kept (`backend/app/core/collectors/orchestrator.py:299-314`). A `SourceUnavailable` exception, or any unexpected exception in a collector (reported as `internal_error`), makes it `unavailable` (`:286-293` , `:316-330`). A buggy collector never stops a scan.

- **Automatic retry (back-off).** After consecutive failures the wait is 1 minute, 5 minutes, 15 minutes, 1 hour, then 6 hours (`BACKOFF_SCHEDULE_SECONDS = (60, 300, 900, 3600, 21600)` , `orchestrator.py:39` , `backoff_delay` `:56-59`). A provider's `Retry-After` is honoured up to 24 hours (`:40` , `:62-68`). Problems fixed by editing configuration (`no_key` , `not_installed` , `not_configured` , `unsafe_base_url`) have no wait; they are re-checked on the next scan (`:42` , `:64-65`). Success resets the counter (`:305-307`). While waiting, cached results from the last good run are served and labelled stale with their age (`:264-277` , `_cached` `:191-204`).
- **Per-platform rows.** `GET /api/sources` builds each row from the `sources` table, the collectors that cover that platform and their state (`backend/app/api/sources.py:76-122`). If a platform has never been checked in the current mode it shows "Not scanned yet. Sample data will be used." (sample) or "Not checked yet." / "No live data source is configured. ..." (live) (`:86-99`). Data written under the other data mode is not presented as current (`:85-88`). `retry_in_minutes` is derived from the earliest `next_retry_at` (`:100-102`).
- **Monitoring toggle.** A platform with `monitored = false` is left out of scans (`run_collectors` intersects the scan scope with monitored platforms, `orchestrator.py:227-230`).
- **Sources are shared by all accounts.** The `sources` and `collector_state` tables have no brand or owner column (`models/ops.py:10-46`), so one admin's toggle or retry affects every account. Retry runs the collectors for the caller's own brand (`api/sources.py:199-203`).

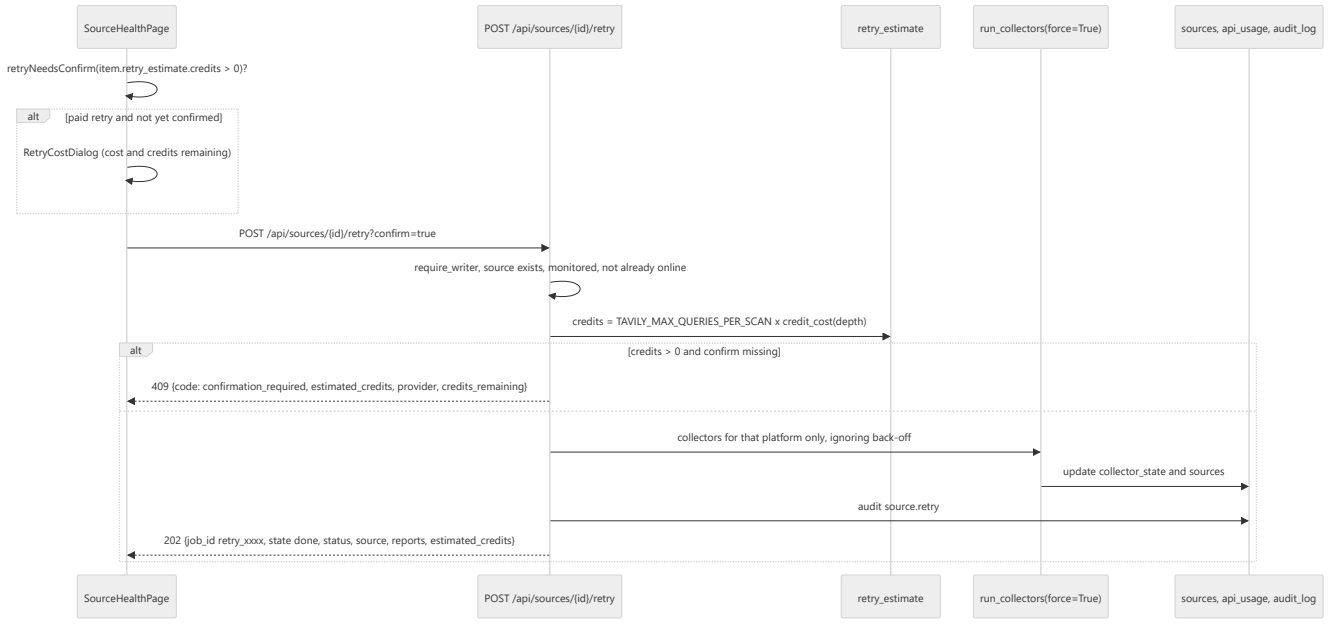
Which collectors run (data mode)

`select_collectors` (`backend/app/core/collectors/selection.py:68-114`):

Requested mode	Result
`sample`	chain is only the `sample` collector: fictional candidates, "never real results" (`:88-89` , `:99-100`)
`live`	chain `tavily` , `searxng` , `itunes` , `google_play` ; entries without a key or package show "unavailable (no_key / not_installed)" instead of falling back to sample (`:102-114`)
`auto` (default)	live when a Tavily key (`TAVILY_API_KEY`), SearXNG URL (`SEARXNG_BASE_URL`) or the `google-play-scraper` package is present; otherwise sample. iTunes needs no key, so on its own it does not switch auto to live (`:77-79` , `:92-97`)

The stored mode is the `data_mode` row of `settings` (default `auto` , `backend/app/services/data_mode.py:9-16`). `GET /api/settings/data-mode` (any signed-in user) returns `{data_mode, effective_mode, reason, chain[]}` ; `PUT /api/settings/data-mode` is admin only, writes the setting and an audit row `settings.data_mode_changed` with `old -> new` (`api/sources.py:233-249` , `data_mode.py:19-31`). The shared demo administrator may only choose `sample` (`api/sources.py:246-247` , see `docs/features/authentication-and-accounts.md`). Changing mode affects every account because the setting is global.

Retry with credit confirmation



- Endpoint `backend/app/api/sources.py:179-230` . Preconditions, each 409: the source is not monitored ("Turn monitoring on first."), the source is already online with a last sync ("This source is online. Nothing to retry."), the caller has no brand ("Create the brand profile before retrying a source."), no collector is active for that platform in the current mode.
- Only Tavily spends credits: the estimate is `TAVILY_MAX_QUERIES_PER_SCAN * credit_cost(TAVILY_SEARCH_DEPTH)` (12 x 1 for `basic` , 12 x 2 for `advanced`) and is an upper bound (`retry_estimate` , `:44-53` ; `credit_cost` `backend/app/core/collectors/credits.py:25-29`). iTunes, SearXNG, Google Play and sample retries are free.
- The retry runs synchronously and answers `state: "done"` ; the UI would poll for up to 10 attempts every 2 seconds if the server ever answered something else (`SourceHealthPage.tsx:97-109` , `:124`).
- The audit detail lists `collector=status` pairs and, if credits were involved, "up to N credits, confirmed" (`api/sources.py:224-225`).

Tavily credit counter

`CreditLedger` (`backend/app/core/collectors/credits.py:36-124`) keeps a monthly counter in `api_usage` (provider `tavily` , month `YYYY-MM`) plus a second row `tavily:reported` for credits the API says it charged. The meter uses the larger of the two (`:57-69`). `reserve(cost)` is one atomic `UPDATE ... WHERE credits + cost <= budget` , so concurrent scans cannot overshoot; `refund` returns credits when a call provably did not run (`:83-109`). Budget is `TAVILY_MONTHLY_BUDGET` (default 900, the

free plan is 1000) (`config.py:92`). `GET /api/sources` returns `tavily_credits` as `{provider, month, used, budget, remaining, reported_used}` (`api/sources.py:154`). The ledger caches its snapshot for 3 seconds when the response cache is on (`selection.py:137`). The scheduler stretches or pauses scans when credits run low (`backend/app/services/scheduler.py:101-144`).

Monitoring toggles

`PATCH /api/sources/{source_id}` with `{monitored: bool}` (`api/sources.py:160-176`): role `admin` or `analyst` (`require_writer`), 404 "Source not found", writes `source.monitor_toggled` only when the value changed. The same toggles can be saved through `PUT /api/settings` (`monitoring.sources`), which accepts only known platform slugs and is admin only (`backend/app/api/settings.py:31-39` , `:160-166`). Default platforms are created on first read by `ensure_sources` (`orchestrator.py:149-167` , `SOURCE_DEFAULTS` `:44-53`).

Provider selection (`backend/app/api/providers.py`)

Endpoint	Role	Behaviour
`GET /api/providers`	any signed-in user	`active_text_provider` , `hosted_text_analysis_enabled` (the `SCAM_LLM_ENABLED` flag), the three text providers (`local` , `gemini` , `huggingface`) with `label, configured, model, needs_key, reason, active` , embeddings status, `image_similarity_backend` ; never a key (`:27-47`)
`PUT /api/providers/active`	admin only; refused for the shared demo admin	body `{provider: "off"

- The active choice is the stored value if valid, otherwise `SCAM_LLM_PROVIDER` (default `gemini`) (`backend/app/core/pipeline/text_analysis.py:32-35`). Hosted analysis (Gemini or Hugging Face) only runs when `SCAM_LLM_ENABLED=true` (default false, so no candidate text leaves the server) (`config.py:66-69` , note in `providers.py:41`). Keys come only from `GEMINI_API_KEY` and `HUGGINGFACE_API_TOKEN` ; the Hugging Face model has no default (`config.py:71-75`). The `local` provider is a deterministic baseline (RapidFuzz and phrase rules) and needs no key (`backend/app/core/providers/text/local.py:18-23`).
- Embeddings stay off unless `TEXT_EMBEDDINGS_ENABLED=true` with a model id and key (`backend/app/core/providers/text/factory.py:34-39`).
- The provider setting is global, not per account.

Google Safe Browsing status

`GET /api/sources` includes `reputation = SafeBrowsingClient.status()` : `{provider, api_version, configured, note, key_env: "GOOGLE_SAFE_BROWSING_API_KEY", quota, last, blocked_until}` ; it contains only whether a key exists (`backend/app/core/providers/safe_browsing.py:116-124` , `backend/app/api/sources.py:156`). `quota` comes from a daily ledger in `api_usage` (provider `safe_browsing:YYYY-MM-DD` , budget `SAFE_BROWSING_DAILY_BUDGET` , default 5000, `safe_browsing.py:50-56`). The note states that URLs are sent to Google and never opened by the app, and that a clean result is not a safety guarantee. Without a key every lookup is `unknown` . The only place the UI shows this is one line on the Settings account tab: "Google Safe Browsing: configured: yes/no" (`SettingsPage.tsx:486-488`); the Source Health page does not render it.

APIs, tables, files

Endpoint	Method	Role
`/api/sources?kind=social|store`	GET	any signed-in user
`/api/sources/{id}`	PATCH	`admin` , `analyst`
`/api/sources/{id}/retry?confirm=`	POST	`admin` , `analyst`
`/api/settings/data-mode`	GET (any user), PUT (admin)	
`/api/providers` , `/api/providers/active`	GET (any user), PUT (admin)	

Tables: `sources` , `collector_state` (status, reason, back-off, last good cache), `api_usage` (Tavily and Safe Browsing counters), `settings` (`data_mode` , `text_provider`), `audit_log` (`source.monitor_toggled` , `source.retry` , `settings.data_mode_changed` , `settings.provider_changed`). External services called: Tavily search (key `TAVILY_API_KEY`), optional SearXNG instance (`SEARXNG_BASE_URL`), iTunes Search (no key, about 20 calls per minute, cached), optional `google-play-scraper` , Google Safe Browsing Lookup API (key optional), and the text providers above.

Files: `backend/app/api/sources.py` , `backend/app/api/providers.py` , `backend/app/core/collectors/` (`orchestrator.py` , `selection.py` , `credits.py` , `tavily.py` , `searxng.py` , `itunes.py` , `google_play.py` , `sample.py`), `backend/app/services/data_mode.py` , `backend/app/services/scheduler.py` , `backend/app/core/providers/` , `frontend/src/pages/SourceHealthPage.tsx` , `frontend/src/components/common/ConfirmDialog.tsx` (`retryNeedsConfirm` , `isConfirmationRequired` , `RetryCostDialog`).

Validation, permissions and edge cases

- The page and the data-mode and provider controls are admin only in the UI. On the server the writes are split: data mode and provider are `require_admin` ; monitoring toggle and retry are `require_writer` , so an analyst could call them directly even though the page is hidden. **Needs confirmation:** confirm that analysts being able to toggle monitoring and trigger retries through the API is intended, given the page is admin-only.
- Auto-retry is driven by scans and the scheduler, not by a timer of its own: the next attempt happens at the next scan after `next_retry_at` passes.
- Back-off and cache are per collector, so a failing Tavily key also shows every platform it covers as unavailable.
- Source Health is global across accounts (see above); credits are global too.
- Web look-alike candidates have no Source Health row; they ride along with a full scan (`orchestrator.py:233-234`).

Related configuration (names only)

`TAVILY_API_KEY`, `TAVILY_MONTHLY_BUDGET`, `TAVILY_MAX_QUERIES_PER_SCAN`, `TAVILY_SEARCH_DEPTH`, `TAVILY_MAX_RESULTS`, `SEARXNG_BASE_URL`, `SEARXNG_ALLOW_PRIVATE_HOSTS`, `SEARXNG_MAX_QUERIES_PER_SCAN`, `ITUNES_COUNTRY`, `ITUNES_CALLS_PER_MINUTE`, `ITUNES_CACHE_TTL_SECONDS`, `ITUNES_MAX_QUERIES_PER_SCAN`, `GOOGLE_PLAY_COUNTRY`, `GOOGLE_PLAY_MAX_QUERIES_PER_SCAN`, `SOURCE_SLOW_THRESHOLD_MS`, `COLLECTOR_TIMEOUT_SECONDS`, `REACHABILITY_CHECKS_PER_SCAN`, `SCAM_LLM_ENABLED`, `SCAM_LLM_PROVIDER`, `GEMINI_API_KEY`, `GEMINI_MODEL`, `HUGGINGFACE_API_TOKEN`, `HUGGINGFACE_MODEL`, `TEXT_EMBEDDINGS_ENABLED`, `GEMINI_EMBEDDING_MODEL`, `TEXT_PROVIDER_TIMEOUT_SECONDS`, `TEXT_PROVIDER_MAX_RETRIES`, `GOOGLE_SAFE_BROWSING_API_KEY`, `SAFE_BROWSING_TIMEOUT_SECONDS`, `SAFE_BROWSING_DAILY_BUDGET`, `SAFE_BROWSING_MAX_URLS_PER_SCAN`, `SAFE_BROWSING_CACHE_TTL_SECONDS`, `IMAGE_SIMILARITY_BACKEND`, `SCAN_INTERVAL_MINUTES`, `SCHEDULER_ENABLED` (all in `backend/app/config/config.py:62-136`; names in `backend/.env.example`).

DOCS/FEATURES/MODEL-EVALUATION.MD

Model evaluation

Plain-language summary

The Evaluation page answers "how good is the look-alike name matcher?". It does this in two ways. First, it runs the matcher over a large set of made-up fake names (for five fictional brands) and counts how many it catches and how many innocent names it wrongly flags; this gives precision, recall and F1 for each trick (for example swapping letters for digits). Second, it shows real results from the people using the product: how many detections reviewers confirmed versus dismissed. The made-up numbers are deliberately labelled optimistic because the test names are made with the same tricks the matcher was built to catch. They show the matcher is consistent, not how it performs on real scammers. The page is administrator-only in the interface.

Sample versus real: the "synthetic" block is entirely generated data (never real impersonation reports). The "reviewer decisions" block is real, from this account's own brand and the current data mode.

Purpose and user-facing behaviour

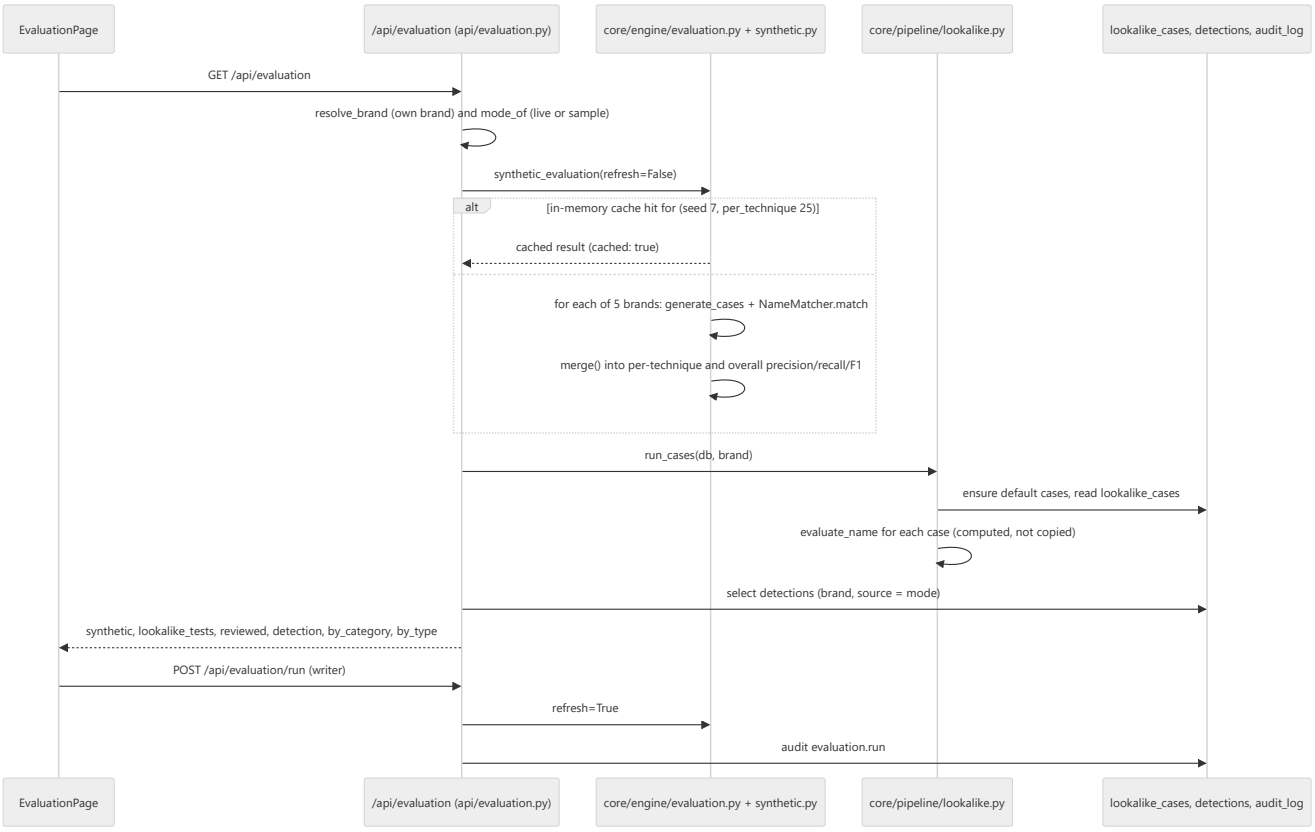
Page `/evaluation` (`frontend/src/pages/EvaluationPage.tsx:16`), wrapped in `RequireAdmin` (`frontend/src/App.tsx:174-179`), menu item for admins only (`frontend/src/components/layout/Sidebar.tsx:143-146`), and also linked from Settings, Account tab, "Open the evaluation" (`frontend/src/pages/SettingsPage.tsx:579-585`).

Sections, top to bottom (`EvaluationPage.tsx`):

1. Dataset banner: "SYNTHETIC DATA: generated, not real impersonation reports." plus dataset name, version, case count, algorithm versions and the split note (`:60-66`).
2. "Read this first" caveat from the server (`LABEL`) (`:67-69`).
3. Four headline tiles: synthetic precision, recall, F1, and reviewer precision (`:72-95`).
4. Confusion matrix and "Hard negatives by group" table (`:97-129`).
5. "Per class: reviewer decisions by category" (`:131-145`).
6. "By technique (synthetic names)" table: precision, recall, F1, found, missed, false alarms, with an Overall row (`:147-191`).
7. "Labelled look-alike test names": the brand's own 12 test cases with expected versus actual verdict (`:193-228`).
8. "Real results: reviewer decisions": confirmed, dismissed, marked official, pending (`:230-241`).

"Re-run evaluation" (`:44-52`) calls `POST /api/evaluation/run` and is disabled for roles that cannot write.

Step-by-step flow



How the synthetic evaluation set is built

- Brands: `BRANDS = ["Acme Bank", "Zenith Pay", "Orbit Telecom", "Zoho", "Northern Lights Mutual Insurance"]` (`backend/app/core/engine/evaluation.py:14`). `Zoho` is a real company name used only as a test string; all five are used as plain text.
- Generation: `generate_cases(brand, seed=7, per_technique=25, other_brands=...)` (`backend/app/core/engine/synthetic.py:235-286`). It is deterministic: the random generator is seeded with `"{seed}:{brand}"` (`:237`).
  - Positives (should be flagged): up to 25 distinct names for each of the 7 techniques plus a `mixed` generator, so up to 200 per brand. Generators: character swap, digit or symbol swap, added word, spacing or separator, transposed letters, missing or extra letter, homoglyph, and mixed (`:63-218` , registry `:223-232` ; techniques list `backend/app/core/engine/name_match.py:58-66`).
  - Negatives (should not be flagged): hard negatives (other businesses sharing the brand's first or last word, benign suffixes such as the `BENIGN_SUFFIXES` list, words like "River Bank"), fan pages (" Fans", "Friends of "), authorized resellers (" Authorized Reseller"), and unrelated names including the other four brands (`:251-285` ; groups `hard_negative` , `fan_page` , `authorized_reseller` , `unrelated`).
  - There is no train/test split: nothing is trained, every case is a test case (`api/evaluation.py:49`).
- Scoring: `evaluate_brand` runs `NameMatcher(brand).match(text)` for each case (`core/engine/evaluation.py:24-62`). A positive counts as found when the verdict is `flagged` ; its label counts as correct (`label_ok`) when the technique that fired matches the generator's technique (for `mixed` , at least two techniques). A negative that is flagged is a false positive. `_prf` turns tp/fp/fn into precision, recall and F1 (`:17-21`). `merge` sums the five brands (`:86-96`), and `confusion` builds the binary matrix, false-positive and false-negative rates, accuracy and per-negative-group rates (`:65-83`).
- Important detail in the code: in `evaluate_brand` every technique row uses the total false-positive count of the brand's negatives (`_prf(tp[t], len(fp_neg), fn[t])` , `:53`), not false positives attributed to that technique, because negatives do not belong to a technique. So per-technique precision values are all computed against the same pool of false alarms; read them as "precision if this were the only technique".
- The synthetic evaluation uses a bare `NameMatcher(brand)` ; it does not use the account's Settings (impersonation words, rules, sensitivity) and does not touch the database (`evaluation.py:27`).

How it is cached

`synthetic_evaluation` stores its result in a module-level dict `_cache` keyed by `(seed, per_technique)` guarded by a lock (`backend/app/api/evaluation.py:30-56`). The inputs are fixed, so the result is stable. `GET /api/evaluation` always passes `refresh=False` (`:94-98`); `POST /api/evaluation/run` passes `refresh=True` and replaces the cached entry (`:101-108`). The cache is per server process, never expires by time, and is shared by all accounts because it does not depend on a brand. The response says `cached: true/false` and the page shows "(cached for this server process)" (`EvaluationPage.tsx:152`). The generic response cache (`RESPONSE_CACHE_SECONDS`) does not include `/api/evaluation` (`backend/app/config/response_cache.py:15-18`). With several server processes each keeps its own copy.

The labelled look-alike test names and reviewer numbers

- `lookalike.run_cases` seeds 12 labelled cases for the brand on first use (7 look-alikes expected `flagged` , 3 other businesses `not_flagged` , 2 `official`) built from the brand's own name by `default_cases` , then runs every case through `evaluate_name` , which uses the real matcher, the brand allowlist and the Settings impersonation words (`backend/app/core/pipeline/lookalike.py:27-70` , `:78-98` (`ensure_default_cases`), `:114-122` , `:155-168`). The result is computed, not copied from the expected value. `confusion` over these cases gives precision and recall for `flagged` (`:171-183`). Admins can add or delete cases in the Look-alike Lab (`backend/app/api/lookalike.py:134` , `:148`); built-in cases are rebuilt when the brand name changes (`:186-190`).
- Reviewer numbers come from `detections` for this brand and the current data mode: `confirmed` , `dismissed` , `marked_official` , pending (`new` or `reviewing`). Precision is `confirmed / (confirmed + dismissed)` , or null when none are reviewed. The API states that recall cannot be measured without labelled ground truth (`api/evaluation.py:59-91`). `by_category` and `by_type` break this down; the page renders `by_category` only (`EvaluationPage.tsx:131-145`), `by_type` is returned by the API but not displayed.

Files, functions, classes

Item	Path
Endpoints, cache, `build` , `LABEL` , dataset constants (`DATASET_NAME = "scammind-synthetic-names"` , `DATASET_VERSION = "2"`)	`backend/app/api/evaluation.py:22-108`
`BRANDS` , `evaluate_brand` , `merge` , `confusion` , `_prf`	`backend/app/core/engine/evaluation.py`
Synthetic generators, `Case` , `MIXED` , `generate_cases`	`backend/app/core/engine/synthetic.py`
`NameMatcher` , `TECHNIQUES`	`backend/app/core/engine/name_match.py`
Labelled cases	`backend/app/core/pipeline/lookalike.py` , model `LookalikeCase` in `backend/app/models/ops.py:144`
Algorithm version constants	`backend/app/core/engine/risk_engine.py` (`ALGORITHM_V1` , `ALGORITHM_V2`), returned as `algorithm_version`
Page	`frontend/src/pages/EvaluationPage.tsx` ; `sm.evaluation` , `sm.runEvaluation` in `frontend/src/api/sm.ts`

APIs, tables, external services

Endpoint	Method	Role
`/api/evaluation?brand_id=`	GET	any signed-in user (the page itself is admin-only in the UI)
`/api/evaluation/run?brand_id=`	POST	`admin` , `analyst` (`require_writer`)

Tables: `lookalike_cases` (read, and written on first use to seed the defaults), `detections` (read), `audit_log` (`evaluation.run` with the look-alike summary, for example "9 of 12 pass"). External services: none. The ML classifier is explicitly deferred: `"ml_classifier": "deferred until labelled analyst data exists"` (`api/evaluation.py:52`).

Validation, permissions, edge cases

- Tenant isolation: the brand is always `resolve_brand(db, brand_id, user)`, so only the caller's own brand data appears; a foreign `brand_id` gives 404 (`api/common.py:22-27`, `:43-50`). The synthetic numbers are the same for every account.
- No brand yet: `resolve_brand` raises 404 "No brand profile yet. Create one first." and the page shows the error box (`EvaluationPage.tsx:56`).
- A plain `GET /api/evaluation` has a side effect: it seeds the 12 default look-alike cases for the brand if none exist (`lookalike.py:78-98`, called by `run_cases` at `:157`).
- Sample data: `data_mode` and `is_sample` are returned; in sample mode the reviewer numbers are over sample detections (`api/evaluation.py:81-84`).
- A failing labelled case is information, not an error (`lookalike.py:31`).
- **Needs confirmation:** the API lets any signed-in role read `/api/evaluation` and lets analysts re-run it, while the UI shows the page to admins only. Confirm that the API access level is intended.

Related configuration (names only)

No environment variable controls the evaluation itself. Indirectly: `SEVERITY_MEDIUM_MIN`, `SEVERITY_HIGH_MIN`, `SEVERITY_CRITICAL_MIN` (risk engine cut-offs), the impersonation words and rules saved in Settings (used by the labelled cases only), and `RESPONSE_CACHE_SECONDS` (does not cover this endpoint).

DOCS/FEATURES/SETTINGS-AND-AUDIT.MD

Settings and audit log

Plain-language summary

The Settings page is where an administrator decides how BrandShield behaves: how often it scans, which platforms it watches, how strict it is about what counts as suspicious, which extra words signal impersonation (such as "support" or "refund"), and where alerts should go. It also holds the choice between real data, sample data or automatic, and the choice of AI text provider. Every change an administrator saves is written to an audit log (who, what, when). Settings are shared by the whole installation, not per account. The page is administrator-only in the interface; alert preferences are stored but nothing is actually sent yet. The shared "demo administrator" account can change harmless settings but is blocked from anything that would affect other people (live data, AI provider, webhook, other accounts' audit rows).

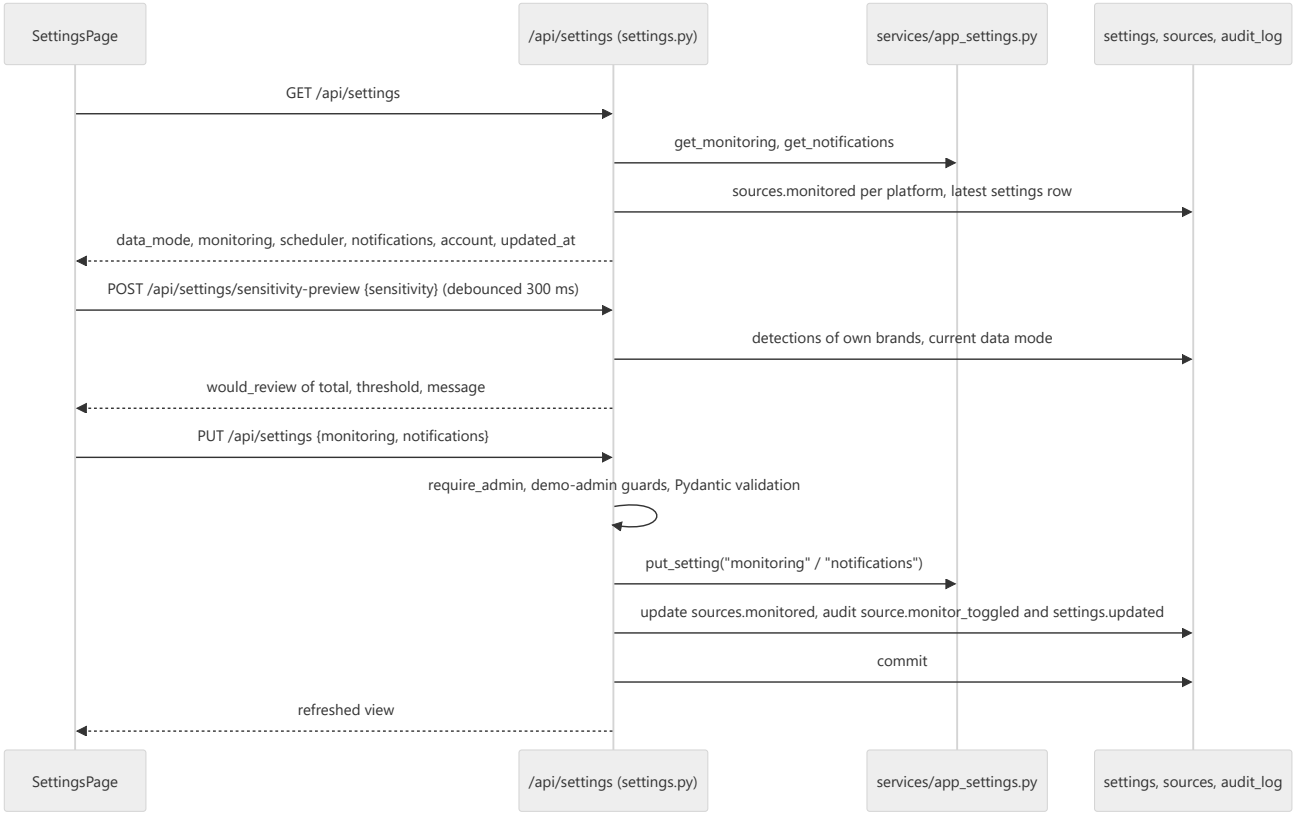
Purpose and user-facing behaviour

Page `/settings` (`frontend/src/pages/SettingsPage.tsx:64`), inside `RequireAdmin` (`frontend/src/App.tsx:174-179`) and shown in the menu only to admins (`frontend/src/components/layout/Sidebar.tsx:33` , `:171`). The page reads `GET /api/settings` , keeps a local draft, and saves with `PUT /api/settings` (`SettingsPage.tsx:66` , `:123-144`). The active tab is remembered per browser (`usePref('settingsTab', ...)` , `:67`). Three tabs (`:14-15`):

Tab	Contents	Code
Monitoring	Scan schedule (5 minutes, 15 minutes, hourly, daily), social platforms and app stores checkboxes, detection sensitivity slider (50 to 95) with a live "N of M candidates would be sent to review" preview, five look-alike rule checkboxes, impersonation words editor, link to the Look-alike Lab	`:204-363`
Notifications	email alerts, weekly digest, webhook switch and URL. A banner states "Preferences are saved, but nothing is delivered yet"	`:365-422`
Account	signed-in user line, "Export Audit Log" button (CSV), Data mode switch, Analysis providers panel with the Safe Browsing line, link to Evaluation	`:424` , `:500-588`

A bar at the bottom shows "You have unsaved changes", Cancel and "Save settings"; non-admins would see "Only administrators can change settings" and disabled controls (`:146-174` ; non-admins cannot reach the page anyway).

Step-by-step flow



1. **Read.** `GET /api/settings` (any signed-in user) returns `data_mode` and `effective_data_mode` , the `monitoring` block (`schedule` , per-platform `sources` , `sensitivity` , derived `threshold` , `rules` , `impersonation_words`), a `scheduler` block, `notifications` (with `delivery: "not_implemented"`), the `account` block (organization = the active brand's name, and the user) and `updated_at` (`backend/app/api/settings.py:96-127`). The webhook URL is returned only to a real admin; the demo admin and all others receive an empty string plus `webhook_configured` (`:118`).
2. **Preview.** `POST /api/settings/sensitivity-preview` (any signed-in user, read-only) counts how many of this account's stored candidates (current data mode, excluding `marked_official`) would be sent to review at a given sensitivity using the same `is_hit` rule as scans, and returns a plain-language message: "Strict"

below 60, "A balanced setting" up to 80, "Sensitive" above (`:186-207` ; `is_hit` in `backend/app/core/pipeline/matching.py:244-250`).

3. **Save.** `PUT /api/settings` (`require_admin` , `:130-183`). Only changed values are written. Monitoring changes are saved under the `monitoring` key of the `settings` table; per-platform toggles update `sources.monitored` and each writes `source.monitor_toggled` ; notification preferences are saved under `notifications` . If anything changed, one audit row `settings.updated` with `changed: <sorted list>` (for example `monitoring.sensitivity, monitoring.rules.added_words`) is written (`:178-179`). A data-mode change in the same request is applied after the commit and audited as `settings.data_mode_changed` (`:181-182`).
4. **Effect.** Saved values apply from the next scan: the scan environment reads sensitivity, rules and words at the start of every scan (`backend/app/core/pipeline/scan_pipeline.py:290-302`), the Look-alike Lab tester reads the words (`backend/app/core/pipeline/lookalike.py:116`), and the scheduler reads the schedule (`backend/app/services/scheduler.py:101-121`).

Settings fields and rules

Field	Allowed values and validation	Default	Code
`monitoring.schedule`	`off` , `5m` , `15m` , `1h` , `1d` (the UI offers all but `off` , which is shown only when already set)	`1d`	`api/settings.py:25` ; `services/app_settings.py:15-24` ; `SettingsPage.tsx:36` , `:223`
`monitoring.sources`	map of platform slug to bool; unknown slug gives 422 "Unknown platform"	all monitored	`api/settings.py:31-39`
`monitoring.sensitivity`	integer 50 to 95; the minimum score to review is `100 - sensitivity` (75 gives 25, 50 gives 50, 95 gives 5)	75	`app_settings.py:17` , `:74-76` ; `api/settings.py:27`
`monitoring.rules`	five booleans: `character_swaps` , `digits_and_symbols` , `added_words` , `spacing_and_separators` , `transposed_or_missing` ; unknown rule gives 422	all true	`backend/app/core/pipeline/matching.py:35-41` ; `api/settings.py:41-49`
`monitoring.impersonation_words`	lower-cased, whitespace-collapsed, de-duplicated, each at most 30 characters, at most 50 words	`support, helpline, official, refund, rewards, kyc`	`api/settings.py:51-67` ; `matching.py:34`
`notifications.email_alerts` , `high_risk_immediate` , `weekly_digest`	booleans	false	`app_settings.py:25-30`
`notifications.webhook_url`	empty or must start with `https://` and contain no spaces; max 500 characters	empty	`api/settings.py:70-83`
`data_mode`	`live` , `sample` , `auto`	`auto`	`api/settings.py:87` ; `services/data_mode.py:9-11`

Stored values are merged over defaults, so unknown stored keys are ignored and a missing key falls back to its default (`app_settings.py:37-56`). The `SCAN_INTERVAL_MINUTES` environment value, when set, overrides the Settings schedule (0 means off) (`app_settings.py:79-83` ; `scheduler.py:101-121`). **Notifications are stored preferences only.** `delivery: "not_implemented"` is hard-coded in the response (`api/settings.py:117-119`); no code reads `webhook_url` to send anything (searched `backend/app` ; the only other hits are in `services/voice.py` , unrelated).

Settings are global. The `settings` table is a key/value table with no account or brand column (`backend/app/models/ops.py:82-91`), and `GET/PUT /api/settings` do not scope by account. One admin's sensitivity, rules, words and data mode apply to every account's scans. Only the `account` block and the sensitivity preview are per account.

Account tab: data mode and providers

- **Data mode.** `GET /api/settings/data-mode` (any user) and `PUT /api/settings/data-mode` (admin) switch between `live` , `sample` and `auto` , and the tab shows the effective mode, the reason and the collector chain (`SettingsPage.tsx:494-577` ; `backend/app/api/sources.py:233-249`). Details of what each mode does are in `docs/features/source-health-and-providers.md` .
- **Providers.** `ProvidersPanel` shows `GET /api/providers` and lets an admin choose `off` , `local` , `gemini` or `huggingface` with `PUT /api/providers/active` ; keys are never displayed (`SettingsPage.tsx:434-491` ; `backend/app/api/providers.py:45-57`). It also shows "Google Safe Browsing: configured: yes/no" taken from `GET /api/sources` (`:486-488`).

Audit log

What is recorded

`audit.record(db, actor, action, target, detail, actor_label=None, source="live")` adds an `AuditLog` row to the caller's own transaction, so it commits or rolls back together with the action (`backend/app/services/audit.py:6-26`). Columns: `ts` , `actor_id` (no foreign key, so the log outlives users), `actor` (name), `action` , `target` (cut at 300 characters), `detail` , `source` (`live` or `sample`) (`backend/app/models/ops.py:129-141`). The table is documented as append-only; there is no delete or edit endpoint.

Actions written by the code (searched `audit.record` in `backend/app`): `auth.login` , `auth.login_failed` , `auth.logout` , `auth.registered` ; `brand.created` , `brand.updated` , `brand.archived` , `brand.restored` ; `logo.added` , `logo.removed` , `logo.replaced` ; `account.added` , `account.removed` ; `app.added` , `app.updated` , `app.removed` ; `publisher.added` , `publisher.removed` ; `asset.verification_changed` ; `candidates.added` , `candidates.imported` , `candidate_image.uploaded` ; `scan.started` , `scan.finished` ; `detection.status_changed` , `detection.resolution_set` , `detection.marked_official` ; `report.generated` , `report.approved` ; `settings.updated` , `settings.data_mode_changed` , `settings.provider_changed` ; `source.monitor_toggled` , `source.retry` ; `lookalike.case_added` , `lookalike.case_removed` , `lookalike.cases_run` ; `evaluation.run` ; `health.canary` .

Reading it

Endpoint	Role	Behaviour
`GET /api/audit?from=&to=&actor=&action=&limit=&offset=`	`admin`, `analyst` (viewers get 403)	newest first, `limit` 1 to 200 (default 50), returns `{total, items[]}`
`GET /api/audit/export.csv`	`admin`, `analyst`	CSV `ts, actor, action, target, detail`; cells starting with `=`, `+`, `-`, `@`, tab or CR get a leading apostrophe (formula-injection guard)

(`backend/app/api/audit.py:52-84`, `_safe_cell` `:70-72`.)

- **Who sees which rows:** `_filtered` shows every row to a real admin; every other caller (analyst) sees only rows whose `actor_id` is their own, and the shared demo administrator is treated the same way (`own_only`) so it cannot read other accounts' activity (`audit.py:36-40`, `:64`, `:80`). The log has no brand column, so a real admin's view mixes all accounts' brand names and handles.
- **UI:** there is no audit-log table page in the frontend. The only UI is the "Export Audit Log" button on the Settings Account tab, which downloads the CSV (`SettingsPage.tsx:521-541`; searched `frontend/src` for `/api/audit`). The button is enabled for analysts and admins; since the page is admin-only in the router it is reachable only by admins. `GET /api/audit` (JSON) is not called by any page.

Demo-administrator restrictions (summary)

The shared demo administrator (`DEMO_ADMIN_EMAIL`) passes `require_admin` but `forbid_demo_admin` (`backend/app/services/seed.py:113-116`) returns 403 "Disabled for the shared demo administrator: ..." for: choosing `live` or `auto` data mode (via `PUT /api/settings` or `PUT /api/settings/data-mode`, `api/settings.py:136-137`, `api/sources.py:246-247`), changing the AI provider (`api/providers.py:53`), and setting a non-empty webhook URL (`api/settings.py:138-139`). An empty webhook value from its form is skipped so it cannot clear the real URL (`:171-172`), the stored URL is hidden from it (`:118`), and its audit view is limited to its own actions (`audit.py:64`, `:80`). Everything else on the page works for it. Full description: `docs/features/authentication-and-accounts.md`.

Files, tables, services

Item	Path
Settings routes and schemas (`MonitoringIn`, `NotificationsIn`, `SettingsIn`, `PreviewBody`)	`backend/app/api/settings.py`
Defaults, merge, `put_setting`, `threshold_for`, `interval_minutes`	`backend/app/services/app_settings.py`
Per-request settings memo (reads the whole `settings` table once)	`backend/app/services/setting_rows.py`, `backend/app/db/request_cache.py:17-21`
Data mode	`backend/app/services/data_mode.py`
Audit routes and helper	`backend/app/api/audit.py`, `backend/app/services/audit.py`
Models `Setting`, `AuditLog`	`backend/app/models/ops.py:82-91`, `:129-141`
Page	`frontend/src/pages/SettingsPage.tsx`, API calls in `frontend/src/api/sm.ts` (`settings`, `saveSettings`, `sensitivityPreview`, `dataMode`, `setDataMode`, `providers`, `setProvider`)

Tables touched: `settings` (keys `monitoring`, `notifications`, `data_mode`, `text_provider`), `sources` (`monitored`), `audit_log`, `detections` (read for the preview), `brands` (read). External services: none from this page itself. `GET /api/settings` and `/api/providers` are in the short-lived response cache; any write request clears it (`backend/app/config/response_cache.py:15-18`, `backend/app/main.py:137-158`).

Permissions, errors and edge cases

Situation	Result
Non-admin calls `PUT /api/settings`	403 "You do not have permission to do this" (`require_admin`)
Unknown platform or rule, sensitivity outside 50 to 95, word over 30 characters or more than 50 words, webhook not `https://`	422 with a plain message
Demo admin tries a guarded change	403 "Disabled for the shared demo administrator: ..."
Save with no actual change	no `settings.updated` row, response is the current view
Two admins editing at once	last write wins; the page does not detect concurrent edits (`SettingsPage.tsx:76-83` only loads once)

`GET /api/settings` is open to any signed-in user (`api/settings.py:125-127`) and returns the rules and words; only the webhook is hidden. **Needs confirmation:** confirm that exposing sensitivity, rules and impersonation words to analysts and viewers through the API is intended (the page itself is admin-only).

Related configuration (names only)

`SCAN_INTERVAL_MINUTES` (overrides the schedule), `SCHEDULER_ENABLED`, `SCHEDULER_POLL_SECONDS`, `SCHEDULER_MAX_BRANDS_PER_RUN`, `SEVERITY_MEDIUM_MIN`, `SEVERITY_HIGH_MIN`, `SEVERITY_CRITICAL_MIN`, `SCAM_LLM_ENABLED`, `SCAM_LLM_PROVIDER`, `RESPONSE_CACHE_SECONDS`, `DEMO_ADMIN_EMAIL`, `DEMO_ADMIN_PASSWORD`, `ADMIN_EMAIL`, `ADMIN_PASSWORD` (`backend/app/config/config.py:45-50`, `:122-127`, `:163-169`).

DOCS/FEATURES/DASHBOARD.MD

Dashboard

Plain-language summary

The Dashboard is the first screen after sign-in. It answers "how risky is my brand right now?" with four headline numbers (total detections, social impersonations, suspicious apps, official assets protected), a donut chart of the kinds of impersonation found, a bar list of where they were found, the five cases most in need of a human decision, the six most recent detections, an explanation of the highest-scoring case, and a small health card for the data sources. A "Run scan now" button starts a new scan. Every number is calculated from stored database rows for the active brand and the current data mode; nothing is hard-coded. To keep the page fast, answers are held for a few seconds and thrown away as soon as anyone changes something.

Real versus sample: when the data mode is `sample` all numbers come from fictional sample candidates, and the page footer says "Sample data for the fictional brand ... Not real detections." (`frontend/src/components/common/States.tsx:53-61`).

Purpose and user-facing behaviour

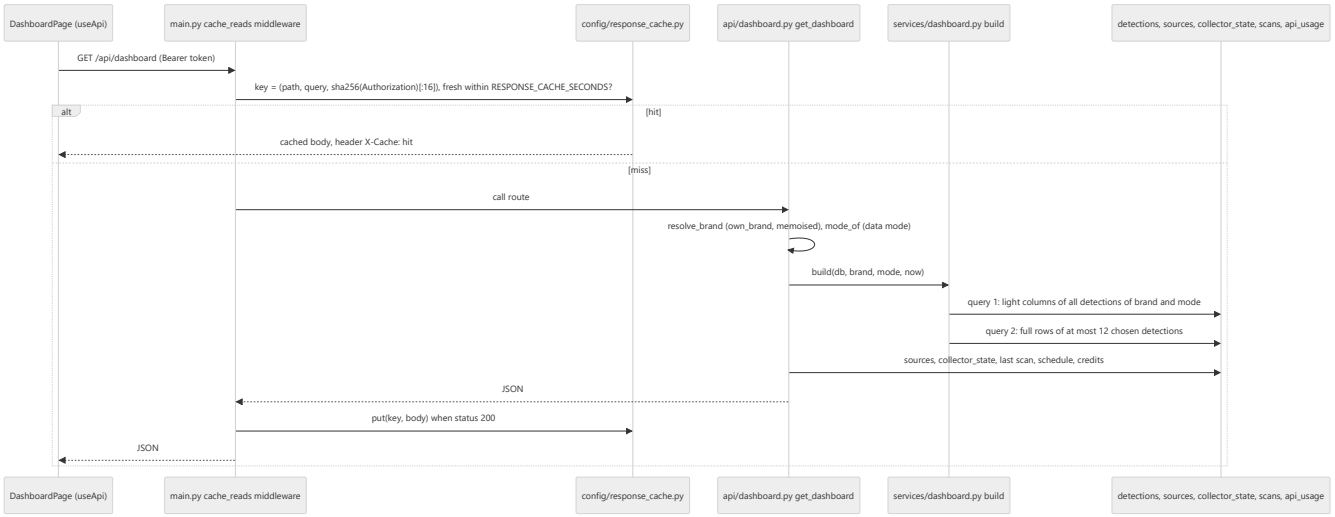
Page `/dashboard` (`frontend/src/pages/DashboardPage.tsx:65` , route `frontend/src/App.tsx:97-105`). Blocks, top to bottom:

Block	Source field in `GET /api/dashboard`	Code
Hero line: "Last scan ... · N of M sources online · Next scan HH:MM", buttons "Run scan now" and "Edit brand profile"	`last_scan_at` , `sources_online` , `sources_total` , `next_scan_at`	`DashboardPage.tsx:112-115` , `:166-201`
Four KPI tiles	`kpis.total_detections` , `social_impersonations` , `suspicious_apps` (each with `value` , `delta_week` , 7-point `trend` sparkline), `official_assets.value`	`:129-155` , `:206-227`
"Impersonation types" donut and legend	`by_type[]` (`type` , `count` , `percent`)	`:117-122` , `:231-255`
"Where we found them" bars	`by_platform[]`	`:123` , `:257-277`
"Needs review" list (max 5) with risk band chip	`needs_review[]`	`:279-308`
"Recent detections" table (max 6)	`recent[]`	`:313-360`
"Why it was flagged" card for the top detection	`top_detection` , `why_flagged`	`:364-397`
"Source health" mini card (first 6 sources), "Manage" link only for admins	`sources[]`	`:399-434`

Behaviour details:

- With no brand, the API answers 404 and the page shows `NoBrand` (`:76` , `:89-98`).
- "Run scan now" calls `useScan('all', ...)` , which starts `POST /api/scan` (analyst or admin only; viewers see the disabled button with the tooltip "Your role can view but not start scans", `:183-192`), polls `GET /api/jobs/{job_id}` every second for up to 10 minutes, then reloads the dashboard and the brand (`frontend/src/hooks/useScan.ts:10-11` , `:44-65` ; `DashboardPage.tsx:70-73`). A 409 "A scan is already running" attaches to the running job (`backend/app/api/scans.py:40-43` , comment at `useScan.ts:2`).
- Clicking a case opens `/apps?detection=<id>` for `fake_app` and `/social?detection=<id>` for everything else (`DashboardPage.tsx:79`). **Needs confirmation:** look-alike web cases (type `lookalike_domain` , category `web`) are included in `needs_review` and `recent` but are sent to `/social` , although web findings have their own page `/web` (`frontend/src/App.tsx:137-144`). Confirm whether `/social` shows them.
- The API also returns `kpis.web_threats` (`backend/app/services/dashboard.py:82`) but the page renders no tile for it.
- `top_detection.on_allowlist` is hard-coded `False` (`services/dashboard.py:114`) because allowlisted assets never appear as detections, so the card always says "It is not on your allowlist".

How the numbers are computed



`get_dashboard` (`backend/app/api/dashboard.py:19-49`): resolves the brand (`resolve_brand` , so only the caller's own brand, optional owned `brand_id`), the effective data mode, then:

1. **`dashboard.build`** (`backend/app/services/dashboard.py:46-120`), "two queries instead of ~60":
   - Query 1 selects only `id, type, category, platform, status, band, severity, score, found_at` for every detection of this brand with `source == mode` (`:49-52`). Everything below is computed in memory from those rows.
   - "Open" rows are statuses `new` , `reviewing` , `confirmed` (`OPEN_STATUSES` , `backend/app/services/detections.py:30`); "visible" rows add `dismissed` . `cleared` , `marked_official` and `retired` never count (`:53-54`).
   - KPIs: `kpi()` returns `value` (count), `delta_week` (rows with `found_at` in the last 7 days) and `trend` (for each of the last 7 days, how many open rows were found up to the end of that day, so it is cumulative) (`:35-43`). `total_detections` uses all open rows; `social_impersonations` the `social` category; `suspicious_apps` the `apps` category; `web_threats` the `web` category. `official_assets` is `len(brand.accounts) + len(brand.apps)` (`:78-84`).
   - `by_type` : counts per detection type among open rows, with whole-number percentages that always add up to 100 (`_percentages` , largest remainder, `:15-24`). The `lookalike_domain` type is listed only once something was found (`:97-99`). `by_platform` is sorted by count descending, then name (`:59`).
   - `counts` : `by_status` (visible rows), `by_band` and `by_severity` (open rows), `by_category` , and `cleared` (rows the engine kept below threshold) (`:85-96`).
   - `needs_review` : the 5 `new` or `reviewing` rows with the highest score (ties by newest id) (`:63`). `recent` : the 6 most recently found open rows (`:64`). `top_detection` : the open row with the highest score (`:65`). Only these at most 12 rows are fetched in full in query 2 (`:66-70`); `why_flagged.reasons` lists the top detection's `signal, label, detail, points` (`:116-119`).
2. **Sources** (`api/dashboard.py:33-40`): `ensure_sources` , the current collector selection and collector states feed `source_item` (the same function as Source Health, `backend/app/api/sources.py:76-122`), giving `id, slug, name, kind, status, monitored, message` , plus `sources_online` and `sources_total` . Sources are global, not per brand.
3. **Last scan and schedule** (`:42-47`): the latest scan of this brand in state `completed` , `partial` or `failed` gives `last_scan_at` and `last_scan` (`scan_view`); `schedule_status` for this brand gives `next_scan_at` and the `schedule` block (enabled, interval, whether credits stretched or paused it, "waiting for official assets") (`backend/app/services/scheduler.py:124-144`).
4. **Credits** (`:48`): `credits` is the Tavily ledger snapshot only when the effective mode is `live` , otherwise `null` .

The response also carries `brand {id, name}` , `data_mode` , `is_sample` and `includes_sample` . The frontend notes any JSON answer containing `is_sample` or `data_mode` to drive the global "sample data" flag (`frontend/src/api/sample.ts:14-33`).

Caching: two layers

1. HTTP response cache (`backend/app/config/response_cache.py` , middleware in `backend/app/main.py:137-158`)

- A short in-process cache for heavy read-only pages. `CACHEABLE` includes `/api/dashboard` , `/api/sources` , `/api/settings` , `/api/detections` (and `/excluded` , `/false-positives`), `/api/brands` , `/api/reports` , `/api/lookalike/test-cases` , `/api/scans` , `/api/scans/latest` , `/api/providers` (`response_cache.py:15-18`). The brand-scoped lists are keyed by the full query string, so a different filter is a different entry.
- Key = `(path, query string, first 16 hex characters of SHA-256 of the Authorization header)` (`:24-25`), so entries are separate per sign-in token. A hit is answered by the middleware itself with header `X-Cache: hit` , before the route and its authentication run (`main.py:143-147`). Only `200` answers are stored (`:149-153`); at most 400 entries, then the whole store is cleared (`response_cache.py:19` , `:36-40`).
- Lifetime `RESPONSE_CACHE_SECONDS` (default 15, `0` = off) (`config.py:123`). It is forced to 0 on Vercel unless set explicitly, because a per-instance cache would go stale across instances (`config.py:184`).
- **Invalidation:** any non-GET, non-HEAD, non-OPTIONS request under `/api` clears the whole cache after it runs (`main.py:155-158`), so switching brand (`POST /api/brands/{id}/select`), reviewing a detection, editing settings and so on show up at once. A scan also clears it when it is queued and when it ends (`_busts_cache` around `request_scan` and the scan finish, `backend/app/core/pipeline/scan_pipeline.py:132-145`).
- Limits: it is per process, so another worker's writes are seen after at most the TTL (`response_cache.py:6-7`); and the Tavily credit snapshot has its own 3-second cache when the response cache is on (`backend/app/core/collectors/selection.py:137`).

2. Per-request memo (`backend/app/db/request_cache.py`)

- `memo(db, key, load)` stores a value in `Session.info` for the lifetime of one request's database session (`:17-21`). The memo is dropped on every `after_flush` , `after_commit` and `after_rollback` , so it can never outlive a write (`:28-30`).
- Used so that repeated identical reads inside one request cost one query: the whole `settings` table is loaded once (`backend/app/services/setting_rows.py` , key `"settings_all"` , used by `data_mode.get_data_mode` and `app_settings`), and brand lookups are memoised per user (`backend/app/api/common.py:24` , `:40` , `:57`).

This matters because the database may be remote (every query is a network round trip; see the pool comments in `backend/app/db/session.py:18-33`).

Files, functions, tables, services

Item	Path
Endpoint	`backend/app/api/dashboard.py` (`get_dashboard`)
Aggregation	`backend/app/services/dashboard.py` (`build`, `kpi`, `_percentages`, `_count_by`)
Detection helpers (`summary_row`, `OPEN_STATUSES`, `ALL_TYPES`)	`backend/app/services/detections.py`
Sources, schedule, credits	`backend/app/api/sources.py` (`source_item`), `backend/app/services/scheduler.py` (`schedule_status`), `backend/app/core/collectors/credits.py`
Caches	`backend/app/config/response_cache.py`, `backend/app/main.py:137-158`, `backend/app/db/request_cache.py`
Page, charts	`frontend/src/pages/DashboardPage.tsx`, `frontend/src/components/charts/Donut.tsx`, `Sparkline.tsx`, `frontend/src/hooks/useScan.ts`, `useApi.ts`

Tables read: `detections`, `official_accounts`, `official_apps` (lazy-loaded through the brand), `sources`, `collector_state`, `scans`, `settings`, `api_usage` (live mode only). Nothing is written by the dashboard endpoint. External services: none called by the dashboard itself; "Run scan now" starts a scan which may call the collectors (see `docs/features/source-health-and-providers.md`).

APIs used by the page

Endpoint	Role	Purpose
`GET /api/dashboard`	any signed-in user	all numbers and lists
`POST /api/scan {scope: "all"}`	`admin`, `analyst`	start a scan; 409 when a scan is running or no brand exists
`GET /api/jobs/{job_id}`	any signed-in user (own brand only)	scan progress
`GET /api/me` and `GET /api/brands/{id}`	any signed-in user	brand refresh after a scan

Permissions, errors, edge cases

- Tenant isolation: the numbers are for `resolve_brand(...)` only; `GET /api/jobs/{id}` answers 404 "Job not found" for another account's scan (`backend/app/api/scans.py:48-53`).
- Roles: everyone can read; viewers cannot start a scan (403 from `require_writer`, button disabled). Admins additionally see the "Manage" link to Source Health (`DashboardPage.tsx:403-407`).
- Switching the active brand in the sidebar clears the browser query cache and reloads the brand, so the dashboard reloads for the new brand (`frontend/src/components/layout/BrandSwitcher.tsx:43-58`; see `docs/features/brand-registry.md`).
- After a brand identity change older detections are `retired` and disappear from every number here until a new scan finds matches (`backend/app/services/detections.py:499-505`).
- Empty states: "No detections yet.", "Nothing found yet.", "Nothing needs review.", "No flagged detection to explain yet." (`DashboardPage.tsx:244`, `:263`, `:288`, `:395`).
- If a scan failed the page shows its message in a notice (`:203-204`, `scanSummary` `:54-63`); partial scans (some source unavailable) are a normal outcome and are summarised as "Some sources were slow or unavailable; results from the others were saved."

Related configuration (names only)

`RESPONSE_CACHE_SECONDS`, `SCAN_EXECUTOR`, `SCHEDULER_ENABLED`, `SCAN_INTERVAL_MINUTES`, `TAVILY_MONTHLY_BUDGET`, `SOURCE_SLOW_THRESHOLD_MS`, `SEVERITY_MEDIUM_MIN`, `SEVERITY_HIGH_MIN`, `SEVERITY_CRITICAL_MIN`, `DB_POOL_SIZE`, `DB_MAX_OVERFLOW`, `DB_POOL_RECYCLE`, `DB_PRE_PING` (`backend/app/config/config.py:105`, `:122-124`, `:166-168`; `backend/app/db/session.py:24-27`).

DOCS/FEATURES/BRAND-STUDIO.MD

Brand Studio

Plain-language summary

Brand Studio is a second web app that sits next to BrandShield and helps a person create a brand: it suggests names, designs logos, picks colour palettes, writes taglines and posts, scores copy, audits a brand and answers branding questions. There is no separate login: if you are signed in to BrandShield you are signed in to Studio, because both apps live on the same web address and share the same saved session. Everything you like can be saved to "Collections" (kept in the BrandShield database), and when you are happy with a brand you can press "Protect in BrandShield" to register it for monitoring. The AI itself runs on the BrandShield server, never in the browser, so the provider keys stay secret and every person has a daily allowance.

1. Purpose and user-facing behavior

Area	What the user sees	Code
Entry	"Brand Studio" button in the BrandShield sidebar, and the "Create a Brand" card on `/protect-brand`. Both do a full page load of `/studio/`.	`frontend/src/components/layout/Sidebar.tsx:156`, `frontend/src/pages/ProtectBrandPage.tsx:12-14`, `frontend/src/utils/studio.ts:3-8`
Sign-in	None of its own. Signed-out visitors are sent to `/login?next=/studio/` and come back after login.	`studio/src/lib/session.ts:71-76`, `frontend/src/utils/studio.ts:15-19`, `frontend/src/pages/LoginPage.tsx:51-52`
Tools (sidebar tabs)	Dashboard, Brand Names, Logo Designer, Content Lab, Color Palette, Sentiment AI, Brand Audit, Protect in BrandShield, AI Consultant, Analytics, Collections	`studio/src/App.tsx:32` (tab type), `studio/src/App.tsx:76-91` (menu)
Extra tool	"Brand Checker" component (name availability opinion and a similar-name suggestion) exists and is imported, but is **not** in the sidebar menu list above	`studio/src/App.tsx:18`, `studio/src/components/BrandChecker.tsx:13`
Remembered inputs	Brand name, industry, audience, etc. typed once and pre-filled in every tool	`studio/src/lib/profile.ts`
Saved items	"Save" buttons store results in Collections; the dashboard counts them	`studio/src/contexts/CollectionsContext.tsx:66-67`, `studio/src/App.tsx:46-51`
Protect	A wizard that creates a **new** BrandShield brand (with optional logo)	`studio/src/components/ProtectBrand.tsx`, `studio/src/lib/brandshield.ts:226-231`

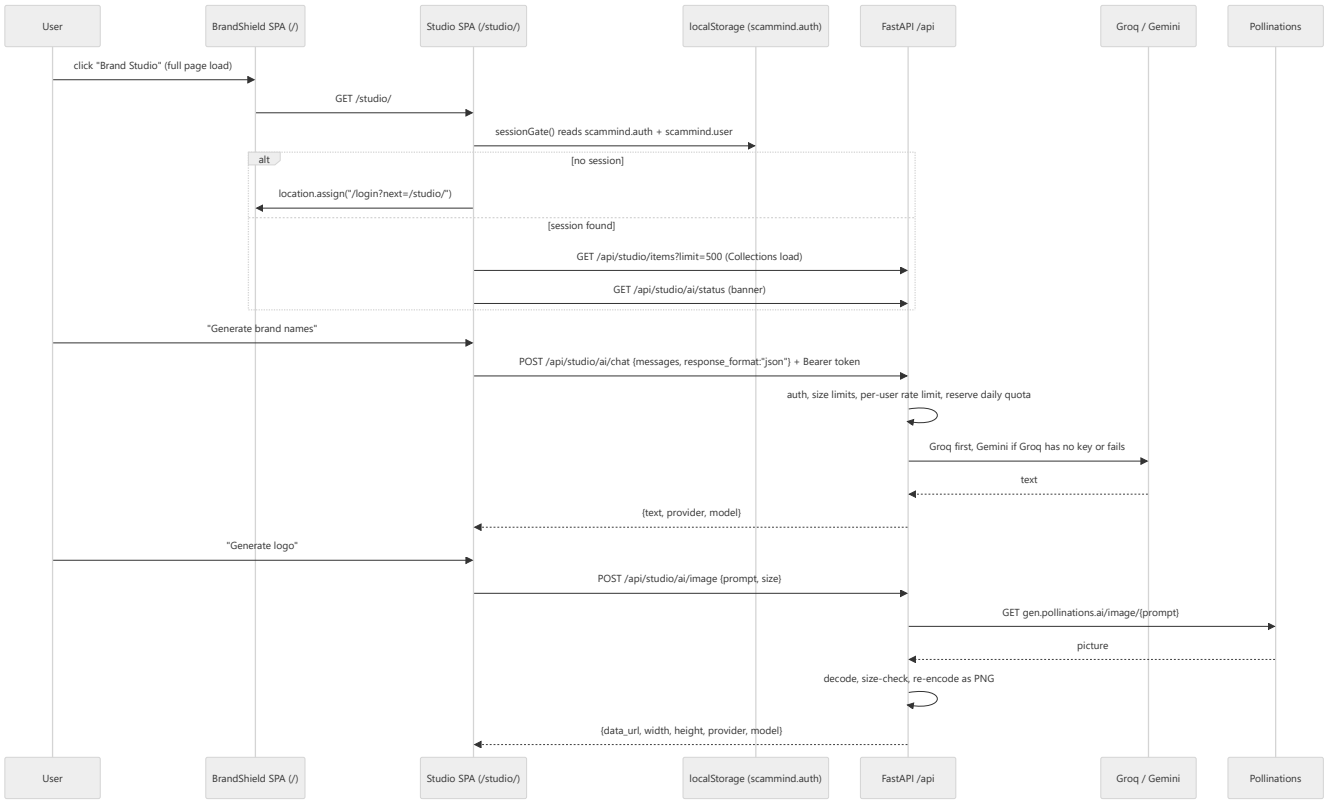
Which AI call each tool makes (all through `studio/src/services/gemini.ts`, despite the file name it calls the BrandShield proxy, see `studio/src/services/gemini.ts:3-18`):

Tool	Service function	Kind	Source
Brand Names	`generateBrandNames`, `suggestNamingContext`	chat (JSON)	`studio/src/services/gemini.ts:41,62`
Logo Designer	`generateLogo`, `suggestLogoIdea`	image + chat	`studio/src/services/gemini.ts:77,272`
Content Lab	`generateContent`	chat (JSON)	`studio/src/services/gemini.ts:138`
Sentiment	`analyzeSentiment`, `analyzeReviews` (Review Analyzer component)	chat (JSON)	`studio/src/services/gemini.ts:157,177`
Color Palette	`generateColorPalette`	chat (JSON)	`studio/src/services/gemini.ts:215`
Brand Audit	`auditBrand`	chat (JSON)	`studio/src/services/gemini.ts:241`
Brand Checker	`checkNameAvailability`, `suggestSimilarName`	chat (JSON)	`studio/src/services/gemini.ts:290,305`
AI Consultant	`createChatSession` (whole history re-sent each turn)	chat (text)	`studio/src/services/gemini.ts:22-37,322`
Analytics	local counters only, no server call	none	`studio/src/utils/analytics.ts:5-25`

The "Analytics" tool reads events from `studio/src/utils/analytics.ts`: `trackEvent` appends `{type, timestamp, label}` to `localStorage` key `bc-analytics`, keeping the last 500 (`analytics.ts:3-11`). It is per browser, not per account, and never sent to the server.

2. Step-by-step flow

2.1 Open Studio and use an AI tool



1. The sidebar link is a plain `<a href="/studio/">` (full page load, not a React route) (`frontend/src/components/layout/Sidebar.tsx:156`).
2. `AuthProvider` reads the BrandShield session with `sessionGate()` ; with no tokens or no user it calls `redirectToLogin()` (`studio/src/contexts/AuthContext.tsx:17-30` , `studio/src/lib/session.ts:79-84`). The loading spinner reads "Checking your BrandShield sign-in" (`studio/src/App.tsx:58-65`).
3. Every call goes through `request()` in `studio/src/lib/api.ts:163` , with relative URLs (same origin), `Authorization: Bearer <access>` (`studio/src/lib/api.ts:123-133`).
4. AI helpers `aiChat` , `aiImage` , `getAiStatus` post to `/api/studio/ai/chat|image|status` (`studio/src/lib/ai.ts:64,76,85`).
5. `useAiStatus()` calls status once per page load and feeds `AiStatusBanner` (`studio/src/lib/useAiStatus.ts:8-20` , `studio/src/components/AiStatusBanner.tsx`).

2.2 One shared login: session, refresh, sign-out

- Same storage keys as BrandShield: `scammind.auth` (tokens) and `scammind.user` ; legacy keys `scammind_auth` , `scammind.offline-session` are wiped on clear (`studio/src/lib/session.ts:3-5` , `frontend/src/api/client.ts:59-61`).
- Read order: `localStorage` then `sessionStorage` (so "keep me signed in" off still works in the same tab) (`studio/src/lib/session.ts:21-31`). Studio never writes a brand-new session; `save()` only keeps the session wherever it already lives (`studio/src/lib/session.ts:55-59`).
- On a 401 the client refreshes **once**. The refresh runs inside `navigator.locks.request('scammind-refresh', ...)` (Web Locks API) so BrandShield, Studio and other tabs never use the same rotating refresh token twice. Inside the lock the stored tokens are re-read: if they differ from the token that failed, another tab already rotated them and the request is just retried (`studio/src/lib/api.ts:7,73-106` ; same lock name in `frontend/src/api/client.ts:227-262`). Without the Locks API it runs directly (`studio/src/lib/api.ts:98-100`).
- Refresh outcomes: `ok` , `denied` (4xx, session cleared and redirect to login), `network` (5xx or fetch failure: keep session, show a network error) (`studio/src/lib/api.ts:84-90,152-160`).
- `endSession()` clears the session, fires `scammind:unauthorized` , and redirects to `/login?next=/studio/` (`studio/src/lib/api.ts:117-121`).
- Other tabs: `AuthProvider` listens to `storage` events on the two keys and to `scammind:unauthorized` ; a sign-out in another tab sends this tab to login (`studio/src/contexts/AuthContext.tsx:31-47`).
- Studio "Log out": best-effort `POST /api/auth/logout` (revokes the refresh token), clear storage, go to `/login` (no `next`) (`studio/src/lib/api.ts:172-187`).
- After login, BrandShield only follows `next` when it matches `^/studio/[A-Za-z0-9\-._~/]*$` with no `..` or `//` (anti open-redirect) (`frontend/src/utils/studio.ts:15-19`).

2.3 AI proxy (server side)

Files: `backend/app/api/studio_ai.py` (routes, validation, quotas), `backend/app/services/studio_ai.py` (providers, retries, quota storage). Router is mounted at `/api/studio/ai` (`backend/app/main.py:180` , `backend/app/api/studio_ai.py:27`). The service objects live on `app.state.studio_ai` and `app.state.studio_ai_limiter` (`backend/app/main.py:102-103`).

Route	Body / query	Success response	Source
`GET /api/studio/ai/status`	none	`{chat:{configured, providers, used_today, daily_limit}, image: {configured, provider:"pollinations", used_today, daily_limit, sizes}, limits: {...}}` . No secrets.	`backend/app/api/studio_ai.py:75-85`
`POST /api/studio/ai/chat`	`{messages[1..200], response_format:"json"|"text", temperature 0..2, max_tokens?}` ; each message `{role: system|user|assistant|model, content}` (`text` accepted as alias)	`{text, provider, model}`	`backend/app/api/studio_ai.py:33-52,88-106`
`POST /api/studio/ai/image?format=json|png`	`{prompt (1..20000), size}`	json: `{image_base64, data_url, media_type, width, height, provider, model}` ; `format=png` returns raw PNG bytes	`backend/app/api/studio_ai.py:54-56,109-136`

Chat order of work (`backend/app/api/studio_ai.py:88-106`):

1. Reject more than `STUDIO_MAX_MESSAGES` messages (422), more than `STUDIO_MAX_CHARS` total characters (413), `max_tokens` above `STUDIO_MAX_TOKENS` (422).
2. 503 "AI chat is not set up on the server." when neither `GROQ_API_KEY` nor `GEMINI_API_KEY` is usable (`backend/app/services/studio_ai.py:105-106`).
3. `_guard` : per-user in-memory limiter `STUDIO_AI_RATE_PER_MINUTE` (429 with `Retry-After: 30`), then daily quota (429 "You have used today's text AI allowance (N). It resets at midnight UTC.") (`backend/app/api/studio_ai.py:63-68`).
4. Provider call. On `StudioAIError` the quota unit is refunded, so a broken provider does not use up the user's day (`backend/app/api/studio_ai.py:102-106` , `backend/app/services/studio_ai.py:305-312`).

Provider selection and fallback (`backend/app/services/studio_ai.py:150-169`): Groq first (`STUDIO_GROQ_MODEL` , default `openai/gpt-oss-120b` , `backend/app/config/config.py:141`), then Gemini (`GEMINI_MODEL` , default `gemini-3.5-flash-lite` , `backend/app/config/config.py:72`) if Groq has no key or fails. In JSON mode the answer is repaired by `_as_json_text` (unwraps a ```json fence or surrounding prose); if it still is not JSON the next provider is tried (`backend/app/services/studio_ai.py:162-166,238-254`). Gemini maps `system` messages to `systemInstruction` and roles `assistant` to `model` (`backend/app/services/studio_ai.py:191-218`).

Image order of work (`backend/app/api/studio_ai.py:109-136`): collapse whitespace; reject prompts over `STUDIO_MAX_PROMPT_CHARS` (default 2000, 413); `size` must be one of `512x512, 768x768, 1024x1024, 1024x768, 768x1024, 1280x720, 720x1280` (422); 503 if `POLLINATIONS_API_KEY` missing; guard (rate limit and image quota); `StudioAI.image` calls `GET https://gen.pollinations.ai/image/{prompt}?model=...&width=..&height=..&seed=-1` with a Bearer key (`backend/app/services/studio_ai.py:222-235`). The picture must be `image/png|jpeg|webp` (SVG refused), is decoded with limits (16 million pixels, 16 to 4096 px per side), and **re-encoded as PNG**, dropping metadata and trailing bytes (`backend/app/services/studio_ai.py:43-46,257-276`).

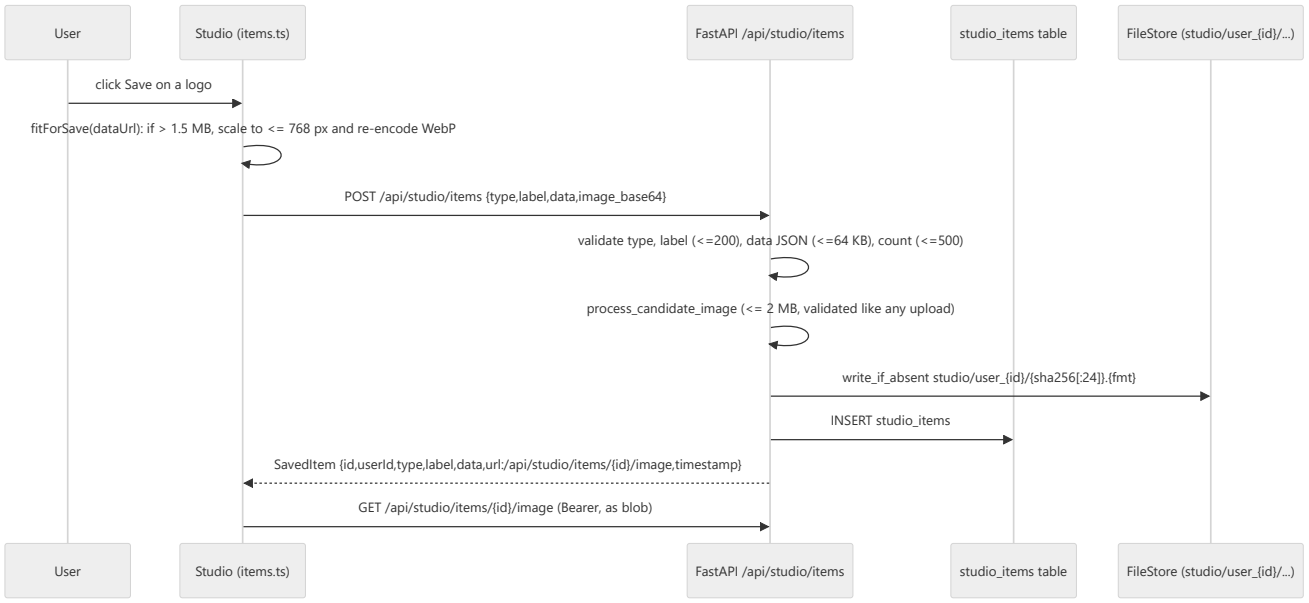
Transport rules for every outbound call (`backend/app/services/studio_ai.py:113-146`): timeout (`STUDIO_AI_TIMEOUT_SECONDS` , images `STUDIO_IMAGE_TIMEOUT_SECONDS`), `1 + STUDIO_AI_MAX_RETRIES` attempts for timeouts, connection errors and 5xx only (never 429 or key errors), backoff `STUDIO_AI_RETRY_BACKOFF_SECONDS * attempt` , response streamed and cut at 512 KB (text) or 12 MB (image), redirects not followed. Prompts, answers and keys are never logged; only provider, status and duration (`backend/app/services/studio_ai.py:1-10,136`).

Status mapping to the browser (`backend/app/services/studio_ai.py:70-80`): provider 401/403 becomes 503 "key was rejected", 402 becomes 503 "out of credits", 429 stays 429 with `Retry-After` , 400/404/413/422 become 422, anything else 502. The provider's own error text is never forwarded.

Daily quota: stored in table `api_usage` with `provider = "studio:{text|image}:u{user_id}:{YYYYMMDD}"` and `month = "YYYY-MM"` ; `reserve_quota` does an atomic `UPDATE ... WHERE credits + 1 <= limit` ; limit `<= 0` means unlimited (`backend/app/services/studio_ai.py:282-317`). Defaults: 200 text and 50 image per user per UTC day (`backend/app/config/config.py:151-152`).

Browser error wording (`studio/src/lib/ai.ts:28-62`): network error; 402 or "out of credits" (image: offers manual upload, `imageUnavailable=true`); 429 (adds "try again in about N seconds"); 503 (image: "not available right now, upload your own logo"); 502; 403 "Your account is not allowed to use this."

2.4 Collections (saved items)



Client (`studio/src/lib/items.ts` , `studio/src/contexts/CollectionsContext.tsx`):

- `listItems` asks for up to 500 items (`studio/src/lib/items.ts:14-21`).
- `createItem` sends `image_base64` only when `url` starts with `data:image/` (`studio/src/lib/items.ts:57-61`).
- **Image size handling (`fitForSave`)**: a generated 1024 px PNG can exceed the server limit of 2 MB. If the decoded size is over `SAVE_LIMIT_BYTES = 1_500_000` the picture is scaled so its longest side is at most 768 px and saved as WebP at quality 0.9; up to 4 attempts, each shrinking the side by 25 percent, until it fits. If it cannot be decoded (tests, odd data) it is sent unchanged and the server decides (`studio/src/lib/items.ts:22-54`).
- Pictures need the `Authorization` header, so `<img src>` cannot load them; `fetchItemImage` fetches a Blob via `request(..., {blob:true})` (`studio/src/lib/items.ts:68-70`).
- `uploadLegacyGuestItems` uploads the old standalone-app guest collection (`localStorage` key `brandcraft_collections` , items with `userId === 'guest'`) one by one, then removes the key; it stops and keeps the rest if the limit, an offline error or any failure occurs (`studio/src/lib/items.ts:72-104`).

Server (`backend/app/api/studio.py` , model `backend/app/models/studio.py:11-22`):

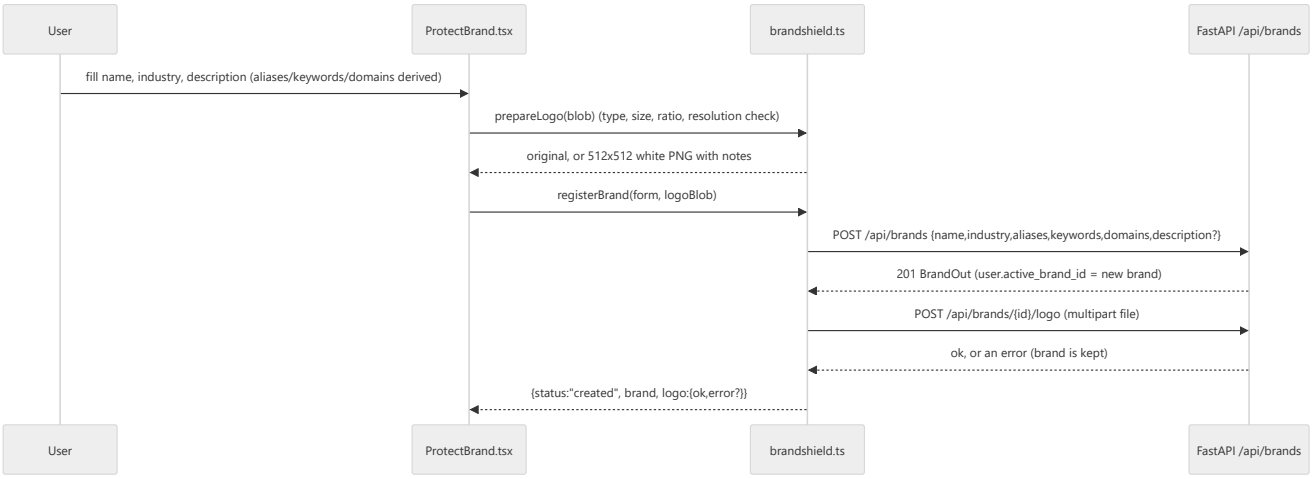
Route	Behavior	Source
`GET /api/studio/items?type&limit(1..500, default 100)&offset`	Owner-only list, newest first, `{total,limit,offset,items}`	`backend/app/api/studio.py:103-118`
`POST /api/studio/items` (201)	JSON `{type,label,data?,image_base64?}` or multipart (`type,label,data` JSON text, `file`)	`backend/app/api/studio.py:121-162`
`DELETE /api/studio/items/{id}`	Owner-only; deletes the stored file only if no other item uses the same content-addressed path	`backend/app/api/studio.py:172-180`
`GET /api/studio/items/{id}/image`	Owner-only; `Cache-Control: private, max-age=300` , `nosniff` , inline	`backend/app/api/studio.py:183-191`

Validation: `type` in `logo, palette, brand_name, content, audit, sentiment` (422); `label` required, printable characters only, whitespace collapsed, max 200 (422); `data` JSON max 64 KB (413); image max 2 MB (413), must be valid base64 (422); body over about 4.06 MB refused before parsing (413); other Content-Type 415; at most 500 items per user (409 "You can keep at most 500 saved items. Delete some first."). Another person's item is a plain 404 (`backend/app/api/studio.py:31-35,51-81,86-87,165-169`). The picture goes through `process_candidate_image` (same validator as uploads) and is stored at `studio/user_{id}/{sha256[:24]}.{format}` (`backend/app/api/studio.py:89-95`).

2.5 Remembered inputs

`studio/src/lib/profile.ts` keeps seven fields: `brandName, industry, audience, description, personality, style, mood` (`profile.ts:6`). They are stored in `localStorage` under `brandshield.studio.profile.{userId|anon}` (`profile.ts:12-14`), so each account has its own set in this browser and nothing is sent to the server. `useRemembered(field, fallback)` is a `useSyncExternalStore` hook, so the same field typed in one tool appears in the others and survives reload (`profile.ts:64-67`). Used by BrandAudit, BrandChecker, BrandNameGenerator, ColorPaletteGenerator, ContentGenerator, LogoGenerator and ProtectBrand (`studio/src/components/*.tsx` , e.g. `BrandAudit.tsx:54-56` , `ProtectBrand.tsx:56-58`). `clearRemembered()` empties it; storage failures are swallowed (value stays in memory).

2.6 "Protect in BrandShield" (brand registration)



- **A new brand is created every time.** `registerBrand` always calls `POST /api/brands` and returns `status: 'created'`; there is no update path, because an account can protect several brands (`studio/src/lib/brandshield.ts:225-231`). The limit is `MAX_BRANDS_PER_ACCOUNT = 10` non-archived brands; the 11th gets a 409 "You can protect up to 10 brands. Archive one first." (`backend/app/api/brands.py:46,143-147`). The new brand also becomes the user's active brand (`backend/app/api/brands.py:150-151`).
- Permissions: `require_writer` (admin or analyst). A viewer gets 403, and the wizard shows "Your account can view BrandShield but cannot create a brand. Ask an administrator for an analyst account." (`backend/app/api/brands.py:143` , `studio/src/components/ProtectBrand.tsx:153`).
- Derivations (client side, pure functions): `deriveAliases("Acme Bank")` gives `acmebank, acme-bank, acme_bank, acme` (max 8, never the name itself); `deriveKeywords` takes name words of 3+ letters plus tagline words of 4+ letters (max 8); `normalizeDomain("https://www.Acme.com/path")` gives `acme.com` or `null` (`studio/src/lib/brandshield.ts:38-70`). `buildPayload` trims, de-duplicates, and only sends `description` when not empty (`brandshield.ts:194-204`).
- Logo rules mirror the backend: PNG, JPEG or WebP; at most 2 MB; at least 256 px; width/height ratio 0.8 to 1.25 (`brandshield.ts:5-12` , `checkLogoMeta` at `89-110`). An unusable file (wrong type, unreadable) is rejected; otherwise, if too small, not square enough or over 2 MB, it is centred on a white 512 x 512 canvas and saved as PNG, with plain-language notes (`prepareLogo` , `brandshield.ts:143-161`).
- The logo upload is a second request; if it fails the brand still exists and the outcome reports `logo: {ok:false, error}` (`brandshield.ts:206-223`).
- 422 errors are mapped to wizard fields through `mapFieldErrors` (`fields.file` becomes the logo field) (`brandshield.ts:234-239`).
- "Back to BrandShield" goes to `/dashboard` (`BRANDSHIELD_HOME` , `studio/src/lib/session.ts:72`). A generated logo saved to Collections can be reused: `dataUrlToBlob` converts a data URL to a Blob (`brandshield.ts:163-170`).

3. Files, functions and classes

Path	Role
`studio/src/main.tsx`	Mounts `ThemeProvider` , `AuthProvider` , `App` ; DEV-only direct-port redirect (`main.tsx:9-12`)
`studio/src/App.tsx`	Sidebar, tabs, dashboard, counts; `BrandProvider` , `CollectionsProvider`
`studio/src/contexts/AuthContext.tsx` , `BrandContext.tsx` , `CollectionsContext.tsx` , `ThemeContext.tsx`	Session user, selected brand/tab navigation, saved items, theme
`studio/src/lib/session.ts` , `api.ts` , `ai.ts` , `items.ts` , `profile.ts` , `brandshield.ts` , `devRedirect.ts` , `useAiStatus.ts`	Shared client logic described above
`studio/src/services/gemini.ts`	Prompts for each tool (calls `aiChat` / `aiImage`)
`studio/src/components/*`	One component per tool, plus `Collections` , `ProtectBrand` , `SuccessFlow` , `LogoUpload` , `AiStatusBanner` , `CustomCursor`
`backend/app/api/studio_ai.py` , `backend/app/services/studio_ai.py`	AI proxy, class `StudioAI` , `StudioAIError` , quota functions
`backend/app/api/studio.py` , `backend/app/models/studio.py`	Collections API, `StudioItem` model, `STUDIO_TYPES`
`backend/app/config/ratelimit.py`	`RequestLimiter` (in-memory sliding window, single process) (`ratelimit.py:35-47`)
`frontend/src/utils/studio.ts`	`STUDIO_URL` , `openStudio()` , `safeStudioNext()`

4. APIs, tables and external services

Kind	Item
APIs used by Studio	`POST /api/auth/refresh`, `POST /api/auth/logout`, `GET/POST /api/studio/items`, `DELETE /api/studio/items/{id}`, `GET /api/studio/items/{id}/image`, `GET /api/studio/ai/status`, `POST /api/studio/ai/chat`, `POST /api/studio/ai/image`, `POST /api/brands`, `POST /api/brands/{id}/logo`
DB tables	`studio_items` (id, user_id FK users ON DELETE CASCADE, type, label, data JSON, image_path, created_at) (`backend/app/models/studio.py:14-22`); `api_usage` (daily quota counters) (`backend/app/models/ops.py:116-119`); `brands` and related tables when protecting a brand
File storage	`FileStore` path `studio/user_{id}/{hash}.{fmt}`
External services	Groq `https://api.groq.com/openai/v1/chat/completions` ; Google Gemini `generativelanguage.googleapis.com/v1beta/models/{model}:generateContent` ; Pollinations `https://gen.pollinations.ai/image/` (`backend/app/services/studio_ai.py:39-41`)
Browser APIs	Web Locks (`navigator.locks`), `localStorage` / `sessionStorage` , Canvas (logo resize, `fitForSave`)

5. Validation, permissions, errors and edge cases

- All Studio endpoints need a valid access token (`get_current_user`); any role may use items and AI. Only brand creation needs analyst or admin.
- Rate limit: `STUDIO_AI_RATE_PER_MINUTE` (default 20) per user, chat and image together, in memory per process; with several workers each counts separately (`backend/app/config/config.py:143` , `backend/app/config/ratelimit.py:35-37`).
- Gemini fallback only applies to chat. Images have a single provider; when Pollinations is out of credits the UI offers uploading your own logo (`studio/src/lib/ai.ts:34-39`).
- Studio-only screens still depend on BrandShield: if the API returns 401 and refresh is denied, the user is redirected to login and loses unsaved results.
- Legacy guest collections are migrated once (section 2.4).
- The brand cap and the 500-item cap are separate (10 brands, 500 items).

6. Dev proxy, redirect and Vercel/Docker layout

- Studio is built with `base: '/studio/'` (`studio/vite.config.ts:11`). Nothing from the environment is injected into the bundle (`studio/vite.config.ts:8-10`).
- Studio dev server: `127.0.0.1:3000` , `strictPort` , proxy `/api` to `STUDIO_API_PROXY` or `http://127.0.0.1:8000` (`studio/vite.config.ts:19-25`).
- BrandShield dev server (port 5173) proxies `/studio` (with `ws: true` for HMR) to `STUDIO_DEV_URL` or `http://127.0.0.1:3000` , and `/api` to `STUDIO_API_PROXY` or `http://127.0.0.1:8000` , so both apps share one origin and one `localStorage` session (`frontend/vite.config.ts:8-14`). This is the address that works in development: `http://localhost:5173/studio/` .
- `devRedirect` : if someone opens Studio directly on `localhost:3000` or `127.0.0.1:3000` , `directDevRedirect()` returns `http://localhost:5173` (or `VITE_SHIELD_URL`) plus the same path (`/studio/` if the path is not under `/studio`), query and hash; any other host or port returns `null` . It runs only when `import.meta.env.DEV` (`studio/src/lib/devRedirect.ts:16-20` , `studio/src/main.tsx:9-12`).
- Vercel (`vercel.json`): three services `backend` (FastAPI, `api.index:app` , `maxDuration` 300), `frontend` (Vite, SPA rewrite) and `studio` (Vite, rewrites under `/studio/`) (`vercel.json:3-44`). Top-level rewrites: `/api/*` and `/health*` to backend, `/studio` and `/studio/*` to studio, everything else to frontend (`vercel.json:45-80`). `/studio/assets/*` is cached immutable for a year (`vercel.json:127-132`). The CSP is `connect-src 'self'` , which is why Studio must use same-origin relative URLs (`vercel.json:114`).
- Docker: the frontend nginx has `location /studio/` proxying to the `studio` container (`http://studio:8080`) and forwards `/api` to the api container (`frontend/nginx/default.conf.template:35-60` ; `docker-compose.yml:146-156`).
- Tests: `studio/src/lib/*.test.ts` (api, brandshield, devRedirect, items, session), `studio/src/contexts/AuthContext.test.tsx` ; cross-app e2e in `frontend/e2e/studio-flow.spec.ts` and `frontend/e2e/multi-brand.spec.ts` (Studio started on port 3100, `frontend/playwright.config.ts:78-87`).

7. Configuration (names only)

Name	Purpose	Default (from `backend/app/config/config.py`)
`GROQ_API_KEY`	Primary chat provider key (secret)	none (`config.py:139`)
`GEMINI_API_KEY`	Chat fallback key (secret); also used by voice	none (`config.py:71`)
`POLLINATIONS_API_KEY`	Image provider key (secret)	none (`config.py:140`)
`STUDIO_GROQ_MODEL`	Groq model id	`openai/gpt-oss-120b`
`GEMINI_MODEL`	Gemini model for chat fallback	`gemini-3.5-flash-lite`
`STUDIO_IMAGE_MODEL`	Pollinations model	`black-forest-labs/flux.1-schnell`
`STUDIO_AI_RATE_PER_MINUTE`	Per-user requests per minute	20
`STUDIO_DAILY_TEXT_QUOTA` , `STUDIO_DAILY_IMAGE_QUOTA`	Per-user per UTC day (0 = unlimited)	200, 50
`STUDIO_AI_TIMEOUT_SECONDS` , `STUDIO_IMAGE_TIMEOUT_SECONDS`	Provider timeouts	45, 120
`STUDIO_AI_MAX_RETRIES` , `STUDIO_AI_RETRY_BACKOFF_SECONDS`	Retry policy	2, 0.5
`STUDIO_MAX_MESSAGES` , `STUDIO_MAX_CHARS` , `STUDIO_MAX_TOKENS` , `STUDIO_MAX_PROMPT_CHARS`	Request size limits	40, 30000, 4096, 2000
`VITE_STUDIO_URL` (frontend)	Only for an odd setup with Studio on another origin	`/studio/`
`VITE_SHIELD_URL` (studio, dev only)	Where the direct-port redirect goes	`http://localhost:5173`
`STUDIO_DEV_URL` , `STUDIO_API_PROXY`	Dev proxy targets	`http://127.0.0.1:3000` , `http://127.0.0.1:8000`

Names are listed in `backend/.env.example:105-120` . No values are reproduced here.

DOCS/FEATURES/VOICE-AGENT.MD

Voice Agent

Plain-language summary

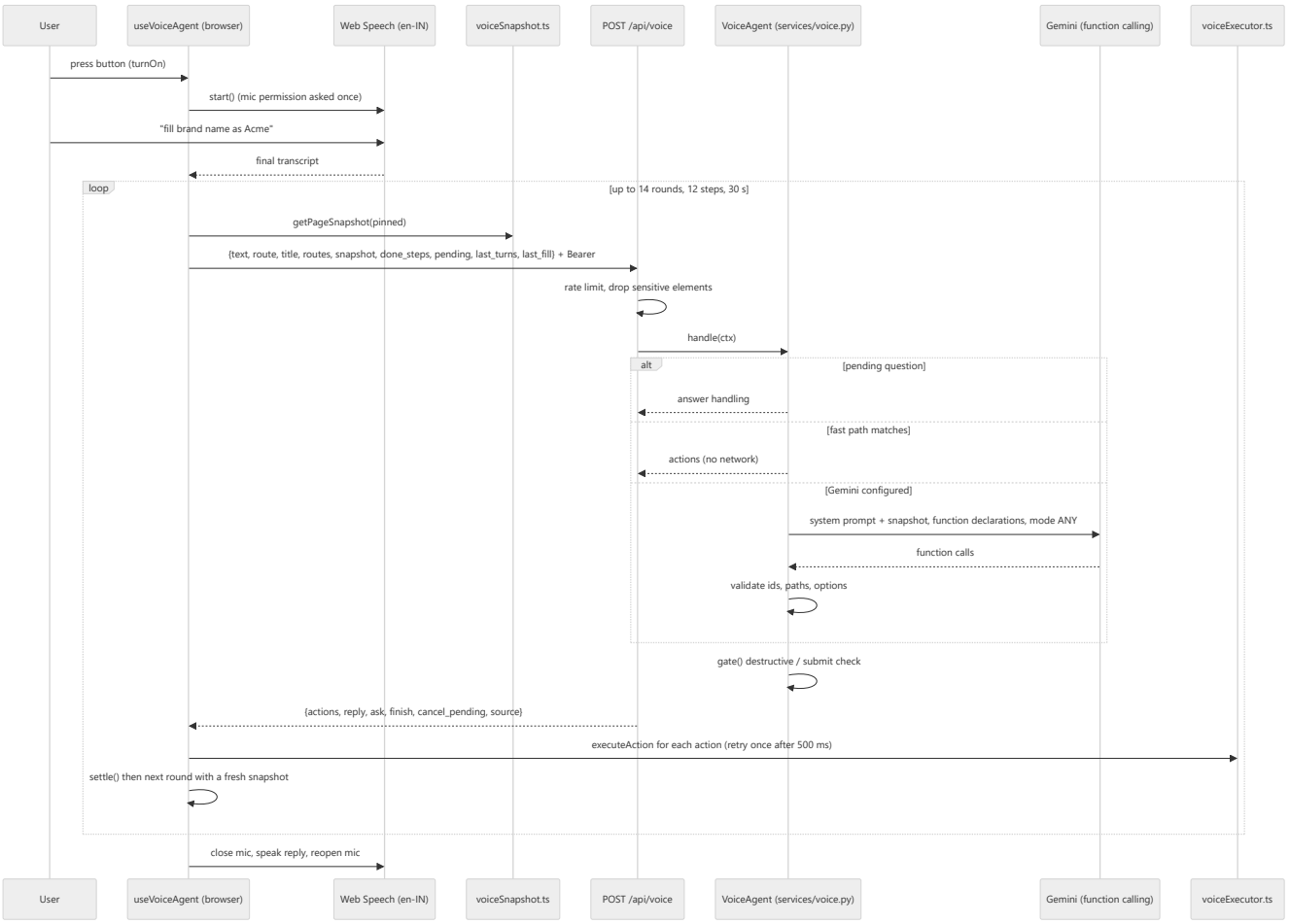
The voice assistant is a round button at the bottom right of every signed-in BrandShield page. When you switch it on, you can speak ("open reports", "fill brand name as Acme", "select high severity") and it will move around the app and fill in forms for you. Simple commands are understood by the server with no AI at all; harder ones are sent to Google Gemini, which picks from a short list of safe actions. The assistant never types into password, one-time-code or card fields, asks before anything destructive (delete, pay, block) or before submitting a form you did not ask to submit, and the microphone is only open while the switch is on.

1. Purpose and user-facing behavior

Behavior	Detail	Source
Button	Floating "voice-fab" with a status panel; mounted once above all signed-in routes so it keeps its state when the route changes. Not shown on `/login`, `/register`, `/`.	`frontend/src/App.tsx:56-78`, `frontend/src/components/voice/VoiceAgentButton.tsx:15-92`
States	`off`, `listening`, `thinking`, `speaking`, `error`; panel texts such as "Listening... try 'scroll down' or 'fill brand name as Acme'"	`frontend/src/hooks/useVoiceAgent.ts:13`, `VoiceAgentButton.tsx:6-12`
Language	Speech recognition and speech synthesis both use `en-IN`	`useVoiceAgent.ts:46,129`
Browser support	Needs the (prefixed) Web Speech API: Chrome or Edge on desktop or Android. Otherwise the panel says "Voice commands need Chrome or Edge on a computer or Android phone."	`useVoiceAgent.ts:41-44,375-379`
Microphone	Requested only when the person presses the button the first time (`rec.start()`); released (`abort()`) on OFF, on error and on unmount	`useVoiceAgent.ts:341-343,353-368,397-398`
Stop	Panel "Stop" button, or saying stop / cancel / abort while it works	`useVoiceAgent.ts:50,177-181,289-296`, `VoiceAgentButton.tsx:57-61`
Follow-up questions	The agent can ask one short question (confirm, choose an option, provide a value). Answers can be spoken or tapped as chips ("Skip", "That's all" for values).	`VoiceAgentButton.tsx:17,45-56`, `useVoiceAgent.ts:387-395`
Silence timer	A pending question is re-asked once after 10 s of silence, then dropped	`useVoiceAgent.ts:49,162-175`

Example phrases that work with no AI key (fast path): "open the look-alike lab", "scroll down", "go back", "what page am I on", "fill test name with acm3bank", "my brand name is Acme", "change it to Acme Bank", "undo", "clear that", "fill the whole form" (`backend/app/services/voice.py:261-281`; the e2e test uses the first and fifth, `frontend/e2e/voice.spec.ts:55-69`).

2. Step-by-step flow



1. **Switch on.** `toggle` calls `turnOn`; with no Speech API it shows the error, otherwise `onRef=true` and `startListening()` (`useVoiceAgent.ts:370-384`).
2. **Listen.** Recognition is `continuous=false`, `interimResults=true`. Interim text is shown in the panel; the first final result is sent to `finalRef` (`useVoiceAgent.ts:298-321`). Chrome ends recognition after a pause, so `onend` restarts it while ON and not speaking (`useVoiceAgent.ts:333-339`). `no-speech` and `aborted` errors are ignored; other errors turn the agent off with a message ("Microphone access was blocked..." for `not-allowed`) (`useVoiceAgent.ts:322-332`).
3. **Run the command (`runCommand`).** Up to 14 request rounds. Each round builds a fresh snapshot and POSTs to `/api/voice` via `sm.voice` (`frontend/src/api/sm.ts:73`). The request includes `routes: voiceRoutes(isAdmin)` so the server only sees pages this user may open (`useVoiceAgent.ts:205-219`).
4. **Execute.** For each returned action: limit of 12 steps and 30 s total ("I stopped here because this is taking too long."); the step is shown in the panel; if it fails it waits 500 ms and tries once more; if it fails again the agent says "I couldn't ..." and stops (`useVoiceAgent.ts:222-243`). Navigations to a path beginning `/studio` use `window.location.assign` (Studio is a separate app); others use React Router (`useVoiceAgent.ts:199`).
5. **Settle.** After actions, `settle()` waits about 250 ms of DOM quiet and no spinners (up to 5 s) before the next snapshot (`frontend/src/utils/voiceExecutor.ts:316-345`).
6. **Finish or ask.** The loop ends on `finish`, on `ask`, on empty actions, on failure or cancel (`useVoiceAgent.ts:252-259`). The reply (and question) is spoken with the microphone closed (so the agent cannot hear itself), then listening resumes; a pending question arms the silence timer (`sayAndListen`, `useVoiceAgent.ts:145-160`). `speak()` has a guard timer in case a browser never fires `end` (`useVoiceAgent.ts:131-132`).
7. **Pending question.** The pending question is stored in `pendingRef` and sent as `pending` on round 0 of the next utterance, which is then treated as its answer (`useVoiceAgent.ts:195,214,276`). `last_turns` (last 6) and `last_fill` (for "undo", "change it to") travel with each request (`useVoiceAgent.ts:215-216,249,280`).
8. **Stop.** `stopRun` sets `cancelRef`, aborts the in-flight fetch and cancels speech; the agent says "Stopped." (`useVoiceAgent.ts:177-181,269-274`).
9. **Switch off.** `turnOff` clears every ref and timer, aborts the fetch, closes recognition and cancels speech (`useVoiceAgent.ts:353-368`). The e2e test asserts `aborts` increases after OFF (`frontend/e2e/voice.spec.ts:66-69`).

2.1 Page snapshot (`frontend/src/utils/voiceSnapshot.ts`)

- Compact keys per element: `i` id (`data-voice-id`, `v1`, `v2` ...), `t` type, `l` label, `n` field name (`data-voice`), `v` value, `r` required, `d` disabled, `e` validation error, `o` options, `c` checked, `off` off-screen (`frontend/src/utils/voice.ts:34-48`).
- What counts as a control: the `CONTROLS` selector (buttons, links, inputs, selects, ARIA roles tab/button/combobox/checkbox/radio/switch/option/menuitem, Radix collection items, table rows, `.detection-row-card`, `[data-voice-row]`) (`voiceSnapshot.ts:10-14`).
- **Where it looks:** the page, **open shadow roots** and **same-origin iframes** (cross-origin frames are skipped) (`roots`, `voiceSnapshot.ts:52-65`), **portals** (dropdown options live under `<body>` and are found because the search is document-wide), and **off-screen** elements: they only need a non-zero size and not `display:none` / `visibility:hidden`; off-screen ones carry `off:1` (`visible`, `isOffscreen`, `voiceSnapshot.ts:34-47`).
- Hidden elements: anything inside `.voice-agent` itself, `[hidden]` or `[aria-hidden="true"]` is ignored (`voiceSnapshot.ts:35`).
- **Open dialogs:** while a `[role="dialog"]` or `[aria-modal="true"]` is visible, only controls inside it are offered, plus dropdown options (`openModal`, `collectAll` at `voiceSnapshot.ts:73-76,206-223`).
- **150 cap:** `getPageSnapshot` keeps at most `MAX = 150` controls, ranked by priority: open dialog (0), form fields (1), buttons/tabs/options (2), rows (3), links (4); at most 40 rows (`MAX_ROWS`); ids found by `find_element` ("pinned") always stay; the result is put back in page order. If there are rows, a first synthetic `info` element `v0` says "N rows (first 40 listed)" (`voiceSnapshot.ts:7-8,225-246`).

- Labels come from `aria-label`, `aria-labelledby`, `label[for]`, wrapping label, placeholder, nearby label, visible text, title, or `data-voice` (`labelOf`, `voiceSnapshot.ts:115-147`); table rows use their first three cells (`:142-145`).
- The server cap matches: `snapshot` max 200 elements (`backend/app/api/voice.py:47`).

2.2 Executor (`frontend/src/utils/voiceExecutor.ts`)

Actions: `navigate`, `back`, `scroll`, `find`, `wait`, `close`, `highlight`, `click`, `select`, `fill`, `toggle` (`frontend/src/utils/voice.ts:9-20`, `voiceExecutor.ts:227-314`).

- **Element resolution:** by `data-voice-id`; if the id is stale (page re-rendered) or the element is hidden, it searches again by the label the planner gave (`voiceExecutor.ts:274-283`).
- **`find_element`:** fuzzy search over the whole live DOM using label, value, name, placeholder, aria-label and (for rows) the first 300 characters of row text; threshold 55, best 3 matches; if nothing is found it scrolls the page down in up to 5 steps, waiting 1.5 s each, in case lists load lazily; the best match is scrolled into view and flashed (`voiceExecutor.ts:56-129`). Result ids are returned as `found` and "pinned" into later snapshots (`useVoiceAgent.ts:197,246`).
- **Fill:** only `<input>` and `<textarea>`, not disabled or read-only. Uses the native value setter and dispatches `input` so React-controlled fields update state (`voiceExecutor.ts:147-151,293-302`).
- **Select (native):** best fuzzy option by text then by value; sets the value and dispatches `change` (`voiceExecutor.ts:173-184`).
- **Select (custom dropdown):** `pickFromDropdown` clicks the trigger, waits for the page to settle, searches the **whole page** (including portals) for option-like elements (`[role=option]`, `[role=menuitem]`, `li`, classes containing option / menu-item / dropdown-item), prefers elements that appeared after the click, clicks the best match, and verifies the trigger's shown value changed. Two attempts; errors are "The options did not open." or "There is no option called X." (`voiceExecutor.ts:188-225`).
- **Toggle:** clicks only if the current state differs from the requested state (`voiceExecutor.ts:303-310`).
- **Close:** clicks a dialog button named close/cancel/dismiss, else dispatches Escape (`voiceExecutor.ts:249-257`).
- **Highlight:** used for sensitive fields; scrolls to, focuses and pulses a password or one-time-code field so the person can type it themselves (`voiceExecutor.ts:258-270`).
- Scrolling targets the open dialog, else `.main-panel`, else the document (`voiceExecutor.ts:153-171`).
- `bestMatch` scores exact 100, prefix 85, contains 75, shared words up to 70, minimum 50 (`voiceExecutor.ts:27-51`).

2.3 `data-voice` attributes

A page makes a field speakable by adding `data-voice="<name>"` (snake_case). The name is spoken as words ("test_name" becomes "test name") and is matched by the fast path. Examples: `brand_name`, `industry`, `description`, `alias`, `keyword`, `domain`, `account_platform`, `account_handle`, `app_name`, `app_store` (`frontend/src/pages/BrandProfilePage.tsx:265-620`); setup wizard fields incl. `scan_depth` (`frontend/src/pages/SetupPage.tsx:408-829`); `status_filter`, `severity_filter`, `resolution_filter`, `note`, `resolution` (`InvestigationsPage.tsx:58-174`); `test_name` (`LookalikeLabPage.tsx:84`); `date_range` (`ReportsPage.tsx:182`); `schedule`, `impersonation_word`, `webhook_url` (`SettingsPage.tsx:213-406`); `search` in the sidebar (`Sidebar.tsx:49`); and dynamic keys in `CandidateTools.tsx:120,128`. `data-voice-row` marks custom clickable rows (`Sidebar.tsx:156` marks the Studio link). `collectVoiceFields()` lists fillable `data-voice` fields (`voice.ts:109-118`). Fields without `data-voice` can still be offered by label, but `fill` helpers in `voice.ts` skip anything without a name (`voice.ts:100-106`).

2.4 Backend `POST /api/voice`

Route: `backend/app/api/voice.py:54-66`, mounted in `backend/app/main.py:181`. Auth: `get_current_user` (any role).

Request field	Type and limit	Source
`text`	string, max 1000	`voice.py:43`
`route`, `title`	max 200, 120	`voice.py:44-45`
`routes[]`	max 50; each `{path (100), spoken_names (max 10)}`	`voice.py:23-25,46`
`snapshot[]`	max 200; `i` must match `^v\d{1,5}$`, `t` max 12, `l/n/v` max 60, `e` max 100, `o` max 60 items	`voice.py:28-39,47`
`done_steps`	max 40, values cut to 200 chars	`voice.py:48,63`
`pending`, `last_fill`	free objects	`voice.py:49,51`
`last_turns`	max 6, text cut to 200	`voice.py:50,64`

Response: `{actions, reply, ask, finish, cancel_pending, source}`; `source` is `fast`, `gemini` or `none` (`backend/app/services/voice.py:66-77`).

Errors: 429 "Too many voice commands. Wait a moment and try again." with `Retry-After: 20` when the per-user limiter (`VOICE_RATE_PER_MINUTE`, default 30, in memory) is exceeded (`voice.py:56-57`, `backend/app/main.py:104`). Gemini failures never produce an error page: the agent answers "The AI service is busy right now. Please try again in a moment." for 429/503/504, else "Sorry, I didn't understand that..." (`services/voice.py:532-539`). The browser shows "Sorry, I didn't catch that. Please try again." on other failures and the server message on 429 (`useVoiceAgent.ts:261-265`).

Order of work in `VoiceAgent.handle` (`backend/app/services/voice.py:514-539`):

1. Normalise text (max 500 characters), keep only routes in `ALLOWED_PATHS` (`:515-516`).
2. If a question is pending, `_answer` interprets the utterance as its answer, unless it is clearly a new command (open, scroll, back, close, stop) which cancels the question (`:519-522,390-447`).
3. Otherwise, if earlier steps exist and a just-filled field now shows a validation error, ask for a corrected value (`validation_follow_up`, `:523-526,480-486`).
4. **Fast path** (regex plus RapidFuzz, no network): `fast_command` (stop, "where am I", scroll, back, close, open page) then `fast_fill` ("fill X as Y", "put Y in X", "X is Y", corrections, undo, clear, "fill the whole form") (`:301-384,527`). Page names match with a score of at least `NAV_SCORE = 82`, field names with `FIELD_SCORE = 80` using `fuzz.ratio` and `token_set_ratio` (`:48-49,174-180,198-206,291-298`).
5. If no Gemini key: "I can open pages, scroll and fill fields with simple commands, like "open reports"." (`:530-531`).
6. **Gemini**: `_gemini` (`:541-613`), then `gate()` (`:533`).

Follow-up questions. After a fill or select, `_follow_up` asks "Do you want me to fill X too?" with options Yes / Skip / That's all for the next empty field (`:245-256`). "Fill the whole form" starts guided mode field by field; `Skip` moves on, "That's all" ends (`:361-365,421-447`). A select with 4 or fewer options asks "Which X? A, B, C?";

otherwise it asks for a value (`_ask_value` , `:231-237`). Ambiguous options give up to 4 candidates (`match_option` , `:183-195`). Ordinals ("first", "second") pick by position (`:186-188`). Spoken email ("at the rate", "dot"), phone ("double five") and URL words are converted to written form (`normalize_value` , `:158-171`).

Safety gate (`gate` , `:457-477`):

Rule	Pattern	Result
Destructive	label matches `delete, remove, block, pay, payment, purchase, buy, archive, discard, revoke, erase, terminate, deactivate` (`:39`)	The click is withheld and the agent asks "Do you want me to ? Say yes or no." The click is stored in `ask.then` and run only after a spoken yes
Submit not requested	click on a label matching `submit, register, create, finish, publish, launch, send, save, confirm` (`:40`) and the spoken text does not contain that word	The agent reads back up to 5 filled values and asks "Should I submit?"
Yes / no words	`YES` , `NO` , `SKIP` , `THATS_ALL` , `STOP` regexes (`:41-45`)	Used when answering a pending confirm

When confirmed, `then` actions are re-checked against the current snapshot ids and only `click` and `toggle` types are accepted (`:403,405-410`).

Gemini call (`_gemini` , `:541-613`): the model gets a system prompt ("You control the BrandShield web UI using the given page snapshot. Use only ids from it..."), a compact JSON of goal, route, done steps, recent turns and elements, and function declarations `navigate` , `scroll` , `find_element` , `wait` , `reply` , `finish` , `ask_user` , plus (only when the snapshot is not empty) `click` , `select_option` , `fill_field` , `toggle` , whose `id` argument is an enum of the ids actually in the snapshot. Settings: `mode="ANY"` (must call a function), automatic function calling disabled, temperature 0, `max_output_tokens=512` , `thinking_level = VOICE_THINKING_LEVEL` (default `minimal`) (`:581-601`). The client uses one attempt and a hard timeout of `max(10, VOICE_TIMEOUT_SECONDS)` seconds (`:503-512`). **Fallback:** if the primary model (`VOICE_GEMINI_MODEL`) raises and less than 4 seconds have passed (`FALLBACK_WITHIN_SECONDS`), the same request is retried once on `VOICE_GEMINI_FALLBACK_MODEL` without a thinking level, unless the fallback is empty or equal to the primary (`:603-613`).

Validation of Gemini output (`_validate` , `:615-669`): at most 14 calls considered; `navigate` only to an allowed path and only once; `find_element` queries that match the sensitive pattern are dropped and a search ends the batch; `click` only on clickable, enabled, known ids; `toggle` only on checkbox/radio/switch; `fill_field` only on enabled non-sensitive inputs with a non-empty value; `select_option` option names are matched against the element's real options (ambiguity becomes a question); `ask_user` options must be known option strings or Yes/No/Skip/That's all, questions are cut to 140 characters; `wait` is capped at 2000 ms; replies are cut to 300 characters. With nothing usable it answers "Sorry, I didn't understand that...".

---

3. Routes the agent may open (`voiceRoutes`) and admin-only filtering

`frontend/src/utils/voice.ts:71-94` defines 14 routes with spoken names; `voiceRoutes(isAdmin)` removes `/sources` , `/settings` , `/evaluation` for non-admins (`ADMIN_ONLY` , `voice.ts:72,92-94`). The hook sends the filtered list on each request, using `session.user()?.role === 'admin'` (`useVoiceAgent.ts:211`). The server independently keeps an allowlist: `ALLOWED_PATHS` (`/dashboard, /brand, /social, /apps, /web, /investigations, /lookalike, /sources, /reports, /settings, /evaluation, /setup, /protect-brand, /studio/`) and only honours routes present in both lists (`backend/app/services/voice.py:34-37,516,618`).

Note: the server allowlist contains the admin pages and trusts the browser's route list to hide them; the real protection is the `RequireAdmin` route guard, which sends a non-admin to `/dashboard` even if the agent navigates there (`frontend/src/App.tsx:38-54,174-179`). Consequence: a non-admin who bypasses the browser list could ask the server to plan a `navigate` , but the page would still redirect.

---

4. Sensitive-field rules (three layers)

Layer	Rule	Source
Snapshot builder	Elements whose label, `data-voice` , id, `name` or `autocomplete` match `pass(word	code) , pwd , secret , token , otp , one-time , pin , cvv , cvc , card , credit , ssn , api key , credential , auth are never described; autocomplete="one-time-code"` too; password, hidden, file, range, color inputs have no type
Request filter (server)	Elements matching the server `SENSITIVE` regex in label or name are dropped from the snapshot before planning "whatever the browser says"	`backend/app/api/voice.py:59-60` , `backend/app/services/voice.py:38`
Planner and validators	`_find_field` , `_next_empty` , `_validate` skip sensitive elements; a request such as "enter my password" is answered "Please type this field yourself." and a `highlight` action focuses the field	`services/voice.py:121-123,198-217,272,359-360,638`
Fill helpers	`usable()` rejects password/hidden/file/checkbox/radio/range inputs and names matching the client `SENSITIVE`	`voice.ts:96-106`
Logging	Snapshots, values and the API key are never logged; only exception type and status code	`services/voice.py:16-17,534-537`

---

5. Debug flag `voiceDebug` (development only)

`dbg()` prints `[voice] ...` to the console only when `import.meta.env.DEV` is true **and** `localStorage.voiceDebug === '1'` . It never runs in a production build (`frontend/src/utils/voiceSnapshot.ts:18-25`). It logs snapshot sizes, id lookup failures, matched elements, `find_element` results and dropdown option counts (`voiceSnapshot.ts:244` , `voiceExecutor.ts:127,209,278,284`). Enable in the browser console: `localStorage.setItem('voiceDebug','1')` .

---

6. Files, APIs, tables and services

Item	Detail
Frontend files	`frontend/src/components/voice/VoiceAgentButton.tsx` (+ `.css`), `frontend/src/hooks/useVoiceAgent.ts`, `frontend/src/utils/voice.ts`, `voiceSnapshot.ts`, `voiceExecutor.ts`, `frontend/src/api/sm.ts:73`
Backend files	`backend/app/api/voice.py`, `backend/app/services/voice.py` (`VoiceAgent`, `Ctx`, `Route`, `VoiceResult`, `fast_command`, `fast_fill`, `gate`), `backend/app/config/ratelimit.py`
API	`POST /api/voice`
DB tables	None. The voice agent does not read or write the database; it only authenticates the user.
External services	Google Gemini through the `google-genai` SDK (imported lazily, `services/voice.py:505-506`); RapidFuzz library (local)
Tests	`frontend/src/utils/voice.test.ts`, `frontend/e2e/voice.spec.ts` (fake `SpeechRecognition`, uses the fast path, no Gemini key; second test checks the unsupported-browser message)

7. Related configuration (names only)

Name	Purpose	Default (`backend/app/config/config.py:144-150`)
`VOICE_GEMINI_API_KEY`	Gemini key for voice (secret); falls back to `GEMINI_API_KEY` if unset (`services/voice.py:495`). With no key, only fast-path commands work	none
`VOICE_GEMINI_MODEL`	Primary model, must support function calling	`gemini-3-flash-preview`
`VOICE_GEMINI_FALLBACK_MODEL`	Tried once when the primary fails within 4 s	`gemini-flash-latest`
`VOICE_THINKING_LEVEL`	`minimal`, `low`, ...; empty = model default	`minimal`
`VOICE_TIMEOUT_SECONDS`	HTTP deadline (floor of 10 s, "Gemini refuses a deadline under 10 s")	10.0
`VOICE_RATE_PER_MINUTE`	Per-user limit, in memory, single process	30

Browser-side limits are constants, not config: 12 steps, 30 s, 14 rounds, 10 s question silence, language `en-IN` (`useVoiceAgent.ts:46-50,205`).

8. Edge cases

- **Mic is deaf while the agent talks** and after it ends if `onRef` is still on; `speak()` guards against a missing `end` event (`useVoiceAgent.ts:131-132`).
- **Anything said while it is working is ignored** except "stop" (`useVoiceAgent.ts:289-296`).
- **Page changes mid-plan:** each round re-snapshots; stale ids fall back to a label search (`voiceExecutor.ts:274-283`).
- **Sign-out or unmount** while ON releases the microphone (`useVoiceAgent.ts:397-398`). The button is only mounted under `RequireAuth`.
- **Studio:** `/studio/` is a route the agent may open; it triggers a full page load (`useVoiceAgent.ts:199`). The voice button does not exist inside Studio (separate app).
- **Not understood:** the stock reply `NOT_UNDERSTOOD` suggests "open reports" or "fill brand name as Acme" (`services/voice.py:47`).
- The `Permissions-Policy` header on Vercel and in the frontend nginx is `microphone=(self)` (`vercel.json:106`, `frontend/nginx/default.conf.template:17`), so the page may use the microphone. It was `microphone=()` (blocked) until it was corrected; the standalone Studio container still sends `microphone=()`. Not verified on a deployed build.

DOCS/FEATURES/LANDING-PAGE.MD

Landing Page

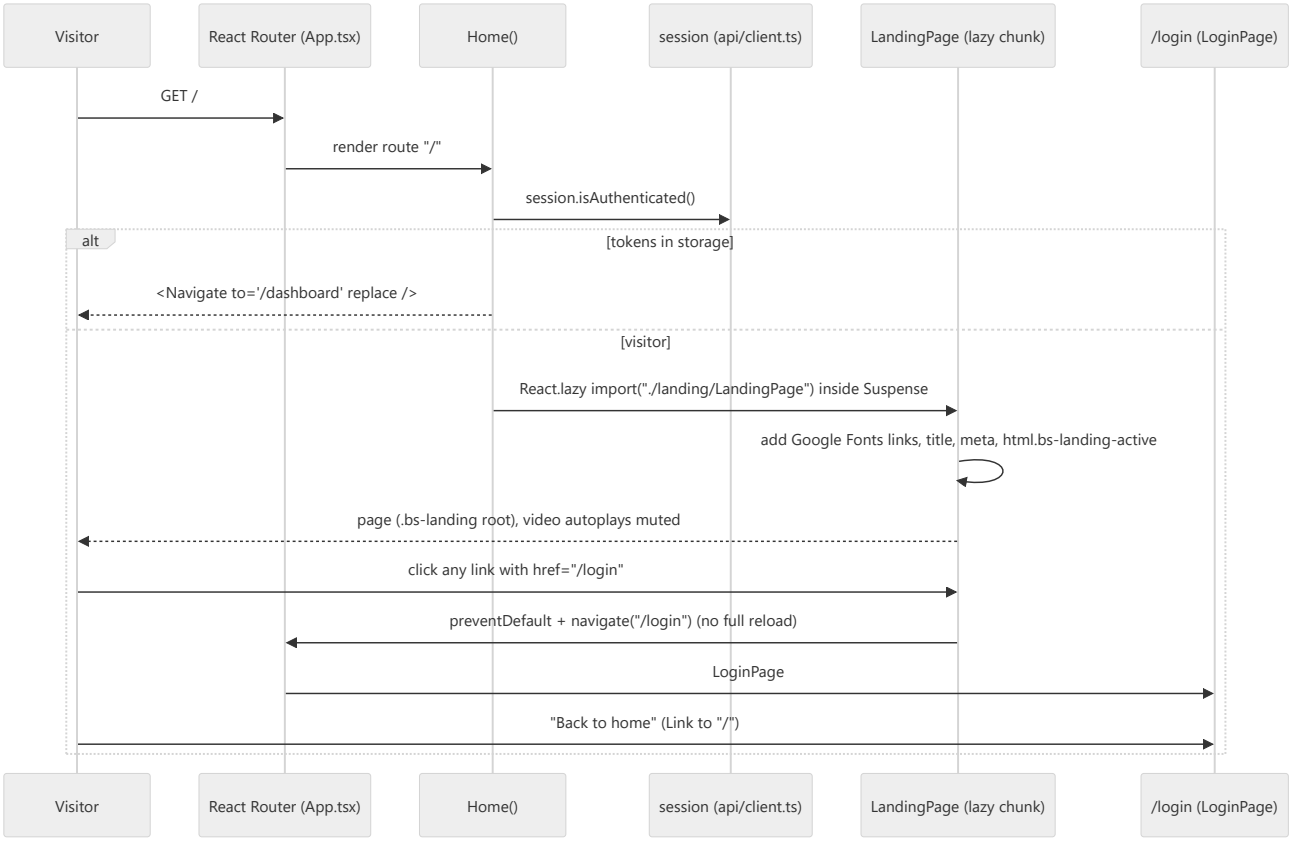
Plain-language summary

The landing page is the public marketing page people see when they open BrandShield's address (`/`) without being signed in. It explains what BrandShield does (finding fake social profiles, cloned apps and look-alike websites), plays a short product video and invites visitors to sign in. It is a ready-made design that was ported into the React app as-is, so it does not use the app's own styling. Every "get started" style button leads to the login page. Visitors who are already signed in never see it: `/` sends them straight to their dashboard.

1. Purpose and user-facing behavior

Behavior	Detail	Source
Who sees it	Signed-out visitors at `/` . Signed-in users are redirected to `/dashboard` .	`frontend/src/App.tsx:28-36,183`
Content	Header with mega menus, hero with search box and floating orb, hero video card, "expertise" scroll section, example cards (fake profile, cloned app, look-alike domain), sources marquee, "about", "how it works" (`#engine`), Brand Studio section (`#studio`), footer	section ids at `frontend/src/landing/LandingPage.tsx:655,820,1002,1239,1324,1508,1664`
Calls to action	All 19 `href="/login"` links (header buttons, pills, round buttons, the hero search "Check" button, the Studio section button, footer) open the login page	`LandingPage.tsx` (for example `:275-346,576-618,704,1679`)
In-page links	`#top` , `#expertise` , `#how` , `#work` , `#marquee` , `#about` , `#engine` , `#studio` scroll within the page; smooth scrolling is turned on only while the page is mounted	`frontend/src/landing/landing.css:209`
External links	GitHub repository and three docs (`USER_GUIDE.md` , `FEATURE_STATUS.md` , `SECURITY.md`) with `target="_blank" rel="noopener"`	`LandingPage.tsx:490-529`
Back to home	The login and register pages show a "Back to home" link to `/`	`frontend/src/pages/LoginPage.tsx:117-120` , `frontend/src/pages/RegisterPage.tsx:126-129`
Page title	Sets `document.title` to "BrandShield — Digital Risk Protection with explainable scores" and a meta description while mounted, restored on leave	`LandingPage.tsx:9-10,21-49`

2. Step-by-step flow



1. **Route.** `<Route path="/" element={<Home />} />` sits outside `RequireAuth`, so it is public (`frontend/src/App.tsx:183`).
2. `Home()` returns `<Navigate to="/dashboard" replace />` when `session.isAuthenticated()` (a token pair is in storage), otherwise renders the lazily loaded landing inside `<Suspense fallback={null}>` (`App.tsx:29-36`). The landing code is a separate chunk loaded with `lazy(() => import('./landing/LandingPage'))` so app users never download it (`App.tsx:26`).
3. **Document-level setup.** `useEffect` sets title and description, appends two `<link rel="stylesheet">` tags (Manrope and Unbounded from Google Fonts), and adds class `bs-landing-active` to `<html>`; the cleanup removes everything (`LandingPage.tsx:5-8,20-49`).
4. **Cursor dot.** A small dot follows the mouse through a ref (no React state per move). It is skipped unless the device supports hover and the user has not asked for reduced motion (`LandingPage.tsx:51-61`).
5. **Hero video.** `<video id="introVideo" src="/brandshield-intro.mp4" autoPlay muted loop playsInline preload="metadata">` (`LandingPage.tsx:638`); the file is `frontend/public/brandshield-intro.mp4`. A shield-icon fallback (`.vfallback`) shows until the `playing` event hides it. On a `error` event the sound button is hidden. With `prefers-reduced-motion: reduce` autoplay is removed, the video is paused and native controls appear. The "Sound on" button toggles `muted`, starts playback if paused and updates `aria-pressed`, `aria-label` and its text (`LandingPage.tsx:63-93`).
6. **CTA navigation.** One `onClick` on the `.bs-landing` root finds the closest `<a>`; if its `href` is exactly `/login` and no meta, ctrl or shift key is held, it calls `navigate('/login')` (client-side navigation). Modified clicks fall through to the browser (open in new tab) (`LandingPage.tsx:98-106`). Because the links are real anchors, the page also works if JavaScript routing fails.
7. **Login.** `/login` is `LoginPage`; after sign-in the user continues to the app (the `?next=/studio/` rule is unrelated to this page; see `docs/features/brand-studio.md`).

---

3. How the page was built (generated from a standalone design)

- The file header says: "Generated from the approved standalone design; markup is a verbatim port (class -> className, style strings -> objects)" (`LandingPage.tsx:1`). It is a 1,851-line component whose JSX is the exported HTML, with inline `style` objects and inline SVG icons. There is no component decomposition and no content from the API.
- **Scoped CSS.** All styles are in `frontend/src/landing/landing.css` (269 lines), imported by the component (`LandingPage.tsx:4`). Every selector is prefixed with `.bs-landing` (196 occurrences), and the design tokens are defined on `.bs-landing` (`--bg`, `--ink`, `--acc`, `--b1`, `--b2`, ...) rather than `:root` (`landing.css:1-5`). This keeps the page from affecting, or being affected by, the app's `tokens.css` and light-mode patches. The only global rule is `html.bs-landing-active{scroll-behavior:smooth}` (`landing.css:209`).
- **"Port addendum" blocks** at the end of the CSS fix things the app's global reset broke (input padding) and shorten the scroll distance of the hero video (`landing.css:187-269` region).
- **Fonts** come from Google Fonts at runtime (added in `useEffect`), so `connect` / `style-src` / `font-src` policies must allow `fonts.googleapis.com` and `fonts.gstatic.com`; the Vercel CSP does (`vercel.json:114`).
- **Test:** `frontend/src/landing/LandingRoute.test.tsx` checks that a signed-out visitor sees the heading matching /Threat detection/ inside a `.bs-landing` element and that a signed-in user is redirected to `/dashboard` (`LandingRoute.test.tsx:15-30`).

---

4. Scroll-driven video card

The hero video sits in a card that grows to full screen as you scroll.

- Markup order: `.herowrap` contains `.vlayer` > `figure.vcard` (the video) and then `section#top.hero` (`LandingPage.tsx:628-655`). A spacer `<div id="how" className="vspacer">` sits after the hero (`LandingPage.tsx:769`).
- Default layout: `.vlayer` is absolutely positioned over the hero (`height:max(100vh,720px)`, `z-index:40`, `pointer-events:none`), and the card is placed bottom right (`right:32px; bottom:100px`) with size `--cw` (`clamp(340px,27vw,520px)`) by `--ch` (16:9) and a rounded shape `110px 24px 24px 24px` (`landing.css:211-215,247`).
- With `@supports (animation-timeline: scroll())`: `.vlayer` becomes `position: sticky; height: 100vh; margin-bottom: -100vh`, and the card runs the `vgrow` keyframes tied to `scroll(root)` from 0 to 420 px of page scroll, growing to `width:100%; height:100vh; border-radius:0`. The "Product tour" chip fades between 260 and 420 px. `.vspacer` adds 420 px of scroll room (`landing.css:224-232,264-268`). The first version used 720 px and 1000 px; an addendum shortened it (`landing.css:261-268`).
- Below 1100 px width the card becomes a normal block under the hero (`position:relative`, `height:56vw`, no animation) (`landing.css:233-236`). With `prefers-reduced-motion` the animation is off and the layer stays absolute (`landing.css:237`). Browsers without scroll-driven animations simply show the static bottom-right card.
- The "expertise" section uses a similar mechanism with `view-timeline` (`.exp{height:1500px; view-timeline-name:--exp}`) for the growing card, text swaps and text colour reveals (`landing.css:88,187-200`).

---

5. Files and dependencies

Path	Role
`frontend/src/landing/LandingPage.tsx`	The page (default export is the named export `LandingPage`)
`frontend/src/landing/landing.css`	All page CSS, scoped under `.bs-landing`
`frontend/src/landing/LandingRoute.test.tsx`	Route test
`frontend/public/brandshield-intro.mp4`	Hero video (served from the site root)
`frontend/src/App.tsx:26-36,183`	`lazy` import and `Home()`
`frontend/src/pages/LoginPage.tsx`, `RegisterPage.tsx`	"Back to home" links (`data-testid="back-to-home"`)

- APIs used: none. DB tables: none. External services: Google Fonts (CSS only); the example cards and logos are inline markup, not fetched.
- Permissions: public route. Nothing is rendered conditionally on role.

- Edge cases: a stale or expired token in storage still counts as authenticated for `Home()`, so the user lands on `/dashboard`, where the API client refreshes or signs them out (`App.tsx:30`, `frontend/src/api/client.ts:108-136`). If the video file is missing the fallback shield stays and the sound button is hidden.
- Related config: none specific (no env var or feature flag). The video path is hard coded as `/brandshield-intro.mp4`.

Needs confirmation: the "standalone HTML design" the page was generated from is not in the repository (only its React port is), so the mapping from design to `LandingPage.tsx` cannot be re-run from this repo; the person who produced the port should confirm where the source design lives.